

```
=====
FILE: IDMS-DB2-Code-Conversion\catalogs\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\catalogs\cobol_reserved_words.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\catalogs\copybook_schema.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\catalogs\db2_naming_catalog.py
ENCODING: utf-8
=====
```

"""

DB2 naming catalog.

This file owns DB2 naming conventions so resolvers and generators do not
hardcode table suffixes or cursor suffix rules.

"""

```
DB2_TABLE_SUFFIX_EQUIVALENTS = [
    ("_TB", "_TV"),
    ("_TV", "_TB"),
    ("TB", "TV"),
    ("TV", "TB"),
]
```

```
DB2_CURSOR_SUFFIX = "C1"
```

```
DB2_DEFAULT_CURSOR_NAME = "DB2CURC1"
```

```
DB2_HOST_REFERENCE_PREFIX = ":"
```

```
=====
FILE: IDMS-DB2-Code-Conversion\catalogs\db2_sql_types.py
ENCODING: utf-8
=====
```

"""

DB2 SQL type catalog.

DCLGEN parsing logic must import SQL type starters from here.

"""

```
DB2_SQL_TYPE_STARTERS = {
    "CHAR",
    "CHARACTER",
    "VARCHAR",
    "LONG",
    "GRAPHIC",
    "VARGRAPHIC",
    "SMALLINT",
    "INTEGER",
    "INT",
}
```

```

    "BIGINT",
    "DECIMAL",
    "DEC",
    "NUMERIC",
    "NUM",
    "FLOAT",
    "REAL",
    "DOUBLE",
    "DATE",
    "TIME",
    "TIMESTAMP",
    "BLOB",
    "CLOB",
    "DBCLOB",
    "XML",
}

```

```

DB2_SQL_SKIP_WORDS = {
    "EXEC",
    "SQL",
    "DECLARE",
    "TABLE",
    "END-EXEC",
    "END",
    "NOT",
    "NULL",
    "WITH",
    "DEFAULT",
    "PRIMARY",
    "FOREIGN",
    "KEY",
    "CONSTRAINT",
    "UNIQUE",
    "CHECK",
    "REFERENCES",
    "CREATE",
    "IN",
    "IS",
    "THE",
    "DCLGEN",
    "COMMAND",
    "THAT",
    "MADE",
    "FOLLOWING",
    "STATEMENTS",
}

```

```

=====
FILE: IDMS-DB2-Code-Conversion\catalogs\dclgen_schema.py
ENCODING: utf-8
=====

```

```

"""
DCLGEN schema catalog.

This file owns DCLGEN-related static names and conventions.
Parser and service classes must import from here instead of hardcoding
DCLGEN names internally.
"""

```

```

DCLGEN_GROUP_PREFIX = "DCL"

```

```

DCLGEN_COLUMN_FIELDS = [
    "table_name",
    "column_name",
    "db2_type",
    "cobol_host_name",
    "cobol_picture",
    "cobol_usage",
    "nullable",
]

```

```
DCLGEN_HOST_FIELD_SUFFIXES_TO_IGNORE = [
    "-NULL",
    "-LEN",
    "-TEXT",
]
```

```
=====
FILE: IDMS-DB2-Code-Conversion\catalogs\input_file_roles.py
ENCODING: utf-8
=====
```

```
"""
Input file role catalog.

Input labels are centralized here so UI, runners, validators, and diagnostics
do not hardcode file role names.
"""
```

```
INPUT_ROLE_SHEET_MAPPING = "Sheet Mapping"
INPUT_ROLE_DCLGEN = "DCLGEN"
INPUT_ROLE_COPYBOOK = "Copybook"
INPUT_ROLE_IDMS_COBOL_SOURCE = "IDMS COBOL Source"
```

```
REQUIRED_INPUT_ROLES = [
    INPUT_ROLE_SHEET_MAPPING,
    INPUT_ROLE_DCLGEN,
    INPUT_ROLE_IDMS_COBOL_SOURCE,
]
```

```
OPTIONAL_INPUT_ROLES = [
    INPUT_ROLE_COPYBOOK,
]
```

```
=====
FILE: IDMS-DB2-Code-Conversion\catalogs\output_sections.py
ENCODING: utf-8
=====
```

```
"""
Output section titles and generated block markers.

Generators and composers must import section labels from this catalog instead
of hardcoding generated block titles inside logic classes.
"""
```

```
DB2_INFRASTRUCTURE_MARKER = (
    "DB2 SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR FLAGS"
)
```

```
DB2_SQL_ERROR_LOCATION_MARKER = "DB2 SQL ERROR LOCATION"
```

```
DB2_CURSOR_FLAGS_MARKER = "DB2 CURSOR END-OF-CURSOR FLAGS"
```

```
DB2_CURSOR_DECLARATIONS_MARKER = "DB2 CURSOR DECLARATIONS"
```

```
DB2_CURSOR_PARAGRAPH_MARKER = "DB2 GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS"
```

```
DB2_GENERATED_TIMESTAMP_MARKER = "DB2 GENERATED TIMESTAMP AND AUDIT WORKING STORAGE"
```

```
SQLERRWS_INCLUDE_NAME = "SQLERRWS"
```

```
SQLCA_INCLUDE_NAME = "SQLCA"
```

```
SQL_LOCATION_FIELD_NAME = "SQL-LOCATION"
```

```
SQL_LOCATION_PICTURE = "PIC X(40) VALUE SPACES."
```

```
=====
FILE: IDMS-DB2-Code-Conversion\catalogs\sheet_mapping_schema.py
ENCODING: utf-8
=====
```

```
"""
Sheet Mapping schema catalog.
```

```
This module owns all Sheet Mapping names, aliases, and model-field mapping.
Parser logic must not hardcode Sheet Mapping column labels.
"""
```

```
SHEET_MAPPING_COLUMN_IDMS_TO_DB2_MAPPING = "IDMS to DB2 Mapping"
SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS = "Cobol Record IDMS"
SHEET_MAPPING_COLUMN_COBOL_ZONE = "Cobol Zone"
SHEET_MAPPING_COLUMN_IDMS_KEY = "IDMS Key"
SHEET_MAPPING_COLUMN_IDMS_PIC_CLAUSE = "IDMS PIC Clause"
SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES = "Length of Field Bytes"
SHEET_MAPPING_COLUMN_FIELD_END_POSITION = "Field end position"
SHEET_MAPPING_COLUMN_DB2_KEY = "DB2 Key"
SHEET_MAPPING_COLUMN_NEW_DB2_RECORD = "New DB2 Record"
SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME = "New DB2 Field name"
SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE = "New DB2 Data Type"
SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK = "Hopex Expression TypeRemark"
SHEET_MAPPING_COLUMN_REMARKS = "Remarks"
SHEET_MAPPING_COLUMN_RELATION = "Relation"
SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK = "Reference Field Name (CopyBook) "
SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE = "Reference Field PIC Clause"
SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE = "Cross Application DB2 table"
SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME = "Cross Application DB2 Field Name"
SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE = "Cross Appln DB2 Data Type"
SHEET_MAPPING_COLUMN_BASETYPE = "Basetype"
```

```
SHEET_MAPPING_CANONICAL_COLUMNS = [
    SHEET_MAPPING_COLUMN_IDMS_TO_DB2_MAPPING,
    SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS,
    SHEET_MAPPING_COLUMN_COBOL_ZONE,
    SHEET_MAPPING_COLUMN_IDMS_KEY,
    SHEET_MAPPING_COLUMN_IDMS_PIC_CLAUSE,
    SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES,
    SHEET_MAPPING_COLUMN_FIELD_END_POSITION,
    SHEET_MAPPING_COLUMN_DB2_KEY,
    SHEET_MAPPING_COLUMN_NEW_DB2_RECORD,
    SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME,
    SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE,
    SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK,
    SHEET_MAPPING_COLUMN_REMARKS,
    SHEET_MAPPING_COLUMN_RELATION,
    SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK,
    SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE,
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE,
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME,
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE,
    SHEET_MAPPING_COLUMN_BASETYPE,
]
```

```
SHEET_MAPPING_FIELD_ALIASES = {
    SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS: [
        "Cobol Record IDMS",
        "COBOL RECORD IDMS",
        "Cobol Record",
        "COBOL Record",
        "IDMS Record",
        "Record IDMS",
        "Cobol Record Name",
```

```

        "IDMS to DB2 Mapping",
        "Cobol Recrd IDMS",
        "COBOL RECRD IDMS",
        "Cobol Rec IDMS",
        "COBOL REC IDMS",
        "Cobol Rord IDMS",
        "COBOL RCRD IDMS",
        "Cobol Recrd",
        "COBOL RECRD",
    ],
    SHEET_MAPPING_COLUMN_COBOL_ZONE: [
        "Cobol Zone",
        "COBOL Zone",
        "COBOL ZONE",
        "Cobol Field",
        "COBOL Field",
        "IDMS Field",
        "IDMS COBOL Zone",
        "IDMS COBOL Field",
        "Zone",
    ],
    SHEET_MAPPING_COLUMN_IDMS_KEY: [
        "IDMS Key",
        "IDMS KEY",
        "Key IDMS",
        "IDMS_Key",
    ],
    SHEET_MAPPING_COLUMN_IDMS_PIC_CLAUSE: [
        "IDMS PIC Clause",
        "IDMS PIC",
        "PIC Clause",
        "Picture",
        "PIC",
    ],
    SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES: [
        "Length of Field Bytes",
        "Length",
        "Field Length",
        "Length Bytes",
        "Length of Field",
    ],
    SHEET_MAPPING_COLUMN_FIELD_END_POSITION: [
        "Field end position",
        "Field End Position",
        "End Position",
        "Field End",
    ],
    SHEET_MAPPING_COLUMN_DB2_KEY: [
        "DB2 Key",
        "DB2 key",
        "DB2 KEY",
        "Key DB2",
        "DB2_key",
        "DB2_Key",
        "DB2KEY",
    ],
    SHEET_MAPPING_COLUMN_NEW_DB2_RECORD: [
        "New DB2 Record",
        "New DB2 Record ",
        "DB2 Record",
        "DB2 Table",
        "New DB2 Table",
        "Table",
        "DB2_Table",
        "New DB2_Record",
    ],
    SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME: [
        "New DB2 Field name",
        "New DB2 Field Name",
        "New DB2_Field name",
        "New DB2_Field Name",
        "New DB2 Field",
        "New DB2_Field",
        "DB2 Field",
        "DB2 Column",
        "New DB2 Column",
    ]

```

```

        "Column",
        "DB2_Field",
        "DB2_Column",
    ],
    SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE: [
        "New DB2 Data Type",
        "New DB2 DataType",
        "New DB2_Data Type",
        "DB2 Data Type",
        "DB2 Type",
        "Data Type",
    ],
    SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK: [
        "Hopex Expression TypeRemark",
        "Hopex Expression Type",
        "Expression Type",
        "Hopex Remark",
    ],
    SHEET_MAPPING_COLUMN_REMARKS: [
        "Remarks",
        "Remark",
        "Comments",
        "Comment",
    ],
    SHEET_MAPPING_COLUMN_RELATION: [
        "Relation",
        "Relationship",
        "Set Relation",
    ],
    SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK: [
        "Reference Field Name (CopyBook) ",
        "Reference Field Name (CopyBook)",
        "Reference Field Name",
        "CopyBook Field",
        "Copybook Field",
        "Reference Field",
    ],
    SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE: [
        "Reference Field PIC Clause",
        "Reference PIC Clause",
        "Reference PIC",
    ],
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE: [
        "Cross Application DB2 table",
        "Cross Application DB2 Table",
        "Cross App DB2 Table",
        "Cross Application Table",
    ],
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME: [
        "Cross Application DB2 Field Name",
        "Cross App DB2 Field Name",
        "Cross Application DB2 Field",
        "Cross App DB2 Field",
    ],
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE: [
        "Cross Appln DB2 Data Type",
        "Cross Application DB2 Data Type",
        "Cross App DB2 Data Type",
    ],
    SHEET_MAPPING_COLUMN_BASETYPE: [
        "Basetype",
        "Base Type",
        "BaseType",
    ],
    ],
}

```

```

SHEET_MAPPING_MODEL_FIELD_MAP = {
    "cobl_record_idms": SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS,
    "cobl_zone": SHEET_MAPPING_COLUMN_COBOL_ZONE,
    "idms_key": SHEET_MAPPING_COLUMN_IDMS_KEY,
    "idms_pic_clause": SHEET_MAPPING_COLUMN_IDMS_PIC_CLAUSE,
    "length_of_field_bytes": SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES,
    "field_end_position": SHEET_MAPPING_COLUMN_FIELD_END_POSITION,
    "db2_key": SHEET_MAPPING_COLUMN_DB2_KEY,
    "new_db2_record": SHEET_MAPPING_COLUMN_NEW_DB2_RECORD,
}

```

```

    "new_db2_field_name": SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME,
    "new_db2_data_type": SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE,
    "hopex_expression_type_remark": SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK,
    "remarks": SHEET_MAPPING_COLUMN_REMARKS,
    "relation": SHEET_MAPPING_COLUMN_RELATION,
    "reference_field_name_copybook": SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK,
    "reference_field_pic_clause": SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE,
    "cross_application_db2_table": SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE,
    "cross_application_db2_field_name": SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME,
    "cross_application_db2_data_type": SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE,
    "basetype": SHEET_MAPPING_COLUMN_BASETYPE,
}

```

```

SHEET_MAPPING_HEADER_DETECTION_GROUPS = [
    [
        SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS,
        "Cobol Recrd IDMS",
        "IDMS Record",
    ],
    [
        SHEET_MAPPING_COLUMN_NEW_DB2_RECORD,
        "DB2 Table",
        "DB2 Record",
    ],
    [
        SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME,
        "New DB2_Field name",
        "DB2 Column",
    ],
]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\config\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\config\app_settings.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\config\logging_settings.py
ENCODING: utf-8
=====

```

```

"""
Logging settings.

Log files are written to the project logs folder with date-time names.
"""

LOG_FILE_PREFIX = "idms_db2_conversion"

LOG_DATE_TIME_FORMAT = "%d-%m-%Y_%H%M%S"

LOG_MESSAGE_FORMAT = "%(asctime)s | %(levelname)s | %(name)s | %(message)s"

LOG_DATE_FORMAT = "%d-%m-%Y %H:%M:%S"

DEFAULT_LOG_LEVEL = "INFO"

```

```
=====
FILE: IDMS-DB2-Code-Conversion\config\path_settings.py
ENCODING: utf-8
=====
```

```
"""
```

```
Path settings for local command-line runs.
```

```
Update these values for your local machine.
```

```
This file is intentionally separate from parser, transformer, generator,
and orchestration logic so file paths are not hardcoded inside business logic.
```

```
"""
```

```
from pathlib import Path
```

```
# -----
# Project folders
# -----
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[1]
```

```
SRC_DIR = PROJECT_ROOT / "src"
```

```
LOGS_DIR = PROJECT_ROOT / "logs"
```

```
# -----
# Default local input/output folders
# -----
# Change this folder if your input files are stored somewhere else.
#
# Expected default layout:
#
# C:\S\S-Input
# |— Excel_Sheet_mapping.csv
# |— Retrieval.txt
# |— Update.txt
# |— DCLGENS_BEFF.txt
# |— DCLGENS_BFAR.txt
# |— DCLGENS_EVEF.txt
# |— Output
#
# -----
```

```
DEFAULT_INPUT_DIR = Path(r"C:\S\S-Input")
```

```
DEFAULT_OUTPUT_DIR = DEFAULT_INPUT_DIR / "Output"
```

```
# -----
# Required input files
# -----
```

```
DEFAULT_SHEET_MAPPING_PATH = DEFAULT_INPUT_DIR / "Excel_Sheet_mapping.csv"
```

```
DEFAULT_RETRIEVAL_SOURCE_PATH = DEFAULT_INPUT_DIR / "Retrieval.txt"
```

```
DEFAULT_UPDATE_SOURCE_PATH = DEFAULT_INPUT_DIR / "Update.txt"
```

```
# -----
# DCLGEN candidate files
# -----
# The runner will load only files that actually exist.
# Add more DCLGEN files here if needed.
# -----
```

```
DEFAULT_DCLGEN_CANDIDATE_PATHS = [
    DEFAULT_INPUT_DIR / "DCLGENS_BEFF.txt",
    DEFAULT_INPUT_DIR / "DCLGENS_BFAR.txt",
    DEFAULT_INPUT_DIR / "DCLGENS_EVEF.txt",
]
```



```

# -----
# Optional copybook candidate files
# -----
# The runner will load only files that actually exist.
# Copybook is optional.
# -----

DEFAULT_COPYBOOK_CANDIDATE_PATHS = [
    DEFAULT_INPUT_DIR / "Copybook.txt",
]

# -----
# Utility helpers
# -----

def existing_files(
    paths: list[Path],
) -> list[Path]:
    """
    Return only paths that exist and are files.
    """

    return [
        path
        for path in paths
        if path.exists() and path.is_file()
    ]

def ensure_output_dir() -> None:
    """
    Ensure the default output directory exists.
    """

    DEFAULT_OUTPUT_DIR.mkdir(
        parents=True,
        exist_ok=True,
    )

=====
FILE: IDMS-DB2-Code-Conversion\logs\idms_db2_conversion_17-08-2026_234612.log
ENCODING: utf-8
=====

17-08-2026 23:46:12 | INFO | idms_db2_update_conversion | Logger initialized.
17-08-2026 23:46:12 | INFO | idms_db2_update_conversion | Log file: C:\VSCode\IDMS-DB2-CC-New\IDMS-DB2-Code-Conversion\
logs\idms_db2_conversion_17-08-2026_234612.log
17-08-2026 23:46:12 | INFO | idms_db2_update_conversion | START LOAD INPUTS
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | Sheet Mapping file: C:\S\S-Input\Excel_Sheet_mapping.csv
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | Sheet Mapping rows: 840
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | DCLGEN file count: 3
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | DCLGEN 1: C:\S\S-Input\DCLGENS_BEFF.txt
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | DCLGEN 1 text length: 3882
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | DCLGEN 2: C:\S\S-Input\DCLGENS_BFAR.txt
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | DCLGEN 2 text length: 7344
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | DCLGEN 3: C:\S\S-Input\DCLGENS_EVEF.txt
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | DCLGEN 3 text length: 3035
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | DCLGEN total columns: 89
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | Copybook file count: 1
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | Copybook 1: C:\S\S-Input\Copybook.txt
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | Copybook 1 text length: 115952
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | Copybook total fields: 1
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | IDMS COBOL source file: C:\S\S-Input\Update.txt
17-08-2026 23:46:13 | INFO | idms_db2_update_conversion | IDMS COBOL source text length: 14764
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DB2 COBOL generation completed.
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Output file created: C:\S\S-Input\Output\Update_17-08-2026_23
4614.cbl
17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Mapping validation: DB2 column mapping is missing. Table=D
Z01FFTB
17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Mapping validation: DB2 column mapping is missing. Table=D
Z01RSTB
17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Program flow analyzer: Sheet Mapping rows received: 840
17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Program flow analyzer: DCLGEN columns received: 89
17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Program flow analyzer: paragraphs detected: 7

```

17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Program flow analyzer: cursor loops detected: 0
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Program flow analyzer: output writes detected: 4
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Program flow analyzer: DB2 date usages detected: 16
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Qualified field rewrite: CT-RK-TGOOD OF VMBFAR -> CT-TGDSE RV-479BFAR OF DCLDZBFARTV
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Qualified field rewrite: NR-CIO-FORM-AS OF VMBFAR -> NR-CI OFMAS-479BFAR OF DCLDZBFARTV
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Qualified field rewrite: DA-CR-FORM-AS OF VMBFAR -> DA-CRF MAS-479BFAR OF DCLDZBFARTV
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Qualified field rewrite: NR-ID-FORM-AS OF VMBFAR -> NR-IDF MAS-479BFAR OF DCLDZBFARTV
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Qualified field rewrite: CT-OPG-FORM-AS OF VMBFAR -> CT-OP GFMAS-479BFAR OF DCLDZBFARTV
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Qualified field rewrite: CT-OPD-FORM-AS OF VMBFAR -> CT-OP DFMAS-479BFAR OF DCLDZBFARTV
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Field usage analyzer: records with usage detected: 2
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Field usage analyzer: record=FFRECAB, condition=0, output=0, move_source=0, move_target=6, dclgen=0
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Field usage analyzer: record=VMBFAR, condition=0, output=0, move_source=0, move_target=0, dclgen=47
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	DB2 infrastructure: no operation-specific DCLGEN includes resolved.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	DB2 infrastructure: generated infrastructure block inserted.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	DB2 cursor paragraphs: no cursor operations found.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Timestamp generator: timestamp working storage and paragraph ensured.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . AM-BINTR-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . AM-BSDEV-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . AM-CNDGFAR-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . AM-CNSGFAR-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . AM-CRNGFAR-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . AM-PUT-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . AM-TPREM-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . AM-UPREM-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . AM-VGL-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CO-CONT-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CO-DEV-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CO-RIGFAR-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CO-RVUPR-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CO-STATGFR-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CO-SUBTP-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CO-TPVGL-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CO-TYP-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CT-OPDFMAS-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CT-OPGMAS-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . CT-TGDSERV-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . DA-CRFMAS-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . DA-INFSDGD-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . DA-UEGDFAR-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV . DA-VAGDFAR-479BFAR was not found in uploaded DCLGEN columns.
17-08-2026 23:46:14	WARNING	idms_db2_update_conversion	Production validation: generated host variable DCLDZBFARTV

. NB-KRDEV-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-CDGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-CIOFMAS-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-CIOFMAS-479SIAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-CIW-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-DSGDFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-IDFMAS-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-IDFMAS-479SIAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-IDRGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-IDXGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-ISGDIF-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-PROD-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-RTRGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-STGDFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NS-DBGDFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NS-IDGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NS-IDR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NS-INFSPD-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. RA-PUT-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. RA-RVTPR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. RA-RVUPR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. RA-TIGDFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. SX-INFSD-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | START LOAD INPUTS

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Sheet Mapping file: C:\S\S-Input\Excel_Sheet_mapping.csv

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Sheet Mapping rows: 840

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Sheet Mapping file name: excel_sheet_mapping.csv

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Sheet Mapping file size bytes: 80425

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | CSV/text decoded length: 80425

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | CSV/text sample: Cobol Recrd IDMS,Cobol Zone,IDMS Key,IDMS PI
C Clause,Length of Field Bytes,Field end position,DB2 key,New DB2 Record ,New DB2_Field name,New DB2 DataType,Hopex Exp
ression Type,Remarks,Relation,Reference Field Name (Copybook),Reference Field PIC Clause,Cross Application DB2 table,Cr
oss Application DB2 Field Name,Cross Appln DB2 DataType,,Basetype\r\nVMBFC, 02 KY-VMBFC.,,9,9,,,,,,,,,,,,,\r\nVMBF
C, 03 CT-RK-TGDSV,CALC,PIC X(4),4,13,Primary Key,DZMBFCTB,CT_RKTGDSC_479MBFC,CHAR(4),Identifier(*) NR

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | CSV detected headers: ['Cobol Recrd IDMS', 'Cobol Zone', 'IDM
S Key', 'IDMS PIC Clause', 'Length of Field Bytes', 'Field end position', 'DB2 key', 'New DB2 Record', 'New DB2_Field n
ame', 'New DB2 DataType', 'Hopex Expression Type', 'Remarks', 'Relation', 'Reference Field Name (Copybook)', 'Reference
Field PIC Clause', 'Cross Application DB2 table', 'Cross Application DB2 Field Name', 'Cross Appln DB2 DataType', '',
'Basetype']

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | CSV parsed useful rows: 840

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Sheet Mapping populated Cobol Record IDMS rows: 835

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Sheet Mapping populated source field rows: 630

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Sheet Mapping populated DB2 table rows: 617

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Sheet Mapping populated DB2 column rows: 560

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Sheet Mapping useful source-to-target rows: 370

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file count: 3

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN 1: C:\S\S-Input\DCLGENS_BEFF.txt

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN 1 text length: 3882

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN 2: C:\S\S-Input\DCLGENS_BFAR.txt

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN 2 text length: 7344

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN 3: C:\S\S-Input\DCLGENS_EVEF.txt

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN 3 text length: 3035

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN total columns: 89

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN input file count: 3

```

17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 1 input length: 3882
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 1: resolved table name: DZBEFFTV
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 1: DECLARE section 1 parsed SQL columns: 22
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 1: COBOL host field scan found 22 field(s).
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 1: SQL columns parsed: 22
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 1: COBOL host fields parsed: 22
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 1: merged DCLGEN columns: 22
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 1 parsed columns: 22
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 2 input length: 7344
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 2: resolved table name: DZBFARTV
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 2: DECLARE section 1 parsed SQL columns: 51
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 2: COBOL host field scan found 51 field(s).
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 2: SQL columns parsed: 51
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 2: COBOL host fields parsed: 51
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 2: merged DCLGEN columns: 51
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 2 parsed columns: 51
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 3 input length: 3035
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 3: resolved table name: DZEVEFTV
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 3: DECLARE section 1 parsed SQL columns: 16
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 3: COBOL host field scan found 16 field(s).
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 3: SQL columns parsed: 16
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 3: COBOL host fields parsed: 16
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 3: merged DCLGEN columns: 16
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN file 3 parsed columns: 16
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | DCLGEN total parsed columns: 89
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Copybook file count: 1
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Copybook 1: C:\S\S-Input\Copybook.txt
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Copybook 1 text length: 115952
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | Copybook total fields: 1
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | IDMS COBOL source file: C:\S\S-Input\Update.txt
17-08-2026 23:46:14 | INFO | idms_db2_update_conversion | IDMS COBOL source text length: 14764

```

```

=====
FILE: IDMS-DB2-Code-Conversion\logs\idms_db2_conversion_17-08-2026_235823.log
ENCODING: utf-8
=====

```

```

17-08-2026 23:58:23 | INFO | idms_db2_update_conversion | Logger initialized.
17-08-2026 23:58:23 | INFO | idms_db2_update_conversion | Log file: C:\VSCode\IDMS-DB2-CC-New\IDMS-DB2-Code-Conversion\
logs\idms_db2_conversion_17-08-2026_235823.log
17-08-2026 23:58:23 | INFO | idms_db2_update_conversion | START LOAD INPUTS
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping file: C:\S\S-Input\Excel_Sheet_mapping.csv
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping rows: 840
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file count: 3
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 1: C:\S\S-Input\DCLGENS_BEFF.txt
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 1 text length: 3882
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 2: C:\S\S-Input\DCLGENS_BFAR.txt
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 2 text length: 7344
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 3: C:\S\S-Input\DCLGENS_EVEF.txt
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 3 text length: 3035
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN total columns: 89
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Copybook file count: 1
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Copybook 1: C:\S\S-Input\Copybook.txt
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Copybook 1 text length: 115952
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Copybook total fields: 1
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | IDMS COBOL source file: C:\S\S-Input\Update.txt
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | IDMS COBOL source text length: 14764
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DB2 COBOL generation completed.
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Output file created: C:\S\S-Input\Output\Update_17-08-2026_23
5824.cbl
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Mapping validation: DB2 column mapping is missing. Table=D
Z01FFTB
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Mapping validation: DB2 column mapping is missing. Table=D
Z01RSTB
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Update restart/control skip: unmapped IDMS record FFRECAB
was skipped for manual redesign.
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Program flow analyzer: Sheet Mapping rows received: 840
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Program flow analyzer: DCLGEN columns received: 89
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Program flow analyzer: paragraphs detected: 7
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Program flow analyzer: cursor loops detected: 0
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Program flow analyzer: output writes detected: 4
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Program flow analyzer: DB2 date usages detected: 16
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Qualified field rewrite: CT-RK-TGOOD OF VMBFAR -> CT-TGDSE
RV-479BFAR OF DCLDZBFARTV
17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Qualified field rewrite: NR-CIO-FORM-AS OF VMBFAR -> NR-CI

```

OFMAS-479BFAR OF DCLDZBFARTV

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Qualified field rewrite: DA-CR-FORM-AS OF VMBFAR -> DA-CRF
MAS-479BFAR OF DCLDZBFARTV

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Qualified field rewrite: NR-ID-FORM-AS OF VMBFAR -> NR-IDF
MAS-479BFAR OF DCLDZBFARTV

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Qualified field rewrite: CT-OPG-FORM-AS OF VMBFAR -> CT-OP
GFMAS-479BFAR OF DCLDZBFARTV

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Qualified field rewrite: CT-OPD-FORM-AS OF VMBFAR -> CT-OP
DFMAS-479BFAR OF DCLDZBFARTV

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Field usage analyzer: records with usage detected: 2

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Field usage analyzer: record=FFRECAB, condition=0, output=
0, move_source=0, move_target=6, dclgen=0

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Field usage analyzer: record=VMBFAR, condition=0, output=0
, move_source=0, move_target=0, dclgen=47

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | DB2 infrastructure: no operation-specific DCLGEN includes
resolved.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | DB2 infrastructure: generated infrastructure block inserte
d.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | DB2 cursor paragraphs: no cursor operations found.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Timestamp generator: timestamp working storage and paragra
ph ensured.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. AM-BINTR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. AM-BSDEV-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. AM-CNDGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. AM-CNSGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. AM-CRNGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. AM-PUT-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. AM-TPREM-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. AM-UPREM-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. AM-VGL-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CO-CONT-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CO-DEV-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CO-RIGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CO-RVUPR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CO-STATGFR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CO-SUBTP-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CO-TPVGL-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CO-TYP-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CT-OPDFMAS-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CT-OPGFMAS-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. CT-TGDSERV-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. DA-CRFMAS-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. DA-INFSDGD-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. DA-UEGDFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. DA-VAGDFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. NB-KRDEV-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. NR-CDGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV
. NR-CIOFMAS-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-CIOFMAS-479SIAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-CIW-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-DSGDFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-IDFMAS-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-IDFMAS-479SIAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-IDRGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-IDXGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-ISGDIF-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-PROD-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-RTRGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NR-STGDFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NS-DBGDFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NS-IDGFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NS-IDR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. NS-INFSPD-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. RA-PUT-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. RA-RVTPR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. RA-RVUPR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. RA-TIGDFAR-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | WARNING | idms_db2_update_conversion | Production validation: generated host variable DCLDZBFARTV

. SX-INFSD-479BFAR was not found in uploaded DCLGEN columns.

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | START LOAD INPUTS

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping file: C:\S\S-Input\Excel_Sheet_mapping.csv

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping rows: 840

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping file name: excel_sheet_mapping.csv

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping file size bytes: 80425

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | CSV/text decoded length: 80425

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | CSV/text sample: Cobol Recrd IDMS,Cobol Zone,IDMS Key,IDMS PI
C Clause,Length of Field Bytes,Field end position,DB2 key,New DB2 Record ,New DB2_Field name,New DB2 DataType,Hopex Exp
ression Type,Remarks,Relation,Reference Field Name (Copybook),Reference Field PIC Clause,Cross Application DB2 table,Cr
oss Application DB2 Field Name,Cross Appln DB2 DataType,,Basetype\r\nVMBFC, 02 KY-VMBFC.,.,9,9,,,,,,,,,,,,,\r\nVMBF
C, 03 CT-RK-TGDSV,CALC,PIC X(4),4,13,Primary Key,DZMBFCTB,CT_RKTGDSC_479MBFC,CHAR(4),Identifier(*) NR

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | CSV detected headers: ['Cobol Recrd IDMS', 'Cobol Zone', 'IDM
S Key', 'IDMS PIC Clause', 'Length of Field Bytes', 'Field end position', 'DB2 key', 'New DB2 Record', 'New DB2_Field n
ame', 'New DB2 DataType', 'Hopex Expression Type', 'Remarks', 'Relation', 'Reference Field Name (Copybook)', 'Reference
Field PIC Clause', 'Cross Application DB2 table', 'Cross Application DB2 Field Name', 'Cross Appln DB2 DataType', '',
'Basetype']

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | CSV parsed useful rows: 840

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping populated Cobol Record IDMS rows: 835

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping populated source field rows: 630

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping populated DB2 table rows: 617

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping populated DB2 column rows: 560

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Sheet Mapping useful source-to-target rows: 370

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file count: 3

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 1: C:\S\S-Input\DCLGENS_BEFF.txt

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 1 text length: 3882

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 2: C:\S\S-Input\DCLGENS_BFAR.txt

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 2 text length: 7344

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 3: C:\S\S-Input\DCLGENS_EVEF.txt

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN 3 text length: 3035

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN total columns: 89

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN input file count: 3

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 1 input length: 3882

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 1: resolved table name: DZBEFFTV

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 1: DECLARE section 1 parsed SQL columns: 22

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 1: COBOL host field scan found 22 field(s).

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 1: SQL columns parsed: 22

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 1: COBOL host fields parsed: 22

```

17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 1: merged DCLGEN columns: 22
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 1 parsed columns: 22
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 2 input length: 7344
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 2: resolved table name: DZBFARTV
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 2: DECLARE section 1 parsed SQL columns: 51
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 2: COBOL host field scan found 51 field(s).
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 2: SQL columns parsed: 51
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 2: COBOL host fields parsed: 51
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 2: merged DCLGEN columns: 51
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 2 parsed columns: 51
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 3 input length: 3035
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 3: resolved table name: DZEVEFTV
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 3: DECLARE section 1 parsed SQL columns: 16
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 3: COBOL host field scan found 16 field(s).
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 3: SQL columns parsed: 16
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 3: COBOL host fields parsed: 16
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 3: merged DCLGEN columns: 16
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN file 3 parsed columns: 16
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | DCLGEN total parsed columns: 89
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Copybook file count: 1
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Copybook 1: C:\S\S-Input\Copybook.txt
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Copybook 1 text length: 115952
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | Copybook total fields: 1
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | IDMS COBOL source file: C:\S\S-Input\Update.txt
17-08-2026 23:58:24 | INFO | idms_db2_update_conversion | IDMS COBOL source text length: 14764

```

```

=====
FILE: IDMS-DB2-Code-Conversion\Mail
ENCODING: utf-8
=====

```

Hi Team,

The Python conversion fixes have been implemented and the latest output was cross-checked against the Iteration-1 feedback.

For the retrieval program, the reported issues are fixed:

COBOL output is now generated in proper 80-column format.
 Right sequence number is placed in columns 73-80.
 Comment indicator is placed in column 7.
 Compiler option is converted to CBL ARITH(EXTEND).
 PROGRAM-ID is replaced correctly.
 DB2 and DCLGEN includes are generated.
 Cursor declarations now include WITH HOLD.
 DCLGEN initialization is generated.
 Cursor EOC handling is moved to fetch paragraphs.
 DB2 date realignment logic is generated.
 DCLGEN field names are corrected.
 SQL-ERROR is replaced with SQLERROR, and EXEC SQL INCLUDE SQLERROR is included.
 For the update program, FFRECB is not available in the Sheet Mapping as a valid mapped business record. So, as discussed, Python does not hardcode FFRECB to any restart DB2 table. The logic now skips FFRECB conversion and adds a manual -redesign comment.

This is intentional because the update restart logic needs proper business mapping and cannot be generated safely from field mapping alone.

Latest outputs are shared for validation.

```

=====
FILE: IDMS-DB2-Code-Conversion\main.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\patterns\cobol_patterns.py
ENCODING: utf-8
=====
```

```
"""
COBOL regex patterns.
```

```
These patterns are shared by parsers, analyzers, transformers, composers,
and formatters. Keep regex definitions centralized here so runtime modules
do not define their own duplicate COBOL patterns.
```

```
"""
```

```
import re
```

```
PROGRAM_ID_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?PROGRAM-ID\.\s*([A-Z0-9- ]+)\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?"
    r"(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)"
    r"\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
)
```

```
IDENTIFICATION_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?IDENTIFICATION\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
ENVIRONMENT_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?ENVIRONMENT\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
DATA_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?DATA\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?PROCEDURE\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
WORKING_STORAGE_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?WORKING-STORAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
LINKAGE_SECTION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?LINKAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
SECTION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?[A-Z0-9][A-Z0-9-]*\s+SECTION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
)
```

```
PARAGRAPH_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?(?<P<name>[A-Z0-9][A-Z0-9-]*)\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
```



```

)

END_PROGRAM_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?END\\s+PROGRAM\\b.*\\.?\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

STOP_RUN_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?STOP\\s+RUN\\.?\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE,
)

GOBACK_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?GOBACK\\.?\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE,
)

COMMENT_OR_SKIP_PATTERN = re.compile(
    r"^\\s*(\\*|/|EJECT\\b|SKIP[0-9]*\\b)",
    flags=re.IGNORECASE,
)

COMMENT_PATTERN = re.compile(
    r"^\\s*\\*",
    flags=re.IGNORECASE,
)

PAGE_EJECT_PATTERN = re.compile(
    r"^\\s*/\\s*$",
    flags=re.IGNORECASE,
)

SEQUENCE_ONLY_PATTERN = re.compile(
    r"^\\s*(\\d{6}|\\d{8})\\s*$",
    flags=re.IGNORECASE,
)

LEFT_SEQUENCE_PATTERN = re.compile(
    r"^\\s*(?P<seq>\\d{6})(?P<body>\\s+.*)$",
    flags=re.IGNORECASE,
)

RIGHT_SEQUENCE_PATTERN = re.compile(
    r"(?P<body>.*?)\\s+(?P<right>\\d{8})\\s*$",
    flags=re.IGNORECASE,
)

DATA_ENTRY_START_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?"
    r"(?P<level>0[1-9]|[1-4][0-9]|66|77|88)"
    r"\\s+(?P<name>[A-Z0-9][A-Z0-9-]*)\\b"
    r"(?P<rest>.*)$",
    flags=re.IGNORECASE,
)

FILE_DESCRIPTOR_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?(FD|SD)\\s+",
    flags=re.IGNORECASE,
)

SELECT_FILE_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?SELECT\\s+",
    flags=re.IGNORECASE,
)

```

```

SQL_ERROR_PARAGRAPH_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?(SQL-ERROR|SQLERROR)\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

SQL_ERROR_DOT_PATTERN = re.compile(
    r"\bPERFORM\s+(SQL-ERROR|SQLERROR)\s*\.\?",
    flags=re.IGNORECASE,
)

DB2_INFRASTRUCTURE_MARKER_PATTERN = re.compile(
    r"DB2 SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR FLAGS",
    flags=re.IGNORECASE,
)

NON_PARAGRAPH_SINGLE_WORDS = {
    "ACCEPT",
    "ADD",
    "ALTER",
    "CALL",
    "CANCEL",
    "CLOSE",
    "COMMIT",
    "COMPUTE",
    "CONTINUE",
    "DELETE",
    "DISPLAY",
    "DIVIDE",
    "ELSE",
    "END-IF",
    "END-EVALUATE",
    "END-PERFORM",
    "END-READ",
    "END-STRING",
    "END-UNSTRING",
    "END-WRITE",
    "END-EXEC",
    "EJECT",
    "EXIT",
    "GOBACK",
    "IF",
    "INITIALIZE",
    "INSPECT",
    "MOVE",
    "MULTIPLY",
    "NEXT",
    "OPEN",
    "PERFORM",
    "READ",
    "RETURN",
    "REWRITE",
    "ROLLBACK",
    "SEARCH",
    "SET",
    "SKIP1",
    "SKIP2",
    "SKIP3",
    "SORT",
    "SPACE",
    "SPACES",
    "START",
    "STOP",
    "STRING",
    "SUBTRACT",
    "UNSTRING",
    "WHEN",
    "WRITE",
}

```

```
=====
FILE: IDMS-DB2-Code-Conversion\patterns\copybook_patterns.py
ENCODING: utf-8
=====
```

```
"""
```

```
Copybook regex patterns.
```

```
Copybook parser logic must import these patterns instead of defining regex
directly inside parser classes.
```

```
"""
```

```
import re
```

```
COPYBOOK_FIELD_PATTERN = re.compile(
    r"^\s*(?P<level>0[1-9]|[1-4][0-9]|66|77|88)\s+"
    r"(?P<name>[A-Z0-9-]+)"
    r"(?P<rest>.*)$",
    flags=re.IGNORECASE,
)
```

```
COPYBOOK_PIC_PATTERN = re.compile(
    r"\bPIC(?:TURE)?\s+"
    r"(?P<pic>[SXA9VZ0-9\(\)\+\-\.\,/\s+])",
    flags=re.IGNORECASE,
)
```

```
COPYBOOK_USAGE_PATTERN = re.compile(
    r"\b(?:USAGE\s+IS\s+|USAGE\s+)"
    r"(?P<usage>COMP-3|COMPUTATIONAL-3|COMP|COMPUTATIONAL|BINARY|DISPLAY|PACKED-DECIMAL)\b",
    flags=re.IGNORECASE,
)
```

```
COPYBOOK_OCCURS_PATTERN = re.compile(
    r"\bOCCURS\s+(?P<occurs>[0-9]+\s+(?:TIMES\b)?",
    flags=re.IGNORECASE,
)
```

```
COPYBOOK_COMMENT_OR_SKIP_PATTERN = re.compile(
    r"^\s*(?:\*|/|EJECT\b|SKIP[0-9]*\b)",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: IDMS-DB2-Code-Conversion\patterns\cursor_flow_patterns.py
ENCODING: utf-8
=====
```

```
"""
```

```
Cursor flow regex patterns.
```

```
"""
```

```
import re
```

```
PERFORM_CURSOR_PARAGRAPH_PATTERN = re.compile(
    r"^\s*PERFORM\s+"
    r"(?P<number>\d{3})-"
    r"(?P<operation>OPEN|FETCH|CLOSE)-"
    r"(?P<cursor>[A-Z0-9-]+)"
    r"\.\?\s*$",
    flags=re.IGNORECASE,
)
```

```
PERFORM_BUSINESS_PARAGRAPH_PATTERN = re.compile(
    r"^\s*PERFORM\s+"
    r"(?P<paragraph>[A-Z0-9][A-Z0-9-]*)"
    r"\.\?\s*$",
    flags=re.IGNORECASE,
```

```

)

UNTIL_SQLCODE_100_PATTERN = re.compile(
    r"^\\s*UNTIL\\s+SQLCODE\\s*=\\s*100\\.?\\s*$",
    flags=re.IGNORECASE,
)

CURSOR_PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^\\s*"
    r"(?P<number>\\d{3})-"
    r"(?P<operation>OPEN|FETCH|CLOSE)-"
    r"(?P<cursor>[A-Z0-9-]+)"
    r"\\.\\s*$",
    flags=re.IGNORECASE,
)

ANY_PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^\\s*[A-Z0-9][A-Z0-9-]*\\.\\s*$",
    flags=re.IGNORECASE,
)

WHEN_ZERO_PATTERN = re.compile(
    r"^\\s*WHEN\\s+ZERO\\s*$",
    flags=re.IGNORECASE,
)

CONTINUE_PATTERN = re.compile(
    r"^\\s*CONTINUE\\.?\\s*$",
    flags=re.IGNORECASE,
)

=====
FILE: IDMS-DB2-Code-Conversion\\patterns\\cursor_rules.py
ENCODING: utf-8
=====

"""
Cursor generation rules.
"""

CURSOR_AUTHORITY_RULES = [
    "Cursor names are derived from Sheet Mapping DB2 record/table names.",
    "Cursor names are not derived from IDMS set names.",
    "Generated cursor OPEN, FETCH, and CLOSE paragraphs are placed near the end.",
    "SQL-ERROR paragraph is required when cursor code performs SQL error handling.",
]

CURSOR_PARAGRAPH_NUMBERING = {
    1: {
        "open": 710,
        "fetch": 720,
        "close": 730,
    },
    2: {
        "open": 810,
        "fetch": 820,
        "close": 830,
    },
    3: {
        "open": 910,
        "fetch": 920,
        "close": 930,
    },
}

def cursor_paragraph_number(
    cursor_order: int,

```

```

        operation: str,
    ) -> int:
        operation_key = str(operation or "").strip().lower()

        if cursor_order not in CURSOR_PARAGRAPH_NUMBERING:
            raise ValueError(f"Unsupported cursor order: {cursor_order}")

        if operation_key not in CURSOR_PARAGRAPH_NUMBERING[cursor_order]:
            raise ValueError(f"Unsupported cursor operation: {operation}")

        return CURSOR_PARAGRAPH_NUMBERING[cursor_order][operation_key]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\db2_date_patterns.py
ENCODING: utf-8
=====

```

```

"""
DB2 date comparison regex patterns.

```

```

These patterns support conservative DB2 date realignment before comparing
DB2 date host fields with COBOL numeric date fields like PARMDATE.
"""

```

```

import re

```

```

DB2_DATE_COMPARISON_PATTERN = re.compile(
    r"^(?P<indent>\s*)IF\s+"
    r"(?P<field>(?:DA|DT)-[A-Z0-9-]+)"
    r"\s+OF\s+"
    r"(?P<group>DCL[A-Z0-9-]+)"
    r"\s+"
    r"(?P<condition>.+\\bPARMDATE\\b.*)$",
    flags=re.IGNORECASE,
)

```

```

WORKING_STORAGE_SECTION_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?WORKING-STORAGE\\s+SECTION\\.\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE,
)

```

```

LINKAGE_SECTION_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?LINKAGE\\s+SECTION\\.\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE,
)

```

```

PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?PROCEDURE\\s+DIVISION\\b.*\\.\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE,
)

```

```

DATE_HELPER_FIELD_PATTERN = re.compile(
    r"\\bHELP-(?:DA|DT)-[A-Z0-9-]+\\b",
    flags=re.IGNORECASE,
)

```

```

DATE_WORKING_STORAGE_MARKER_PATTERN = re.compile(
    r"DB2 DATE COMPARISON WORKING STORAGE",
    flags=re.IGNORECASE,
)

```

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\db2_patterns.py
ENCODING: utf-8
=====

```

```

"""
DB2 infrastructure and generated block regex patterns.

```

DB2 infrastructure generators, validators, transformers, and composers must import these patterns instead of defining DB2 regex internally.

This file must contain regex patterns only.

Do not place validation token lists, business rules, conversion rules, or layout constants in this file.

"""

```
import re
```

```
SQLCA_TOKEN_PATTERN = re.compile(  
    r"\bSQLCA\b",  
    flags=re.IGNORECASE,  
)
```

```
SQLERRWS_TOKEN_PATTERN = re.compile(  
    r"\bSQLERRWS\b",  
    flags=re.IGNORECASE,  
)
```

```
SQL_LOCATION_PATTERN = re.compile(  
    r"\bSQL-LOCATION\b",  
    flags=re.IGNORECASE,  
)
```

```
SQL_ERROR_PARAGRAPH_PATTERN = re.compile(  
    r"^\s*(?:\d{6}\s+)?(SQL-ERROR|SQLERROR)\s*\.\s*(?:\d{8})?\s*$",  
    flags=re.IGNORECASE | re.MULTILINE,  
)
```

```
SQL_ERROR_PERFORM_PATTERN = re.compile(  
    r"\bPERFORM\s+(SQL-ERROR|SQLERROR)\s*\.\s*?",  
    flags=re.IGNORECASE,  
)
```

```
DB2_CURSOR_BLOCK_MARKER_PATTERN = re.compile(  
    r"DB2 GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS",  
    flags=re.IGNORECASE,  
)
```

```
DB2_INFRASTRUCTURE_MARKER_PATTERN = re.compile(  
    r"DB2 SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR FLAGS",  
    flags=re.IGNORECASE,  
)
```

```
DB2_CURSOR_DECLARATIONS_MARKER_PATTERN = re.compile(  
    r"DB2 GENERATED CURSOR DECLARATIONS",  
    flags=re.IGNORECASE,  
)
```

```
DB2_CURSOR_FLAGS_MARKER_PATTERN = re.compile(  
    r"DB2 GENERATED CURSOR FLAGS",  
    flags=re.IGNORECASE,  
)
```

```
DB2_SQL_ERROR_LOCATION_MARKER_PATTERN = re.compile(  
    r"DB2 GENERATED SQL ERROR LOCATION",  
    flags=re.IGNORECASE,  
)
```

```
PROCEDURE_DIVISION_PATTERN = re.compile(  
    r"(\s*(?:\d{6}\s+)?PROCEDURE\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$)",  
    flags=re.IGNORECASE | re.MULTILINE,  
)
```

```
LINKAGE_SECTION_PATTERN = re.compile(  
    r"(\s*(?:\d{6}\s+)?LINKAGE\s+SECTION\.\s*(?:\d{8})?\s*$)",  
    flags=re.IGNORECASE | re.MULTILINE,  
)
```

```
WORKING_STORAGE_PATTERN = re.compile(  
    r"(\s*(?:\d{6}\s+)?WORKING-STORAGE\s+SECTION\.\s*(?:\d{8})?\s*$)",  
    flags=re.IGNORECASE | re.MULTILINE,  
)
```

```

DATA_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?DATA\s+DIVISION\.\s*(?:\d{8})?\s*$)",
    flags=re.IGNORECASE | re.MULTILINE,
)

END_PROGRAM_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?END\s+PROGRAM\b.*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

EXEC_SQL_START_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?END-EXEC\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
)

SQL_INCLUDE_PATTERN = re.compile(
    r"\bINCLUDE\s+(?P<include_name>[A-Z0-9_-]+\)\b",
    flags=re.IGNORECASE,
)

SQLCA_INCLUDE_PATTERN = re.compile(
    r"\bINCLUDE\s+SQLCA\b",
    flags=re.IGNORECASE,
)

SQLERRWS_INCLUDE_PATTERN = re.compile(
    r"\bINCLUDE\s+SQLERRWS\b",
    flags=re.IGNORECASE,
)

DECLARE_CURSOR_PATTERN = re.compile(
    r"\bDECLARE\s+(?P<cursor_name>[A-Z0-9_-]+\)\s+Cursors\b",
    flags=re.IGNORECASE,
)

OPEN_CURSOR_PATTERN = re.compile(
    r"\bOPEN\s+(?P<cursor_name>[A-Z0-9_-]+\)\b",
    flags=re.IGNORECASE,
)

FETCH_CURSOR_PATTERN = re.compile(
    r"\bFETCH\s+(?P<cursor_name>[A-Z0-9_-]+\)\b",
    flags=re.IGNORECASE,
)

CLOSE_CURSOR_PATTERN = re.compile(
    r"\bCLOSE\s+(?P<cursor_name>[A-Z0-9_-]+\)\b",
    flags=re.IGNORECASE,
)

SQLCODE_PATTERN = re.compile(
    r"\bSQLCODE\b",
    flags=re.IGNORECASE,
)

SQLCODE_SUCCESS_PATTERN = re.compile(
    r"\bSQLCODE\s*=\s*0\b",
    flags=re.IGNORECASE,
)

SQLCODE_NOT_FOUND_PATTERN = re.compile(
    r"\bSQLCODE\s*=\s*100\b",
    flags=re.IGNORECASE,
)

SQLCODE_ERROR_PATTERN = re.compile(
    r"\bSQLCODE\s*(?:NOT\s*=\s*0|<\s*0|>\s*0)\b",
    flags=re.IGNORECASE,
)

```

```

COMMIT_PATTERN = re.compile(
    r"\bCOMMIT\b",
    flags=re.IGNORECASE,
)

ROLLBACK_PATTERN = re.compile(
    r"\bROLLBACK\b",
    flags=re.IGNORECASE,
)

=====
FILE: IDMS-DB2-Code-Conversion\patterns\dclgen_patterns.py
ENCODING: utf-8
=====

"""
DCLGEN regex patterns.

Parser classes must import these patterns instead of defining regex inside
parser classes.
"""

import re

DECLARE_TABLE_PATTERN = re.compile(
    r"EXEC\s+SQL\s+DECLARE\s+([A-Z0-9_\. #\$@]+)\s+TABLE",
    flags=re.IGNORECASE,
)

DECLARE_TABLE_ALT_PATTERN = re.compile(
    r"\bDECLARE\s+([A-Z0-9_\. #\$@]+)\s+TABLE\b",
    flags=re.IGNORECASE,
)

DCLGEN_TABLE_COMMENT_PATTERN = re.compile(
    r"\bDCLGEN\s+TABLE\s*\$*\s*([A-Z0-9_\. #\$@]+)\s*\$",
    flags=re.IGNORECASE,
)

COBOL_GROUP_PATTERN = re.compile(
    r"^\s*\*?\s*\01\s+(DCL[A-Z0-9- ]+)\.?\s*\$",
    flags=re.IGNORECASE,
)

COBOL_FIELD_PATTERN = re.compile(
    r"^\s*(0[2-9]|[1-4][0-9]|77)\s+"
    r"([A-Z][A-Z0-9- ]*)\b(?:P<body>.*)$",
    flags=re.IGNORECASE,
)

DCLGEN_PIC_PATTERN = re.compile(
    r"\bPIC(?:TURE)?\s+(?:IS\s+)?"
    r"?(?P<pic>[A-Z0-9SV() .,+\- ]+)",
    flags=re.IGNORECASE,
)

DCLGEN_USAGE_PATTERN = re.compile(
    r"\b(?:USAGE\s+(?:IS\s+)?)"
    r"(COMP-3|COMP|COMP-1|COMP-2|COMP-4|COMP-5|BINARY|PACKED-DECIMAL|DISPLAY)\b",
    flags=re.IGNORECASE,
)

SQL_NAME_CLEANUP_PATTERN = re.compile(
    r"^[A-Z0-9_#\$@]+",
    flags=re.IGNORECASE,
)

```



```
MULTIPLE_UNDERSCORE_PATTERN = re.compile(
    r"_+",
    flags=re.IGNORECASE,
)
```

```
VALID_SQL_COLUMN_NAME_PATTERN = re.compile(
    r"^[A-Z][A-Z0-9_#@$]*$",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: IDMS-DB2-Code-Conversion\patterns\field_reference_patterns.py
ENCODING: utf-8
=====
```

```
import re
```

```
QUALIFIED_REFERENCE_PATTERN = re.compile(
    r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+"
    r"(?:P<qualifier>OF|IN)\s+"
    r"(?:P<record>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)
```

```
REDEFINES_PATTERN = re.compile(
    r"\b(?:P<alias>[A-Z][A-Z0-9-]*)\s+REDEFINES\s+"
    r"(?:P<base>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)
```

```
STRING_LITERAL_PATTERN = re.compile(
    r"'[^\']*'|\"[^\"]*\"'",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: IDMS-DB2-Code-Conversion\patterns\field_usage_patterns.py
ENCODING: utf-8
=====
```

```
"""
Field usage regex patterns.
```

```
This module contains regex patterns only.
```

```
Runtime analyzers, transformers, validators, composers, and services must
import these patterns instead of defining duplicate regex internally.
```

```
Rules:
- No business rules in this file.
- No service logic in this file.
- No parsing orchestration in this file.
- Regex patterns only.
"""
```

```
import re
```

```
QUALIFIED_REFERENCE_PATTERN = re.compile(
    r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+(?:OF|IN)\s+"
    r"(?:P<record>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)
```

```
DCLGEN_OF_PATTERN = re.compile(
    r":?\s*(?:P<field>[A-Z][A-Z0-9-]*)\s+OF\s+"
    r"(?:P<group>DCL[A-Z0-9-]*)",
    flags=re.IGNORECASE,
)
```

```
DCLGEN_DOT_PATTERN = re.compile(

```

```

r":\s*(?P<group>DCL[A-Z0-9-]+\).(?P<field>[A-Z][A-Z0-9-]*)",
flags=re.IGNORECASE,
)

MOVE_STATEMENT_PATTERN = re.compile(
r"^\\s*MOVE\\s+(?P<source>.+?)\\s+TO\\s+(?P<target>.+?)(?:\\.|\\s*)$",
flags=re.IGNORECASE,
)

CONDITION_STATEMENT_PATTERN = re.compile(
r"^\\s*(IF|WHEN|UNTIL|EVALUATE)\\b",
flags=re.IGNORECASE,
)

OUTPUT_STATEMENT_PATTERN = re.compile(
r"^\\s*(DISPLAY|WRITE|REWRITE|RETURN|GOBACK|EXIT|CALL)\\b",
flags=re.IGNORECASE,
)

COMMENT_OR_BLANK_PATTERN = re.compile(
r"^\\s*(?:$|\\s*)",
flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
r"^\\s*EXEC\\s+SQL\\b",
flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
r"^\\s*END-EXEC\\.?\\s*$",
flags=re.IGNORECASE,
)

```

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\fixed_format_patterns.py
ENCODING: utf-8
=====

```

```

"""
Fixed-format COBOL regex patterns.
These patterns are used by fixed-format composers and helpers.
Keep regex definitions here instead of defining them inside composer classes.
"""

```

```

import re
LEFT_SEQUENCE_PATTERN = re.compile(
r"^\\s*(?P<left>\\d{6})(?P<rest>.*)$",
flags=re.IGNORECASE,
)
RIGHT_SEQUENCE_PATTERN = re.compile(
r"^(?P<body>.*?)(?P<right>\\d{8})\\s*$",
flags=re.IGNORECASE,
)
SEQUENCE_ONLY_PATTERN = re.compile(
r"^\\s*(\\d{6}|\\d{8})\\s*$",
flags=re.IGNORECASE,
)
DIVISION_PATTERN = re.compile(
r"^(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\\s+DIVISION\\b.*\\.?$",
flags=re.IGNORECASE,
)
SECTION_PATTERN = re.compile(
r"^[A-Z0-9-]+\\s+SECTION\\.?$",
flags=re.IGNORECASE,
)
PARAGRAPH_PATTERN = re.compile(
r"^[A-Z0-9][A-Z0-9-]*\\.?$",
flags=re.IGNORECASE,
)
AREA_A_PREFIX_PATTERN = re.compile(
(
r"^(
r"PROGRAM-ID\\.|"
r"AUTHOR\\.|"
r"INSTALLATION\\.|"

```

```

r"DATE-WRITTEN\."|
r"DATE-COMPILED\."|
r"SECURITY\."|
r"FD\s+|"
r"SD\s+|"
r"01\s+|"
r"66\s+|"
r"77\s+"
r")"
),
flags=re.IGNORECASE,
)
DATA_LEVEL_PATTERN = re.compile(
r"^(0[1-9]|[1-4][0-9]|66|77|88)\s+",
flags=re.IGNORECASE,
)
EXEC_SQL_START_PATTERN = re.compile(
r"^EXEC\s+SQL\b",
flags=re.IGNORECASE,
)
EXEC_SQL_END_PATTERN = re.compile(
r"^END-EXEC\.$",
flags=re.IGNORECASE,
)
BOOLEAN_OPERATOR_END_PATTERN = re.compile(
r"\b(AND|OR)\s*$",
flags=re.IGNORECASE,
)
IF_BOOLEAN_SPLIT_PATTERN = re.compile(
r"(\s+AND\s+|\s+OR\s+)",
flags=re.IGNORECASE,
)
MOVE_STATEMENT_PATTERN = re.compile(
r"^MOVE\s+(?P<src>.+?)\s+TO\s+(?P<tgt>.+?\.)$",
flags=re.IGNORECASE,
)
DEBUG_LINE_PATTERN = re.compile(
r"^[Dd]\s+",
flags=re.IGNORECASE,
)

=====
FILE: IDMS-DB2-Code-Conversion\patterns\fixed_format_patterns.py.broken_before_pdf_restore
ENCODING: utf-8
=====

"""
Fixed-format COBOL regex patterns.

This file contains regex patterns only.
"""

import re

SEQUENCE_ONLY_PATTERN = re.compile(
    r"^\s*\d{6}\s*$",
    flags=re.IGNORECASE,
)

DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?[A-Z-]+\s+DIVISION\b",
    flags=re.IGNORECASE,
)

FIXED_FORMAT_LINE_PATTERN = re.compile(
    r"^(?P<left>\d{6})(?P<indicator>.)?(?P<body>.{0,65})(?P<right>\d{0,8})$",
    flags=re.IGNORECASE,
)

```

```
=====
FILE: IDMS-DB2-Code-Conversion\patterns\fixed_format_patterns.py.broken_before_pdf_restore_1
ENCODING: utf-8
=====
```

```
=====
"""
Fixed-format COBOL regex patterns.
These patterns are used by fixed-format composers and helpers.
Keep regex definitions here instead of defining them inside composer classes.
"""
```

```
import re
LEFT_SEQUENCE_PATTERN = re.compile(
    r"^\s*(?P<left>\d{6})(?P<rest>.*)$",
    flags=re.IGNORECASE,
)
RIGHT_SEQUENCE_PATTERN = re.compile(
    r"^(?P<body>.*?)(?P<right>\d{8})\s*$",
    flags=re.IGNORECASE,
)
SEQUENCE_ONLY_PATTERN = re.compile(
    r"^\s*(\d{6}|\d{8})\s*$",
    flags=re.IGNORECASE,
)
DIVISION_PATTERN = re.compile(
    r"^(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\s+DIVISION\b.*\.$",
    flags=re.IGNORECASE,
)
SECTION_PATTERN = re.compile(
    r"^[A-Z0-9-]+\s+SECTION\.\.$",
    flags=re.IGNORECASE,
)
PARAGRAPH_PATTERN = re.compile(
    r"^[A-Z0-9][A-Z0-9-]*\.\.$",
    flags=re.IGNORECASE,
)
AREA_A_PREFIX_PATTERN = re.compile(
    (
        r"^("
        r"PROGRAM-ID\." |
        r"AUTHOR\." |
        r"Page 19"
        r"INSTALLATION\." |
        r"DATE-WRITTEN\." |
        r"DATE-COMPILED\." |
        r"SECURITY\." |
        r"FD\s+" |
        r"SD\s+" |
        r"01\s+" |
        r"66\s+" |
        r"77\s+"
        r")"
    ),
    flags=re.IGNORECASE,
)
DATA_LEVEL_PATTERN = re.compile(
    r"^(0[1-9]|[1-4][0-9]|66|77|88)\s+",
    flags=re.IGNORECASE,
)
EXEC_SQL_START_PATTERN = re.compile(
    r"^EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)
EXEC_SQL_END_PATTERN = re.compile(
    r"^END-EXEC\.\.$",
    flags=re.IGNORECASE,
)
BOOLEAN_OPERATOR_END_PATTERN = re.compile(
    r"\b(AND|OR)\s*$",
    flags=re.IGNORECASE,
)
IF_BOOLEAN_SPLIT_PATTERN = re.compile(
    r"(\s+AND\s+|\s+OR\s+)",
    flags=re.IGNORECASE,
)
MOVE_STATEMENT_PATTERN = re.compile(
```

```

r"^MOVE\s+(?P<src>.+?)\s+TO\s+(?P<tgt>.+?\.)?$",
flags=re.IGNORECASE,
)
DEBUG_LINE_PATTERN = re.compile(
r"^[Dd]\s+",
flags=re.IGNORECASE,
)
=====

=====
FILE: IDMS-DB2-Code-Conversion\patterns\fixed_format_patterns.py.broken_before_pdf_restore_2
ENCODING: utf-8
=====

"""
Fixed-format COBOL regex patterns.
These patterns are used by fixed-format composers and helpers.
Keep regex definitions here instead of defining them inside composer classes.
"""

import re
LEFT_SEQUENCE_PATTERN = re.compile(
r"^\s*(?P<left>\d{6})(?P<rest>.*)$",
flags=re.IGNORECASE,
)
RIGHT_SEQUENCE_PATTERN = re.compile(
r"^(?P<body>.*?)(?P<right>\d{8})\s*$",
flags=re.IGNORECASE,
)
SEQUENCE_ONLY_PATTERN = re.compile(
r"^\s*(\d{6}|\d{8})\s*$",
flags=re.IGNORECASE,
)
DIVISION_PATTERN = re.compile(
r"^(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\s+DIVISION\b.*\.$",
flags=re.IGNORECASE,
)
SECTION_PATTERN = re.compile(
r"^[A-Z0-9-]+\s+SECTION\.$",
flags=re.IGNORECASE,
)
PARAGRAPH_PATTERN = re.compile(
r"^[A-Z0-9][A-Z0-9-]*\.$",
flags=re.IGNORECASE,
)
AREA_A_PREFIX_PATTERN = re.compile(
(
r"^(
r"PROGRAM-ID\."|
r"AUTHOR\."|
Page 19
r"INSTALLATION\."|
r"DATE-WRITTEN\."|
r"DATE-COMPILED\."|
r"SECURITY\."|
r"FD\s+|"
r"SD\s+|"
r"01\s+|"
r"66\s+|"
r"77\s+"
r")"
),
flags=re.IGNORECASE,
)
DATA_LEVEL_PATTERN = re.compile(
r"^(0[1-9]|[1-4][0-9]|66|77|88)\s+",
flags=re.IGNORECASE,
)
EXEC_SQL_START_PATTERN = re.compile(
r"^EXEC\s+SQL\b",
flags=re.IGNORECASE,
)
EXEC_SQL_END_PATTERN = re.compile(
r"^END-EXEC\.$",
flags=re.IGNORECASE,
)

```

```

BOOLEAN_OPERATOR_END_PATTERN = re.compile(
    r"\b(AND|OR)\s*$",
    flags=re.IGNORECASE,
)
IF_BOOLEAN_SPLIT_PATTERN = re.compile(
    r"(\s+AND\s+|\s+OR\s+)",
    flags=re.IGNORECASE,
)
MOVE_STATEMENT_PATTERN = re.compile(
    r"^MOVE\s+(?P<src>.+?)\s+TO\s+(?P<tgt>.+?\.\.?)$",
    flags=re.IGNORECASE,
)
DEBUG_LINE_PATTERN = re.compile(
    r"^[Dd]\s+",
    flags=re.IGNORECASE,
)

```

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\fixed_format_rules.py
ENCODING: utf-8
=====

```

```

"""
Fixed-format COBOL layout rules.
These constants define the final physical COBOL layout:
- Columns 1-6 : left sequence number
- Column 7 : indicator area
- Columns 8-72 : COBOL body
- Columns 73-80 : right sequence number
"""

```

```

TOTAL_WIDTH = 80
BODY_WIDTH = 65
LEFT_SEQUENCE_WIDTH = 6
INDICATOR_WIDTH = 1
RIGHT_SEQUENCE_WIDTH = 8
DEFAULT_LEFT_START = 10
DEFAULT_LEFT_STEP = 10
# COBOL team expected:
# 000010 body 00010000
# 000020 body 00020000
DEFAULT_RIGHT_START = 10000
DEFAULT_RIGHT_STEP = 10000
AREA_A_INDENT = ""
AREA_B_INDENT = " "
SQL_INDENT = " "
SQL_CONTINUATION_INDENT = " "
SPACE_INDICATOR = " "
COMMENT_INDICATOR = "*"
PAGE_INDICATOR = "/"
DEBUG_INDICATOR = "D"
CONTINUATION_INDICATOR = "-"
VALID_INDICATORS = {
    SPACE_INDICATOR,
    COMMENT_INDICATOR,
    PAGE_INDICATOR,
    DEBUG_INDICATOR,
    "d",
    CONTINUATION_INDICATOR,
}
NON_PARAGRAPH_WORDS = {
    "ACCEPT",
    "ADD",
    "CALL",
    "CLOSE",
    "COMMIT",
    "CONTINUE",
    "DISPLAY",
    "ELSE",
    "END-EXEC",
    "END-EVALUATE",
    "END-IF",
    "END-PERFORM",
    "EVALUATE",
    "EXEC",
    "EXIT",
}

```

```

"FETCH",
"IF",
"INITIALIZE",
"MOVE",
"OPEN",
"PERFORM",
"READ",
"SET",
"STOP",
"WHEN",
"WRITE",
}
PROCEDURE_VERBS = (
"ACCEPT ",
"ADD ",
"CALL ",
"CLOSE ",
"COMMIT",
"CONTINUE",
"DISPLAY ",
"ELSE",
"END-EVALUATE",
"END-IF",
"EVALUATE ",
"EXEC SQL",
"EXIT",
"IF ",
"INITIALIZE ",
"MOVE ",
"OPEN ",
"PERFORM ",
"READ ",
"SET ",
"STOP ",
"WHEN ",
"WRITE ",
)
SQL_LEVEL_1_KEYWORDS = (
"SELECT ",
"INTO ",
"FROM ",
"WHERE ",
"ORDER BY ",
"GROUP BY ",
"HAVING ",
"FETCH ",
"OPEN ",
"CLOSE ",
"COMMIT",
"ROLLBACK",
"INSERT ",
"UPDATE ",
"DELETE ",
"SET ",
"VALUES ",
"FOR READ ONLY",
"QUERYNO ",
)
SQL_LEVEL_2_KEYWORDS = (
"AND ",
"OR ",
",",
)
AREA_A_HEADER_PREFIXES = (
"PROGRAM-ID.",
"AUTHOR.",
"INSTALLATION.",
"DATE-WRITTEN.",
"DATE-COMPILED.",
"SECURITY.",
)
AREA_A_DATA_LEVELS = {
"01",
"66",
"77",
}

```

```

AREA_B_DATA_LEVELS = {
"02",
"03",
"04",
"05",
"06",
"07",
"08",
"09",
"10",
"11",
"12",
"13",
"14",
"15",
"16",
"17",
"18",
"19",
"20",
"21",
"22",
"23",
"24",
"25",
"26",
"27",
"28",
"29",
"30",
"31",
"32",
"33",
"34",
"35",
"36",
"37",
"38",
"39",
"40",
"41",
"42",
"43",
"44",
"45",
"46",
"47",
"48",
"49",
"88",
}

```

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\fixed_format_rules.py.broken_before_pdf_restore
ENCODING: utf-8
=====

```

```

"""
Fixed-format COBOL layout rules.

```

```

Columns:
- 1-6   : left sequence number
- 7     : indicator area
- 8-72  : COBOL body
- 73-80 : right sequence number

```

```

This file contains constants only.
No regex should be defined here.
"""

```

```

TOTAL_WIDTH = 80
BODY_WIDTH = 65

LEFT_SEQUENCE_WIDTH = 6
INDICATOR_WIDTH = 1

```



```
RIGHT_SEQUENCE_WIDTH = 8
```

```
DEFAULT_LEFT_START = 10
```

```
DEFAULT_LEFT_STEP = 10
```

```
SPACE_INDICATOR = " "
```

```
COMMENT_INDICATOR = "*"
```

```
DEBUG_INDICATOR = "D"
```

```
PAGE_INDICATOR = "/"
```

```
VALID_INDICATORS = {  
    SPACE_INDICATOR,  
    COMMENT_INDICATOR,  
    DEBUG_INDICATOR,  
    PAGE_INDICATOR,  
}
```

```
=====
```

```
FILE: IDMS-DB2-Code-Conversion\patterns\fixed_format_rules.py.broken_before_pdf_restore_1
```

```
ENCODING: utf-8
```

```
=====
```

```
=====
```

```
"""
```

```
Fixed-format COBOL layout rules.
```

```
These constants define the final physical COBOL layout:
```

- Columns 1-6 : left sequence number
- Column 7 : indicator area
- Columns 8-72 : COBOL body
- Columns 73-80 : right sequence number

```
=====
```

```
TOTAL_WIDTH = 80
```

```
BODY_WIDTH = 65
```

```
Page 20
```

```
LEFT_SEQUENCE_WIDTH = 6
```

```
INDICATOR_WIDTH = 1
```

```
RIGHT_SEQUENCE_WIDTH = 8
```

```
DEFAULT_LEFT_START = 10
```

```
DEFAULT_LEFT_STEP = 10
```

```
# COBOL team expected:
```

```
# 000010 body 00010000
```

```
# 000020 body 00020000
```

```
DEFAULT_RIGHT_START = 10000
```

```
DEFAULT_RIGHT_STEP = 10000
```

```
AREA_A_INDENT = ""
```

```
AREA_B_INDENT = " "
```

```
SQL_INDENT = " "
```

```
SQL_CONTINUATION_INDENT = " "
```

```
SPACE_INDICATOR = " "
```

```
COMMENT_INDICATOR = "*"
```

```
PAGE_INDICATOR = "/"
```

```
DEBUG_INDICATOR = "D"
```

```
CONTINUATION_INDICATOR = "-"
```

```
VALID_INDICATORS = {
```

```
    SPACE_INDICATOR,
```

```
    COMMENT_INDICATOR,
```

```
    PAGE_INDICATOR,
```

```
    DEBUG_INDICATOR,
```

```
    "d",
```

```
    CONTINUATION_INDICATOR,
```

```
}
```

```
NON_PARAGRAPH_WORDS = {
```

```
    "ACCEPT",
```

```
    "ADD",
```

```
    "CALL",
```

```
    "CLOSE",
```

```
    "COMMIT",
```

```
    "CONTINUE",
```

```
    "DISPLAY",
```

```
    "ELSE",
```

```
    "END-EXEC",
```

```
    "END-EVALUATE",
```

```
    "END-IF",
```

```
    "END-PERFORM",
```

```
    "EVALUATE",
```

```

"EXEC",
"EXIT",
"FETCH",
"IF",
"INITIALIZE",
"MOVE",
"OPEN",
"PERFORM",
"READ",
"SET",
"STOP",
"WHEN",
"WRITE",
}
PROCEDURE_VERBS = (
"ACCEPT ",
"ADD ",
"CALL ",
"CLOSE ",
"COMMIT",
"CONTINUE",
"DISPLAY ",
"ELSE",
"END-EVALUATE",
"END-IF",
Page 21
"EVALUATE ",
"EXEC SQL",
"EXIT",
"IF ",
"INITIALIZE ",
"MOVE ",
"OPEN ",
"PERFORM ",
"READ ",
"SET ",
"STOP ",
"WHEN ",
"WRITE ",
)
SQL_LEVEL_1_KEYWORDS = (
"SELECT ",
"INTO ",
"FROM ",
"WHERE ",
"ORDER BY ",
"GROUP BY ",
"HAVING ",
"FETCH ",
"OPEN ",
"CLOSE ",
"COMMIT",
"ROLLBACK",
"INSERT ",
"UPDATE ",
"DELETE ",
"SET ",
"VALUES ",
"FOR READ ONLY",
"QUERYNO ",
)
SQL_LEVEL_2_KEYWORDS = (
"AND ",
"OR ",
",",
)
AREA_A_HEADER_PREFIXES = (
"PROGRAM-ID.",
"AUTHOR.",
"INSTALLATION.",
"DATE-WRITTEN.",
"DATE-COMPILED.",
"SECURITY.",
)
AREA_A_DATA_LEVELS = {
"01",

```

```

"66",
"77",
}
AREA_B_DATA_LEVELS = {
"02",
"03",
"04",
"05",
"06",
"07",
"08",
"09",
"10",
"11",
"12",
"13",
Page 22
"14",
"15",
"16",
"17",
"18",
"19",
"20",
"21",
"22",
"23",
"24",
"25",
"26",
"27",
"28",
"29",
"30",
"31",
"32",
"33",
"34",
"35",
"36",
"37",
"38",
"39",
"40",
"41",
"42",
"43",
"44",
"45",
"46",
"47",
"48",
"49",
"88",
}

```

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\fixed_format_rules.py.broken_before_pdf_restore_2
ENCODING: utf-8
=====

```

```

"""
Fixed-format COBOL layout rules.
These constants define the final physical COBOL layout:
- Columns 1-6 : left sequence number
- Column 7 : indicator area
- Columns 8-72 : COBOL body
- Columns 73-80 : right sequence number
"""

TOTAL_WIDTH = 80
BODY_WIDTH = 65
Page 20
LEFT_SEQUENCE_WIDTH = 6
INDICATOR_WIDTH = 1

```

```

RIGHT_SEQUENCE_WIDTH = 8
DEFAULT_LEFT_START = 10
DEFAULT_LEFT_STEP = 10
# COBOL team expected:
# 000010 body 00010000
# 000020 body 00020000
DEFAULT_RIGHT_START = 10000
DEFAULT_RIGHT_STEP = 10000
AREA_A_INDENT = ""
AREA_B_INDENT = " "
SQL_INDENT = " "
SQL_CONTINUATION_INDENT = " "
SPACE_INDICATOR = " "
COMMENT_INDICATOR = "*"
PAGE_INDICATOR = "/"
DEBUG_INDICATOR = "D"
CONTINUATION_INDICATOR = "-"
VALID_INDICATORS = {
SPACE_INDICATOR,
COMMENT_INDICATOR,
PAGE_INDICATOR,
DEBUG_INDICATOR,
"d",
CONTINUATION_INDICATOR,
}
NON_PARAGRAPH_WORDS = {
"ACCEPT",
"ADD",
"CALL",
"CLOSE",
"COMMIT",
"CONTINUE",
"DISPLAY",
"ELSE",
"END-EXEC",
"END-EVALUATE",
"END-IF",
"END-PERFORM",
"EVALUATE",
"EXEC",
"EXIT",
"FETCH",
"IF",
"INITIALIZE",
"MOVE",
"OPEN",
"PERFORM",
"READ",
"SET",
"STOP",
"WHEN",
"WRITE",
}
PROCEDURE_VERBS = (
"ACCEPT ",
"ADD ",
"CALL ",
"CLOSE ",
"COMMIT",
"CONTINUE",
"DISPLAY ",
"ELSE",
"END-EVALUATE",
"END-IF",
Page 21
"EVALUATE ",
"EXEC SQL",
"EXIT",
"IF ",
"INITIALIZE ",
"MOVE ",
"OPEN ",
"PERFORM ",
"READ ",
"SET ",
"STOP ",

```

```

"WHEN ",
"WRITE ",
)
SQL_LEVEL_1_KEYWORDS = (
"SELECT ",
"INTO ",
"FROM ",
"WHERE ",
"ORDER BY ",
"GROUP BY ",
"HAVING ",
"FETCH ",
"OPEN ",
"CLOSE ",
"COMMIT",
"ROLLBACK",
"INSERT ",
"UPDATE ",
"DELETE ",
"SET ",
"VALUES ",
"FOR READ ONLY",
"QUERYNO ",
)
SQL_LEVEL_2_KEYWORDS = (
"AND ",
"OR ",
",",
)
AREA_A_HEADER_PREFIXES = (
"PROGRAM-ID.",
"AUTHOR.",
"INSTALLATION.",
"DATE-WRITTEN.",
"DATE-COMPILED.",
"SECURITY.",
)
AREA_A_DATA_LEVELS = {
"01",
"66",
"77",
}
AREA_B_DATA_LEVELS = {
"02",
"03",
"04",
"05",
"06",
"07",
"08",
"09",
"10",
"11",
"12",
"13",
Page 22
"14",
"15",
"16",
"17",
"18",
"19",
"20",
"21",
"22",
"23",
"24",
"25",
"26",
"27",
"28",
"29",
"30",
"31",
"32",
"33",

```

```

"34",
"35",
"36",
"37",
"38",
"39",
"40",
"41",
"42",
"43",
"44",
"45",
"46",
"47",
"48",
"49",
"88",
}

```

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\idms_patterns.py
ENCODING: utf-8
=====

```

```

"""
IDMS regex patterns.

COBOL parser and IDMS transformers must import these patterns instead of
defining IDMS operation regex inside parser or transformer classes.
"""

```

```

import re

```

```

OBTAIN_CALC_PATTERN = re.compile(
    r"\bOBTAIN\s+(?:KEEP\s+)?(?:P<record>[A-Z0-9-]+\s+)?CALC\b",
    flags=re.IGNORECASE,
)

```

```

OBTAIN_CALC_REVERSED_PATTERN = re.compile(
    r"\bOBTAIN\s+(?:KEEP\s+)?CALC\s+(?:P<record>[A-Z0-9-]+\s+)\b",
    flags=re.IGNORECASE,
)

```

```

OBTAIN_FIRST_NEXT_PATTERN = re.compile(
    r"\bOBTAIN\s+(?:KEEP\s+)?(?:P<mode>FIRST|NEXT)\s+"
    r"?(?:P<record>[A-Z0-9-]+\s+)?\s+WITHIN\s+(?:P<set>[A-Z0-9-]+\s+)\b",
    flags=re.IGNORECASE,
)

```

```

OBTAIN_OWNER_PATTERN = re.compile(
    r"\bOBTAIN\s+OWNER\s+WITHIN\s+(?:P<set>[A-Z0-9-]+\s+)\b",
    flags=re.IGNORECASE,
)

```

```

FIND_FIRST_PATTERN = re.compile(
    r"\bFIND\s+FIRST\s+(?:P<record>[A-Z0-9-]+\s+)?\s*WITHIN\s+(?:P<set>[A-Z0-9-]+\s+)\b",
    flags=re.IGNORECASE,
)

```

```

STORE_PATTERN = re.compile(
    r"\bSTORE\s+(?:P<record>[A-Z0-9-]+\s+)\b",
    flags=re.IGNORECASE,
)

```

```

MODIFY_PATTERN = re.compile(
    r"\bMODIFY\s+(?:P<record>[A-Z0-9-]+\s+)\b",
    flags=re.IGNORECASE,
)

```

```

ERASE_PATTERN = re.compile(
    r"\bERASE\s+(?P<record>[A-Z0-9- ]+)\b",
    flags=re.IGNORECASE,
)

READY_PATTERN = re.compile(
    r"^s*READY\b",
    flags=re.IGNORECASE,
)

READY_UPDATE_PATTERN = re.compile(
    r"\bREADY\s+AREA\s+(?P<record>[A-Z0-9- ]+).*UPDATE\b",
    flags=re.IGNORECASE,
)

FINISH_PATTERN = re.compile(
    r"^s*FINISH\b",
    flags=re.IGNORECASE,
)

COMMIT_PATTERN = re.compile(
    r"^s*COMMIT\b",
    flags=re.IGNORECASE,
)

BIND_STATEMENT_PATTERN = re.compile(
    r"^s*BIND\b",
    flags=re.IGNORECASE,
)

FIND_CURRENT_PATTERN = re.compile(
    r"^s*FIND\s+CURRENT\b",
    flags=re.IGNORECASE,
)

CONNECT_STATEMENT_PATTERN = re.compile(
    r"^s*CONNECT\b",
    flags=re.IGNORECASE,
)

DISCONNECT_STATEMENT_PATTERN = re.compile(
    r"^s*DISCONNECT\b",
    flags=re.IGNORECASE,
)

USAGE_MODE_PATTERN = re.compile(
    r"\bUSAGE-MODE\s+IS\s+(UPDATE|RETRIEVAL)\b",
    flags=re.IGNORECASE,
)

IDMS_STATUS_PERFORM_PATTERN = re.compile(
    r"\bPERFORM\b.*\bIDMS-STATUS\b",
    flags=re.IGNORECASE,
)

IDMS_ABORT_PERFORM_PATTERN = re.compile(
    r"\bPERFORM\b.*\bIDMS-ABORT\b",
    flags=re.IGNORECASE,
)

DB_REC_NOT_FOUND_TOKEN_PATTERN = re.compile(
    r"\bDB-REC-NOT-FOUND\b",

```

```

        flags=re.IGNORECASE,
    )

DB_END_OF_SET_TOKEN_PATTERN = re.compile(
    r"\bDB-END-OF-SET\b",
    flags=re.IGNORECASE,
)

ON_DB_REC_NOT_FOUND_PATTERN = re.compile(
    r"^\\s*ON\\s+DB-REC-NOT-FOUND\\s+(?P<statement>.+?)\\.?\\s*$",
    flags=re.IGNORECASE,
)

IDMS_DECLARATIVE_OR_CONTROL_PATTERNS = [
    re.compile(r"^\\s*IDMS-CONTROL\\s+SECTION\b", flags=re.IGNORECASE),
    re.compile(r"^\\s*PROTOCOL\b", flags=re.IGNORECASE),
    re.compile(r"^\\s*IDMS-RECORDS\\s+WITHIN\b", flags=re.IGNORECASE),
    re.compile(r"^\\s*SCHEMA\\s+SECTION\b", flags=re.IGNORECASE),
    re.compile(r"^\\s*DB\\s+[A-Z0-9-]+\\s+WITHIN\\s+[A-Z0-9-]+\b", flags=re.IGNORECASE),
    re.compile(r"^\\s*COPY\\s+IDMS\b", flags=re.IGNORECASE),
]

=====
FILE: IDMS-DB2-Code-Conversion\patterns\naming_patterns.py
ENCODING: utf-8
=====

"""
Naming patterns and naming helpers.

Name normalization logic must use these patterns instead of repeating
regex cleanup expressions across services.
"""

import re

NON_DB2_NAME_CHARACTER_PATTERN = re.compile(
    r"^[^A-Z0-9_]+",
    flags=re.IGNORECASE,
)

NON_COMPACT_NAME_CHARACTER_PATTERN = re.compile(
    r"^[^A-Z0-9_]+",
    flags=re.IGNORECASE,
)

MULTIPLE_UNDERSCORE_PATTERN = re.compile(
    r"_+",
    flags=re.IGNORECASE,
)

DCL_PREFIX_PATTERN = re.compile(
    r"^DCL_?",
    flags=re.IGNORECASE,
)

FOUR_DIGIT_RECORD_SUFFIX_PATTERN = re.compile(
    r"_[0-9]{4}$",
    flags=re.IGNORECASE,
)

```



```
=====
FILE: IDMS-DB2-Code-Conversion\patterns\pic_patterns.py
ENCODING: utf-8
=====
```

```
"""
```

```
PIC and MOVE regex patterns.
```

```
PIC length auto-fixer and production validators must import these patterns
instead of defining regex internally.
```

```
"""
```

```
import re
```

```
DATA_ENTRY_START_PATTERN = re.compile(
    r"^\s*(?P<level>0[1-9]|[1-4][0-9]|77)\s+"
    r"(?P<name>[A-Z][A-Z0-9-]*)\b"
    r"(?P<rest>.*)$",
    flags=re.IGNORECASE,
)
```

```
NUMERIC_PIC_PATTERN = re.compile(
    r"\bPIC(?:TURE)?\s+(?:IS\s+)"
    r"(?P<pic>S?9(?:$(?P<len>\d+)?)?"
    r"(?P<trailing>[^\n]*)"
    r"(?P<dot>\.)?)",
    flags=re.IGNORECASE,
)
```

```
MOVE_PATTERN = re.compile(
    r"\bMOVE\s+"
    r"(?P<source>[A-Z][A-Z0-9-]*(?:\.[A-Z][A-Z0-9-]*)?)"
    r"(?:\s+OF\s+[A-Z][A-Z0-9-]*)?"
    r"\s+TO\s+"
    r"(?P<target>[A-Z][A-Z0-9-]*(?:\.[A-Z][A-Z0-9-]*)?)\b",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: IDMS-DB2-Code-Conversion\patterns\sequence_patterns.py
ENCODING: utf-8
=====
```

```
"""
```

```
Manual COBOL sequence number patterns.
```

```
The converter must support both:
```

- unsequenced COBOL
- manual-style sequenced COBOL:
000010 COBOL STATEMENT 00010000

```
"""
```

```
import re
```

```
LEFT_SEQUENCE_PATTERN = re.compile(
    r"^\s*\d{6}\s+(?P<body>.*)$",
    flags=re.IGNORECASE,
)
```

```
RIGHT_SEQUENCE_PATTERN = re.compile(
    r"(?P<body>.*?)(?:\s+(?P<right>\d{8}))\s*$",
    flags=re.IGNORECASE,
)
```

```
FULL_SEQUENCE_PATTERN = re.compile(
    r"^\s*(?P<left>\d{6})\s+(?P<body>.*?)(?:\s+(?P<right>\d{8}))?\s*$",
    flags=re.IGNORECASE,
)
```

```

SEQUENCE_ONLY_PATTERN = re.compile(
    r"^\s*(\d{6}|\d{8})\s*$",
    flags=re.IGNORECASE,
)

def strip_sequence_numbers(
    line: str,
) -> str:
    text = str(line or "").rstrip()

    right_match = RIGHT_SEQUENCE_PATTERN.match(text)

    if right_match:
        text = right_match.group("body").rstrip()

    left_match = LEFT_SEQUENCE_PATTERN.match(text)

    if left_match:
        text = left_match.group("body")

    return text.strip()

def compose_manual_sequence_line(
    left_sequence: str,
    body: str,
    right_sequence: str,
) -> str:
    clean_body = str(body or "")

    if not clean_body:
        return f"{left_sequence} {right_sequence}"

    if len(clean_body) >= 64:
        return f"{left_sequence} {clean_body} {right_sequence}"

    return f"{left_sequence} {clean_body:<64} {right_sequence}"

```

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\sheet_mapping_patterns.py
ENCODING: utf-8
=====

```

```

"""

```

Sheet Mapping regex patterns.

Sheet Mapping parser logic must import these patterns instead of defining
regex directly inside parser classes.

```

"""

```

```

import re

```

```

HEADER_NON_ALPHANUMERIC_PATTERN = re.compile(
    r"^[A-Z0-9]+",
    flags=re.IGNORECASE,
)

```

```

HEADER_WHITESPACE_PATTERN = re.compile(
    r"\s+",
    flags=re.IGNORECASE,
)

```

```

CELL_WHITESPACE_PATTERN = re.compile(
    r"[\t]+",
    flags=re.IGNORECASE,
)

```

```
=====
FILE: IDMS-DB2-Code-Conversion\patterns\sql_patterns.py
ENCODING: utf-8
=====
```

```
"""
```

```
SQL regex patterns and formatting keyword groups.
```

```
SQL generators, formatters, validators, and timestamp/audit components must
import SQL patterns from here instead of defining regex internally.
"""
```

```
import re
```

```
EXEC_SQL_PATTERN = re.compile(
    r"^\s*EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)
```

```
END_EXEC_PATTERN = re.compile(
    r"^\s*END-EXEC\.\s*$",
    flags=re.IGNORECASE,
)
```

```
INCLUDE_PATTERN = re.compile(
    r"\bINCLUDE\s+(?P<table>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)
```

```
SELECT_SQL_PATTERN = re.compile(
    r"\bSELECT\b",
    flags=re.IGNORECASE,
)
```

```
FROM_SQL_PATTERN = re.compile(
    r"\bFROM\s+(?P<table>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)
```

```
WHERE_SQL_PATTERN = re.compile(
    r"\bWHERE\b",
    flags=re.IGNORECASE,
)
```

```
UPDATE_SQL_PATTERN = re.compile(
    r"\bUPDATE\s+(?P<table>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)
```

```
INSERT_SQL_PATTERN = re.compile(
    r"\bINSERT\s+INTO\s+(?P<table>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)
```

```
DELETE_SQL_PATTERN = re.compile(
    r"\bDELETE\s+FROM\s+(?P<table>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)
```

```
OPEN_CURSOR_PATTERN = re.compile(
    r"\bOPEN\s+(?P<cursor>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)
```

```
FETCH_CURSOR_PATTERN = re.compile(
```

```

        r"\bFETCH\s+(?P<cursor>[A-Z][A-Z0-9-]*)\b",
        flags=re.IGNORECASE,
    )

CLOSE_CURSOR_PATTERN = re.compile(
    r"\bCLOSE\s+(?P<cursor>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

HOST_REFERENCE_PATTERN = re.compile(
    r":\s*(?P<group>DCL[A-Z0-9-]+\s*\.\s*(?P<field>[A-Z][A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

SQL_KEYWORDS_ZERO_INDENT = (
    "EXEC SQL",
    "END-EXEC",
)

```

```

SQL_KEYWORDS_LEVEL_1 = (
    "INCLUDE ",
    "DECLARE ",
    "SELECT",
    "INTO",
    "FROM ",
    "WHERE",
    "ORDER BY",
    "GROUP BY",
    "HAVING",
    "FETCH ",
    "OPEN ",
    "CLOSE ",
    "COMMIT",
    "ROLLBACK",
    "INSERT ",
    "UPDATE ",
    "DELETE ",
    "SET ",
    "VALUES",
    "FOR READ ONLY",
)

```

```

SQL_KEYWORDS_LEVEL_2 = (
    "AND ",
    "OR ",
)

```

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\validation_patterns.py
ENCODING: utf-8
=====

```

```

"""
Validation regex patterns.

Validators must import these patterns instead of defining regex internally.
"""

```

```
import re
```

```

TODO_HOST_VARIABLE_PATTERN = re.compile(
    r":\s*TODO-HOST-VARIABLE",
    flags=re.IGNORECASE,
)

```

```

TODO_DB2_PATTERN = re.compile(
    r"TODO\s+DB2",
    flags=re.IGNORECASE,
)

```

)

```
ERROR_DB2_PATTERN = re.compile(  
    r"ERROR\s+DB2\s*:",  
    flags=re.IGNORECASE,  
)
```

```
UNABLE_TO_DECLARE_CURSOR_PATTERN = re.compile(  
    r"UNABLE\s+TO\s+DECLARE\s+Cursors",  
    flags=re.IGNORECASE,  
)
```

```
NO_FETCH_HOST_VARIABLES_PATTERN = re.compile(  
    r"NO\s+FETCH\s+HOST\s+VARIABLES\s+MAPPED",  
    flags=re.IGNORECASE,  
)
```

```
FORBIDDEN_EXECUTABLE_IDMS_PATTERNS = [  
    re.compile(r"^BIND\b", flags=re.IGNORECASE),  
    re.compile(r"^READY\b", flags=re.IGNORECASE),  
    re.compile(r"^OBTAIN\b", flags=re.IGNORECASE),  
    re.compile(r"^FIND\s+CURRENT\b", flags=re.IGNORECASE),  
    re.compile(r"^FIND\s+FIRST\b", flags=re.IGNORECASE),  
    re.compile(r"^STORE\b", flags=re.IGNORECASE),  
    re.compile(r"^MODIFY\b", flags=re.IGNORECASE),  
    re.compile(r"^ERASE\b", flags=re.IGNORECASE),  
    re.compile(r"^CONNECT\b", flags=re.IGNORECASE),  
    re.compile(r"^DISCONNECT\b", flags=re.IGNORECASE),  
    re.compile(  
        r"^PERFORM\s+[A-Z0-9-]*IDMS-STATUS\b",  
        flags=re.IGNORECASE,  
    ),  
    re.compile(  
        r"^PERFORM\s+[A-Z0-9-]*IDMS-ABORT\b",  
        flags=re.IGNORECASE,  
    ),  
    re.compile(  
        r"\bUSAGE-MODE\s+IS\s+UPDATE\b",  
        flags=re.IGNORECASE,  
    ),  
    re.compile(  
        r"\bUSAGE-MODE\s+IS\s+RETRIEVAL\b",  
        flags=re.IGNORECASE,  
    ),  
)
```

```
FORBIDDEN_IDMS_DECLARATIVE_PATTERNS = [  
    re.compile(  
        r"^IDMS-CONTROL\s+SECTION\b",  
        flags=re.IGNORECASE,  
    ),  
    re.compile(  
        r"^PROTOCOL\b",  
        flags=re.IGNORECASE,  
    ),  
    re.compile(  
        r"^IDMS-RECORDS\s+WITHIN\s+WORKING-STORAGE\s+SECTION\b",  
        flags=re.IGNORECASE,  
    ),  
    re.compile(  
        r"^SCHEMA\s+SECTION\b",  
        flags=re.IGNORECASE,  
    ),  
    re.compile(  
        r"^DB[A-Z0-9-]+\s+WITHIN\s+[A-Z0-9-]+\b",  
        flags=re.IGNORECASE,  
    ),  
    re.compile(  
        r"^COPY\s+IDMS\b",  
        flags=re.IGNORECASE,  
    ),  
)
```

```

re.compile(
    r"^COPY\s+IDMS\s+IDMS-STATUS\b",
    flags=re.IGNORECASE,
),
re.compile(
    r"^COPY\s+IDMS\s+SUBSCHEMA-BINDS\b",
    flags=re.IGNORECASE,
),
]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\patterns\validation_rules.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\pyproject.toml
ENCODING: utf-8
=====

```

```

[project]
name = "idms-db2-code-conversion"
version = "0.1.0"
description = "IDMS COBOL to DB2 COBOL Phase 2 converter"
readme = "README.md"
requires-python = ">=3.14"
dependencies = [
    "openpyxl>=3.1.5",
    "pdfplumber>=0.11.10",
    "pydantic>=2.13.4",
    "pydantic-settings>=2.15.0",
    "streamlit>=1.61.1",
]

```

```

[tool.setuptools]
package-dir = {"" = "src"}

```

```

[tool.setuptools.packages.find]
where = ["src"]

```

```

[dependency-groups]
dev = [
    "pytest>=9.1.1",
]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\README.md
ENCODING: utf-8
=====

```

IDMS DB2 Phase 2 Converter

Standalone Phase 2 project for converting IDMS COBOL programs into DB2 embedded SQL COBOL.

Inputs

- Sheet Mapping Excel or CSV
- DCLGEN text file or multiple DCLGEN files
- Copybook text file
- Optional Copybook PDF
- IDMS COBOL source code

Output

- Converted DB2 COBOL code
- Validation messages
- Metadata overview
- Record summary
- Column summary
- Set / relationship summary
- Sheet Mapping preview

Clean Architecture Rule

Hardcoded names, aliases, regex patterns, and business rules must not live inside parsers or services.

Use:

- `catalogs/` for column names, aliases, SQL type lists, labels, and static catalog values
- `patterns/` for regex patterns
- `rules/` for business and conversion rules
- `src/idms\_db2\_phase2/parsers/` for parsing logic only
- `src/idms\_db2\_phase2/services/`, `generators/`, `resolvers/`, `transformers/`, and `validators/` for logic only

Run

From the project root:

```
```bash
set PYTHONPATH=src
python -m streamlit run src/idms_db2_phase2/app.py --server.port 8502

python src\idms_db2_phase2\testing\run_retrieval.py
```

```
=====
FILE: IDMS-DB2-Code-Conversion\rules__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\rules\authority_rules.py
ENCODING: utf-8
=====
```

```
"""
Authority rules.
```

```
These are the central project authority rules confirmed by the existing
conversion service comments.
"""
```

```
INPUT_AUTHORITY_RULES = [
 "Sheet Mapping is the authority for DB2 record/table names.",
 "Sheet Mapping is the authority for DB2 column names.",
 "DCLGEN is the authority for COBOL host variable names and PIC clauses.",
 "Original COBOL is the authority for business flow.",
 "Final output is resequenced in manual-style COBOL format.",
]
```

```
NO_HARDCODE_RULES = [
 "Do not hardcode business records in services.",
 "Do not hardcode DB2 tables in services.",
 "Do not hardcode DB2 columns in services.",
 "Do not hardcode DCLGEN names in services.",
 "Do not hardcode host variables in services.",
 "Do not hardcode parser column aliases inside parser classes.",
 "Do not hardcode regex patterns inside parser or service classes.",
]
```

```
=====
FILE: IDMS-DB2-Code-Conversion\rules\business_flow_rules.py
ENCODING: utf-8
=====
```

```
"""
Business flow preservation rules.
```

```
These constants describe how original COBOL business logic should be preserved.
No regex or runtime logic belongs in this file.
```

```

"""
BUSINESS_FLOW_RULES = [
 "Preserve original COBOL business logic unless replacing IDMS database statements.",
 "Do not rewrite unrelated COBOL statements.",
 "Do not corrupt COBOL headers such as DATE-WRITTEN, AUTHOR, or PROGRAM-ID.",
 "Replace only IDMS database access logic with DB2 embedded SQL logic.",
 "Retain paragraph structure unless a database replacement requires a local block.",
]

=====
FILE: IDMS-DB2-Code-Conversion\rules\business_rules.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: IDMS-DB2-Code-Conversion\rules\cobol_statement_rules.py
ENCODING: utf-8
=====

"""
COBOL statement classification rules.

This file contains COBOL statement rule constants only.

Rules belong in rules/, not patterns/.

No regex patterns should be defined here.
No parser, service, transformer, generator, validator, or composer logic
should be placed here.

Use this file for generic COBOL statement classification values that are
shared by formatters, composers, parsers, transformers, or validators.
"""

NON_PARAGRAPH_SINGLE_WORDS = {
 "ACCEPT",
 "ADD",
 "ALTER",
 "CALL",
 "CANCEL",
 "CLOSE",
 "COMMIT",
 "COMPUTE",
 "CONTINUE",
 "DELETE",
 "DISPLAY",
 "DIVIDE",
 "ELSE",
 "END-ADD",
 "END-CALL",
 "END-DELETE",
 "END-DIVIDE",
 "END-EVALUATE",
 "END-EXEC",
 "END-IF",
 "END-MULTIPLY",
 "END-PERFORM",
 "END-READ",
 "END-RETURN",
 "END-REWRITE",
 "END-SEARCH",
 "END-START",
 "END-STRING",
 "END-SUBTRACT",
 "END-UNSTRING",
 "END-WRITE",
 "EJECT",
 "EVALUATE",
 "EXEC",
 "FETCH",
 "EXIT",

```



```

"GOBACK",
"IF",
"INITIALIZE",
"INSPECT",
"MOVE",
"MULTIPLY",
"NEXT",
"OPEN",
"PERFORM",
"READ",
"RETURN",
"REWRITE",
"ROLLBACK",
"SEARCH",
"SET",
"SKIP1",
"SKIP2",
"SKIP3",
"SORT",
"SPACE",
"SPACES",
"START",
"STOP",
"STRING",
"SUBTRACT",
"UNSTRING",
"WHEN",
"WRITE",
}

```

```
NON_PARAGRAPH_WORDS = NON_PARAGRAPH_SINGLE_WORDS
```

```

PROCEDURE_VERBS = (
 "ACCEPT ",
 "ADD ",
 "ALTER ",
 "CALL ",
 "CANCEL ",
 "CLOSE ",
 "COMMIT",
 "COMPUTE ",
 "CONTINUE",
 "DELETE ",
 "DISPLAY ",
 "DIVIDE ",
 "ELSE",
 "END-ADD",
 "END-CALL",
 "END-DELETE",
 "END-DIVIDE",
 "END-EVALUATE",
 "END-EXEC",
 "END-IF",
 "END-MULTIPLY",
 "END-PERFORM",
 "END-READ",
 "END-RETURN",
 "END-REWRITE",
 "END-SEARCH",
 "END-START",
 "END-STRING",
 "END-SUBTRACT",
 "END-UNSTRING",
 "END-WRITE",
 "EVALUATE ",
 "EXEC SQL",
 "EXIT",
 "FETCH ",
 "GOBACK",
 "IF ",
 "INITIALIZE ",
 "INSPECT ",
 "MOVE ",
 "MULTIPLY ",
 "NEXT ",
 "OPEN ",

```

```

"PERFORM ",
"READ ",
"RETURN ",
"REWRITE ",
"ROLLBACK",
"SEARCH ",
"SET ",
"SORT ",
"START ",
"STOP ",
"STRING ",
"SUBTRACT ",
"UNSTRING ",
"WHEN ",
"WRITE ",
)

SQL_LEVEL_1_KEYWORDS = (
"SELECT ",
"INTO ",
"FROM ",
"WHERE",
"ORDER BY ",
"GROUP BY ",
"HAVING ",
"FETCH ",
"OPEN ",
"CLOSE ",
"COMMIT",
"ROLLBACK",
"INSERT ",
"UPDATE ",
"DELETE ",
"SET ",
"VALUES ",
"FOR READ ONLY",
"QUERYNO ",
)

SQL_LEVEL_2_KEYWORDS = (
"AND ",
"OR ",
)

def is_non_paragraph_word(
 word: str,
) -> bool:
 """
 Return True when the supplied word is a COBOL verb or keyword that should
 not be treated as a paragraph name.
 """
 clean_word = str(word or "").strip().upper()
 return clean_word in NON_PARAGRAPH_WORDS

def starts_with_procedure_verb(
 statement: str,
) -> bool:
 """
 Return True when the statement starts with a known PROCEDURE DIVISION verb.
 """
 clean_statement = str(statement or "").strip().upper()

 if not clean_statement:
 return False

 return clean_statement.startswith(PROCEDURE_VERBS)

def starts_with_sql_level_1_keyword(
 statement: str,
) -> bool:
 """
 Return True when the SQL statement starts with a level-1 SQL keyword.
 """

```

```

clean_statement = str(statement or "").strip().upper()

if not clean_statement:
 return False

return clean_statement.startswith(SQL_LEVEL_1_KEYWORDS)

def starts_with_sql_level_2_keyword(
 statement: str,
) -> bool:
 """
 Return True when the SQL statement starts with a level-2 SQL keyword.
 """
 clean_statement = str(statement or "").strip().upper()

 if not clean_statement:
 return False

 return clean_statement.startswith(SQL_LEVEL_2_KEYWORDS)

=====
FILE: IDMS-DB2-Code-Conversion\rules\conversion_rules.py
ENCODING: utf-8
=====

"""
General IDMS to DB2 conversion rules.
"""

CONVERSION_RULES = [
 "Preserve original COBOL business flow.",
 "Replace IDMS database operations with DB2-compatible SQL logic.",
 "Do not fabricate DB2 SQL when Sheet Mapping metadata is missing.",
 "When mapping metadata is missing, generate a clear DB2 conversion-skipped comment.",
 "When removing executable PROCEDURE DIVISION IDMS code, add CONTINUE.",
]

MISSING_MAPPING_RULES = [
 "FFRECB conversion requires Sheet Mapping and DCLGEN metadata.",
 "If Sheet Mapping entry is missing, conversion must be skipped with a clear comment.",
 "Missing Sheet Mapping metadata is an input-data blocker, not a Python logic failure.",
]

=====
FILE: IDMS-DB2-Code-Conversion\rules\cursor_rules.py
ENCODING: utf-8
=====

"""
Cursor generation rules.

This file contains cursor authority rules and paragraph numbering rules.

Rules belong in rules/, not patterns/.

No regex patterns should be defined here.
No parser, service, transformer, or generator logic should be placed here.
"""

CURSOR_AUTHORITY_RULES = [
 "Cursor names are derived from Sheet Mapping DB2 record/table names.",
 "Cursor names are not derived from IDMS set names.",
 "Generated cursor OPEN, FETCH, and CLOSE paragraphs are placed near the end.",
 "SQL-ERROR paragraph is required when cursor code performs SQL error handling.",
]

CURSOR_PARAGRAPH_NUMBERING = {
 1: {
 "open": 710,
 "fetch": 720,
 "close": 730,
 }

```

```

 },
 2: {
 "open": 810,
 "fetch": 820,
 "close": 830,
 },
 3: {
 "open": 910,
 "fetch": 920,
 "close": 930,
 },
}

def cursor_paragraph_number(
 cursor_order: int,
 operation: str,
) -> int:
 operation_key = str(operation or "").strip().lower()

 if cursor_order not in CURSOR_PARAGRAPH_NUMBERING:
 raise ValueError(f"Unsupported cursor order: {cursor_order}")

 if operation_key not in CURSOR_PARAGRAPH_NUMBERING[cursor_order]:
 raise ValueError(f"Unsupported cursor operation: {operation}")

 return CURSOR_PARAGRAPH_NUMBERING[cursor_order][operation_key]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\rules\db2_date_conversion_rules.py
ENCODING: utf-8
=====

```

```

"""
DB2 date conversion rules.

This file contains reusable templates for DB2 date comparison conversion.

No program name is hardcoded.
No table name is hardcoded.
No DCLGEN group name is hardcoded.
No business field name is hardcoded.

The composer supplies dynamic values such as:
- field_name
- group_name
- helper
- condition
- indent
"""

DB2_DATE_COMPARISON_RULES = [
 "DB2 DATE values must be realigned before numeric comparison.",
 "Generate date conversion only when a DA-/DT- DCLGEN host field is compared with PARMDATE.",
 "Generated helper field names are derived from the detected DB2 host field name.",
 "Date conversion logic must not hardcode program names, table names, record names, or business fields.",
]

```

```

DB2_DATE_COMPARISON_WS_MARKER = "DB2 DATE COMPARISON WORKING STORAGE"

```

```

DB2_DATE_BASE_WORKING_STORAGE_LINES = [
 f"* {DB2_DATE_COMPARISON_WS_MARKER}",
 "01 WS-DATUMVELDEN.",
 " 03 DA-CCYYMMDD.",
 " 05 CC PIC 99 VALUE ZERO.",
 " 05 YY PIC 99 VALUE ZERO.",
 " 05 MM PIC 99 VALUE ZERO.",
 " 05 DD PIC 99 VALUE ZERO.",
 " 03 DA-CCYYMMDD-R REDEFINES DA-CCYYMMDD.",
 " 05 CCYY PIC 9999.",
 " 05 MM PIC 99.",
 " 05 DD PIC 99.",
]

```

```

" 03 DA-DD-MM-CCYY.",
" 05 DD PIC 9(02).",
" 05 TE-MARKER5 PIC X VALUE '."',
" 05 MM PIC 9(02).",
" 05 TE-MARKER6 PIC X VALUE '."',
" 05 CCYY PIC 9(04).",
]

```

```

DB2_DATE_HELPER_FIELD_TEMPLATE = (
 " 02 {helper:<30} PIC 9(8) VALUE ZEROES."
)

```

```

DB2_DATE_LOW_VALUE_LITERAL = "01.01.0001"
DB2_DATE_HIGH_VALUE_LITERAL = "31.12.9999"
DB2_DATE_HIGH_NUMERIC_LITERAL = "99999999"

```

```

DB2_DATE_CONVERSION_LINE_TEMPLATES = [
 "{indent}MOVE ZEROES TO DA-CCYYMMDD",
 "{indent} {helper}",
 "{indent}MOVE {field_name} OF {group_name} TO DA-DD-MM-CCYY",
 "{indent}EVALUATE TRUE",
 "{indent} WHEN DA-DD-MM-CCYY = '{low_value}'",
 "{indent} MOVE ZEROES TO DA-CCYYMMDD-R",
 "{indent} WHEN DA-DD-MM-CCYY = '{high_value}'",
 "{indent} MOVE '{high_numeric}' TO DA-CCYYMMDD-R",
 "{indent} WHEN OTHER",
 "{indent} MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF DA-CCYYMMDD-R",
 "{indent} MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R",
 "{indent} MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R",
 "{indent}END-EVALUATE",
 "{indent}MOVE DA-CCYYMMDD-R TO {helper}",
]

```

```

DB2_DATE_IF_REPLACEMENT_TEMPLATE = "{indent}IF {helper} {condition}"

```

```

=====
FILE: IDMS-DB2-Code-Conversion\rules\db2_infrastructure_rules.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\rules\db2_validation_rules.py
ENCODING: utf-8
=====

```

```

"""
DB2 validation rules.

```

This file contains DB2 validation constants only.

Rules belong in rules/, not patterns/.

No regex patterns should be defined here.

No parser, service, transformer, generator, validator, or composer logic should be placed here.

Use this file for DB2 validation rule values that are shared by validators or other runtime modules.

```

"""
REQUIRED_DB2_TOKENS = (
 "EXEC SQL",
 "SQLCA",
 "END-EXEC",
)

```

```
=====
FILE: IDMS-DB2-Code-Conversion\rules\field_mapping_rules.py
ENCODING: utf-8
=====
```

```
"""
Field mapping rules.
```

```
These constants describe authority and rewrite behavior for IDMS-to-DB2 field
mapping. No regex or runtime logic belongs in this file.
"""
```

```
FIELD_MAPPING_RULES = [
 "Sheet Mapping is the authority for DB2 column names.",
 "DCLGEN is the authority for COBOL host variable spelling.",
 "Do not invent DB2 field names.",
 "Do not use IDMS record names in final DB2 procedure logic when a Sheet Mapping and DCLGEN mapping exists.",
 "Rewrite only safe qualified references unless strong context exists.",
]
```

```
QUALIFIED_REFERENCE_REWRITE_RULES = [
 "Rewrite FIELD OF IDMS-RECORD when a Sheet Mapping and DCLGEN mapping exists.",
 "Rewrite FIELD IN IDMS-RECORD when a Sheet Mapping and DCLGEN mapping exists.",
 "Do not rewrite bare words globally.",
 "When mapping metadata is missing, leave a clear conversion-skipped comment.",
]
```

```
=====
FILE: IDMS-DB2-Code-Conversion\rules\fixed_format_rules.py
ENCODING: utf-8
=====
```

```
"""
Fixed-format COBOL layout rules.
These constants define final physical COBOL layout:
- Columns 1-6 : left sequence number
- Column 7 : indicator area
- Columns 8-72 : COBOL body
- Columns 73-80 : right sequence number
This file contains rules/constants only.
No regex should be defined here.
"""
```

```
TOTAL_WIDTH = 80
BODY_WIDTH = 65
LEFT_SEQUENCE_WIDTH = 6
INDICATOR_WIDTH = 1
RIGHT_SEQUENCE_WIDTH = 8
DEFAULT_LEFT_START = 10
DEFAULT_LEFT_STEP = 10
Generic fallback for manual-style COBOL when input has no valid
manual right sequence, or when input has old/small right sequence.
#
Example:
left 000010 -> right 00010000
left 000020 -> right 00020000
#
This is not program-specific hardcoding.
It is a layout fallback rule.
DEFAULT_RIGHT_START = 10000
DEFAULT_RIGHT_STEP = 10000
AREA_A_INDENT = ""
AREA_B_INDENT = " "
SQL_INDENT = " "
SQL_CONTINUATION_INDENT = " "
SPACE_INDICATOR = " "
COMMENT_INDICATOR = "*"
PAGE_INDICATOR = "/"
DEBUG_INDICATOR = "D"
CONTINUATION_INDICATOR = "- "
VALID_INDICATORS = {
 SPACE_INDICATOR,
 COMMENT_INDICATOR,
 PAGE_INDICATOR,
 DEBUG_INDICATOR,
 "d",
```

```

CONTINUATION_INDICATOR,
}
NON_PARAGRAPH_WORDS = {
"ACCEPT",
"ADD",
"ALTER",
"CALL",
"CANCEL",
"CLOSE",
"COMMIT",
"COMPUTE",
"CONTINUE",
"DELETE",
"DISPLAY",
"DIVIDE",
"ELSE",
"END-ADD",
"END-CALL",
"END-DELETE",
"END-DIVIDE",
"END-EVALUATE",
"END-EXEC",
"END-IF",
"END-MULTIPLY",
"END-PERFORM",
"END-READ",
"END-RETURN",
"END-REWRITE",
"END-SEARCH",
"END-START",
"END-STRING",
"END-SUBTRACT",
"END-UNSTRING",
"END-WRITE",
"EVALUATE",
"EXEC",
"EXIT",
"FETCH",
"GOBACK",
"IF",
"INITIALIZE",
"INSPECT",
"MOVE",
"MULTIPLY",
"NEXT",
"OPEN",
"PERFORM",
"READ",
"RETURN",
"REWRITE",
"ROLLBACK",
"SEARCH",
"SET",
"SKIP1",
"SKIP2",
"SKIP3",
"SORT",
"SPACE",
"SPACES",
"START",
"STOP",
"STRING",
"SUBTRACT",
"UNSTRING",
"WHEN",
"WRITE",
}
PROCEDURE_VERBS = (
"ACCEPT ",
"ADD ",
"ALTER ",
"CALL ",
"CANCEL ",
"CLOSE ",
"COMMIT",
"COMPUTE ",

```

```

"CONTINUE",
"DELETE ",
"DISPLAY ",
"DIVIDE ",
"ELSE",
"END-ADD",
"END-CALL",
"END-DELETE",
"END-DIVIDE",
"END-EVALUATE",
"END-EXEC",
"END-IF",
"END-MULTIPLY",
"END-PERFORM",
"END-READ",
"END-RETURN",
"END-REWRITE",
"END-SEARCH",
"END-START",
"END-STRING",
"END-SUBTRACT",
"END-UNSTRING",
"END-WRITE",
"EVALUATE ",
"EXEC SQL",
"EXIT",
"FETCH ",
"GOBACK",
"IF ",
"INITIALIZE ",
"INSPECT ",
"MOVE ",
"MULTIPLY ",
"NEXT ",
"OPEN ",
"PERFORM ",
"READ ",
"RETURN ",
"REWRITE ",
"ROLLBACK",
"SEARCH ",
"SET ",
"SORT ",
"START ",
"STOP ",
"STRING ",
"SUBTRACT ",
"UNSTRING ",
"WHEN ",
"WRITE ",
)
SQL_LEVEL_1_KEYWORDS = (
"SELECT ",
"INTO ",
"FROM ",
"WHERE ",
"ORDER BY ",
"GROUP BY ",
"HAVING ",
"FETCH ",
"OPEN ",
"CLOSE ",
"COMMIT",
"ROLLBACK",
"INSERT ",
"UPDATE ",
"DELETE ",
"SET ",
"VALUES ",
"FOR READ ONLY",
"QUERYNO ",
)
SQL_LEVEL_2_KEYWORDS = (
"AND ",
"OR ",
",",

```



```

)
AREA_A_HEADER_PREFIXES = (
"PROGRAM-ID.",
"AUTHOR.",
"INSTALLATION.",
"DATE-WRITTEN.",
"DATE-COMPILED.",
"SECURITY.",
)
AREA_A_DATA_LEVELS = {
"01",
"66",
"77",
}
AREA_B_DATA_LEVELS = {
"02",
"03",
"04",
"05",
"06",
"07",
"08",
"09",
"10",
"11",
"12",
"13",
"14",
"15",
"16",
"17",
"18",
"19",
"20",
"21",
"22",
"23",
"24",
"25",
"26",
"27",
"28",
"29",
"30",
"31",
"32",
"33",
"34",
"35",
"36",
"37",
"38",
"39",
"40",
"41",
"42",
"43",
"44",
"45",
"46",
"47",
"48",
"49",
"88",
}

```

```

=====
FILE: IDMS-DB2-Code-Conversion\rules\fixed_format_rules.py.broken_before_pdf_restore
ENCODING: utf-8
=====

```

```

"""
Fixed-format COBOL layout rules.

```

```

Final physical COBOL layout:

```

- Columns 1-6 : left sequence number
- Column 7 : indicator area
- Columns 8-72 : COBOL body
- Columns 73-80 : right sequence number

This file contains constants only.

No regex should be defined here.

No formatter, parser, composer, service, or transformer logic belongs here.

"""

```
TOTAL_WIDTH = 80
LEFT_SEQUENCE_WIDTH = 6
INDICATOR_WIDTH = 1
BODY_WIDTH = 65
RIGHT_SEQUENCE_WIDTH = 8
```

```
DEFAULT_LEFT_START = 10
DEFAULT_LEFT_STEP = 10
```

```
DEFAULT_RIGHT_START = 10000
DEFAULT_RIGHT_STEP = 10000
```

```
SPACE_INDICATOR = " "
COMMENT_INDICATOR = "*"
DEBUG_INDICATOR = "D"
PAGE_INDICATOR = "/"
CONTINUATION_INDICATOR = "-"
```

```
VALID_INDICATORS = {
 SPACE_INDICATOR,
 COMMENT_INDICATOR,
 DEBUG_INDICATOR,
 DEBUG_INDICATOR.lower(),
 PAGE_INDICATOR,
 CONTINUATION_INDICATOR,
}
```

```
AREA_A_INDENT = ""
AREA_B_INDENT = " "
```

```
MIN_MANUAL_RIGHT_SEQUENCE = 10000
MIN_MANUAL_RIGHT_STEP = 10000
```

```
MAX_BLANK_LINES = 3
```

```
=====
FILE: IDMS-DB2-Code-Conversion\rules\fixed_format_rules.py.broken_before_pdf_restore_1
ENCODING: utf-8
=====
```

"""

Fixed-format COBOL layout rules.

These constants define final physical COBOL layout:

- Columns 1-6 : left sequence number
- Column 7 : indicator area
- Columns 8-72 : COBOL body
- Columns 73-80 : right sequence number

This file contains rules/constants only.

No regex should be defined here.

"""

```
TOTAL_WIDTH = 80
BODY_WIDTH = 65
LEFT_SEQUENCE_WIDTH = 6
INDICATOR_WIDTH = 1
RIGHT_SEQUENCE_WIDTH = 8
DEFAULT_LEFT_START = 10
DEFAULT_LEFT_STEP = 10
```

```
Generic fallback for manual-style COBOL when input has no valid
manual right sequence, or when input has old/small right sequence.
```

```
#
```

```
Example:
```

```
left 000010 -> right 00010000
```

```
left 000020 -> right 00020000
```

```
#
```

```
This is not program-specific hardcoding.
It is a layout fallback rule.
```

```
Page 39
```

```
DEFAULT_RIGHT_START = 10000
DEFAULT_RIGHT_STEP = 10000
AREA_A_INDENT = ""
AREA_B_INDENT = " "
SQL_INDENT = " "
SQL_CONTINUATION_INDENT = " "
SPACE_INDICATOR = " "
COMMENT_INDICATOR = "*" "
PAGE_INDICATOR = "/" "
DEBUG_INDICATOR = "D"
CONTINUATION_INDICATOR = "- "
VALID_INDICATORS = {
 SPACE_INDICATOR,
 COMMENT_INDICATOR,
 PAGE_INDICATOR,
 DEBUG_INDICATOR,
 "d",
 CONTINUATION_INDICATOR,
}
NON_PARAGRAPH_WORDS = {
 "ACCEPT",
 "ADD",
 "ALTER",
 "CALL",
 "CANCEL",
 "CLOSE",
 "COMMIT",
 "COMPUTE",
 "CONTINUE",
 "DELETE",
 "DISPLAY",
 "DIVIDE",
 "ELSE",
 "END-ADD",
 "END-CALL",
 "END-DELETE",
 "END-DIVIDE",
 "END-EVALUATE",
 "END-EXEC",
 "END-IF",
 "END-MULTIPLY",
 "END-PERFORM",
 "END-READ",
 "END-RETURN",
 "END-REWRITE",
 "END-SEARCH",
 "END-START",
 "END-STRING",
 "END-SUBTRACT",
 "END-UNSTRING",
 "END-WRITE",
 "EVALUATE",
 "EXEC",
 "EXIT",
 "FETCH",
 "GOBACK",
 "IF",
 "INITIALIZE",
 "INSPECT",
 "MOVE",
 "MULTIPLY",
 "NEXT",
 "OPEN",
 "PERFORM",
 "READ",
 "RETURN",
 "REWRITE",
 "ROLLBACK",
 "SEARCH",
 "SET",
 "SKIP1",
 "SKIP2",
}
```

```
Page 40
```

```
"SKIP2",
```

```

"SKIP3",
"SORT",
"SPACE",
"SPACES",
"START",
"STOP",
"STRING",
"SUBTRACT",
"UNSTRING",
"WHEN",
"WRITE",
}
PROCEDURE_VERBS = (
"ACCEPT ",
"ADD ",
"ALTER ",
"CALL ",
"CANCEL ",
"CLOSE ",
"COMMIT",
"COMPUTE ",
"CONTINUE",
"DELETE ",
"DISPLAY ",
"DIVIDE ",
"ELSE",
"END-ADD",
"END-CALL",
"END-DELETE",
"END-DIVIDE",
"END-EVALUATE",
"END-EXEC",
"END-IF",
"END-MULTIPLY",
"END-PERFORM",
"END-READ",
"END-RETURN",
"END-REWRITE",
"END-SEARCH",
"END-START",
"END-STRING",
"END-SUBTRACT",
"END-UNSTRING",
"END-WRITE",
"EVALUATE ",
"EXEC SQL",
"EXIT",
"FETCH ",
"GOBACK",
"IF ",
"INITIALIZE ",
"INSPECT ",
"MOVE ",
"MULTIPLY ",
"NEXT ",
"OPEN ",
"PERFORM ",
"READ ",
"RETURN ",
"REWRITE ",
"ROLLBACK",
"SEARCH ",
"SET ",
"SORT ",
"START ",
"STOP ",
"STRING ",
"SUBTRACT ",
"UNSTRING ",
"WHEN ",
"WRITE ",
)
Page 41
SQL_LEVEL_1_KEYWORDS = (
"SELECT ",
"INTO ",

```

```

"FROM ",
"WHERE ",
"ORDER BY ",
"GROUP BY ",
"HAVING ",
"FETCH ",
"OPEN ",
"CLOSE ",
"COMMIT",
"ROLLBACK",
"INSERT ",
"UPDATE ",
"DELETE ",
"SET ",
"VALUES ",
"FOR READ ONLY",
"QUERYNO ",
)
SQL_LEVEL_2_KEYWORDS = (
"AND ",
"OR ",
",",
)
AREA_A_HEADER_PREFIXES = (
"PROGRAM-ID.",
"AUTHOR.",
"INSTALLATION.",
"DATE-WRITTEN.",
"DATE-COMPILED.",
"SECURITY.",
)
AREA_A_DATA_LEVELS = {
"01",
"66",
"77",
}
AREA_B_DATA_LEVELS = {
"02",
"03",
"04",
"05",
"06",
"07",
"08",
"09",
"10",
"11",
"12",
"13",
"14",
"15",
"16",
"17",
"18",
"19",
"20",
"21",
"22",
"23",
"24",
"25",
"26",
"27",
"28",
Page 42
"29",
"30",
"31",
"32",
"33",
"34",
"35",
"36",
"37",
"38",
"39",

```

```

"40",
"41",
"42",
"43",
"44",
"45",
"46",
"47",
"48",
"49",
"88",
}
=====

=====
FILE: IDMS-DB2-Code-Conversion\rules\fixed_format_rules.py.broken_before_pdf_restore_2
ENCODING: utf-8
=====

"""
Fixed-format COBOL layout rules.
These constants define final physical COBOL layout:
- Columns 1-6 : left sequence number
- Column 7 : indicator area
- Columns 8-72 : COBOL body
- Columns 73-80 : right sequence number
This file contains rules/constants only.
No regex should be defined here.
"""

TOTAL_WIDTH = 80
BODY_WIDTH = 65
LEFT_SEQUENCE_WIDTH = 6
INDICATOR_WIDTH = 1
RIGHT_SEQUENCE_WIDTH = 8
DEFAULT_LEFT_START = 10
DEFAULT_LEFT_STEP = 10
Generic fallback for manual-style COBOL when input has no valid
manual right sequence, or when input has old/small right sequence.
#
Example:
left 000010 -> right 00010000
left 000020 -> right 00020000
#
This is not program-specific hardcoding.
It is a layout fallback rule.
Page 39
DEFAULT_RIGHT_START = 10000
DEFAULT_RIGHT_STEP = 10000
AREA_A_INDENT = ""
AREA_B_INDENT = " "
SQL_INDENT = " "
SQL_CONTINUATION_INDENT = " "
SPACE_INDICATOR = " "
COMMENT_INDICATOR = "*"
PAGE_INDICATOR = "/"
DEBUG_INDICATOR = "D"
CONTINUATION_INDICATOR = "-"
VALID_INDICATORS = {
SPACE_INDICATOR,
COMMENT_INDICATOR,
PAGE_INDICATOR,
DEBUG_INDICATOR,
"d",
CONTINUATION_INDICATOR,
}
NON_PARAGRAPH_WORDS = {
"ACCEPT",
"ADD",
"ALTER",
"CALL",
"CANCEL",
"CLOSE",
"COMMIT",
"COMPUTE",
"CONTINUE",

```

```

"DELETE",
"DISPLAY",
"DIVIDE",
"ELSE",
"END-ADD",
"END-CALL",
"END-DELETE",
"END-DIVIDE",
"END-EVALUATE",
"END-EXEC",
"END-IF",
"END-MULTIPLY",
"END-PERFORM",
"END-READ",
"END-RETURN",
"END-REWRITE",
"END-SEARCH",
"END-START",
"END-STRING",
"END-SUBTRACT",
"END-UNSTRING",
"END-WRITE",
"EVALUATE",
"EXEC",
"EXIT",
"FETCH",
"GOBACK",
"IF",
"INITIALIZE",
"INSPECT",
"MOVE",
"MULTIPLY",
"NEXT",
"OPEN",
"PERFORM",
"READ",
"RETURN",
"REWRITE",
"ROLLBACK",
"SEARCH",
"SET",
"SKIP1",
Page 40
"SKIP2",
"SKIP3",
"SORT",
"SPACE",
"SPACES",
"START",
"STOP",
"STRING",
"SUBTRACT",
"UNSTRING",
"WHEN",
"WRITE",
}
PROCEDURE_VERBS = (
"ACCEPT ",
"ADD ",
"ALTER ",
"CALL ",
"CANCEL ",
"CLOSE ",
"COMMIT",
"COMPUTE ",
"CONTINUE",
"DELETE ",
"DISPLAY ",
"DIVIDE ",
"ELSE",
"END-ADD",
"END-CALL",
"END-DELETE",
"END-DIVIDE",
"END-EVALUATE",
"END-EXEC",

```

```

"END-IF",
"END-MULTIPLY",
"END-PERFORM",
"END-READ",
"END-RETURN",
"END-REWRITE",
"END-SEARCH",
"END-START",
"END-STRING",
"END-SUBTRACT",
"END-UNSTRING",
"END-WRITE",
"EVALUATE ",
"EXEC SQL",
"EXIT",
"FETCH ",
"GOBACK",
"IF ",
"INITIALIZE ",
"INSPECT ",
"MOVE ",
"MULTIPLY ",
"NEXT ",
"OPEN ",
"PERFORM ",
"READ ",
"RETURN ",
"REWRITE ",
"ROLLBACK",
"SEARCH ",
"SET ",
"SORT ",
"START ",
"STOP ",
"STRING ",
"SUBTRACT ",
"UNSTRING ",
"WHEN ",
"WRITE ",
)
Page 41
SQL_LEVEL_1_KEYWORDS = (
"SELECT ",
"INTO ",
"FROM ",
"WHERE ",
"ORDER BY ",
"GROUP BY ",
"HAVING ",
"FETCH ",
"OPEN ",
"CLOSE ",
"COMMIT",
"ROLLBACK",
"INSERT ",
"UPDATE ",
"DELETE ",
"SET ",
"VALUES ",
"FOR READ ONLY",
"QUERYNO ",
)
SQL_LEVEL_2_KEYWORDS = (
"AND ",
"OR ",
",",
)
AREA_A_HEADER_PREFIXES = (
"PROGRAM-ID.",
"AUTHOR.",
"INSTALLATION.",
"DATE-WRITTEN.",
"DATE-COMPILED.",
"SECURITY.",
)
AREA_A_DATA_LEVELS = {

```



```

"01",
"66",
"77",
}
AREA_B_DATA_LEVELS = {
"02",
"03",
"04",
"05",
"06",
"07",
"08",
"09",
"10",
"11",
"12",
"13",
"14",
"15",
"16",
"17",
"18",
"19",
"20",
"21",
"22",
"23",
"24",
"25",
"26",
"27",
"28",
Page 42
"29",
"30",
"31",
"32",
"33",
"34",
"35",
"36",
"37",
"38",
"39",
"40",
"41",
"42",
"43",
"44",
"45",
"46",
"47",
"48",
"49",
"88",
}

```

```

=====
FILE: IDMS-DB2-Code-Conversion\rules>manual_layout_rules.py
ENCODING: utf-8
=====

```

"""

Manual-style COBOL layout rules.

These constants describe final manual-style sequencing behavior.

No regex or runtime logic belongs in this file.

"""

```

MANUAL_LAYOUT_RULES = [
 "Final COBOL should be manual-style sequenced.",
 "Columns 1-6 contain the left sequence number.",
 "Columns 8-72 contain the COBOL body.",
 "Columns 73-80 contain the right sequence number.",
 "Generated output must preserve COBOL headers and division structure.",

```

]

```
=====
FILE: IDMS-DB2-Code-Conversion\rules\sql_generation_rules.py
ENCODING: utf-8
=====
```

```
"""
SQL generation rules.
```

```
SQL services and generators must import these rules instead of embedding
business rules directly in generator classes.
"""
```

```
SQL_GENERATION_RULES = [
 "Sheet Mapping is the authority for DB2 record/table names.",
 "Sheet Mapping is the authority for DB2 column names.",
 "DCLGEN is the authority for COBOL host variable spelling and group names.",
 "If Sheet Mapping uses TB but uploaded DCLGEN uses TV, generated SQL uses TV.",
 "UPDATE generation is conservative/manual-style, not broad all-column update.",
]
```

```
TABLE_RESOLUTION_RULES = [
 "If DZBFARTB is found in Sheet Mapping but DCLGEN has DZBFARTV, resolve to DZBFARTV.",
 "Generated SELECT FROM table name must match resolved DCLGEN table name.",
 "Generated UPDATE table name must match resolved DCLGEN table name.",
]
```

```
=====
FILE: IDMS-DB2-Code-Conversion\rules\timestamp_audit_rules.py
ENCODING: utf-8
=====
```

```
"""
Timestamp and audit rules.
```

```
TIMESTAMP_AUDIT_RULES = [
 "Sheet Mapping decides DB2 table and column names.",
 "DCLGEN supplies COBOL host variable spelling and group names.",
 "DCLGEN must not introduce audit fields absent from Sheet Mapping.",
 "For UPDATE flows, generate TS_UPDATE and USER-ID moves only.",
 "TS_CREATE is insert-only and is not generated for update-only flows.",
 "Timestamp paragraph must be a safe paragraph with a terminating boundary.",
]
```

```
AUDIT_COLUMN_PREFIXES = [
 "TS_CREATE",
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
 "NS_IDMSKEY",
]
```

```
UPDATE_AUDIT_COLUMN_PREFIXES = [
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
]
```

```
INSERT_EXCLUDE_AUDIT_PREFIXES = [
 "TS_UPDATE",
]
```

```
DATE_COLUMN_PREFIXES = [
 "DA_",
 "DT_",
]
```

```
=====
FILE: IDMS-DB2-Code-Conversion\rules\validation_rules.py
ENCODING: utf-8
=====
```

```
"""
Validation rule messages.
Validators import these constants so validation message text stays centralized.
"""
INPUT_VALIDATION_MESSAGES = {
 "missing_sheet_mapping": "Sheet Mapping is required and must contain rows.",
 "missing_dclgen": "At least one DCLGEN file is required.",
 "missing_dolgen": "At least one DCLGEN file is required.",
 "missing_idms_cobol": "IDMS COBOL source text file is required and must contain readable COBOL code.",
}
MAPPING_VALIDATION_MESSAGES = {
 "missing_mapping_rows": "Mapping validation: Sheet Mapping rows are missing.",
 "missing_dclgen_columns": "Mapping validation: DCLGEN columns are missing.",
 "missing_record_mapping": "Mapping validation: DB2 table mapping is missing.",
 "missing_column_mapping": "Mapping validation: DB2 column mapping is missing.",
}
PRODUCTION_VALIDATION_MESSAGES = {
 "todo_host_variable": "Production validation: TODO host variable remains in generated COBOL.",
 "todo_db2": "Production validation: TODO DB2 conversion marker remains in generated COBOL.",
 "error_db2": "Production validation: generated DB2 error marker remains in generated COBOL.",
 "unable_to_declare_cursor": "Production validation: unable-to-declare-cursor marker remains in generated COBOL.",
 "no_fetch_host_variables": "Production validation: cursor FETCH has no host variables.",
 "no_dclgen_hosts": "Production validation: no valid DCLGEN host variables are available for validation.",
}
```

```
=====
FILE: IDMS-DB2-Code-Conversion\rules\validation_rules.py.broken_before_pdf_restore
ENCODING: utf-8
=====
```

```
"""
Validation rule messages.

Validators and validation services import these constants so validation
message text stays centralized.

Rules:
- No regex in this file.
- No validation service logic in this file.
- No parser logic in this file.
"""
```

```
INPUT_VALIDATION_MESSAGES = {
 "missing_sheet_mapping": "Sheet Mapping is required and must contain rows.",
 "missing_dclgen": "At least one DCLGEN file is required.",
 "missing_dolgen": "At least one DCLGEN file is required.",
 "missing_idms_cobol": (
 "IDMS COBOL source text file is required and must contain readable "
 "COBOL code."
),
}

MAPPING_VALIDATION_MESSAGES = {
 "missing_mapping_rows": "Mapping validation: Sheet Mapping rows are missing.",
 "missing_dclgen_columns": "Mapping validation: DCLGEN columns are missing.",
 "missing_record_mapping": "Mapping validation: DB2 table mapping is missing.",
 "missing_column_mapping": "Mapping validation: DB2 column mapping is missing.",
}

PRODUCTION_VALIDATION_MESSAGES = {
 "todo_host_variable": (
 "Production validation: TODO host variable remains in generated COBOL."
),
}
```

```

"todo_db2": (
 "Production validation: TODO DB2 conversion marker remains in generated COBOL."
),
"error_db2": (
 "Production validation: generated DB2 error marker remains in generated COBOL."
),
"unable_to_declare_cursor": (
 "Production validation: unable-to-declare-cursor marker remains in "
 "generated COBOL."
),
"no_fetch_host_variables": (
 "Production validation: cursor FETCH has no host variables."
),
"no_dclgen_hosts": (
 "Production validation: no valid DCLGEN host variables are available "
 "for validation."
),
"missing_exec_sql": (
 "Production validation: required DB2 token missing: EXEC SQL"
),
"missing_sqlca": (
 "Production validation: required DB2 token missing: SQLCA"
),
"missing_end_exec": (
 "Production validation: required DB2 token missing: END-EXEC"
),
"residual_idms_statement": (
 "Production validation: residual executable IDMS statement remains."
),
"unknown_host_reference": (
 "Production validation: host reference was not found in uploaded "
 "DCLGEN columns."
),
}

```

```

=====
FILE: IDMS-DB2-Code-Conversion\rules\validation_rules.py.broken_before_pdf_restore_1
ENCODING: utf-8
=====

```

```

=====
"""
Validation rule messages.
Validators import these constants so validation message text stays centralized.
"""

INPUT_VALIDATION_MESSAGES = {
 "missing_sheet_mapping": "Sheet Mapping is required and must contain rows.",
 "missing_dclgen": "At least one DCLGEN file is required.",
 "missing_dolgen": "At least one DCLGEN file is required.",
 "missing_idms_cobol": "IDMS COBOL source text file is required and must contain readable COBOL code.",
}

MAPPING_VALIDATION_MESSAGES = {
 "missing_mapping_rows": "Mapping validation: Sheet Mapping rows are missing.",
 "missing_dclgen_columns": "Mapping validation: DCLGEN columns are missing.",
 "missing_record_mapping": "Mapping validation: DB2 table mapping is missing.",
 "missing_column_mapping": "Mapping validation: DB2 column mapping is missing.",
}

PRODUCTION_VALIDATION_MESSAGES = {
 "todo_host_variable": "Production validation: TODO host variable remains in generated COBOL.",
 "todo_db2": "Production validation: TODO DB2 conversion marker remains in generated COBOL.",
 "error_db2": "Production validation: generated DB2 error marker remains in generated COBOL.",
 "unable_to_declare_cursor": "Production validation: unable-to-declare-cursor marker remains in generated COBOL.",
 "no_fetch_host_variables": "Production validation: cursor FETCH has no host variables.",
 "no_dclgen_hosts": "Production validation: no valid DCLGEN host variables are available for validation.",
}

```

Page 44

```

=====
FILE: IDMS-DB2-Code-Conversion\rules\validation_rules.py.broken_before_pdf_restore_2
ENCODING: utf-8
=====
"""

```

```

Validation rule messages.
Validators import these constants so validation message text stays centralized.
"""

INPUT_VALIDATION_MESSAGES = {
 "missing_sheet_mapping": "Sheet Mapping is required and must contain rows.",
 "missing_dclgen": "At least one DCLGEN file is required.",
 "missing_dolgen": "At least one DCLGEN file is required.",
 "missing_idms_cobol": "IDMS COBOL source text file is required and must contain readable COBOL code.",
}

MAPPING_VALIDATION_MESSAGES = {
 "missing_mapping_rows": "Mapping validation: Sheet Mapping rows are missing.",
 "missing_dclgen_columns": "Mapping validation: DCLGEN columns are missing.",
 "missing_record_mapping": "Mapping validation: DB2 table mapping is missing.",
 "missing_column_mapping": "Mapping validation: DB2 column mapping is missing.",
}

PRODUCTION_VALIDATION_MESSAGES = {
 "todo_host_variable": "Production validation: TODO host variable remains in generated COBOL.",
 "todo_db2": "Production validation: TODO DB2 conversion marker remains in generated COBOL.",
 "error_db2": "Production validation: generated DB2 error marker remains in generated COBOL.",
 "unable_to_declare_cursor": "Production validation: unable-to-declare-cursor marker remains in generated COBOL.",
 "no_fetch_host_variables": "Production validation: cursor FETCH has no host variables.",
 "no_dclgen_hosts": "Production validation: no valid DCLGEN host variables are available for validation.",
}

```

Page 44

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\analyzers__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\analyzers\field_usage_analyzer.py
ENCODING: utf-8
=====

```

```

import re
from dataclasses import dataclass, field

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.sequence_patterns import strip_sequence_numbers

```

```

@dataclass
class FieldUsage:
 record_name: str
 condition_fields: set[str] = field(default_factory=set)
 output_fields: set[str] = field(default_factory=set)
 move_source_fields: set[str] = field(default_factory=set)
 move_target_fields: set[str] = field(default_factory=set)
 dclgen_host_fields: set[str] = field(default_factory=set)
 all_fields: set[str] = field(default_factory=set)

@dataclass
class FieldUsageAnalysis:
 usage_by_record: dict[str, FieldUsage] = field(default_factory=dict)
 diagnostics: list[str] = field(default_factory=list)

```

```

class FieldUsageAnalyzer:
 """
 Generic COBOL field-usage analyzer.

```

Detects:

- IDMS qualified references: FIELD OF RECORD / FIELD IN RECORD
- DCLGEN qualified references: FIELD OF DCLGROUP
- DB2 host references: :DCLGROUP.FIELD / :FIELD OF DCLGROUP
- MOVE source fields used for output
- condition fields in IF / WHEN / UNTIL / EVALUATE

This analyzer does not rewrite COBOL.

"""

```
QUALIFIED_REFERENCE_PATTERN = re.compile(
 r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+(?:OF|IN)\s+"
 r"(?:P<record>[A-Z][A-Z0-9-]*)\b",
 flags=re.IGNORECASE,
)

DCLGEN_OF_PATTERN = re.compile(
 r":?\s*(?:P<field>[A-Z][A-Z0-9-]*)\s+OF\s+"
 r"(?:P<group>DCL[A-Z0-9-]+)",
 flags=re.IGNORECASE,
)

DCLGEN_DOT_PATTERN = re.compile(
 r":?\s*(?:P<group>DCL[A-Z0-9-]+)\.?(?:P<field>[A-Z][A-Z0-9-]*)",
 flags=re.IGNORECASE,
)

MOVE_PATTERN = re.compile(
 r"\bMOVE\s+(?:P<source>.+?)\s+TO\s+(?:P<target>.+?)(?:\.|$)",
 flags=re.IGNORECASE,
)

CONDITION_PATTERN = re.compile(
 r"^\s*(IF|WHEN|UNTIL|EVALUATE)\b",
 flags=re.IGNORECASE,
)

DIVISION_PATTERN = re.compile(
 r"^\s*(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\s+DIVISION\b",
 flags=re.IGNORECASE,
)

COMMENT_PATTERN = re.compile(
 r"^\s*[/*]",
 flags=re.IGNORECASE,
)

def __init__(
 self,
 mapping_repository: MappingRepository,
 table_name_resolver: TableNameResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver
 self.group_to_record = self._build_group_to_record_map()

def analyze(
 self,
 cobol_text: str,
) -> FieldUsageAnalysis:
 result = FieldUsageAnalysis()
 inside_procedure = False
 inside_exec_sql = False

 for line_number, raw_line in enumerate(
 str(cobol_text or "").splitlines(),
 start=1,
):
 logical = strip_sequence_numbers(raw_line).strip()

 if not logical:
 continue

 if self.COMMENT_PATTERN.match(logical):
 continue
```

```

division_match = self.DIVISION_PATTERN.match(logical)

if division_match:
 inside_procedure = division_match.group(1).upper() == "PROCEDURE"
 continue

if logical.upper().startswith("EXEC SQL"):
 inside_exec_sql = True

if inside_exec_sql:
 self._capture_dclgen_references(
 logical=logical,
 result=result,
 is_condition=False,
 is_output=False,
)

 if logical.upper().startswith("END-EXEC"):
 inside_exec_sql = False

 continue

if not inside_procedure:
 continue

is_condition = bool(self.CONDITION_PATTERN.match(logical))

self._capture_idms_qualified_references(
 logical=logical,
 result=result,
 is_condition=is_condition,
)

self._capture_dclgen_references(
 logical=logical,
 result=result,
 is_condition=is_condition,
 is_output=False,
)

self._capture_move_usage(
 logical=logical,
 result=result,
)

for usage in result.usage_by_record.values():
 usage.all_fields.update(usage.condition_fields)
 usage.all_fields.update(usage.output_fields)
 usage.all_fields.update(usage.move_source_fields)
 usage.all_fields.update(usage.move_target_fields)
 usage.all_fields.update(usage.dclgen_host_fields)

result.diagnostics.append(
 f"Field usage analyzer: records with usage detected: {len(result.usage_by_record)}"
)

for record_name, usage in sorted(result.usage_by_record.items()):
 result.diagnostics.append(
 "Field usage analyzer: "
 f"record={record_name}, "
 f"condition={len(usage.condition_fields)}, "
 f"output={len(usage.output_fields)}, "
 f"move_source={len(usage.move_source_fields)}, "
 f"move_target={len(usage.move_target_fields)}, "
 f"dclgen={len(usage.dclgen_host_fields)}"
)

return result

def _capture_idms_qualified_references(
 self,
 logical: str,
 result: FieldUsageAnalysis,
 is_condition: bool,
) -> None:

```

```

for match in self.QUALIFIED_REFERENCE_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 record_name = NameNormalizer.normalize(match.group("record"))

 if not field_name or not record_name:
 continue

 if record_name.startswith("DCL"):
 continue

 if record_name not in self._mapping_records():
 continue

 usage = self._usage_for_record(result, record_name)
 usage.all_fields.add(field_name)

 if is_condition:
 usage.condition_fields.add(field_name)

def _capture_dclgen_references(
 self,
 logical: str,
 result: FieldUsageAnalysis,
 is_condition: bool,
 is_output: bool,
) -> None:
 for match in self.DCLGEN_OF_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 group_name = NameNormalizer.normalize(match.group("group"))
 record_name = self.group_to_record.get(group_name, "")

 if not field_name or not record_name:
 continue

 usage = self._usage_for_record(result, record_name)
 usage.dclgen_host_fields.add(field_name)

 if is_condition:
 usage.condition_fields.add(field_name)

 if is_output:
 usage.output_fields.add(field_name)

 for match in self.DCLGEN_DOT_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 group_name = NameNormalizer.normalize(match.group("group"))
 record_name = self.group_to_record.get(group_name, "")

 if not field_name or not record_name:
 continue

 usage = self._usage_for_record(result, record_name)
 usage.dclgen_host_fields.add(field_name)

 if is_condition:
 usage.condition_fields.add(field_name)

 if is_output:
 usage.output_fields.add(field_name)

def _capture_move_usage(
 self,
 logical: str,
 result: FieldUsageAnalysis,
) -> None:
 match = self.MOVE_PATTERN.search(logical)

 if not match:
 return

 source_text = match.group("source")
 target_text = match.group("target")
 is_output = "UIT-" in target_text.upper()

 for source_match in self.QUALIFIED_REFERENCE_PATTERN.finditer(source_text):
 field_name = NameNormalizer.to_cobol(source_match.group("field"))

```



```

 record_name = NameNormalizer.normalize(source_match.group("record"))

 if not field_name or not record_name:
 continue

 if record_name.startswith("DCL"):
 continue

 if record_name not in self._mapping_records():
 continue

 usage = self._usage_for_record(result, record_name)
 usage.move_source_fields.add(field_name)

 if is_output:
 usage.output_fields.add(field_name)

 self._capture_dclgen_references(
 logical=source_text,
 result=result,
 is_condition=False,
 is_output=is_output,
)

 for target_match in self.QUALIFIED_REFERENCE_PATTERN.finditer(target_text):
 field_name = NameNormalizer.to_cobol(target_match.group("field"))
 record_name = NameNormalizer.normalize(target_match.group("record"))

 if not field_name or not record_name:
 continue

 if record_name.startswith("DCL"):
 continue

 if record_name not in self._mapping_records():
 continue

 usage = self._usage_for_record(result, record_name)
 usage.move_target_fields.add(field_name)

def _usage_for_record(
 self,
 result: FieldUsageAnalysis,
 record_name: str,
) -> FieldUsage:
 record = NameNormalizer.normalize(record_name)

 if record not in result.usage_by_record:
 result.usage_by_record[record] = FieldUsage(record_name=record)

 return result.usage_by_record[record]

def _build_group_to_record_map(
 self,
) -> dict[str, str]:
 output: dict[str, str] = {}

 for record in self._mapping_records():
 normalized_record = NameNormalizer.normalize(record)
 table = self.table_name_resolver.table_for_record(normalized_record)

 if not table:
 continue

 group = "DCL" + NameNormalizer.to_cobol(table)
 output[NameNormalizer.normalize(group)] = normalized_record

 return output

def _mapping_records(
 self,
) -> set[str]:
 try:
 return {
 NameNormalizer.normalize(record)
 for record in self.mapping_repository.records()
 }
 except:
 return set()

```

```

 if NameNormalizer.normalize(record)
 }
except Exception:
 return set()

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\analyzers\field_usage_analyzer.py.broken_before_pdf_restore
ENCODING: utf-8
=====

```

```

"""
Generic COBOL field-usage analyzer.

```

```

Detects:
- IDMS qualified references: FIELD OF RECORD / FIELD IN RECORD
- DCLGEN qualified references: FIELD OF DCLGROUP
- DB2 host references: :DCLGROUP.FIELD
- MOVE source and target fields
- condition fields in IF / WHEN / UNTIL / EVALUATE
- output fields in output-style statements

```

This analyzer does not rewrite COBOL.

```

Architecture rule:
- Regex patterns live in patterns/field_usage_patterns.py.
- This file contains analyzer logic only.
"""

```

```

from dataclasses import dataclass, field

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.field_usage_patterns import (
 COMMENT_OR_BLANK_PATTERN,
 CONDITION_STATEMENT_PATTERN,
 DCLGEN_DOT_PATTERN,
 DCLGEN_OF_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 MOVE_STATEMENT_PATTERN,
 OUTPUT_STATEMENT_PATTERN,
 QUALIFIED_REFERENCE_PATTERN,
)
from patterns.sequence_patterns import strip_sequence_numbers

```

```

@dataclass
class FieldUsage:
 """
 Field usage collected for one COBOL record or DCLGEN group.
 """

 record_name: str
 condition_fields: set[str] = field(default_factory=set)
 output_fields: set[str] = field(default_factory=set)
 move_source_fields: set[str] = field(default_factory=set)
 move_target_fields: set[str] = field(default_factory=set)
 dclgen_host_fields: set[str] = field(default_factory=set)
 all_fields: set[str] = field(default_factory=set)

```

```

@dataclass
class FieldUsageAnalysis:
 """
 Complete field usage analysis for a COBOL program.
 """

 usage_by_record: dict[str, FieldUsage] = field(default_factory=dict)
 diagnostics: list[str] = field(default_factory=list)

```

```

class FieldUsageAnalyzer:
 """
 Analyzes COBOL text for field usage.

```

This analyzer is intentionally conservative:

- It detects qualified references only.
  - It does not rewrite bare words globally.
  - It does not invent DB2 table, column, or host variable names.
  - It preserves the original COBOL business flow.
- """

```
def __init__(
 self,
 mapping_repository: MappingRepository | None = None,
 table_name_resolver: TableNameResolver | None = None,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver

def analyze(
 self,
 cobol_text: str,
) -> FieldUsageAnalysis:
 """
 Analyze COBOL source text and return usage by record/group.
 """

 analysis = FieldUsageAnalysis()

 if not cobol_text:
 analysis.diagnostics.append(
 "Field usage analyzer: COBOL text is empty."
)
 return analysis

 in_exec_sql = False

 for raw_line in str(cobol_text or "").splitlines():
 logical_line = self._logical_line(raw_line)

 if self._should_skip_line(logical_line):
 continue

 if EXEC_SQL_START_PATTERN.search(logical_line):
 in_exec_sql = True

 self._capture_qualified_references(
 line=logical_line,
 analysis=analysis,
)

 self._capture_dclgen_references(
 line=logical_line,
 analysis=analysis,
)

 if not in_exec_sql:
 self._capture_move_references(
 line=logical_line,
 analysis=analysis,
)

 self._capture_condition_references(
 line=logical_line,
 analysis=analysis,
)

 self._capture_output_references(
 line=logical_line,
 analysis=analysis,
)

 if EXEC_SQL_END_PATTERN.search(logical_line):
 in_exec_sql = False

 self._add_summary_diagnostics(analysis)
 return analysis

def _logical_line(
```

```

 self,
 line: str,
) -> str:
 """
 Strip COBOL sequence numbers and normalize the line.
 """

 text = str(line or "")
 try:
 text = strip_sequence_numbers(text)
 except Exception:
 text = text[6:] if len(text) > 6 and text[:6].isdigit() else text

 return text.strip()

def _should_skip_line(
 self,
 line: str,
) -> bool:
 """
 Skip blank and comment lines.
 """

 return bool(COMMENT_OR_BLANK_PATTERN.match(str(line or "")))

def _capture_qualified_references(
 self,
 line: str,
 analysis: FieldUsageAnalysis,
) -> None:
 """
 Capture FIELD OF RECORD and FIELD IN RECORD references.
 """

 for match in QUALIFIED_REFERENCE_PATTERN.finditer(line):
 field_name = self._normalize(match.group("field"))
 record_name = self._normalize(match.group("record"))

 if not field_name or not record_name:
 continue

 usage = self._usage_for_record(
 analysis=analysis,
 record_name=record_name,
)

 usage.all_fields.add(field_name)

 if CONDITION_STATEMENT_PATTERN.match(line):
 usage.condition_fields.add(field_name)

 if OUTPUT_STATEMENT_PATTERN.match(line):
 usage.output_fields.add(field_name)

def _capture_dclgen_references(
 self,
 line: str,
 analysis: FieldUsageAnalysis,
) -> None:
 """
 Capture DCLGEN host references:
 - FIELD OF DCLGROUP
 - :DCLGROUP.FIELD
 """

 for match in DCLGEN_OF_PATTERN.finditer(line):
 field_name = self._normalize(match.group("field"))
 group_name = self._normalize(match.group("group"))

 if not field_name or not group_name:
 continue

 usage = self._usage_for_record(
 analysis=analysis,
 record_name=group_name,
)

```

```

 usage.dclgen_host_fields.add(field_name)
 usage.all_fields.add(field_name)

 for match in DCLGEN_DOT_PATTERN.finditer(line):
 field_name = self._normalize(match.group("field"))
 group_name = self._normalize(match.group("group"))

 if not field_name or not group_name:
 continue

 usage = self._usage_for_record(
 analysis=analysis,
 record_name=group_name,
)

 usage.dclgen_host_fields.add(field_name)
 usage.all_fields.add(field_name)

def _capture_move_references(
 self,
 line: str,
 analysis: FieldUsageAnalysis,
) -> None:
 """
 Capture qualified references from MOVE source and target sides.
 """

 match = MOVE_STATEMENT_PATTERN.match(line)

 if not match:
 return

 source_text = match.group("source") or ""
 target_text = match.group("target") or ""

 self._capture_move_side(
 text=source_text,
 analysis=analysis,
 is_source=True,
)

 self._capture_move_side(
 text=target_text,
 analysis=analysis,
 is_source=False,
)

def _capture_move_side(
 self,
 text: str,
 analysis: FieldUsageAnalysis,
 is_source: bool,
) -> None:
 """
 Capture qualified references from one side of a MOVE statement.
 """

 for match in QUALIFIED_REFERENCE_PATTERN.finditer(text):
 field_name = self._normalize(match.group("field"))
 record_name = self._normalize(match.group("record"))

 if not field_name or not record_name:
 continue

 usage = self._usage_for_record(
 analysis=analysis,
 record_name=record_name,
)

 if is_source:
 usage.move_source_fields.add(field_name)
 else:
 usage.move_target_fields.add(field_name)

 usage.all_fields.add(field_name)

```

```

def _capture_condition_references(
 self,
 line: str,
 analysis: FieldUsageAnalysis,
) -> None:
 """
 Capture qualified references from condition statements.
 """

 if not CONDITION_STATEMENT_PATTERN.match(line):
 return

 for match in QUALIFIED_REFERENCE_PATTERN.finditer(line):
 field_name = self._normalize(match.group("field"))
 record_name = self._normalize(match.group("record"))

 if not field_name or not record_name:
 continue

 usage = self._usage_for_record(
 analysis=analysis,
 record_name=record_name,
)

 usage.condition_fields.add(field_name)
 usage.all_fields.add(field_name)

def _capture_output_references(
 self,
 line: str,
 analysis: FieldUsageAnalysis,
) -> None:
 """
 Capture qualified references from output-style statements.
 """

 if not OUTPUT_STATEMENT_PATTERN.match(line):
 return

 for match in QUALIFIED_REFERENCE_PATTERN.finditer(line):
 field_name = self._normalize(match.group("field"))
 record_name = self._normalize(match.group("record"))

 if not field_name or not record_name:
 continue

 usage = self._usage_for_record(
 analysis=analysis,
 record_name=record_name,
)

 usage.output_fields.add(field_name)
 usage.all_fields.add(field_name)

def _usage_for_record(
 self,
 analysis: FieldUsageAnalysis,
 record_name: str,
) -> FieldUsage:
 """
 Return existing usage for a record/group or create a new entry.
 """

 normalized_record_name = self._normalize(record_name)

 if normalized_record_name not in analysis.usage_by_record:
 analysis.usage_by_record[normalized_record_name] = FieldUsage(
 record_name=normalized_record_name,
)

 return analysis.usage_by_record[normalized_record_name]

def _normalize(
 self,
 value: str,

```

```

) -> str:
 """
 Normalize names using project NameNormalizer when available.
 """

 text = str(value or "").strip()

 if not text:
 return ""

 try:
 return NameNormalizer.normalize(text)
 except Exception:
 return text.upper().replace("_", "-")

def _add_summary_diagnostics(
 self,
 analysis: FieldUsageAnalysis,
) -> None:
 """
 Add concise diagnostics for observability.
 """

 record_count = len(analysis.usage_by_record)

 analysis.diagnostics.append(
 f"Field usage analyzer: records/groups detected: {record_count}."
)

 for record_name in sorted(analysis.usage_by_record):
 usage = analysis.usage_by_record[record_name]
 field_count = len(usage.all_fields)

 analysis.diagnostics.append(
 "Field usage analyzer: "
 f"{record_name} fields detected: {field_count}."
)

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\analyzers\field_usage_analyzer.py.broken_before_pdf_restore_1
ENCODING: utf-8
=====

```

```

=====
import re
from dataclasses import dataclass, field
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.sequence_patterns import strip_sequence_numbers
@dataclass
class FieldUsage:
 record_name: str
 condition_fields: set[str] = field(default_factory=set)
 output_fields: set[str] = field(default_factory=set)
 move_source_fields: set[str] = field(default_factory=set)
 move_target_fields: set[str] = field(default_factory=set)
 dclgen_host_fields: set[str] = field(default_factory=set)
 all_fields: set[str] = field(default_factory=set)
@dataclass
class FieldUsageAnalysis:
 usage_by_record: dict[str, FieldUsage] = field(default_factory=dict)
 diagnostics: list[str] = field(default_factory=list)
class FieldUsageAnalyzer:
 """
 Generic COBOL field-usage analyzer.
 Detects:
 - IDMS qualified references: FIELD OF RECORD / FIELD IN RECORD
 - DCLGEN qualified references: FIELD OF DCLGROUP
 - DB2 host references: :DCLGROUP.FIELD / :FIELD OF DCLGROUP
 - MOVE source fields used for output
 - condition fields in IF / WHEN / UNTIL / EVALUATE
 This analyzer does not rewrite COBOL.
 """
 QUALIFIED_REFERENCE_PATTERN = re.compile(

```

```

r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+(?:OF|IN)\s+(?:P<record>[A-Z][A-Z0-9-]*)\b",
flags=re.IGNORECASE,
)
DCLGEN_OF_PATTERN = re.compile(
r":?\s*(?:P<field>[A-Z][A-Z0-9-]*)\s+OF\s+(?:P<group>DCL[A-Z0-9-]+)",
flags=re.IGNORECASE,
)
DCLGEN_DOT_PATTERN = re.compile(
r":\s*(?:P<group>DCL[A-Z0-9-]+)\.?(?:P<field>[A-Z][A-Z0-9-]*)",
flags=re.IGNORECASE,
)
Page 45
MOVE_PATTERN = re.compile(
r"\bMOVE\s+(?:P<source>.+?)\s+TO\s+(?:P<target>.+?)(?:\.\|$\)",
flags=re.IGNORECASE,
)
CONDITION_PATTERN = re.compile(
r"^\s*(IF|WHEN|UNTIL|EVALUATE)\b",
flags=re.IGNORECASE,
)
DIVISION_PATTERN = re.compile(
r"^\s*(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\s+DIVISION\b",
flags=re.IGNORECASE,
)
COMMENT_PATTERN = re.compile(
r"^\s*[\s*/]",
flags=re.IGNORECASE,
)
def __init__(
self,
mapping_repository: MappingRepository,
table_name_resolver: TableNameResolver,
) -> None:
self.mapping_repository = mapping_repository
self.table_name_resolver = table_name_resolver
self.group_to_record = self._build_group_to_record_map()
def analyze(
self,
cobol_text: str,
) -> FieldUsageAnalysis:
result = FieldUsageAnalysis()
inside_procedure = False
inside_exec_sql = False
for line_number, raw_line in enumerate(
str(cobol_text or "").splitlines(),
start=1,
):
logical = strip_sequence_numbers(raw_line).strip()
if not logical:
continue
if self.COMMENT_PATTERN.match(logical):
continue
division_match = self.DIVISION_PATTERN.match(logical)
if division_match:
inside_procedure = division_match.group(1).upper() == "PROCEDURE"
continue
if logical.upper().startswith("EXEC SQL"):
inside_exec_sql = True
if inside_exec_sql:
self._capture_dclgen_references(
logical=logical,
result=result,
is_condition=False,
is_output=False,
)
if logical.upper().startswith("END-EXEC"):
inside_exec_sql = False
continue
if not inside_procedure:
continue
is_condition = bool(self.CONDITION_PATTERN.match(logical))
Page 46
self._capture_idms_qualified_references(
logical=logical,
result=result,
is_condition=is_condition,

```



```

)
self._capture_dclgen_references(
 logical=logical,
 result=result,
 is_condition=is_condition,
 is_output=False,
)
self._capture_move_usage(
 logical=logical,
 result=result,
)
for usage in result.usage_by_record.values():
 usage.all_fields.update(usage.condition_fields)
 usage.all_fields.update(usage.output_fields)
 usage.all_fields.update(usage.move_source_fields)
 usage.all_fields.update(usage.move_target_fields)
 usage.all_fields.update(usage.dclgen_host_fields)
 result.diagnostics.append(
 f"Field usage analyzer: records with usage detected: {len(result.usage_by_record)}"
)
for record_name, usage in sorted(result.usage_by_record.items()):
 result.diagnostics.append(
 "Field usage analyzer: "
 f"record={record_name}, "
 f"condition={len(usage.condition_fields)}, "
 f"output={len(usage.output_fields)}, "
 f"move_source={len(usage.move_source_fields)}, "
 f"move_target={len(usage.move_target_fields)}, "
 f"dclgen={len(usage.dclgen_host_fields)}"
)
return result

def _capture_idms_qualified_references(
 self,
 logical: str,
 result: FieldUsageAnalysis,
 is_condition: bool,
) -> None:
 for match in self.QUALIFIED_REFERENCE_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 record_name = NameNormalizer.normalize(match.group("record"))
 if not field_name or not record_name:
 continue
 if record_name.startswith("DCL"):
 continue
 if record_name not in self.mapping_repository.records():
 continue
 usage = self._usage_for_record(result, record_name)
 usage.all_fields.add(field_name)
 if is_condition:
 usage.condition_fields.add(field_name)
 def _capture_dclgen_references(
 self,
 logical: str,
 result: FieldUsageAnalysis,
 is_condition: bool,
 is_output: bool,
) -> None:
 for match in self.DCLGEN_OF_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 group_name = NameNormalizer.normalize(match.group("group"))
 record_name = self.group_to_record.get(group_name, "")
 if not field_name or not record_name:
 continue
 usage = self._usage_for_record(result, record_name)
 usage.dclgen_host_fields.add(field_name)
 if is_condition:
 usage.condition_fields.add(field_name)
 if is_output:
 usage.output_fields.add(field_name)
 for match in self.DCLGEN_DOT_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 group_name = NameNormalizer.normalize(match.group("group"))
 record_name = self.group_to_record.get(group_name, "")
 if not field_name or not record_name:
 continue

```

```

usage = self._usage_for_record(result, record_name)
usage.dclgen_host_fields.add(field_name)
if is_condition:
usage.condition_fields.add(field_name)
if is_output:
usage.output_fields.add(field_name)
def _capture_move_usage(
self,
logical: str,
result: FieldUsageAnalysis,
) -> None:
match = self.MOVE_PATTERN.search(logical)
if not match:
return
source_text = match.group("source")
target_text = match.group("target")
is_output = "UIT-" in target_text.upper()
for source_match in self.QUALIFIED_REFERENCE_PATTERN.finditer(source_text):
field_name = NameNormalizer.to_cobol(source_match.group("field"))
record_name = NameNormalizer.normalize(source_match.group("record"))
if not field_name or not record_name:
continue
if record_name.startswith("DCL"):
continue
if record_name not in self.mapping_repository.records():
continue
usage = self._usage_for_record(result, record_name)
usage.move_source_fields.add(field_name)
if is_output:
usage.output_fields.add(field_name)
self._capture_dclgen_references(
logical=source_text,
result=result,
is_condition=False,
is_output=is_output,
)
for target_match in self.QUALIFIED_REFERENCE_PATTERN.finditer(target_text):
field_name = NameNormalizer.to_cobol(target_match.group("field"))
record_name = NameNormalizer.normalize(target_match.group("record"))
Page 48
if not field_name or not record_name:
continue
if record_name.startswith("DCL"):
continue
if record_name not in self.mapping_repository.records():
continue
usage = self._usage_for_record(result, record_name)
usage.move_target_fields.add(field_name)
def _usage_for_record(
self,
result: FieldUsageAnalysis,
record_name: str,
) -> FieldUsage:
record = NameNormalizer.normalize(record_name)
if record not in result.usage_by_record:
result.usage_by_record[record] = FieldUsage(record_name=record)
return result.usage_by_record[record]
def _build_group_to_record_map(
self,
) -> dict[str, str]:
output: dict[str, str] = {}
for record in self.mapping_repository.records():
normalized_record = NameNormalizer.normalize(record)
table = self.table_name_resolver.table_for_record(normalized_record)
if not table:
continue
group = "DCL" + NameNormalizer.to_cobol(table)
output[NameNormalizer.normalize(group)] = normalized_record
return output
=====

```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\analyzers\field_usage_analyzer.py.broken_before_pdf_restore_2
ENCODING: utf-8
=====
```

```
import re
from dataclasses import dataclass, field
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.sequence_patterns import strip_sequence_numbers
@dataclass
class FieldUsage:
 record_name: str
 condition_fields: set[str] = field(default_factory=set)
 output_fields: set[str] = field(default_factory=set)
 move_source_fields: set[str] = field(default_factory=set)
 move_target_fields: set[str] = field(default_factory=set)
 dclgen_host_fields: set[str] = field(default_factory=set)
 all_fields: set[str] = field(default_factory=set)
@dataclass
class FieldUsageAnalysis:
 usage_by_record: dict[str, FieldUsage] = field(default_factory=dict)
 diagnostics: list[str] = field(default_factory=list)
class FieldUsageAnalyzer:
 """
 Generic COBOL field-usage analyzer.
 Detects:
 - IDMS qualified references: FIELD OF RECORD / FIELD IN RECORD
 - DCLGEN qualified references: FIELD OF DCLGROUP
 - DB2 host references: :DCLGROUP.FIELD / :FIELD OF DCLGROUP
 - MOVE source fields used for output
 - condition fields in IF / WHEN / UNTIL / EVALUATE
 This analyzer does not rewrite COBOL.
 """
 QUALIFIED_REFERENCE_PATTERN = re.compile(
 r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+(?:OF|IN)\s+(?P<record>[A-Z][A-Z0-9-]*)\b",
 flags=re.IGNORECASE,
)
 DCLGEN_OF_PATTERN = re.compile(
 r":\s*(?P<field>[A-Z][A-Z0-9-]*)\s+OF\s+(?P<group>DCL[A-Z0-9-]+)",
 flags=re.IGNORECASE,
)
 DCLGEN_DOT_PATTERN = re.compile(
 r":\s*(?P<group>DCL[A-Z0-9-]+)\.(?P<field>[A-Z][A-Z0-9-]*)",
 flags=re.IGNORECASE,
)
 Page 45
 MOVE_PATTERN = re.compile(
 r"\bMOVE\s+(?P<source>.+?)\s+TO\s+(?P<target>.+?)(?:\.|$)",
 flags=re.IGNORECASE,
)
 CONDITION_PATTERN = re.compile(
 r"^\s*(IF|WHEN|UNTIL|EVALUATE)\b",
 flags=re.IGNORECASE,
)
 DIVISION_PATTERN = re.compile(
 r"^\s*(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\s+DIVISION\b",
 flags=re.IGNORECASE,
)
 COMMENT_PATTERN = re.compile(
 r"^\s*[/*]",
 flags=re.IGNORECASE,
)
 def __init__(
 self,
 mapping_repository: MappingRepository,
 table_name_resolver: TableNameResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver
 self.group_to_record = self._build_group_to_record_map()
 def analyze(
 self,
 cobol_text: str,
) -> FieldUsageAnalysis:
```

```

result = FieldUsageAnalysis()
inside_procedure = False
inside_exec_sql = False
for line_number, raw_line in enumerate(
 str(cobol_text or "").splitlines(),
 start=1,
):
 logical = strip_sequence_numbers(raw_line).strip()
 if not logical:
 continue
 if self.COMMENT_PATTERN.match(logical):
 continue
 division_match = self.DIVISION_PATTERN.match(logical)
 if division_match:
 inside_procedure = division_match.group(1).upper() == "PROCEDURE"
 continue
 if logical.upper().startswith("EXEC SQL"):
 inside_exec_sql = True
 if inside_exec_sql:
 self._capture_dclgen_references(
 logical=logical,
 result=result,
 is_condition=False,
 is_output=False,
)
 if logical.upper().startswith("END-EXEC"):
 inside_exec_sql = False
 continue
 if not inside_procedure:
 continue
 is_condition = bool(self.CONDITION_PATTERN.match(logical))
 self._capture_idms_qualified_references(
 logical=logical,
 result=result,
 is_condition=is_condition,
)
 self._capture_dclgen_references(
 logical=logical,
 result=result,
 is_condition=is_condition,
 is_output=False,
)
 self._capture_move_usage(
 logical=logical,
 result=result,
)
 for usage in result.usage_by_record.values():
 usage.all_fields.update(usage.condition_fields)
 usage.all_fields.update(usage.output_fields)
 usage.all_fields.update(usage.move_source_fields)
 usage.all_fields.update(usage.move_target_fields)
 usage.all_fields.update(usage.dclgen_host_fields)
 result.diagnostics.append(
 f"Field usage analyzer: records with usage detected: {len(result.usage_by_record)}"
)
 for record_name, usage in sorted(result.usage_by_record.items()):
 result.diagnostics.append(
 "Field usage analyzer: "
 f"record={record_name}, "
 f"condition={len(usage.condition_fields)}, "
 f"output={len(usage.output_fields)}, "
 f"move_source={len(usage.move_source_fields)}, "
 f"move_target={len(usage.move_target_fields)}, "
 f"dclgen={len(usage.dclgen_host_fields)}"
)
 return result
def _capture_idms_qualified_references(
 self,
 logical: str,
 result: FieldUsageAnalysis,
 is_condition: bool,
) -> None:
 for match in self.QUALIFIED_REFERENCE_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 record_name = NameNormalizer.normalize(match.group("record"))

```

```

if not field_name or not record_name:
 continue
if record_name.startswith("DCL"):
 continue
if record_name not in self.mapping_repository.records():
 continue
usage = self._usage_for_record(result, record_name)
usage.all_fields.add(field_name)
if is_condition:
 usage.condition_fields.add(field_name)
def _capture_dclgen_references(
 self,
 logical: str,
 result: FieldUsageAnalysis,
 is_condition: bool,
 is_output: bool,
) -> None:
 for match in self.DCLGEN_OF_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 group_name = NameNormalizer.normalize(match.group("group"))
 record_name = self.group_to_record.get(group_name, "")
 if not field_name or not record_name:
 continue
 usage = self._usage_for_record(result, record_name)
 usage.dclgen_host_fields.add(field_name)
 if is_condition:
 usage.condition_fields.add(field_name)
 if is_output:
 usage.output_fields.add(field_name)
 for match in self.DCLGEN_DOT_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 group_name = NameNormalizer.normalize(match.group("group"))
 record_name = self.group_to_record.get(group_name, "")
 if not field_name or not record_name:
 continue
 usage = self._usage_for_record(result, record_name)
 usage.dclgen_host_fields.add(field_name)
 if is_condition:
 usage.condition_fields.add(field_name)
 if is_output:
 usage.output_fields.add(field_name)
def _capture_move_usage(
 self,
 logical: str,
 result: FieldUsageAnalysis,
) -> None:
 match = self.MOVE_PATTERN.search(logical)
 if not match:
 return
 source_text = match.group("source")
 target_text = match.group("target")
 is_output = "UIT-" in target_text.upper()
 for source_match in self.QUALIFIED_REFERENCE_PATTERN.finditer(source_text):
 field_name = NameNormalizer.to_cobol(source_match.group("field"))
 record_name = NameNormalizer.normalize(source_match.group("record"))
 if not field_name or not record_name:
 continue
 if record_name.startswith("DCL"):
 continue
 if record_name not in self.mapping_repository.records():
 continue
 usage = self._usage_for_record(result, record_name)
 usage.move_source_fields.add(field_name)
 if is_output:
 usage.output_fields.add(field_name)
 self._capture_dclgen_references(
 logical=source_text,
 result=result,
 is_condition=False,
 is_output=is_output,
)
 for target_match in self.QUALIFIED_REFERENCE_PATTERN.finditer(target_text):
 field_name = NameNormalizer.to_cobol(target_match.group("field"))
 record_name = NameNormalizer.normalize(target_match.group("record"))

```

Page 48

```

if not field_name or not record_name:
 continue
if record_name.startswith("DCL"):
 continue
if record_name not in self.mapping_repository.records():
 continue
usage = self._usage_for_record(result, record_name)
usage.move_target_fields.add(field_name)
def _usage_for_record(
 self,
 result: FieldUsageAnalysis,
 record_name: str,
) -> FieldUsage:
 record = NameNormalizer.normalize(record_name)
 if record not in result.usage_by_record:
 result.usage_by_record[record] = FieldUsage(record_name=record)
 return result.usage_by_record[record]
def _build_group_to_record_map(
 self,
) -> dict[str, str]:
 output: dict[str, str] = {}
 for record in self.mapping_repository.records():
 normalized_record = NameNormalizer.normalize(record)
 table = self.table_name_resolver.table_for_record(normalized_record)
 if not table:
 continue
 group = "DCL" + NameNormalizer.to_cobol(table)
 output[NameNormalizer.normalize(group)] = normalized_record
 return output

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\analyzers\metadata_service.py
ENCODING: utf-8
=====

```

```

from collections import defaultdict

from idms_db2_phase2.domain.models import (
 RelationshipSummary,
 RecordSummary,
 SheetMappingRow,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer

class MetadataService:
 """
 Builds metadata summaries from Sheet Mapping rows.

 Used by UI and diagnostics to show:
 - Record summaries
 - Column rows
 - Relationship summaries
 - Sheet Mapping preview rows
 """

 def record_summaries(
 self,
 rows: list[SheetMappingRow],
) -> list[RecordSummary]:
 grouped: dict[str, list[SheetMappingRow]] = defaultdict(list)

 for row in rows:
 record = NameNormalizer.normalize(row.cobol_record_idms)

 if not record:
 continue

 grouped[record].append(row)

 output: list[RecordSummary] = []

 for record_name, record_rows in sorted(grouped.items()):
 db2_table = self._first_db2_table(record_rows)

```

```

 columns = [
 NameNormalizer.normalize(row.new_db2_field_name)
 for row in record_rows
 if NameNormalizer.normalize(row.new_db2_field_name)
]

 key_columns = [
 NameNormalizer.normalize(row.new_db2_field_name)
 for row in record_rows
 if NameNormalizer.normalize(row.new_db2_field_name)
 and (
 NameNormalizer.normalize(row.idms_key)
 or NameNormalizer.normalize(row.db2_key)
)
]

 output.append(
 RecordSummary(
 record_name=record_name,
 db2_table=db2_table,
 column_count=len(set(columns)),
 key_columns=sorted(set(key_columns)),
)
)

 return output

def column_rows(
 self,
 rows: list[SheetMappingRow],
) -> list[dict[str, str]]:
 output: list[dict[str, str]] = []

 for row in rows:
 record = NameNormalizer.normalize(row.cobol_record_idms)
 table = NameNormalizer.normalize(row.new_db2_record)
 column = NameNormalizer.normalize(row.new_db2_field_name)

 if not record and not table and not column:
 continue

 output.append(
 {
 "IDMS Record": record,
 "COBOL Zone": row.cobol_zone,
 "IDMS Key": row.idms_key,
 "IDMS PIC": row.idms_pic_clause,
 "DB2 Key": row.db2_key,
 "DB2 Table": table,
 "DB2 Column": column,
 "DB2 Type": row.new_db2_data_type,
 "Hopex Expression Type": row.hopex_expression_type_remark,
 "Remarks": row.remarks,
 "Relation": row.relation,
 "Reference Field": row.reference_field_name_copybook,
 "Reference PIC": row.reference_field_pic_clause,
 "Cross App DB2 Table": row.cross_application_db2_table,
 "Cross App DB2 Field": row.cross_application_db2_field_name,
 "Cross App DB2 Type": row.cross_application_db2_data_type,
 "Basetype": row.basetype,
 }
)

 return output

def relationship_summaries(
 self,
 rows: list[SheetMappingRow],
) -> list[RelationshipSummary]:
 output: list[RelationshipSummary] = []
 seen: set[tuple[str, str, str, str]] = set()

 for row in rows:
 relation = NameNormalizer.normalize(row.relation)

 if not relation:

```

```

 continue

 child_record = NameNormalizer.normalize(row.cobol_record_idms)
 child_key = NameNormalizer.normalize(row.new_db2_field_name)
 parent_key = NameNormalizer.normalize(
 row.cross_application_db2_field_name
 or row.new_db2_field_name
)
 parent_record = self._infer_parent_record_from_reference(row)

 key = (
 relation,
 parent_record,
 child_record,
 child_key,
)

 if key in seen:
 continue

 seen.add(key)

 output.append(
 RelationshipSummary(
 relation=relation,
 parent_record=parent_record,
 child_record=child_record,
 parent_key=parent_key,
 child_key=child_key,
)
)

 return output

def mapping_preview_rows(
 self,
 rows: list[SheetMappingRow],
) -> list[dict[str, str]]:
 output: list[dict[str, str]] = []

 for row in rows:
 output.append(
 {
 "Cobol Record IDMS": row.cobol_record_idms,
 "Cobol Zone": row.cobol_zone,
 "IDMS Key": row.idms_key,
 "IDMS PIC Clause": row.idms_pic_clause,
 "Length of Field Bytes": row.length_of_field_bytes,
 "Field end position": row.field_end_position,
 "DB2 Key": row.db2_key,
 "New DB2 Record": row.new_db2_record,
 "New DB2 Field name": row.new_db2_field_name,
 "New DB2 Data Type": row.new_db2_data_type,
 "Hopex Expression Type": row.hopex_expression_type_remark,
 "Remarks": row.remarks,
 "Relation": row.relation,
 "Reference Field Name": row.reference_field_name_copybook,
 "Reference Field PIC Clause": row.reference_field_pic_clause,
 "Cross Application DB2 Table": row.cross_application_db2_table,
 "Cross Application DB2 Field Name": row.cross_application_db2_field_name,
 "Cross Appln DB2 Data Type": row.cross_application_db2_data_type,
 "Basetype": row.basetype,
 }
)

 return output

def _first_db2_table(
 self,
 rows: list[SheetMappingRow],
) -> str:
 for row in rows:
 table = NameNormalizer.normalize(row.new_db2_record)

 if table:
 return table

```



```

 return ""

def _infer_parent_record_from_reference(
 self,
 row: SheetMappingRow,
) -> str:
 if row.cross_application_db2_table:
 return NameNormalizer.normalize(row.cross_application_db2_table)

 value = row.cross_application_db2_field_name

 if not value:
 return ""

 normalized = NameNormalizer.normalize(value)
 parts = normalized.split("_")

 if len(parts) >= 2:
 return parts[0]

 return ""

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\analyzers\program_flow_analyzer.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass, field

```

```

from idms_db2_phase2.domain.models import DclgenColumn, IdmsOperation, SheetMappingRow
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.cobol_patterns import PARAGRAPH_PATTERN, PROCEDURE_DIVISION_PATTERN
from patterns.sequence_patterns import strip_sequence_numbers

```

```

@dataclass
class ParagraphSpan:
 name: str
 start_line: int
 end_line: int
 lines: list[str] = field(default_factory=list)

```

```

@dataclass
class CursorLoop:
 record_name: str = ""
 set_name: str = ""
 operation: str = ""
 operation_line: int = 0
 process_paragraph: str = ""
 perform_line: int = 0
 until_line: int = 0
 loop_type: str = "unknown"
 cursor_name: str = ""
 open_paragraph: str = ""
 fetch_paragraph: str = ""
 close_paragraph: str = ""

```

```

@dataclass
class OutputWrite:
 output_record: str = ""
 paragraph_name: str = ""
 write_line: int = 0
 move_lines: list[str] = field(default_factory=list)

```

```

@dataclass
class DateUsage:
 host_field: str = ""
 dclgen_group: str = ""
 idms_record: str = ""
 line_number: int = 0

```

```

line_text: str = ""
usage_type: str = ""

```

```
@dataclass
```

```

class ProgramFlowAnalysis:
 paragraphs: list[ParagraphSpan] = field(default_factory=list)
 cursor_loops: list[CursorLoop] = field(default_factory=list)
 output_writes: list[OutputWrite] = field(default_factory=list)
 date_usages: list[DateUsage] = field(default_factory=list)
 diagnostics: list[str] = field(default_factory=list)

 def process_paragraph_for_cursor(
 self,
 cursor_name: str,
) -> str:
 cursor = NameNormalizer.to_cobol(NameNormalizer.normalize(cursor_name))

 for loop in self.cursor_loops:
 if NameNormalizer.to_cobol(NameNormalizer.normalize(loop.cursor_name)) == cursor:
 return loop.process_paragraph

 return ""

 def loop_for_set(
 self,
 set_name: str,
) -> CursorLoop | None:
 target = NameNormalizer.normalize(set_name)

 for loop in self.cursor_loops:
 if NameNormalizer.normalize(loop.set_name) == target:
 return loop

 return None

 def loop_for_record(
 self,
 record_name: str,
) -> CursorLoop | None:
 target = NameNormalizer.normalize(record_name)

 for loop in self.cursor_loops:
 if NameNormalizer.normalize(loop.record_name) == target:
 return loop

 return None

```

```

class ProgramFlowAnalyzer:
 """
 Generic flow analyzer for diagnostics.

 This analyzer does not rewrite COBOL. It detects:
 - procedure paragraphs
 - cursor-like operations from parsed IDMS operations
 - output write statements
 - basic date usage hints
 """

 CURSOR_OPERATIONS = {
 "OBTAIN_FIRST",
 "OBTAIN_NEXT",
 "FIND_FIRST",
 }

 WRITE_TOKENS = {
 "WRITE",
 "STORE",
 "MODIFY",
 "ERASE",
 }

 DATE_TOKENS = {
 "DATE",
 "TIME",
 }

```

```

 "TIMESTAMP",
 "TS-",
 }

def __init__(
 self,
 mapping_rows: list[SheetMappingRow] | None = None,
 dclgen_columns: list[DclgenColumn] | None = None,
) -> None:
 self.mapping_rows = mapping_rows or []
 self.dclgen_columns = dclgen_columns or []
 self.cursor_name_resolver = CursorNameResolver()

def analyze(
 self,
 cobol_text: str,
 operations: list[IdmsOperation] | None = None,
) -> ProgramFlowAnalysis:
 diagnostics: list[str] = []

 if not str(cobol_text or "").strip():
 return ProgramFlowAnalysis(
 diagnostics=["Program flow analyzer: COBOL text is empty."]
)

 logical_lines = self._logical_lines_with_numbers(cobol_text)
 paragraphs = self._paragraph_spans(logical_lines)
 cursor_loops = self._cursor_loops(operations or [])
 output_writes = self._output_writes(logical_lines, paragraphs)
 date_usages = self._date_usages(logical_lines)

 diagnostics.append(
 f"Program flow analyzer: Sheet Mapping rows received: {len(self.mapping_rows)}"
)
 diagnostics.append(
 f"Program flow analyzer: DCLGEN columns received: {len(self.dclgen_columns)}"
)
 diagnostics.append(
 f"Program flow analyzer: paragraphs detected: {len(paragraphs)}"
)
 diagnostics.append(
 f"Program flow analyzer: cursor loops detected: {len(cursor_loops)}"
)
 diagnostics.append(
 f"Program flow analyzer: output writes detected: {len(output_writes)}"
)
 diagnostics.append(
 f"Program flow analyzer: DB2 date usages detected: {len(date_usages)}"
)

 for loop in cursor_loops[:5]:
 diagnostics.append(
 "Program flow analyzer: loop sample: "
 f"record={loop.record_name}, set={loop.set_name}, "
 f"type={loop.loop_type}, cursor={loop.cursor_name}"
)

 return ProgramFlowAnalysis(
 paragraphs=paragraphs,
 cursor_loops=cursor_loops,
 output_writes=output_writes,
 date_usages=date_usages,
 diagnostics=diagnostics,
)

def _logical_lines_with_numbers(
 self,
 cobol_text: str,
) -> list[tuple[int, str, str]]:
 output: list[tuple[int, str, str]] = []

 for line_number, raw_line in enumerate(cobol_text.splitlines(), start=1):
 logical = strip_sequence_numbers(raw_line)
 output.append((line_number, logical, raw_line.rstrip()))

 return output

```

```

def _paragraph_spans(
 self,
 logical_lines: list[tuple[int, str, str]],
) -> list[ParagraphSpan]:
 paragraphs: list[ParagraphSpan] = []
 current: ParagraphSpan | None = None
 inside_procedure = False

 for line_number, logical, raw_line in logical_lines:
 if PROCEDURE_DIVISION_PATTERN.match(logical):
 inside_procedure = True

 if not inside_procedure:
 continue

 match = PARAGRAPH_PATTERN.match(logical.strip())

 if match:
 if current is not None:
 current.end_line = line_number - 1
 paragraphs.append(current)

 current = ParagraphSpan(
 name=match.group("name").upper(),
 start_line=line_number,
 end_line=line_number,
 lines=[raw_line],
)
 continue

 if current is not None:
 current.lines.append(raw_line)
 current.end_line = line_number

 if current is not None:
 paragraphs.append(current)

 return paragraphs

def _cursor_loops(
 self,
 operations: list[IdmsOperation],
) -> list[CursorLoop]:
 output: list[CursorLoop] = []
 cursor_order_by_name: dict[str, int] = {}

 for operation in operations:
 operation_name = str(operation.operation or "").upper()

 if operation_name not in self.CURSOR_OPERATIONS:
 continue

 record_name = NameNormalizer.normalize(operation.record_name)
 set_name = NameNormalizer.normalize(operation.set_name)

 if not record_name:
 continue

 cursor_name = self.cursor_name_resolver.cursor_name_from_table(record_name)

 if cursor_name not in cursor_order_by_name:
 cursor_order_by_name[cursor_name] = len(cursor_order_by_name) + 1

 paragraph_spec = self.cursor_name_resolver.paragraph_names(
 cursor_order=cursor_order_by_name[cursor_name],
 cursor_name=cursor_name,
)

 output.append(
 CursorLoop(
 record_name=record_name,
 set_name=set_name,
 operation=operation_name,
 operation_line=operation.line_number,
 loop_type="cursor",
)
)

```

```

 cursor_name=cursor_name,
 open_paragraph=paragraph_spec["open"],
 fetch_paragraph=paragraph_spec["fetch"],
 close_paragraph=paragraph_spec["close"],
)
)

return output

def _output_writes(
 self,
 logical_lines: list[tuple[int, str, str]],
 paragraphs: list[ParagraphSpan],
) -> list[OutputWrite]:
 output: list[OutputWrite] = []
 paragraph_by_line = self._paragraph_name_by_line(paragraphs)

 for line_number, logical, _raw_line in logical_lines:
 upper = logical.strip().upper()

 if not any(upper.startswith(token) for token in self.WRITE_TOKENS):
 continue

 output.append(
 OutputWrite(
 output_record=self._first_word_after_operation(upper),
 paragraph_name=paragraph_by_line.get(line_number, ""),
 write_line=line_number,
)
)

 return output

def _date_usages(
 self,
 logical_lines: list[tuple[int, str, str]],
) -> list[DateUsage]:
 output: list[DateUsage] = []

 for line_number, logical, raw_line in logical_lines:
 upper = logical.upper()

 if not any(token in upper for token in self.DATE_TOKENS):
 continue

 output.append(
 DateUsage(
 line_number=line_number,
 line_text=raw_line,
 usage_type="date-token",
)
)

 return output

def _paragraph_name_by_line(
 self,
 paragraphs: list[ParagraphSpan],
) -> dict[int, str]:
 output: dict[int, str] = {}

 for paragraph in paragraphs:
 for line_number in range(paragraph.start_line, paragraph.end_line + 1):
 output[line_number] = paragraph.name

 return output

def _first_word_after_operation(
 self,
 upper_line: str,
) -> str:
 parts = upper_line.split()

 if len(parts) < 2:
 return ""

```

```
return NameNormalizer.normalize(parts[1].rstrip("."))
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\app.py
ENCODING: utf-8
=====
```

```
from pathlib import Path
import sys
```

```
import streamlit as st
```

```
CURRENT_FILE = Path(__file__).resolve()
SRC_DIR = CURRENT_FILE.parents[1]
PROJECT_ROOT = CURRENT_FILE.parents[2]
```

```
for path in [PROJECT_ROOT, SRC_DIR]:
 if str(path) not in sys.path:
 sys.path.insert(0, str(path))
```

```
from idms_db2_phase2.ui.main_page import render_main_page
```

```
def main() -> None:
 st.set_page_config(
 page_title="IDMS > DB2 Phase 2 Converter",
 layout="wide",
)

 st.title("IDMS > DB2 Phase 2 Converter")

 st.caption(
 "Upload Sheet Mapping, DCLGEN, optional Copybook files, "
 "and IDMS COBOL source code to generate DB2 embedded SQL COBOL."
)

 render_main_page()
```

```
if __name__ == "__main__":
 main()
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers__init__.py
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\cobol_formatter.py
ENCODING: utf-8
=====
```

```
class CobolFormatter:
 """
 Preserve-first COBOL formatter.

 This formatter intentionally does NOT re-indent or restructure the full
 COBOL program.

 Reason:
 - The original IDMS COBOL source is the authority for manual style.
 - The converter must preserve existing indentation, comments, sequence-like
 lines, paragraph spacing, and business-flow layout.
 - Only generated DB2 blocks should be inserted or normalized by generators
 and composers.

 This class therefore performs only safe text cleanup:
 - Normalize Windows/Unix line endings.
 - Remove trailing spaces.
 - Limit excessive blank lines.
```

- Ensure one final newline.

It must not:

- Strip sequence numbers.
- Re-indent PROCEDURE DIVISION.
- Reformat original business logic.
- Move paragraphs.
- Add or remove periods.
- Change field references.

"""

```
def format(
 self,
 text: str,
) -> str:
 if not text:
 return ""

 normalized = self._normalize_line_endings(text)
 normalized = self._rstrip_lines(normalized)
 normalized = self._normalize_excess_blank_lines(normalized)

 return normalized.rstrip() + "\n"

def _normalize_line_endings(
 self,
 text: str,
) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

def _rstrip_lines(
 self,
 text: str,
) -> str:
 return "\n".join(
 line.rstrip()
 for line in str(text or "").splitlines()
)

def _normalize_excess_blank_lines(
 self,
 text: str,
) -> str:
 output = str(text or "")

 while "\n\n\n\n" in output:
 output = output.replace("\n\n\n\n", "\n\n\n")

 return output
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\cursor_flow_composer.py
ENCODING: utf-8
=====
```

"""

Cursor flow composer.

Normalizes generated cursor execution flow.

It converts mechanical flow:

```
PERFORM 710-OPEN-CURSOR.
PERFORM 720-FETCH-CURSOR.
PERFORM BUSINESS-PARA
 UNTIL SQLCODE = 100.
```

to:

```
PERFORM 710-OPEN-CURSOR.
PERFORM 720-FETCH-CURSOR UNTIL CURSOR-EOC.
PERFORM 730-CLOSE-CURSOR.
```

It also converts generated FETCH paragraphs:

```
WHEN ZERO
 CONTINUE
```

to:

```
WHEN ZERO
 PERFORM BUSINESS-PARA
```

It also removes leftover generated OBTAIN NEXT cursor calls after a cursor has already been converted to FETCH UNTIL EOC flow.

This class is generic. It does not hardcode program names, record names, table names, cursor names, or business paragraph names.

"""

```
import re
from dataclasses import dataclass

from patterns.sequence_patterns import strip_sequence_numbers
```

```
@dataclass
class CursorFlowPlan:
 cursor_name: str
 open_number: int
 fetch_number: int
 close_number: int
 open_paragraph: str
 fetch_paragraph: str
 close_paragraph: str
 eoc_condition: str
 business_paragraph: str
 open_index: int
 fetch_index: int
 business_index: int
 until_index: int

class CursorFlowComposer:
 LOOKAHEAD_LIMIT = 120

 PERFORM_CURSOR_PATTERN = re.compile(
 r"^\s*PERFORM\s+"
 r"(?P<number>\d{3})-"
 r"(?P<operation>OPEN|FETCH|CLOSE)-"
 r"(?P<cursor>[A-Z0-9-]+)"
 r"\.\s*$",
 flags=re.IGNORECASE,
)

 PERFORM_BUSINESS_PATTERN = re.compile(
 r"^\s*PERFORM\s+"
 r"(?P<paragraph>[A-Z0-9][A-Z0-9-]*)"
 r"\.\s*$",
 flags=re.IGNORECASE,
)

 UNTIL_SQLCODE_100_PATTERN = re.compile(
 r"^\s*UNTIL\s+SQLCODE\s*=\s*100\.\s*$",
 flags=re.IGNORECASE,
)

 CURSOR_PARAGRAPH_HEADER_PATTERN = re.compile(
 r"^\s*"
 r"(?P<number>\d{3})-"
 r"(?P<operation>OPEN|FETCH|CLOSE)-"
 r"(?P<cursor>[A-Z0-9-]+)"
 r"\.\s*$",
 flags=re.IGNORECASE,
)

 ANY_PARAGRAPH_HEADER_PATTERN = re.compile(
 r"^\s*[A-Z0-9][A-Z0-9-]*\.\s*$",
 flags=re.IGNORECASE,
)
```



```

WHEN_ZERO_PATTERN = re.compile(
 r"^\\s*WHEN\\s+ZERO\\s*$",
 flags=re.IGNORECASE,
)

CONTINUE_PATTERN = re.compile(
 r"^\\s*CONTINUE\\.?\\s*$",
 flags=re.IGNORECASE,
)

CONVERTED_OBTAIN_NEXT_COMMENT_PATTERN = re.compile(
 r"^\\s**\\s*DB2:\\s*Converted\\s+OBTAIN\\s+NEXT\\b",
 flags=re.IGNORECASE,
)

CONVERTED_OBTAIN_COMMENT_PATTERN = re.compile(
 r"^\\s**\\s*DB2:\\s*Converted\\s+OBTAIN\\b",
 flags=re.IGNORECASE,
)

NON_PARAGRAPH_DOTTED_LINES = {
 "CONTINUE.",
 "END-EXEC.",
 "END-EVALUATE.",
 "END-IF.",
 "END-PERFORM.",
 "EXIT.",
 "GOBACK.",
 "STOP.",
}

def compose(
 self,
 text: str,
) -> str:
 if not text:
 return ""

 lines = self._normalize_line_endings(text).splitlines()

 plans = self._discover_plans(lines)

 if plans:
 lines = self._rewrite_main_flow(
 lines=lines,
 plans=plans,
)

 lines = self._rewrite_fetch_paragraphs(
 lines=lines,
 plans=plans,
)

 lines = self._remove_leftover_obtain_next_cursor_calls(lines)

 return "\\n".join(lines).rstrip() + "\\n"

def _normalize_line_endings(
 self,
 text: str,
) -> str:
 return str(text or "").replace("\\r\\n", "\\n").replace("\\r", "\\n")

def _logical(
 self,
 line: str,
) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

def _discover_plans(
 self,
 lines: list[str],
) -> list[CursorFlowPlan]:
 plans: list[CursorFlowPlan] = []
 index = 0

```

```

while index < len(lines):
 logical = self._logical(lines[index])
 open_match = self.PERFORM_CURSOR_PATTERN.match(logical)

 if not open_match:
 index += 1
 continue

 if open_match.group("operation").upper() != "OPEN":
 index += 1
 continue

 cursor_name = open_match.group("cursor").upper()
 open_number = int(open_match.group("number"))
 fetch_number = open_number + 10
 close_number = open_number + 20

 fetch_index = self._find_fetch_after_open(
 lines=lines,
 start_index=index + 1,
 cursor_name=cursor_name,
 fetch_number=fetch_number,
)

 if fetch_index < 0:
 index += 1
 continue

 business_info = self._find_business_loop_after_fetch(
 lines=lines,
 start_index=fetch_index + 1,
)

 if business_info is None:
 index += 1
 continue

 business_index, until_index, business_paragraph = business_info

 plans.append(
 CursorFlowPlan(
 cursor_name=cursor_name,
 open_number=open_number,
 fetch_number=fetch_number,
 close_number=close_number,
 open_paragraph=f"{open_number:03d}-OPEN-{cursor_name}",
 fetch_paragraph=f"{fetch_number:03d}-FETCH-{cursor_name}",
 close_paragraph=f"{close_number:03d}-CLOSE-{cursor_name}",
 eoc_condition=f"{cursor_name}-EOC",
 business_paragraph=business_paragraph,
 open_index=index,
 fetch_index=fetch_index,
 business_index=business_index,
 until_index=until_index,
)
)

 index = until_index + 1

return plans

def _find_fetch_after_open(
 self,
 lines: list[str],
 start_index: int,
 cursor_name: str,
 fetch_number: int,
) -> int:
 end_index = min(
 len(lines),
 start_index + self.LOOKAHEAD_LIMIT,
)

 for index in range(start_index, end_index):
 logical = self._logical(lines[index])
 match = self.PERFORM_CURSOR_PATTERN.match(logical)

```

```

 if not match:
 continue

 if match.group("operation").upper() != "FETCH":
 continue

 if match.group("cursor").upper() != cursor_name:
 continue

 if int(match.group("number")) != fetch_number:
 continue

 return index

 return -1

def _find_business_loop_after_fetch(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[int, int, str] | None:
 end_index = min(
 len(lines),
 start_index + self.LOOKAHEAD_LIMIT,
)

 for index in range(start_index, end_index):
 logical = self._logical(lines[index])

 if not logical:
 continue

 if logical.startswith("*") or logical.startswith("/"):
 continue

 if self.PERFORM_CURSOR_PATTERN.match(logical):
 continue

 business_match = self.PERFORM_BUSINESS_PATTERN.match(logical)

 if not business_match:
 continue

 business_paragraph = business_match.group("paragraph").upper()

 until_index = self._find_until_sqlcode_100(
 lines=lines,
 start_index=index + 1,
)

 if until_index < 0:
 continue

 return index, until_index, business_paragraph

 return None

def _find_until_sqlcode_100(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 end_index = min(
 len(lines),
 start_index + 10,
)

 for index in range(start_index, end_index):
 logical = self._logical(lines[index])

 if self.UNTIL_SQLCODE_100_PATTERN.match(logical):
 return index

 return -1

```

```

def _rewrite_main_flow(
 self,
 lines: list[str],
 plans: list[CursorFlowPlan],
) -> list[str]:
 plan_by_open_index = {
 plan.open_index: plan
 for plan in plans
 }

 indexes_to_skip: set[int] = set()

 for plan in plans:
 indexes_to_skip.add(plan.fetch_index)
 indexes_to_skip.add(plan.business_index)
 indexes_to_skip.add(plan.until_index)

 output: list[str] = []

 for index, line in enumerate(lines):
 if index in indexes_to_skip:
 continue

 plan = plan_by_open_index.get(index)

 if plan is None:
 output.append(line)
 continue

 output.append(line)
 output.append(
 self._format_like_line(
 reference_line=line,
 replacement_body=(
 f"PERFORM {plan.fetch_paragraph} "
 f"UNTIL {plan.eoc_condition}."
),
)
)
 output.append(
 self._format_like_line(
 reference_line=line,
 replacement_body=f"PERFORM {plan.close_paragraph}.",
)
)

 return output

def _rewrite_fetch_paragraphs(
 self,
 lines: list[str],
 plans: list[CursorFlowPlan],
) -> list[str]:
 business_by_fetch_paragraph = {
 plan.fetch_paragraph.upper(): plan.business_paragraph
 for plan in plans
 }

 output: list[str] = []
 index = 0

 while index < len(lines):
 logical = self._logical(lines[index])
 header_match = self.CURSOR_PARAGRAPH_HEADER_PATTERN.match(logical)

 if not header_match:
 output.append(lines[index])
 index += 1
 continue

 if header_match.group("operation").upper() != "FETCH":
 output.append(lines[index])
 index += 1
 continue

 fetch_paragraph = (

```

```

 f"{header_match.group('number')}"-
 f"FETCH-"
 f"{header_match.group('cursor')}"
).upper()

 business_paragraph = business_by_fetch_paragraph.get(fetch_paragraph)

 if not business_paragraph:
 output.append(lines[index])
 index += 1
 continue

 paragraph_lines, next_index = self._collect_paragraph(
 lines=lines,
 start_index=index,
)

 rewritten = self._replace_when_zero_continue(
 paragraph_lines=paragraph_lines,
 business_paragraph=business_paragraph,
)

 output.extend(rewritten)
 index = next_index

 return output

def _collect_paragraph(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[list[str], int]:
 output = [lines[start_index]]
 index = start_index + 1

 while index < len(lines):
 logical = self._logical(lines[index])

 if self.CURSOR_PARAGRAPH_HEADER_PATTERN.match(logical):
 break

 if self._is_non_cursor_paragraph_header(logical):
 break

 output.append(lines[index])
 index += 1

 return output, index

def _replace_when_zero_continue(
 self,
 paragraph_lines: list[str],
 business_paragraph: str,
) -> list[str]:
 output: list[str] = []
 index = 0

 while index < len(paragraph_lines):
 line = paragraph_lines[index]
 logical = self._logical(line)

 output.append(line)

 if not self.WHEN_ZERO_PATTERN.match(logical):
 index += 1
 continue

 next_index = self._next_non_blank_index(
 lines=paragraph_lines,
 start_index=index + 1,
)

 if next_index < 0:
 index += 1
 continue

```

```

next_line = paragraph_lines[next_index]
next_logical = self._logical(next_line)

if not self.CONTINUE_PATTERN.match(next_logical):
 index += 1
 continue

output.extend(
 paragraph_lines[index + 1 : next_index]
)
output.append(
 self._format_like_line(
 reference_line=next_line,
 replacement_body=f"PERFORM {business_paragraph}",
)
)

index = next_index + 1

return output

def _remove_leftover_obtain_next_cursor_calls(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 index = 0

 while index < len(lines):
 logical = self._logical(lines[index])

 if not self.CONVERTED_OBTAIN_NEXT_COMMENT_PATTERN.match(logical):
 output.append(lines[index])
 index += 1
 continue

 skipped_lines = [lines[index]]
 index += 1

 while index < len(lines):
 next_logical = self._logical(lines[index])

 if not next_logical:
 skipped_lines.append(lines[index])
 index += 1
 continue

 if next_logical.startswith("*") and not self.CONVERTED_OBTAIN_NEXT_COMMENT_PATTERN.match(next_logical):
 break

 if self.CONVERTED_OBTAIN_NEXT_COMMENT_PATTERN.match(next_logical):
 break

 if self.PERFORM_CURSOR_PATTERN.match(next_logical):
 skipped_lines.append(lines[index])
 index += 1
 continue

 break

 output.append(
 self._replacement_comment_for_removed_obtain_next(
 reference_line=skipped_lines[0],
)
)
 output.append(
 self._format_like_line(
 reference_line=skipped_lines[0],
 replacement_body="CONTINUE.",
)
)

 return output

def _replacement_comment_for_removed_obtain_next(
 self,

```

```

 reference_line: str,
) -> str:
 return self._format_like_line(
 reference_line=reference_line,
 replacement_body="* DB2: Removed redundant OBTAIN NEXT after cursor EOC loop.",
)

def _next_non_blank_index(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 for index in range(start_index, len(lines)):
 logical = self._logical(lines[index])

 if logical:
 return index

 return -1

def _is_non_cursor_paragraph_header(
 self,
 logical: str,
) -> bool:
 normalized = str(logical or "").strip().upper()

 if not normalized:
 return False

 if normalized in self.NON_PARAGRAPH_DOTTED_LINES:
 return False

 if normalized.startswith("WHEN "):
 return False

 if normalized.startswith("END-"):
 return False

 if normalized.startswith("EXEC "):
 return False

 if normalized.startswith("MOVE "):
 return False

 if normalized.startswith("DISPLAY "):
 return False

 if normalized.startswith("PERFORM "):
 return False

 if normalized.startswith("SET "):
 return False

 if normalized.startswith("OPEN "):
 return False

 if normalized.startswith("CLOSE "):
 return False

 if normalized.startswith("FETCH "):
 return False

 if normalized.startswith("INTO"):
 return False

 if normalized.startswith(":"):
 return False

 if not self.ANY_PARAGRAPH_HEADER_PATTERN.match(normalized):
 return False

 if self.CURSOR_PARAGRAPH_HEADER_PATTERN.match(normalized):
 return False

 return True

```

```

def _format_like_line(
 self,
 reference_line: str,
 replacement_body: str,
) -> str:
 line = str(reference_line or "").rstrip()

 if self._is_fixed_format_line(line):
 left = line[:6]
 indicator = line[6]
 right = line[72:80]
 body = self._body_with_reference_indent(
 reference_line=line,
 replacement_body=replacement_body,
)

 return f"{left}{indicator}{body[:65].ljust(65)}{right}"

 indent = self._leading_spaces(
 reference_line,
 default=" ",
)

 return f"{indent}{replacement_body}"

def _body_with_reference_indent(
 self,
 reference_line: str,
 replacement_body: str,
) -> str:
 body = reference_line[7:72].rstrip()
 indent = self._leading_spaces(
 body,
 default=" ",
)

 return f"{indent}{replacement_body}"

def _is_fixed_format_line(
 self,
 line: str,
) -> bool:
 if len(line) < 80:
 return False

 if not line[:6].isdigit():
 return False

 if not line[72:80].isdigit():
 return False

 return True

def _leading_spaces(
 self,
 text: str,
 default: str = "",
) -> str:
 value = str(text or "")

 if not value:
 return default

 count = len(value) - len(value.lstrip(" "))

 if count <= 0:
 return default

 return value[:count]

```



```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\cursor_order_cleanup_composer.py
ENCODING: utf-8
=====
```

```
"""
```

```
Cursor ORDER BY cleanup composer.
```

```
This composer removes unintended ORDER BY clauses from parent/root cursor
declarations.
```

```
Generic rule:
```

- A cursor declaration without a WHERE clause is treated as a parent/root cursor.
- Parent/root cursor ORDER BY is removed unless explicit ordering is already handled upstream.
- A cursor declaration with a WHERE clause is treated as a child/dependent cursor and keeps ORDER BY.

```
This class does not hardcode cursor names, table names, record names, or
business fields.
```

```
"""
```

```
from patterns.sequence_patterns import strip_sequence_numbers
```

```
class CursorOrderCleanupComposer:
 def compose(
 self,
 text: str,
) -> str:
 if not text:
 return ""

 lines = self._normalize_line_endings(text).splitlines()

 output: list[str] = []
 index = 0

 while index < len(lines):
 if not self._is_exec_sql_start(lines[index]):
 output.append(lines[index])
 index += 1
 continue

 block, next_index = self._collect_exec_sql_block(
 lines=lines,
 start_index=index,
)

 if not self._is_cursor_declare_block(block):
 output.extend(block)
 index = next_index
 continue

 output.extend(
 self._cleanup_cursor_declare_block(block)
)
 index = next_index

 return "\n".join(output).rstrip() + "\n"

 def _normalize_line_endings(
 self,
 text: str,
) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

 def _logical(
 self,
 line: str,
) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

 def _logical_upper(
 self,
 line: str,
```

```

) -> str:
 return self._logical(line).upper()

def _is_exec_sql_start(
 self,
 line: str,
) -> bool:
 return self._logical_upper(line) == "EXEC SQL"

def _is_exec_sql_end(
 self,
 line: str,
) -> bool:
 logical = self._logical_upper(line).rstrip(".")

 return logical == "END-EXEC"

def _collect_exec_sql_block(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[list[str], int]:
 block: list[str] = []
 index = start_index

 while index < len(lines):
 block.append(lines[index])

 if self._is_exec_sql_end(lines[index]):
 return block, index + 1

 index += 1

 return block, index

def _is_cursor_declare_block(
 self,
 block: list[str],
) -> bool:
 for line in block:
 logical = self._logical_upper(line)

 if "DECLARE " in logical and "CURSOR " in logical:
 return True

 return False

def _cleanup_cursor_declare_block(
 self,
 block: list[str],
) -> list[str]:
 if self._has_where_clause(block):
 return list(block)

 if not self._has_order_by_clause(block):
 return list(block)

 return self._remove_order_by_block(block)

def _has_where_clause(
 self,
 block: list[str],
) -> bool:
 for line in block:
 logical = self._logical_upper(line)

 if logical == "WHERE":
 return True

 if logical.startswith("WHERE "):
 return True

 return False

def _has_order_by_clause(
 self,

```

```

 block: list[str],
) -> bool:
 for line in block:
 logical = self._logical_upper(line)

 if logical == "ORDER BY":
 return True

 if logical.startswith("ORDER BY "):
 return True

 return False

def _remove_order_by_block(
 self,
 block: list[str],
) -> list[str]:
 output: list[str] = []
 skipping_order_by = False

 for line in block:
 logical = self._logical_upper(line)

 if self._is_order_by_start(logical):
 skipping_order_by = True
 continue

 if skipping_order_by:
 if self._is_order_by_end(logical):
 skipping_order_by = False
 output.append(line)

 continue

 output.append(line)

 return output

def _is_order_by_start(
 self,
 logical: str,
) -> bool:
 if logical == "ORDER BY":
 return True

 if logical.startswith("ORDER BY "):
 return True

 return False

def _is_order_by_end(
 self,
 logical: str,
) -> bool:
 if logical == "FOR READ ONLY":
 return True

 if logical.startswith("FOR READ ONLY"):
 return True

 if logical == "END-EXEC":
 return True

 if logical == "END-EXEC.":
 return True

 return False

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\db2_date_comparison_composer.py
ENCODING: utf-8
=====

```

```

"""
DB2 date comparison composer.

```

This composer realigns DB2 date host fields before comparing them with numeric COBOL date fields such as PARMDATE.

The conversion template is stored in:  
rules/db2\_date\_conversion\_rules.py

This class is generic:

- No program name is hardcoded.
- No table name is hardcoded.
- No DCLGEN group name is hardcoded.
- No business field name is hardcoded.
- DA-/DT- field names are detected from the COBOL being converted.

"""

```
from patterns.db2_date_patterns import (
 DATE_WORKING_STORAGE_MARKER_PATTERN,
 DB2_DATE_COMPARISON_PATTERN,
 LINKAGE_SECTION_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
)
from patterns.sequence_patterns import strip_sequence_numbers
from rules.db2_date_conversion_rules import (
 DB2_DATE_BASE_WORKING_STORAGE_LINES,
 DB2_DATE_COMPARISON_WS_MARKER,
 DB2_DATE_CONVERSION_LINE_TEMPLATES,
 DB2_DATE_HELPER_FIELD_TEMPLATE,
 DB2_DATE_HIGH_NUMERIC_LITERAL,
 DB2_DATE_HIGH_VALUE_LITERAL,
 DB2_DATE_IF_REPLACEMENT_TEMPLATE,
 DB2_DATE_LOW_VALUE_LITERAL,
)

class Db2DateComparisonComposer:
 WS_MARKER = DB2_DATE_COMPARISON_WS_MARKER

 def compose(
 self,
 text: str,
) -> str:
 if not text:
 return ""

 lines = self._normalize_line_endings(text).splitlines()

 date_fields = self._date_fields_used_in_comparisons(lines)

 if not date_fields:
 return text.rstrip() + "\n"

 lines = self._ensure_date_working_storage(
 lines=lines,
 date_fields=date_fields,
)

 lines = self._rewrite_date_comparisons(lines)

 return "\n".join(lines).rstrip() + "\n"

 def _normalize_line_endings(
 self,
 text: str,
) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

 def _logical(
 self,
 line: str,
) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

 def _date_fields_used_in_comparisons(
 self,
 lines: list[str],
) -> list[str]:
```

```

output: list[str] = []
seen: set[str] = set()

for line in lines:
 logical = self._logical(line)

 if logical.startswith("*") or logical.startswith("/"):
 continue

 match = DB2_DATE_COMPARISON_PATTERN.match(logical)

 if not match:
 continue

 field_name = self._clean_cobol_name(match.group("field"))

 if not field_name:
 continue

 if field_name in seen:
 continue

 seen.add(field_name)
 output.append(field_name)

return output

def _ensure_date_working_storage(
 self,
 lines: list[str],
 date_fields: list[str],
) -> list[str]:
 if self._has_date_working_storage(lines):
 return lines

 block = self._date_working_storage_block(date_fields)

 insert_index = self._date_working_storage_insert_index(lines)

 if insert_index < 0:
 return block + [""] + lines

 return lines[:insert_index] + block + [""] + lines[insert_index:]

def _has_date_working_storage(
 self,
 lines: list[str],
) -> bool:
 for line in lines:
 if DATE_WORKING_STORAGE_MARKER_PATTERN.search(line):
 return True

 logical = self._logical(line).upper()

 if logical.startswith("01 WS-DATUMVELDEN"):
 return True

 if logical.startswith("01 WS-DATUMVELDEN"):
 return True

 return False

def _date_working_storage_insert_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 logical = self._logical(line)

 if LINKAGE_SECTION_PATTERN.match(logical):
 return index

 for index, line in enumerate(lines):
 logical = self._logical(line)

 if PROCEDURE_DIVISION_PATTERN.match(logical):

```

```

 return index

 return -1

def _date_working_storage_block(
 self,
 date_fields: list[str],
) -> list[str]:
 lines = list(DB2_DATE_BASE_WORKING_STORAGE_LINES)

 for field_name in date_fields:
 helper = self._helper_name(field_name)

 lines.append(
 DB2_DATE_HELPER_FIELD_TEMPLATE.format(
 helper=helper,
)
)

 return lines

def _rewrite_date_comparisons(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []

 for line in lines:
 logical = self._logical(line)

 if logical.startswith("*") or logical.startswith("/"):
 output.append(line)
 continue

 match = DB2_DATE_COMPARISON_PATTERN.match(logical)

 if not match:
 output.append(line)
 continue

 field_name = self._clean_cobol_name(match.group("field"))
 group_name = self._clean_cobol_name(match.group("group"))
 condition = str(match.group("condition") or "").strip()

 if not field_name or not group_name or not condition:
 output.append(line)
 continue

 indent = self._leading_spaces_from_line(
 line=line,
 default=" ",
)

 output.extend(
 self._date_conversion_lines(
 indent=indent,
 field_name=field_name,
 group_name=group_name,
)
)

 output.append(
 self._replace_date_field_in_if(
 indent=indent,
 field_name=field_name,
 condition=condition,
)
)

 return output

def _date_conversion_lines(
 self,
 indent: str,
 field_name: str,
 group_name: str,

```

```

) -> list[str]:
 helper = self._helper_name(field_name)

 output: list[str] = []

 for template in DB2_DATE_CONVERSION_LINE_TEMPLATES:
 output.append(
 template.format(
 indent=indent,
 field_name=field_name,
 group_name=group_name,
 helper=helper,
 low_value=DB2_DATE_LOW_VALUE_LITERAL,
 high_value=DB2_DATE_HIGH_VALUE_LITERAL,
 high_numeric=DB2_DATE_HIGH_NUMERIC_LITERAL,
)
)

 return output

def _replace_date_field_in_if(
 self,
 indent: str,
 field_name: str,
 condition: str,
) -> str:
 helper = self._helper_name(field_name)

 return DB2_DATE_IF_REPLACEMENT_TEMPLATE.format(
 indent=indent,
 helper=helper,
 condition=condition,
)

def _helper_name(
 self,
 field_name: str,
) -> str:
 return f"HELP-{self._clean_cobol_name(field_name)}"

def _clean_cobol_name(
 self,
 value: str,
) -> str:
 text = str(value or "").strip().upper()
 text = text.replace("_", "-")

 while " " in text:
 text = text.replace(" ", "")

 return text

def _leading_spaces_from_line(
 self,
 line: str,
 default: str,
) -> str:
 text = str(line or "")

 if self._is_fixed_format_line(text):
 body = text[7:72]
 return self._leading_spaces(body, default)

 return self._leading_spaces(text, default)

def _is_fixed_format_line(
 self,
 line: str,
) -> bool:
 text = str(line or "")

 if len(text) < 80:
 return False

 if not text[:6].isdigit():
 return False

```

```

 if not text[72:80].isdigit():
 return False

 return True

def _leading_spaces(
 self,
 text: str,
 default: str,
) -> str:
 value = str(text or "")

 if not value:
 return default

 count = len(value) - len(value.lstrip(" "))

 if count <= 0:
 return default

 return value[:count]

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\fixed_format_body_formatter.py
ENCODING: utf-8
=====

"""
Fixed-format COBOL body formatter.

Responsibilities:
- Decide Area A vs Area B.
- Preserve original indentation wherever possible.
- Keep original business COBOL indentation.
- Normalize generated SQL and generated procedure statements.
"""

from __future__ import annotations

import re

try:
 from patterns.fixed_format_patterns import (
 AREA_A_PREFIX_PATTERN,
 DATA_LEVEL_PATTERN,
 DIVISION_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 PARAGRAPH_PATTERN,
 SECTION_PATTERN,
)
except Exception:
 AREA_A_PREFIX_PATTERN = re.compile(
 r"^(PROGRAM-ID\.|AUTHOR\.|INSTALLATION\.|DATE-WRITTEN\.|DATE-COMPILED\.|SECURITY\.)",
 flags=re.IGNORECASE,
)

 DATA_LEVEL_PATTERN = re.compile(
 r"^(0[1-9]|[1-4][0-9]|66|77|88)\s+",
 flags=re.IGNORECASE,
)

 DIVISION_PATTERN = re.compile(
 r"^(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\s+DIVISION\b",
 flags=re.IGNORECASE,
)

 EXEC_SQL_START_PATTERN = re.compile(
 r"^EXEC\s+SQL\b",
 flags=re.IGNORECASE,
)

 EXEC_SQL_END_PATTERN = re.compile(

```



```

 r"^END-EXEC\.$$",
 flags=re.IGNORECASE,
)

 PARAGRAPH_PATTERN = re.compile(
 r"^[A-Z0-9][A-Z0-9-]*\.\s*$",
 flags=re.IGNORECASE,
)

 SECTION_PATTERN = re.compile(
 r"^[A-Z0-9-]+\s+SECTION\.$$",
 flags=re.IGNORECASE,
)

try:
 from rules.cobol_statement_rules import (
 NON_PARAGRAPH_WORDS,
 PROCEDURE_VERBS,
 SQL_LEVEL_1_KEYWORDS,
 SQL_LEVEL_2_KEYWORDS,
)
except Exception:
 NON_PARAGRAPH_WORDS = {
 "ACCEPT",
 "ADD",
 "CALL",
 "CLOSE",
 "COMPUTE",
 "CONTINUE",
 "DISPLAY",
 "ELSE",
 "END-IF",
 "EVALUATE",
 "EXEC",
 "EXIT",
 "GO",
 "GOBACK",
 "IF",
 "INITIALIZE",
 "MOVE",
 "OPEN",
 "PERFORM",
 "READ",
 "SET",
 "STOP",
 "WHEN",
 "WRITE",
 }

 PROCEDURE_VERBS = (
 "ACCEPT ",
 "ADD ",
 "CALL ",
 "CLOSE ",
 "COMPUTE ",
 "CONTINUE",
 "DISPLAY ",
 "ELSE",
 "END-IF",
 "EVALUATE ",
 "EXEC SQL",
 "EXIT",
 "GO ",
 "GOBACK",
 "IF ",
 "INITIALIZE ",
 "MOVE ",
 "OPEN ",
 "PERFORM ",
 "READ ",
 "SET ",
 "STOP ",
 "WHEN ",
 "WRITE ",
)

```

```

SQL_LEVEL_1_KEYWORDS = (
 "SELECT ",
 "INTO ",
 "FROM ",
 "WHERE ",
 "ORDER BY ",
 "GROUP BY ",
 "HAVING ",
 "FETCH ",
 "OPEN ",
 "CLOSE ",
 "COMMIT",
 "ROLLBACK",
 "INSERT ",
 "UPDATE ",
 "DELETE ",
 "SET ",
 "VALUES ",
 "FOR READ ONLY",
 "QUERYNO ",
)

SQL_LEVEL_2_KEYWORDS = (
 "AND ",
 "OR ",
)

try:
 from rules.fixed_format_rules import (
 AREA_A_DATA_LEVELS,
 AREA_B_INDENT,
 COMMENT_INDICATOR,
 DEBUG_INDICATOR,
 PAGE_INDICATOR,
 SPACE_INDICATOR,
 SQL_INDENT,
)
except Exception:
 AREA_A_DATA_LEVELS = {
 "01",
 "66",
 "77",
 }
 AREA_B_INDENT = " "
 COMMENT_INDICATOR = "*"
 DEBUG_INDICATOR = "D"
 PAGE_INDICATOR = "/"
 SPACE_INDICATOR = " "
 SQL_INDENT = " "

class FixedFormatBodyFormatter:
 def area_body(
 self,
 body: str,
 logical: str,
 current_division: str,
 inside_exec_sql: bool,
 indicator: str,
 previous_procedure_indent: str,
) -> str:
 original = str(body or "").rstrip()
 clean_statement = str(logical or "").strip()

 if indicator in {
 COMMENT_INDICATOR,
 PAGE_INDICATOR,
 DEBUG_INDICATOR,
 "-",
 }:
 return original

 if not clean_statement:
 return ""

```

```

 if original.startswith(" "):
 return original

 if self.is_area_a_statement(clean_statement):
 return clean_statement

 if inside_exec_sql:
 return self.sql_area_body(clean_statement)

 if current_division == "PROCEDURE":
 return self.procedure_area_b_body(
 clean_statement=clean_statement,
 previous_procedure_indent=previous_procedure_indent,
)

 if DATA_LEVEL_PATTERN.match(clean_statement):
 return self.data_area_body(clean_statement)

 return clean_statement

def procedure_area_b_body(
 self,
 clean_statement: str,
 previous_procedure_indent: str,
) -> str:
 upper = clean_statement.upper()

 if self.is_area_a_statement(clean_statement):
 return clean_statement

 if upper.startswith(PROCEDURE_VERBS):
 return AREA_B_INDENT + clean_statement

 if previous_procedure_indent:
 return previous_procedure_indent + clean_statement

 return AREA_B_INDENT + clean_statement

def data_area_body(
 self,
 clean_statement: str,
) -> str:
 parts = clean_statement.split(maxsplit=1)

 if not parts:
 return clean_statement

 level = parts[0]

 if level in AREA_A_DATA_LEVELS:
 return clean_statement

 return AREA_B_INDENT + clean_statement

def sql_area_body(
 self,
 clean_statement: str,
) -> str:
 upper = clean_statement.upper()

 if EXEC_SQL_START_PATTERN.match(clean_statement):
 return AREA_B_INDENT + clean_statement

 if EXEC_SQL_END_PATTERN.match(clean_statement):
 return AREA_B_INDENT + clean_statement

 if upper.startswith(SQL_LEVEL_1_KEYWORDS):
 return SQL_INDENT + clean_statement

 if upper.startswith(SQL_LEVEL_2_KEYWORDS):
 return SQL_INDENT + clean_statement

 return SQL_INDENT + clean_statement

def is_area_a_statement(

```

```

 self,
 statement: str,
) -> bool:
 clean_statement = str(statement or "").strip()

 if not clean_statement:
 return False

 if DIVISION_PATTERN.match(clean_statement):
 return True

 if SECTION_PATTERN.match(clean_statement):
 return True

 if AREA_A_PREFIX_PATTERN.match(clean_statement):
 return True

 if DATA_LEVEL_PATTERN.match(clean_statement):
 first_word = clean_statement.split(maxsplit=1)[0]
 return first_word in AREA_A_DATA_LEVELS

 if not PARAGRAPH_PATTERN.match(clean_statement):
 return False

 paragraph_name = clean_statement.rstrip(".").strip().upper()

 if paragraph_name in NON_PARAGRAPH_WORDS:
 return False

 return True

def is_exec_sql_start(
 self,
 statement: str,
) -> bool:
 return bool(
 EXEC_SQL_START_PATTERN.match(
 str(statement or "").strip(),
)
)

def is_exec_sql_end(
 self,
 statement: str,
) -> bool:
 return bool(
 EXEC_SQL_END_PATTERN.match(
 str(statement or "").strip(),
)
)

def leading_spaces(
 self,
 text: str,
 default: str = "",
) -> str:
 value = str(text or "")

 if not value:
 return default

 count = len(value) - len(value.lstrip(" "))

 if count <= 0:
 return default

 return value[:count]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\fixed_format_composer.py
ENCODING: utf-8
=====

```

```

"""
Fixed-format COBOL composer.

```

This composer orchestrates final physical COBOL formatting.

Detailed responsibilities are split into:

- fixed\_format\_line\_parser.py
- fixed\_format\_body\_formatter.py
- fixed\_format\_wrapper.py
- fixed\_format\_sequence\_manager.py
- patterns/fixed\_format\_patterns.py
- rules/fixed\_format\_rules.py

Final physical layout:

- Columns 1-6 : left sequence number
  - Column 7 : indicator area
  - Columns 8-72 : COBOL body
  - Columns 73-80 : right sequence number
- """

```
from idms_db2_phase2.composers.fixed_format_body_formatter import (
 FixedFormatBodyFormatter,
)
from idms_db2_phase2.composers.fixed_format_line_parser import (
 FixedFormatLineParser,
)
from idms_db2_phase2.composers.fixed_format_sequence_manager import (
 FixedFormatSequenceManager,
)
from idms_db2_phase2.composers.fixed_format_wrapper import FixedFormatWrapper
from patterns.fixed_format_patterns import (
 DIVISION_PATTERN,
 SEQUENCE_ONLY_PATTERN,
)
from rules.fixed_format_rules import (
 BODY_WIDTH,
 COMMENT_INDICATOR,
 DEBUG_INDICATOR,
 PAGE_INDICATOR,
 SPACE_INDICATOR,
 TOTAL_WIDTH,
 VALID_INDICATORS,
)
```

```
class FixedFormatComposer:
```

```
 """
```

```
 Final fixed-format COBOL composer.
```

```
 This class does not own regex patterns, layout constants, or sequence
 detection rules. It only coordinates the helper classes.
```

```
 Sequence numbering is generic:
```

- Existing valid left sequence pattern is detected.
  - Existing valid manual-style right sequence pattern is detected.
  - Invalid small right sequence patterns are normalized by the sequence manager.
- ```
    """
```

```
    def __init__(
```

```
        self,
```

```
    ) -> None:
```

```
        self.line_parser = FixedFormatLineParser()
```

```
        self.body_formatter = FixedFormatBodyFormatter()
```

```
        self.wrapper = FixedFormatWrapper()
```

```
        self.sequence_manager = FixedFormatSequenceManager()
```

```
    def format(
```

```
        self,
```

```
        text: str,
```

```
        left_start: int | None = None,
```

```
        left_step: int | None = None,
```

```
        right_start: int | None = None,
```

```
        right_step: int | None = None,
```

```
        preserve_blank_lines: bool = True,
```

```
    ) -> str:
```

```
        if not text:
```

```
            return ""
```

```

lines = self._normalize_line_endings(text).splitlines()
lines = self._merge_dangling_boolean_lines(lines)

sequence_state = self.sequence_manager.create_state(
    lines=lines,
    left_start=left_start,
    left_step=left_step,
    right_start=right_start,
    right_step=right_step,
)

output_lines: list[str] = []
current_division = ""
inside_exec_sql = False
previous_procedure_indent = " "

for raw_line in lines:
    raw_text = str(raw_line or "").rstrip()

    if not raw_text.strip():
        if preserve_blank_lines:
            output_lines.append(
                self._compose_line(
                    left_seq=sequence_state.current_left(),
                    indicator=SPACE_INDICATOR,
                    area_body="",
                    right_seq=sequence_state.current_right(),
                )
            )
            sequence_state.advance()
            continue

    if SEQUENCE_ONLY_PATTERN.match(raw_text.strip()):
        continue

    parsed = self.line_parser.parse_line(raw_text)

    indicator = str(parsed["indicator"])
    body = str(parsed["body"])
    logical = body.strip()

    if not body.strip() and indicator == SPACE_INDICATOR:
        if preserve_blank_lines:
            output_lines.append(
                self._compose_line(
                    left_seq=sequence_state.current_left(),
                    indicator=SPACE_INDICATOR,
                    area_body="",
                    right_seq=sequence_state.current_right(),
                )
            )
            sequence_state.advance()
            continue

    division_match = DIVISION_PATTERN.match(logical)

    if division_match:
        current_division = division_match.group(1).upper()
        previous_procedure_indent = " "

    if self.body_formatter.is_exec_sql_start(logical):
        inside_exec_sql = True

    area_body = self.body_formatter.area_body(
        body=body,
        logical=logical,
        current_division=current_division,
        inside_exec_sql=inside_exec_sql,
        indicator=indicator,
        previous_procedure_indent=previous_procedure_indent,
    )

    physical_bodies = self.wrapper.wrap_body(
        body=area_body,
        indicator=indicator,
        inside_exec_sql=inside_exec_sql,

```

```

        current_division=current_division,
        previous_procedure_indent=previous_procedure_indent,
    )

    physical_bodies = self.wrapper.repair_boolean_operator_only_lines(
        physical_bodies
    )

    for index, physical_body in enumerate(physical_bodies):
        physical_indicator = indicator

        if index > 0:
            physical_indicator = self._continuation_indicator(indicator)

        output_lines.append(
            self._compose_line(
                left_seq=sequence_state.current_left(),
                indicator=physical_indicator,
                area_body=physical_body,
                right_seq=sequence_state.current_right(),
            )
        )

        sequence_state.advance()

    if (
        current_division == "PROCEDURE"
        and indicator == SPACE_INDICATOR
        and physical_bodies
        and not self.body_formatter.is_area_a_statement(logical)
    ):
        previous_procedure_indent = self.body_formatter.leading_spaces(
            physical_bodies[0],
            default=" ",
        )

    if self.body_formatter.is_exec_sql_end(logical):
        inside_exec_sql = False

    return "\n".join(output_lines).rstrip() + "\n"

def validate_fixed_format(
    self,
    text: str,
) -> list[str]:
    messages: list[str] = []

    for line_number, line in enumerate(
        str(text or "").splitlines(),
        start=1,
    ):
        if not line:
            continue

        if len(line) != TOTAL_WIDTH:
            messages.append(
                f"Line {line_number}: expected 80 columns, found {len(line)}."
            )
            continue

        left_seq = line[0:6]
        indicator = line[6:7]
        right_seq = line[72:80]

        if not left_seq.isdigit():
            messages.append(
                f"Line {line_number}: left sequence is not numeric."
            )

        if indicator not in VALID_INDICATORS:
            messages.append(
                f"Line {line_number}: invalid indicator column value."
            )

        if not right_seq.isdigit():
            messages.append(

```

```

        f"Line {line_number}: right sequence is not numeric."
    )

    return messages

def _normalize_line_endings(
    self,
    text: str,
) -> str:
    return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

def _merge_dangling_boolean_lines(
    self,
    lines: list[str],
) -> list[str]:
    output: list[str] = []
    index = 0

    while index < len(lines):
        current = str(lines[index] or "").rstrip()

        if index + 1 >= len(lines):
            output.append(current)
            index += 1
            continue

        next_line = str(lines[index + 1] or "").rstrip()

        current_body = self.line_parser.body_for_boolean_merge(current)
        next_body = self.line_parser.body_for_boolean_merge(next_line)

        if not current_body.strip() or not next_body.strip():
            output.append(current)
            index += 1
            continue

        if self.wrapper.is_comment_or_page_line(current_body):
            output.append(current)
            index += 1
            continue

        if self.wrapper.is_comment_or_page_line(next_body):
            output.append(current)
            index += 1
            continue

        if self.wrapper.ends_with_boolean_operator(current_body):
            merged_body = f"{current_body.rstrip()} {next_body.strip()}"

            output.append(
                self.line_parser.replace_body_preserving_sequence(
                    original_line=current,
                    new_body=merged_body,
                )
            )
            index += 2
            continue

        output.append(current)
        index += 1

    return output

def _continuation_indicator(
    self,
    indicator: str,
) -> str:
    if indicator in {COMMENT_INDICATOR, PAGE_INDICATOR}:
        return indicator

    if indicator in {DEBUG_INDICATOR, DEBUG_INDICATOR.lower()}:
        return DEBUG_INDICATOR

    return SPACE_INDICATOR

def _compose_line(

```



```

self,
left_seq: str,
indicator: str,
area_body: str,
right_seq: str,
) -> str:
    safe_left = str(left_seq or "").zfill(6)[-6:]
    safe_right = str(right_seq or "").zfill(8)[-8:]
    safe_indicator = str(indicator or SPACE_INDICATOR)[:1]

    if safe_indicator not in VALID_INDICATORS:
        safe_indicator = SPACE_INDICATOR

    if safe_indicator == "d":
        safe_indicator = DEBUG_INDICATOR

    safe_body = str(area_body or "").rstrip()

    if len(safe_body) > BODY_WIDTH:
        safe_body = safe_body[:BODY_WIDTH]

    body_area = safe_body.ljust(BODY_WIDTH)
    line = f"{safe_left}{safe_indicator}{body_area}{safe_right}"

    if len(line) > TOTAL_WIDTH:
        return line[:TOTAL_WIDTH]

    if len(line) < TOTAL_WIDTH:
        return line.ljust(TOTAL_WIDTH)

    return line

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\fixed_format_line_parser.py
ENCODING: utf-8
=====

```

```

"""

```

Fixed-format COBOL line parser.

Responsibilities:

- Parse already sequenced 80-column COBOL lines.
- Parse unsequenced COBOL lines.
- Strip old left and right sequence numbers.
- Preserve COBOL body indentation.
- Detect indicator column values.

Important:

- A true left sequence number must start in column 1 and must be exactly six digits.
- COBOL level numbers such as 01, 03, 05, 10, 20, 77, and 88 must never be treated as sequence numbers.
- Old right sequence numbers may appear separated by spaces or accidentally attached to the COBOL body. Both cases are handled conservatively.

```

"""

```

```

from __future__ import annotations

```

```

import re

```

```

try:
    from patterns.fixed_format_patterns import DEBUG_LINE_PATTERN
except Exception:
    DEBUG_LINE_PATTERN = re.compile(
        r"^[Dd]\s+",
        flags=re.IGNORECASE,
    )

```

```

try:
    from rules.fixed_format_rules import (
        BODY_WIDTH,
        COMMENT_INDICATOR,
        DEBUG_INDICATOR,
    )

```

```

        PAGE_INDICATOR,
        SPACE_INDICATOR,
        TOTAL_WIDTH,
        VALID_INDICATORS,
    )
except Exception:
    BODY_WIDTH = 65
    COMMENT_INDICATOR = "*"
    DEBUG_INDICATOR = "D"
    PAGE_INDICATOR = "/"
    SPACE_INDICATOR = " "
    TOTAL_WIDTH = 80
    VALID_INDICATORS = {
        SPACE_INDICATOR,
        COMMENT_INDICATOR,
        DEBUG_INDICATOR,
        DEBUG_INDICATOR.lower(),
        PAGE_INDICATOR,
        "-",
    }

class FixedFormatLineParser:
    """
    Parses fixed-format and loose COBOL lines safely.
    """

    TRUE_LEFT_SEQUENCE_PATTERN = re.compile(
        r"^(?P<left>\d{6})(?P<body>\s+.*?)$",
        flags=re.IGNORECASE,
    )

    RIGHT_SEQUENCE_WITH_SPACES_PATTERN = re.compile(
        r"^(?P<body>.*?)(?P<spaces>\s+)(?P<right>\d{8})\s*$",
        flags=re.IGNORECASE,
    )

    TRAILING_TIGHT_RIGHT_SEQUENCE_PATTERN = re.compile(
        r"^(?P<body>.*\S)(?P<right>\d{8})\s*$",
        flags=re.IGNORECASE,
    )

    COBOL_STATEMENT_STARTERS = (
        "ACCEPT ",
        "ADD ",
        "CALL ",
        "CLOSE ",
        "COMPUTE ",
        "CONTINUE",
        "DISPLAY ",
        "ELSE",
        "END-",
        "EVALUATE ",
        "EXEC ",
        "EXIT",
        "IF ",
        "INITIALIZE ",
        "MOVE ",
        "OPEN ",
        "PERFORM ",
        "READ ",
        "SET ",
        "STOP ",
        "TO ",
        "WHEN ",
        "WRITE ",
    )

    def parse_line(
        self,
        line: str,
    ) -> dict[str, str | bool]:
        text = str(line or "").rstrip()

        fixed = self.parse_fixed_80_line(text)

```

```

    if fixed is not None:
        return fixed

    body = self.remove_loose_sequence_numbers(text)
    indicator = self.indicator_for_body(body)
    body = self.strip_inline_indicator(
        body=body,
        indicator=indicator,
    )

    return {
        "left_sequence": "",
        "indicator": indicator,
        "body": body,
        "right_sequence": "",
        "is_fixed_format": False,
    }

def parse_fixed_80_line(
    self,
    line: str,
) -> dict[str, str | bool] | None:
    text = str(line or "")

    if not self._is_fixed_format_line(text):
        return None

    left_seq = text[:6]
    indicator = text[6:7]
    body = text[7:72].rstrip()
    right_seq = text[72:80]

    if indicator not in VALID_INDICATORS:
        indicator = SPACE_INDICATOR

    return {
        "left_sequence": left_seq,
        "indicator": indicator,
        "body": body,
        "right_sequence": right_seq,
        "is_fixed_format": True,
    }

def body_for_boolean_merge(
    self,
    line: str,
) -> str:
    parsed = self.parse_line(line)
    return str(parsed.get("body", ""))

def remove_loose_sequence_numbers(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()
    text = self.remove_loose_right_sequence(text)
    text = self.remove_loose_left_sequence(text)
    return text.rstrip()

def remove_loose_left_sequence(
    self,
    line: str,
) -> str:
    text = str(line or "")
    match = self.TRUE_LEFT_SEQUENCE_PATTERN.match(text)

    if not match:
        return text

    left = str(match.group("left") or "")
    body = str(match.group("body") or "")

    if not left.isdigit() or len(left) != 6:
        return text

    return body.lstrip()

```

```

def remove_loose_right_sequence(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    spaced = self.RIGHT_SEQUENCE_WITH_SPACES_PATTERN.match(text)

    if spaced:
        body = str(spaced.group("body") or "").rstrip()
        spaces = str(spaced.group("spaces") or "")
        right = str(spaced.group("right") or "")

        if self._looks_like_spaced_right_sequence(
            body=body,
            spaces=spaces,
            right=right,
            original=text,
        ):
            return body

    tight = self.TRAILING_TIGHT_RIGHT_SEQUENCE_PATTERN.match(text)

    if tight:
        body = str(tight.group("body") or "").rstrip()
        right = str(tight.group("right") or "")

        if self._looks_like_tight_right_sequence(
            body=body,
            right=right,
            original=text,
        ):
            return body

    return text

def _looks_like_spaced_right_sequence(
    self,
    body: str,
    spaces: str,
    right: str,
    original: str,
) -> bool:
    if not self._is_eight_digit_sequence(right):
        return False

    if not body:
        return False

    if len(original) >= TOTAL_WIDTH:
        return True

    if len(spaces) >= 2:
        return True

    return False

def _looks_like_tight_right_sequence(
    self,
    body: str,
    right: str,
    original: str,
) -> bool:
    if not self._is_eight_digit_sequence(right):
        return False

    if not body:
        return False

    if len(original) >= TOTAL_WIDTH and original[72:80].isdigit():
        return True

    body_upper = body.strip().upper()

    if not body_upper:

```

```

        return False

    if body_upper.startswith(self.COBOL_STATEMENT_STARTERS):
        return True

    if right.startswith("0000") and self._body_ends_like_cobol_identifier(body):
        return True

    return False

def _is_eight_digit_sequence(
    self,
    value: str,
) -> bool:
    text = str(value or "")
    return len(text) == 8 and text.isdigit()

def _body_ends_like_cobol_identifier(
    self,
    body: str,
) -> bool:
    text = str(body or "").rstrip()

    if not text:
        return False

    if text[-1].isdigit():
        return False

    if text[-1] in {'"', "'"}:
        return False

    last_token = text.split()[-1]

    return bool(
        re.fullmatch(
            r"[A-Z][A-Z0-9-]*",
            last_token,
            flags=re.IGNORECASE,
        )
    )

def indicator_for_body(
    self,
    body: str,
) -> str:
    text = str(body or "")

    if not text:
        return SPACE_INDICATOR

    stripped = text.lstrip()

    if stripped.startswith(COMMENT_INDICATOR):
        return COMMENT_INDICATOR

    if stripped.startswith(PAGE_INDICATOR):
        return PAGE_INDICATOR

    if DEBUG_LINE_PATTERN.match(text):
        return DEBUG_INDICATOR

    return SPACE_INDICATOR

def strip_inline_indicator(
    self,
    body: str,
    indicator: str,
) -> str:
    text = str(body or "")

    if indicator in {COMMENT_INDICATOR, PAGE_INDICATOR}:
        stripped = text.lstrip()

        if stripped.startswith(indicator):
            return stripped[1:].lstrip()

```

```

if indicator == DEBUG_INDICATOR and DEBUG_LINE_PATTERN.match(text):
    stripped = text.lstrip()

    if stripped[:1].upper() == DEBUG_INDICATOR:
        return stripped[1:].lstrip()

return text.rstrip()

def replace_body_preserving_sequence(
    self,
    original_line: str,
    new_body: str,
) -> str:
    text = str(original_line or "").rstrip()
    parsed = self.parse_line(text)

    if bool(parsed.get("is_fixed_format")):
        left = str(parsed.get("left_sequence") or "").zfill(6)[-6:]
        indicator = str(parsed.get("indicator") or SPACE_INDICATOR)[:1]
        right = str(parsed.get("right_sequence") or "").zfill(8)[-8:]

        if indicator not in VALID_INDICATORS:
            indicator = SPACE_INDICATOR

        body = str(new_body or "").rstrip()
        return f"{left}{indicator}{body[:BODY_WIDTH].ljust(BODY_WIDTH)}{right}"

    match = self.TRUE_LEFT_SEQUENCE_PATTERN.match(text)

    if match:
        left = str(match.group("left") or "").zfill(6)[-6:]
        return f"{left} {new_body}"

    return str(new_body or "").rstrip()

def is_debug_line(
    self,
    stripped: str,
) -> bool:
    return bool(DEBUG_LINE_PATTERN.match(str(stripped or "")))

def _is_fixed_format_line(
    self,
    line: str,
) -> bool:
    text = str(line or "")

    if len(text) < TOTAL_WIDTH:
        return False

    if not text[:6].isdigit():
        return False

    if not text[72:80].isdigit():
        return False

    return True

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\fixed_format_sequence_manager.py
ENCODING: utf-8
=====

```

```

"""
Fixed-format COBOL sequence manager.

```

Responsibilities:

- Detect left sequence start and step from input when available.
- Detect manual-style right sequence start and step from input when available.
- Reject old/generated right sequence styles like 00000010, 00000020.
- Provide generic fallback manual sequence values when source cannot provide them.

```

"""

```

```

from __future__ import annotations

```

```

import re
from dataclasses import dataclass

try:
    from patterns.fixed_format_patterns import (
        LEFT_SEQUENCE_PATTERN,
        RIGHT_SEQUENCE_PATTERN,
    )
except Exception:
    LEFT_SEQUENCE_PATTERN = re.compile(
        r"^\s*(?P<left>\d{6})\s+(?P<body>.*)$",
        flags=re.IGNORECASE,
    )

    RIGHT_SEQUENCE_PATTERN = re.compile(
        r"^(?P<body>.*?)(?P<right>\d{8})\s*$",
        flags=re.IGNORECASE,
    )

try:
    from rules.fixed_format_rules import (
        DEFAULT_LEFT_START,
        DEFAULT_LEFT_STEP,
        DEFAULT_RIGHT_START,
        DEFAULT_RIGHT_STEP,
        TOTAL_WIDTH,
    )
except Exception:
    DEFAULT_LEFT_START = 10
    DEFAULT_LEFT_STEP = 10
    DEFAULT_RIGHT_START = 10000
    DEFAULT_RIGHT_STEP = 10000
    TOTAL_WIDTH = 80

@dataclass
class FixedFormatSequenceState:
    left_current: int
    left_step: int
    right_current: int
    right_step: int

    def current_left(
        self,
    ) -> str:
        return f"{self.left_current:06d}"

    def current_right(
        self,
    ) -> str:
        return f"{self.right_current:08d}"

    def advance(
        self,
    ) -> None:
        self.left_current += self.left_step
        self.right_current += self.right_step

class FixedFormatSequenceManager:
    """
    Generic sequence manager.

    Manual right sequence style:
    - 00010000
    - 00020000
    - 00030000

    Old/generated right sequence style:
    - 00000010
    - 00000020

    The old style is intentionally not preserved.

```

"""

MIN_MANUAL_RIGHT_SEQUENCE = 10000

MIN_MANUAL_RIGHT_STEP = 10000

```
def create_state(
    self,
    lines: list[str],
    left_start: int | None = None,
    left_step: int | None = None,
    right_start: int | None = None,
    right_step: int | None = None,
) -> FixedFormatSequenceState:
    detected_left_values = self._left_sequence_values(lines)
    detected_right_values = self._right_sequence_values(lines)

    effective_left_start = (
        left_start
        if left_start is not None
        else self._first_positive(
            detected_left_values,
            DEFAULT_LEFT_START,
        )
    )

    effective_left_step = (
        left_step
        if left_step is not None
        else self._detect_step(
            detected_left_values,
            DEFAULT_LEFT_STEP,
        )
    )

    effective_right_start = (
        right_start
        if right_start is not None
        else self._detect_manual_right_start(
            detected_right_values,
        )
    )

    effective_right_step = (
        right_step
        if right_step is not None
        else self._detect_manual_right_step(
            detected_right_values,
        )
    )

    return FixedFormatSequenceState(
        left_current=effective_left_start,
        left_step=effective_left_step,
        right_current=effective_right_start,
        right_step=effective_right_step,
    )

def _left_sequence_values(
    self,
    lines: list[str],
) -> list[int]:
    values: list[int] = []

    for line in lines:
        text = str(line or "").rstrip()

        if len(text) >= TOTAL_WIDTH and text[:6].isdigit():
            values.append(int(text[:6]))
            continue

        match = LEFT_SEQUENCE_PATTERN.match(text)

        if match:
            left = str(match.group("left") or "")

            if left.isdigit():
```



```

        values.append(int(left))

    return values

def _right_sequence_values(
    self,
    lines: list[str],
) -> list[int]:
    values: list[int] = []

    for line in lines:
        text = str(line or "").rstrip()

        if len(text) >= TOTAL_WIDTH and text[72:80].isdigit():
            values.append(int(text[72:80]))
            continue

        match = RIGHT_SEQUENCE_PATTERN.match(text)

        if match:
            right = str(match.group("right") or "")

            if right.isdigit():
                values.append(int(right))

    return values

def _first_positive(
    self,
    values: list[int],
    default_value: int,
) -> int:
    for value in values:
        if value > 0:
            return value

    return default_value

def _detect_step(
    self,
    values: list[int],
    default_step: int,
) -> int:
    differences = self._positive_differences(values)

    if not differences:
        return default_step

    return self._most_common_difference(
        differences,
        default_step,
    )

def _detect_manual_right_start(
    self,
    values: list[int],
) -> int:
    manual_values = [
        value
        for value in values
        if value >= self.MIN_MANUAL_RIGHT_SEQUENCE
    ]

    if not manual_values:
        return DEFAULT_RIGHT_START

    return manual_values[0]

def _detect_manual_right_step(
    self,
    values: list[int],
) -> int:
    manual_values = [
        value
        for value in values
        if value >= self.MIN_MANUAL_RIGHT_SEQUENCE
    ]

```

```

]

differences = self._positive_differences(manual_values)

if not differences:
    return DEFAULT_RIGHT_STEP

step = self._most_common_difference(
    differences,
    DEFAULT_RIGHT_STEP,
)

if step < self.MIN_MANUAL_RIGHT_STEP:
    return DEFAULT_RIGHT_STEP

return step

def _positive_differences(
    self,
    values: list[int],
) -> list[int]:
    differences: list[int] = []

    for index in range(1, len(values)):
        difference = values[index] - values[index - 1]

        if difference > 0:
            differences.append(difference)

    return differences

def _most_common_difference(
    self,
    differences: list[int],
    default_step: int,
) -> int:
    if not differences:
        return default_step

    counts: dict[int, int] = {}

    for difference in differences:
        counts[difference] = counts.get(difference, 0) + 1

    return sorted(
        counts.items(),
        key=lambda item: (-item[1], item[0]),
    )[0][0]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\fixed_format_wrapper.py
ENCODING: utf-8
=====

```

```

"""

```

Fixed-format COBOL body wrapper.

Responsibilities:

- Split long comments.
- Split long MOVE statements.
- Split IF statements with AND/OR safely.
- Split generic long statements by word boundaries.
- Keep each body within columns 8-72.

```

"""

```

```

from __future__ import annotations

```

```

import re

```

```

try:

```

```

    from patterns.fixed_format_patterns import (
        BOOLEAN_OPERATOR_END_PATTERN,
        IF_BOOLEAN_SPLIT_PATTERN,
        MOVE_STATEMENT_PATTERN,

```

```

    )
except Exception:
    BOOLEAN_OPERATOR_END_PATTERN = re.compile(
        r"\b(?:AND|OR)\s*$",
        flags=re.IGNORECASE,
    )

    IF_BOOLEAN_SPLIT_PATTERN = re.compile(
        r"(\s+AND\s+|\s+OR\s+)",
        flags=re.IGNORECASE,
    )

    MOVE_STATEMENT_PATTERN = re.compile(
        r"^MOVE\s+(?P<src>.+?)\s+TO\s+(?P<tgt>.+?\.\.?)$",
        flags=re.IGNORECASE,
    )

try:
    from rules.fixed_format_rules import (
        BODY_WIDTH,
        COMMENT_INDICATOR,
        PAGE_INDICATOR,
    )
except Exception:
    BODY_WIDTH = 65
    COMMENT_INDICATOR = "*"
    PAGE_INDICATOR = "/"

class FixedFormatWrapper:
    def wrap_body(
        self,
        body: str,
        indicator: str,
        inside_exec_sql: bool,
        current_division: str,
        previous_procedure_indent: str,
    ) -> list[str]:
        text = str(body or "").rstrip()

        if len(text) <= BODY_WIDTH:
            return [text]

        if indicator in {COMMENT_INDICATOR, PAGE_INDICATOR}:
            return self.wrap_comment_body(text)

        return self.wrap_cobol_body(
            text=text,
            inside_exec_sql=inside_exec_sql,
            current_division=current_division,
            previous_procedure_indent=previous_procedure_indent,
        )

    def wrap_comment_body(
        self,
        text: str,
    ) -> list[str]:
        clean = str(text or "").strip()

        if not clean:
            return [""]

        return self.wrap_by_words(
            text=clean,
            continuation_indent="",
        )

    def wrap_cobol_body(
        self,
        text: str,
        inside_exec_sql: bool,
        current_division: str,
        previous_procedure_indent: str,
    ) -> list[str]:
        raw = str(text or "").rstrip()

```

```

logical = raw.strip()

indent = self.leading_spaces(
    raw,
    default=" " if current_division == "PROCEDURE" else "",
)

if inside_exec_sql:
    indent = self.leading_spaces(
        raw,
        default=" ",
    )

if_lines = self.wrap_if_statement(
    logical=logical,
    indent=indent,
)

if if_lines:
    return if_lines

move_lines = self.wrap_move_statement(
    logical=logical,
    indent=indent,
)

if move_lines:
    return move_lines

return self.wrap_by_words(
    text=indent + logical,
    continuation_indent=indent + " ",
)

def wrap_if_statement(
    self,
    logical: str,
    indent: str,
) -> list[str]:
    text = str(logical or "").strip()

    if not text.upper().startswith("IF "):
        return []

    split_result = self.split_condition_on_boolean_operator(text)

    if not split_result:
        return self.wrap_by_words(
            text=indent + text,
            continuation_indent=indent + " ",
        )

    left_part, operator, right_part = split_result

    first_line = f"{indent}{left_part.rstrip()}"
    second_line = f"{indent}{operator.strip().upper()} {right_part.strip()}"

    return self.force_lines_to_body_width(
        lines=[first_line, second_line],
        continuation_indent=indent + " ",
    )

def split_condition_on_boolean_operator(
    self,
    logical: str,
) -> tuple[str, str, str] | None:
    text = str(logical or "").strip()

    tokens = IF_BOOLEAN_SPLIT_PATTERN.split(
        text,
        maxsplit=1,
    )

    if len(tokens) < 3:
        return None

```

```

left_part = tokens[0].rstrip()
operator = tokens[1].strip().upper()
right_part = tokens[2].strip()

if not left_part or not operator or not right_part:
    return None

return left_part, operator, right_part

def wrap_move_statement(
    self,
    logical: str,
    indent: str,
) -> list[str]:
    match = MOVE_STATEMENT_PATTERN.match(str(logical or "").strip())

    if not match:
        return []

    src = match.group("src").strip()
    tgt = match.group("tgt").strip()

    first = f"{indent}MOVE {src}"
    second = f"{indent}TO {tgt}"

    if len(first) <= BODY_WIDTH and len(second) <= BODY_WIDTH:
        return [first, second]

    return self.wrap_by_words(
        text=indent + logical,
        continuation_indent=indent + " ",
    )

def wrap_by_words(
    self,
    text: str,
    continuation_indent: str,
) -> list[str]:
    raw = str(text or "").rstrip()

    if len(raw) <= BODY_WIDTH:
        return [raw]

    leading_indent = self.leading_spaces(
        raw,
        default="",
    )
    content = raw.strip()
    words = content.split()

    lines: list[str] = []
    current = leading_indent

    for word in words:
        if current.strip():
            candidate = current.rstrip() + " " + word
        else:
            candidate = leading_indent + word

        if len(candidate) <= BODY_WIDTH:
            current = candidate
            continue

        if current.strip():
            lines.append(current.rstrip())

        current = continuation_indent + word

    if current.strip():
        lines.append(current.rstrip())

    return self.force_lines_to_body_width(
        lines=lines,
        continuation_indent=continuation_indent,
    )

```

```

def force_lines_to_body_width(
    self,
    lines: list[str],
    continuation_indent: str,
) -> list[str]:
    output: list[str] = []

    for line in lines:
        text = str(line or "").rstrip()

        if len(text) <= BODY_WIDTH:
            output.append(text)
            continue

        indent = self.leading_spaces(
            text,
            default=continuation_indent,
        )
        available_width = max(1, BODY_WIDTH - len(indent))
        content = text.strip()

        while content:
            part = content[:available_width].rstrip()

            if not part:
                part = content[:available_width]

            output.append(indent + part)
            content = content[len(part):].strip()

    return self.repair_boolean_operator_only_lines(output)

def repair_boolean_operator_only_lines(
    self,
    lines: list[str],
) -> list[str]:
    output: list[str] = []
    index = 0

    while index < len(lines):
        current = str(lines[index] or "").rstrip()
        current_stripped = current.strip().upper()

        if current_stripped in {"AND", "OR"} and index + 1 < len(lines):
            next_line = str(lines[index + 1] or "").rstrip()
            next_stripped = next_line.strip()

            if next_stripped:
                indent = self.leading_spaces(
                    current,
                    default=" ",
                )
                repaired = f"{indent}{current_stripped} {next_stripped}"

                if len(repaired) <= BODY_WIDTH:
                    output.append(repaired)
                    index += 2
                    continue

            output.append(current)
            index += 1

    return output

def ends_with_boolean_operator(
    self,
    body: str,
) -> bool:
    return bool(
        BOOLEAN_OPERATOR_END_PATTERN.search(
            str(body or "").strip(),
        )
    )

def is_comment_or_page_line(
    self,

```

```

        logical: str,
    ) -> bool:
        stripped = str(logical or "").strip()
        return stripped.startswith("*") or stripped.startswith("/")

def leading_spaces(
    self,
    text: str,
    default: str = "",
) -> str:
    value = str(text or "")

    if not value:
        return default

    count = len(value) - len(value.lstrip(" "))

    if count <= 0:
        return default

    return value[:count]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\manual_layout_composer.py
ENCODING: utf-8
=====

```

```
import re
```

```

from catalogs.output_sections import (
    DB2_CURSOR_PARAGRAPH_MARKER,
    DB2_INFRASTRUCTURE_MARKER,
)
from patterns.db2_patterns import END_PROGRAM_PATTERN

```

```
class ManuallayoutComposer:
```

```

    """
    Keeps generated DB2 blocks in a manual-style production layout.

```

```
    Required output order:
```

1. Original COBOL header starts first.
2. Original DATA and PROCEDURE structure remains.
3. Generated DB2 infrastructure remains one complete block.
4. Generated cursor OPEN/FETCH/CLOSE paragraphs are near the end.
5. SQL-ERROR paragraph is near the end.
6. END PROGRAM remains last.

```

    This composer is intentionally conservative. It normalizes spacing and
    keeps END PROGRAM as the final program boundary without rewriting business
    logic.
    """

```

```

def compose(
    self,
    text: str,
) -> str:
    if not text:
        return ""

    lines = self._normalize_line_endings(text).splitlines()
    lines = self._rstrip_blank_lines(lines)

    if not lines:
        return ""

    end_program_line = ""
    end_program_index = self._find_end_program_index(lines)

    if end_program_index >= 0:
        end_program_line = lines[end_program_index]
        lines = lines[:end_program_index] + lines[end_program_index + 1:]

    lines = self._normalize_generated_block_spacing(lines)

```

```

    if end_program_line:
        lines = self._append_end_program(lines, end_program_line)

    result = "\n".join(lines)
    result = self._normalize_blank_lines(result)

    return result.strip() + "\n"

def _normalize_line_endings(
    self,
    text: str,
) -> str:
    return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

def _normalize_generated_block_spacing(
    self,
    lines: list[str],
) -> list[str]:
    output: list[str] = []

    for line in lines:
        logical = str(line or "").upper()

        if DB2_INFRASTRUCTURE_MARKER in logical:
            output = self._rstrip_blank_lines(output)
            output.append("")

        if DB2_CURSOR_PARAGRAPH_MARKER in logical:
            output = self._rstrip_blank_lines(output)
            output.append("")

        output.append(line)

    return output

def _append_end_program(
    self,
    lines: list[str],
    end_program_line: str,
) -> list[str]:
    if not end_program_line:
        return lines

    trimmed = self._rstrip_blank_lines(lines)

    if not trimmed:
        return [end_program_line]

    return trimmed + ["", end_program_line]

def _find_end_program_index(
    self,
    lines: list[str],
) -> int:
    for index, line in enumerate(lines):
        logical = self._logical_line(line)

        if END_PROGRAM_PATTERN.match(logical):
            return index

    return -1

def _logical_line(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    text = self._remove_right_sequence(text)
    text = self._remove_left_sequence(text)

    return text.strip()

def _remove_left_sequence(
    self,
    line: str,

```



```

) -> str:
    text = str(line or "")

    if len(text) > 6 and text[:6].strip().isdigit():
        return text[6:].strip()

    return text

def _remove_right_sequence(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    match = re.match(
        r"^(?P<body>.*?)(?:\s+(?P<right>\d{8}))\s*$",
        text,
        flags=re.IGNORECASE,
    )

    if match:
        return match.group("body").rstrip()

    return text

def _rstrip_blank_lines(
    self,
    lines: list[str],
) -> list[str]:
    index = len(lines)

    while index > 0 and not str(lines[index - 1] or "").strip():
        index -= 1

    return lines[:index]

def _lstrip_blank_lines(
    self,
    lines: list[str],
) -> list[str]:
    index = 0

    while index < len(lines) and not str(lines[index] or "").strip():
        index += 1

    return lines[index:]

def _trim_blank_edges(
    self,
    lines: list[str],
) -> list[str]:
    return self._lstrip_blank_lines(
        self._rstrip_blank_lines(lines)
    )

def _normalize_blank_lines(
    self,
    text: str,
) -> str:
    normalized = str(text or "")

    normalized = re.sub(
        r"\n{4,}",
        "\n\n\n",
        normalized,
    )

    normalized = re.sub(
        r"\n[ \t]+\n",
        "\n\n",
        normalized,
    )

    return normalized

```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers>manual_style_preserver.py
ENCODING: utf-8
=====
```

```
class ManualStylePreserver:
    """
    Preserves manual COBOL style by avoiding broad formatting changes.

    Requirement:
    - Output should remain close to original IDMS COBOL style.
    - Only DB2 conversion changes should be carried out.
    - Existing comments should remain where possible.
    """

    def preserve(
        self,
        original_text: str,
        converted_text: str,
    ) -> str:
        if not converted_text:
            return ""

        text = self._preserve_comment_density(converted_text)
        text = self._normalize_excess_blank_lines(text)

        return text.rstrip() + "\n"

    def _preserve_comment_density(
        self,
        text: str,
    ) -> str:
        output_lines: list[str] = []

        for line in str(text or "").splitlines():
            output_lines.append(line.rstrip())

        return "\n".join(output_lines)

    def _normalize_excess_blank_lines(
        self,
        text: str,
    ) -> str:
        while "\n\n\n\n" in text:
            text = text.replace("\n\n\n\n", "\n\n\n")

        return text
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\sqlcode_wrapper_cleanup_composer.py
ENCODING: utf-8
=====
```

```
"""
SQLCODE wrapper cleanup composer.

This composer removes contradictory SQLCODE wrappers left after redundant
OBTAIN NEXT cursor calls have been removed.

It targets this generated pattern:
```

```
IF NOT SQLCODE = 100
    * DB2: Removed redundant OBTAIN NEXT after cursor EOC loop.
    CONTINUE.
IF SQLCODE = 100
    ...
END-IF
END-IF.
```

and rewrites it to:

```
* DB2: Removed redundant SQLCODE wrapper after cursor EOC loop.
IF WS-STATUS = 'C'
    ...
END-IF
```

It also wraps MOVE statements inside the recovered block:

```
MOVE SOURCE OF DCLGROUP TO TARGET
```

into:

```
MOVE SOURCE OF DCLGROUP  
TO TARGET
```

```
"""
```

```
import re
```

```
from patterns.sequence_patterns import strip_sequence_numbers
```

```
class SqlcodeWrapperCleanupComposer:  
    OUTER_IF_TEXT = "IF NOT SQLCODE = 100"  
    INNER_IF_TEXT = "IF SQLCODE = 100"  
    REMOVED_OBTAIN_NEXT_TEXT = (  
        "DB2: REMOVED REDUNDANT OBTAIN NEXT AFTER CURSOR EOC LOOP"  
    )  
    CLEANUP_COMMENT_TEXT = (  
        "DB2: Removed redundant SQLCODE wrapper after cursor EOC loop."  
    )  
  
    BASE_INDENT = "    "  
    CHILD_INDENT = "        "  
    BODY_WIDTH = 65  
  
    MOVE_TO_PATTERN = re.compile(  
        r"^MOVE\s+(?P<source>.+?)\s+TO\s+(?P<target>.+?\.\.?)$",  
        flags=re.IGNORECASE,  
    )  
  
    def compose(  
        self,  
        text: str,  
    ) -> str:  
        if not text:  
            return ""  
  
        lines = self._normalize_line_endings(text).splitlines()  
  
        output: list[str] = []  
        index = 0  
  
        while index < len(lines):  
            logical = self._logical(lines[index]).upper()  
  
            if self.OUTER_IF_TEXT not in logical:  
                output.append(lines[index])  
                index += 1  
                continue  
  
            block = self._try_extract_redundant_wrapper_block(  
                lines=lines,  
                start_index=index,  
            )  
  
            if block is None:  
                output.append(lines[index])  
                index += 1  
                continue  
  
            replacement_lines, next_index = block  
  
            output.extend(replacement_lines)  
            index = next_index  
  
        return "\n".join(output).rstrip() + "\n"  
  
    def _normalize_line_endings(  
        self,  
        text: str,  
    ) -> str:
```

```

        return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

def _logical(
    self,
    line: str,
) -> str:
    return strip_sequence_numbers(str(line or "")).strip()

def _try_extract_redundant_wrapper_block(
    self,
    lines: list[str],
    start_index: int,
) -> tuple[list[str], int] | None:
    outer_line = lines[start_index]

    marker_index = self._find_marker_after_outer_if(
        lines=lines,
        start_index=start_index + 1,
    )

    if marker_index < 0:
        return None

    inner_if_index = self._find_inner_if_sqlcode_100(
        lines=lines,
        start_index=marker_index + 1,
    )

    if inner_if_index < 0:
        return None

    inner_end_index = self._find_matching_end_if(
        lines=lines,
        if_index=inner_if_index,
    )

    if inner_end_index < 0:
        return None

    outer_end_index = self._find_matching_end_if(
        lines=lines,
        if_index=start_index,
    )

    if outer_end_index < 0:
        return None

    if outer_end_index <= inner_end_index:
        return None

    inner_body = lines[inner_if_index + 1 : inner_end_index]

    normalized_inner_body = self._normalize_inner_body(
        lines=inner_body,
    )

    replacement: list[str] = [
        self._format_comment_like_line(
            reference_line=outer_line,
            comment_text=self.CLEANUP_COMMENT_TEXT,
        )
    ]

    replacement.extend(normalized_inner_body)

    return replacement, outer_end_index + 1

def _find_marker_after_outer_if(
    self,
    lines: list[str],
    start_index: int,
) -> int:
    end_index = min(
        len(lines),
        start_index + 8,
    )

```

```

    for index in range(start_index, end_index):
        logical = self._logical(lines[index]).upper()

        if self.REMOVED_OBTAIN_NEXT_TEXT in logical:
            return index

    return -1

def _find_inner_if_sqlcode_100(
    self,
    lines: list[str],
    start_index: int,
) -> int:
    end_index = min(
        len(lines),
        start_index + 12,
    )

    for index in range(start_index, end_index):
        logical = self._logical(lines[index]).upper()

        if logical.startswith(self.INNER_IF_TEXT):
            return index

    return -1

def _find_matching_end_if(
    self,
    lines: list[str],
    if_index: int,
) -> int:
    depth = 0

    for index in range(if_index, len(lines)):
        logical = self._logical(lines[index]).upper().rstrip(".")

        if logical.startswith("IF "):
            depth += 1
            continue

        if logical == "END-IF":
            depth -= 1

        if depth == 0:
            return index

    return -1

def _normalize_inner_body(
    self,
    lines: list[str],
) -> list[str]:
    output: list[str] = []
    first_content_seen = False

    for line in lines:
        logical = self._logical(line)

        if not logical:
            output.append(line)
            continue

        if logical.startswith("*") or logical.startswith("/"):
            output.append(line)
            continue

        if not first_content_seen:
            output.extend(
                self._normalized_lines_for_logical(
                    original_line=line,
                    logical=logical,
                    indent=self.BASE_INDENT,
                )
            )
        first_content_seen = True

```

```

        continue

    if logical.upper().rstrip(".") == "END-IF":
        output.extend(
            self._normalized_lines_for_logical(
                original_line=line,
                logical="END-IF",
                indent=self.BASE_INDENT,
            )
        )
        continue

    output.extend(
        self._normalized_lines_for_logical(
            original_line=line,
            logical=logical,
            indent=self.CHILD_INDENT,
        )
    )

    return output

def _normalized_lines_for_logical(
    self,
    original_line: str,
    logical: str,
    indent: str,
) -> list[str]:
    wrapped_bodies = self._wrap_logical_body(
        logical=logical,
        indent=indent,
    )

    return [
        self._replace_body_part(
            original_line=original_line,
            new_body=body,
        )
        for body in wrapped_bodies
    ]

def _wrap_logical_body(
    self,
    logical: str,
    indent: str,
) -> list[str]:
    text = str(logical or "").strip()

    if not text:
        return [indent.rstrip()]

    move_lines = self._wrap_move_statement(
        logical=text,
        indent=indent,
    )

    if move_lines:
        return move_lines

    body = indent + text

    if len(body) <= self.BODY_WIDTH:
        return [body]

    return self._wrap_by_words(
        body=body,
        continuation_indent=indent + "    ",
    )

def _wrap_move_statement(
    self,
    logical: str,
    indent: str,
) -> list[str]:
    match = self.MOVE_TO_PATTERN.match(str(logical or "").strip())

```

```

if not match:
    return []

source = str(match.group("source") or "").strip()
target = str(match.group("target") or "").strip()

if not source or not target:
    return []

force_split = " OF DCL" in source.upper()

one_line = f"{indent}MOVE {source} TO {target}"

if len(one_line) <= self.BODY_WIDTH and not force_split:
    return [one_line]

first_line = f"{indent}MOVE {source}"
second_line = f"{indent}TO {target}"

output: list[str] = []

if len(first_line) <= self.BODY_WIDTH:
    output.append(first_line)
else:
    output.extend(
        self._wrap_by_words(
            body=first_line,
            continuation_indent=indent + "    ",
        )
    )

if len(second_line) <= self.BODY_WIDTH:
    output.append(second_line)
else:
    output.extend(
        self._wrap_by_words(
            body=second_line,
            continuation_indent=indent + "    ",
        )
    )

return output

def _wrap_by_words(
    self,
    body: str,
    continuation_indent: str,
) -> list[str]:
    text = str(body or "").rstrip()

    if len(text) <= self.BODY_WIDTH:
        return [text]

    words = text.strip().split()
    output: list[str] = []
    current = ""

    for word in words:
        candidate = word if not current else current.rstrip() + " " + word

        if len(candidate) <= self.BODY_WIDTH:
            current = candidate
            continue

        if current:
            if output:
                output.append(continuation_indent + current.strip())
            else:
                output.append(current.strip())

        current = word

    if current:
        if output:
            output.append(continuation_indent + current.strip())
        else:

```

```

        output.append(current.strip())

    if not output:
        return [text[: self.BODY_WIDTH]]

    normalized: list[str] = []

    for index, line in enumerate(output):
        if index == 0:
            normalized.append(line[: self.BODY_WIDTH])
            continue

        if line.startswith(continuation_indent):
            normalized.append(line[: self.BODY_WIDTH])
        else:
            normalized.append(
                (continuation_indent + line.strip())[: self.BODY_WIDTH]
            )

    return normalized

def _replace_body_part(
    self,
    original_line: str,
    new_body: str,
) -> str:
    line = str(original_line or "").rstrip()

    if self._is_fixed_format_line(line):
        left = line[:6]
        indicator = line[6]
        right = line[72:80]
        body = str(new_body or "").rstrip()[: self.BODY_WIDTH].ljust(
            self.BODY_WIDTH
        )

        return f"{left}{indicator}{body}{right}"

    return str(new_body or "").rstrip()

def _format_comment_like_line(
    self,
    reference_line: str,
    comment_text: str,
) -> str:
    line = str(reference_line or "").rstrip()
    clean_comment = str(comment_text or "").strip()

    if not clean_comment.upper().startswith("DB2:"):
        clean_comment = f"DB2: {clean_comment}"

    body = f" {clean_comment}"

    if self._is_fixed_format_line(line):
        left = line[:6]
        right = line[72:80]
        body_area = body[: self.BODY_WIDTH].ljust(self.BODY_WIDTH)

        return f"{left}*{body_area}{right}"

    return f"*{body}"

def _is_fixed_format_line(
    self,
    line: str,
) -> bool:
    text = str(line or "")

    if len(text) < 80:
        return False

    if not text[:6].isdigit():
        return False

    if not text[72:80].isdigit():
        return False

```



```
return True
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\composers\update_restart_skip_composer.py
ENCODING: utf-8
=====
```

```
"""
```

```
Update restart/control skip composer.
```

```
Purpose:
```

- Do not hardcode FFRECAP to any DB2 restart table.
- Do not invent restart DB2 SQL.
- When generated output contains missing-mapping conversion blocks for restart/control-like IDMS records, replace those blocks with a clean manual-redesign comment and CONTINUE.

```
This is generic:
```

- If a record has valid Sheet Mapping + DCLGEN metadata, normal conversion should already have happened and this composer does nothing.
- If a record has missing mapping metadata, this composer removes the invalid generated SQLCODE checks because no SQL was executed.

```
"""
```

```
from __future__ import annotations
```

```
import re
```

```
from patterns.sequence_patterns import strip_sequence_numbers
```

```
class UpdateRestartSkipComposer:
```

```
    """
```

```
    Cleans generated missing-mapping blocks for unmapped IDMS records.
```

```
    This is intentionally conservative and only acts when the generated converter already emitted clear missing-mapping diagnostics.
```

```
    """
```

```
    CONVERTED_FOR_PATTERN = re.compile(
        r"^\s*\*\s*DB2:\s*Converted\s+"
        r"(?P<operation>OBTAIN\s+CALC|STORE|MODIFY|ERASE|DELETE|UPDATE|INSERT)"
        r"\s+(?:for\s+)?(?:P<record>[A-Z0-9-]+\.)?\s*$",
        flags=re.IGNORECASE,
    )
```

```
    MISSING_MAPPING_MARKERS = (
        "CONVERSION SKIPPED BECAUSE SHEET MAPPING ENTRY DOES NOT EXIST",
        "MISSING SHEET MAPPING METADATA",
        "MISSING SELECT",
        "MISSING INSERT",
        "MISSING UPDATE",
        "MISSING DELETE",
        "MISSING MAPPING",
    )
```

```
    RESTART_CONTROL_HINTS = (
        "RECAP",
        "RESTART",
        "RST",
        "CONTROL",
        "CTRL",
        "CHECKPOINT",
        "CHKPT",
    )
```

```
    SQLCODE_IF_PATTERN = re.compile(
        r"^\s*IF\s+SQLCODE\b",
        flags=re.IGNORECASE,
    )
```

```
    END_IF_PATTERN = re.compile(
        r"^\s*END-IF\.\?\s*$",
        flags=re.IGNORECASE,
```

```

)

CONTINUE_PATTERN = re.compile(
    r"^\s*CONTINUE\.\?\s*$",
    flags=re.IGNORECASE,
)

def __init__(
    self,
) -> None:
    self.messages: list[str] = []

def compose(
    self,
    text: str,
) -> str:
    self.messages = []

    if not text:
        return ""

    lines = self._normalize_line_endings(text).splitlines()
    output: list[str] = []
    index = 0

    while index < len(lines):
        line = lines[index]
        logical = self._logical(line)

        converted_match = self.CONVERTED_FOR_PATTERN.match(logical)

        if not converted_match:
            output.append(line)
            index += 1
            continue

        record_name = str(converted_match.group("record") or "").upper()
        block_end = self._missing_mapping_block_end(
            lines=lines,
            start_index=index,
        )

        if block_end <= index:
            output.append(line)
            index += 1
            continue

        block_lines = lines[index:block_end]

        if not self._contains_missing_mapping_marker(block_lines):
            output.append(line)
            index += 1
            continue

        output.extend(
            self._replacement_block(record_name)
        )

        index = block_end

        index = self._skip_following_sqlcode_blocks(
            lines=lines,
            start_index=index,
        )

        self.messages.append(
            f"Update restart/control skip: unmapped IDMS record {record_name} was skipped for manual redesign."
        )

    return "\n".join(output).rstrip() + "\n"

def _missing_mapping_block_end(
    self,
    lines: list[str],
    start_index: int,
) -> int:

```

```

index = start_index + 1

while index < len(lines):
    logical = self._logical(lines[index])
    upper = logical.upper()

    if not logical:
        index += 1
        continue

    if upper.startswith("*DB2:") or upper.startswith("* DB2:"):
        index += 1
        continue

    if self.CONTINUE_PATTERN.match(logical):
        index += 1
        break

    break

return index

def _contains_missing_mapping_marker(
    self,
    lines: list[str],
) -> bool:
    combined = "\n".join(
        self._logical(line).upper()
        for line in lines
    )

    return any(
        marker in combined
        for marker in self.MISSING_MAPPING_MARKERS
    )

def _replacement_block(
    self,
    record_name: str,
) -> list[str]:
    if self._looks_like_restart_or_control(record_name):
        return [
            f"* DB2: IDMS record {record_name} was not converted automatically.",
            "* DB2: Missing Sheet Mapping and DCLGEN metadata.",
            "* DB2: Restart/control logic requires manual DB2 redesign.",
            "CONTINUE.",
        ]

    return [
        f"* DB2: IDMS record {record_name} was not converted automatically.",
        "* DB2: Missing Sheet Mapping and DCLGEN metadata.",
        "* DB2: No one-to-one DB2 SQL conversion generated.",
        "CONTINUE.",
    ]

def _looks_like_restart_or_control(
    self,
    record_name: str,
) -> bool:
    normalized = str(record_name or "").upper()

    return any(
        hint in normalized
        for hint in self.RESTART_CONTROL_HINTS
    )

def _skip_following_sqlcode_blocks(
    self,
    lines: list[str],
    start_index: int,
) -> int:
    index = start_index

    while index < len(lines):
        logical = self._logical(lines[index])

```

```

        if not logical:
            index += 1
            continue

        if not self.SQLCODE_IF_PATTERN.match(logical):
            break

        index = self._skip_if_block(
            lines=lines,
            start_index=index,
        )

    return index

def _skip_if_block(
    self,
    lines: list[str],
    start_index: int,
) -> int:
    index = start_index
    depth = 0

    while index < len(lines):
        logical = self._logical(lines[index])
        upper = logical.upper()

        if upper.startswith("IF "):
            depth += 1

        if self.END_IF_PATTERN.match(logical):
            depth -= 1
            index += 1

            if depth <= 0:
                return index

            continue

        index += 1

    return index

def _normalize_line_endings(
    self,
    text: str,
) -> str:
    return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

def _logical(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    try:
        stripped = strip_sequence_numbers(text)
        if stripped:
            return stripped.strip()
    except Exception:
        pass

    if len(text) >= 80 and text[:6].isdigit() and text[72:80].isdigit():
        return text[7:72].strip()

    if len(text) > 6 and text[:6].isdigit():
        text = text[6:].strip()

    if len(text) >= 8 and text[-8:].isdigit():
        text = text[:-8].rstrip()

    return text.strip()

```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\config\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\config\app_config.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\config\app_settings.py
ENCODING: utf-8
=====
```

"""

Application settings.

Keep app-level settings outside parser, transformer, generator, and service logic.

APP_NAME = "IDMS DB2 Phase 2 Converter"

APP_DESCRIPTION = (
 "Standalone Phase 2 project for converting IDMS COBOL programs "
 "into DB2 embedded SQL COBOL."
)

DEFAULT_STREAMLIT_PORT = 8502

DEFAULT_AUTO_FIX_PIC_LENGTH_MISMATCHES = False

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\config\logging_settings.py
ENCODING: utf-8
=====
```

"""

Logging settings.

Log files are written to the project logs folder with date-time names.

LOG_FILE_PREFIX = "idms_db2_conversion"

LOG_DATE_TIME_FORMAT = "%d-%m-%Y %H%M%S"

LOG_MESSAGE_FORMAT = "%(asctime)s | %(levelname)s | %(name)s | %(message)s"

LOG_DATE_FORMAT = "%d-%m-%Y %H:%M:%S"

DEFAULT_LOG_LEVEL = "INFO"

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\config\path_config.py
ENCODING: utf-8
=====
```

"""

Path settings for local command-line runs.

These paths are intentionally centralized here instead of being embedded in runner, parser, or conversion logic.

Update these values for your local environment.

"""

```
from pathlib import Path
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[1]
```

```
SRC_DIR = PROJECT_ROOT / "src"
```

```
LOGS_DIR = PROJECT_ROOT / "logs"
```

```
DEFAULT_INPUT_DIR = Path(r"C:\S-Input")
```

```
DEFAULT_OUTPUT_DIR = DEFAULT_INPUT_DIR / "Output"
```

```
DEFAULT_SHEET_MAPPING_PATH = DEFAULT_INPUT_DIR / "Excel_Sheet_mapping.csv"
```

```
DEFAULT_RETRIEVAL_SOURCE_PATH = DEFAULT_INPUT_DIR / "Retrieval.txt"
```

```
DEFAULT_UPDATE_SOURCE_PATH = DEFAULT_INPUT_DIR / "Update.txt"
```

```
DEFAULT_DCLGEN_CANDIDATE_PATHS = [  
    DEFAULT_INPUT_DIR / "DCLGENS_BEFF.txt",  
    DEFAULT_INPUT_DIR / "DCLGENS_BFAR.txt",  
    DEFAULT_INPUT_DIR / "DCLGENS_EVEF.txt",  
]
```

```
DEFAULT_COPYBOOK_CANDIDATE_PATHS = [  
    DEFAULT_INPUT_DIR / "Copybook.txt",  
]
```

```
=====
```

FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\config\path_settings.py

ENCODING: utf-8

```
=====
```

[Empty file]

```
=====
```

FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\domain__init__.py

ENCODING: utf-8

```
=====
```

[Empty file]

```
=====
```

FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\domain\enums.py

ENCODING: utf-8

```
=====
```

[Empty file]

```
=====
```

FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\domain\models.py

ENCODING: utf-8

```
=====
```

```
from dataclasses import dataclass, field
```

```
@dataclass
```

```

class SheetMappingRow:
    cobol_record_idms: str = ""
    cobol_zone: str = ""
    idms_key: str = ""
    idms_pic_clause: str = ""
    length_of_field_bytes: str = ""
    field_end_position: str = ""
    db2_key: str = ""
    new_db2_record: str = ""
    new_db2_field_name: str = ""
    new_db2_data_type: str = ""
    hopex_expression_type_remark: str = ""
    remarks: str = ""
    relation: str = ""
    reference_field_name_copybook: str = ""
    reference_field_pic_clause: str = ""
    cross_application_db2_table: str = ""
    cross_application_db2_field_name: str = ""
    cross_application_db2_data_type: str = ""
    basetype: str = ""

@dataclass
class DclgenColumn:
    table_name: str = ""
    column_name: str = ""
    db2_type: str = ""
    cobol_host_name: str = ""
    cobol_picture: str = ""
    cobol_usage: str = ""
    nullable: bool = True

@dataclass
class CopybookField:
    level: str = ""
    name: str = ""
    picture: str = ""
    usage: str = ""
    occurs: str = ""

@dataclass
class IdmsOperation:
    operation: str
    record_name: str = ""
    set_name: str = ""
    line_number: int = 0
    raw_line: str = ""

@dataclass
class RelationshipSummary:
    relation: str = ""
    parent_record: str = ""
    child_record: str = ""
    parent_key: str = ""
    child_key: str = ""

@dataclass
class RecordSummary:
    record_name: str
    db2_table: str
    column_count: int
    key_columns: list[str] = field(default_factory=list)

@dataclass
class ConversionInput:
    sheet_mapping_rows: list[SheetMappingRow] = field(default_factory=list)
    dclgen_columns: list[DclgenColumn] = field(default_factory=list)
    copybook_fields: list[CopybookField] = field(default_factory=list)
    idms_cobol_text: str = ""
    target_program_id: str = ""
    auto_fix_pic_length_mismatches: bool = False

```

```

@dataclass
class ConversionResult:
    converted_cobol: str = ""
    validation_messages: list[str] = field(default_factory=list)
    operations: list[IdmsOperation] = field(default_factory=list)

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\domain\result_models.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\generators\__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\generators\cursor_paragraph_generator.py
ENCODING: utf-8
=====

import re

from catalogs.output_sections import DB2_CURSOR_PARAGRAPH_MARKER
from idms_db2_phase2.domain.models import IdmsOperation
from idms_db2_phase2.generators.db2_infrastructure_generator import (
    Db2InfrastructureGenerator,
)
from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from patterns.db2_patterns import END_PROGRAM_PATTERN

class CursorParagraphGenerator:
    """
    Generates DB2 cursor OPEN / FETCH / CLOSE paragraphs.

    Important:
    - Cursor declarations are generated by Db2InfrastructureGenerator.
    - FETCH INTO host variables must match the SELECT columns used in those
      declarations.
    - Therefore this generator reuses
      db2_infrastructure_generator.last_cursor_specs whenever available.
    """

    SQLERROR_PARAGRAPH_NAME = "SQLERROR"

    def __init__(
        self,
        db2_infrastructure_generator: Db2InfrastructureGenerator,
        host_variable_resolver: HostVariableResolver,
        sql_error_generator: SqlErrorGenerator,
    ) -> None:
        self.db2_infrastructure_generator = db2_infrastructure_generator
        self.host_variable_resolver = host_variable_resolver
        self.sql_error_generator = sql_error_generator
        self.messages: list[str] = []

    def apply(
        self,
        cobol_text: str,
        operations: list[IdmsOperation],
    ) -> tuple[str, list[str]]:
        self.messages = []

        text = str(cobol_text or "")

```



```

if not text:
    return text, self.messages

cursor_specs = self._cursor_specs(
    operations=operations,
)

if not cursor_specs:
    self.messages.append(
        "DB2 cursor paragraphs: no cursor operations found."
    )
    return text, self.messages

sql_error_paragraph = self.SQLERROR_PARAGRAPH_NAME

updated_text = self._normalize_existing_sql_error_references(text)

updated_text = self._rewrite_cursor_perform_calls(
    text=updated_text,
    cursor_specs=cursor_specs,
)

if DB2_CURSOR_PARAGRAPH_MARKER in updated_text:
    self.messages.append(
        "DB2 cursor paragraphs: generated cursor paragraph block already exists."
    )
    return updated_text, self.messages

block = self.paragraph_block(
    cursor_specs=cursor_specs,
    sql_error_paragraph=sql_error_paragraph,
)

updated_text = self._insert_paragraph_block(
    text=updated_text,
    block=block,
)

self.messages.append(
    f"DB2 cursor paragraphs: generated {len(cursor_specs)} cursor paragraph set(s)."
)

for spec in cursor_specs:
    if not list(spec.get("host_variables", [])):
        self.messages.append(
            "DB2 cursor paragraphs: no FETCH host variables resolved for "
            f"cursor {spec.get('cursor_name', '')} "
            f"record {spec.get('record_name', '')} "
            f"table {spec.get('table_name', '')}."
        )

return updated_text, self.messages

def _cursor_specs(
    self,
    operations: list[IdmsOperation],
) -> list[dict[str, object]]:
    cached_specs = getattr(
        self.db2_infrastructure_generator,
        "last_cursor_specs",
        [],
    )

    if cached_specs:
        return cached_specs

    return self.db2_infrastructure_generator.cursor_specs(
        operations
    )

def paragraph_block(
    self,
    cursor_specs: list[dict[str, object]],
    sql_error_paragraph: str,
) -> str:
    lines: list[str] = []

```

```

lines.extend(
    self._comment_block(DB2_CURSOR_PARAGRAPH_MARKER)
)

lines.append("")

for spec in cursor_specs:
    cursor_name = str(spec.get("cursor_name", ""))
    open_paragraph = str(spec.get("open_paragraph", ""))
    fetch_paragraph = str(spec.get("fetch_paragraph", ""))
    close_paragraph = str(spec.get("close_paragraph", ""))
    host_variables = list(spec.get("host_variables", []))

    if not cursor_name:
        continue

    if open_paragraph:
        lines.extend(
            self.open_paragraph(
                cursor_name=cursor_name,
                paragraph_name=open_paragraph,
                sql_error_paragraph=sql_error_paragraph,
            )
        )
        lines.append("")

    if fetch_paragraph:
        lines.extend(
            self.fetch_paragraph(
                cursor_name=cursor_name,
                paragraph_name=fetch_paragraph,
                host_variables=host_variables,
                sql_error_paragraph=sql_error_paragraph,
            )
        )
        lines.append("")

    if close_paragraph:
        lines.extend(
            self.close_paragraph(
                cursor_name=cursor_name,
                paragraph_name=close_paragraph,
                sql_error_paragraph=sql_error_paragraph,
            )
        )
        lines.append("")

return "\n".join(lines).rstrip() + "\n"

def open_paragraph(
    self,
    cursor_name: str,
    paragraph_name: str,
    sql_error_paragraph: str,
) -> list[str]:
    return [
        f"{paragraph_name}.",
        f"MOVE '{paragraph_name}' TO SQL-LOCATION.",
        "EXEC SQL",
        f"    OPEN {cursor_name}",
        "END-EXEC.",
        "EVALUATE SQLCODE",
        "    WHEN ZERO",
        f"        SET {cursor_name}-NOT-EOC TO TRUE",
        "    WHEN OTHER",
        f"        DISPLAY 'ERROR WHILE OPENING CURSOR {cursor_name}'.",
        f"        PERFORM {sql_error_paragraph}.",
        "END-EVALUATE.",
    ]

def fetch_paragraph(
    self,
    cursor_name: str,
    paragraph_name: str,
    host_variables: list[str],

```

```

        sql_error_paragraph: str,
    ) -> list[str]:
        lines: list[str] = [
            f"{paragraph_name}.",
            f"MOVE '{paragraph_name}' TO SQL-LOCATION.",
            "EXEC SQL",
            f"    FETCH {cursor_name}",
        ]

        clean_hosts = self._clean_host_variables(host_variables)

        if clean_hosts:
            lines.append("    INTO")
            lines.extend(
                self._fetch_host_lines(clean_hosts)
            )

        lines.extend(
            [
                "END-EXEC.",
                "EVALUATE SQLCODE",
                "    WHEN ZERO",
                "        CONTINUE",
                "    WHEN 100",
                f"        SET {cursor_name}-EOC TO TRUE",
                "    WHEN OTHER",
                f"        DISPLAY 'ERROR WHILE FETCHING CURSOR {cursor_name}'.",
                f"        PERFORM {sql_error_paragraph}.",
                "END-EVALUATE.",
            ]
        )

        return lines

def close_paragraph(
    self,
    cursor_name: str,
    paragraph_name: str,
    sql_error_paragraph: str,
) -> list[str]:
    return [
        f"{paragraph_name}.",
        f"MOVE '{paragraph_name}' TO SQL-LOCATION.",
        "EXEC SQL",
        f"    CLOSE {cursor_name}",
        "END-EXEC.",
        "EVALUATE SQLCODE",
        "    WHEN ZERO",
        "        CONTINUE",
        "    WHEN OTHER",
        f"        DISPLAY 'ERROR WHILE CLOSING CURSOR {cursor_name}'.",
        f"        PERFORM {sql_error_paragraph}.",
        "END-EVALUATE.",
    ]

def _rewrite_cursor_perform_calls(
    self,
    text: str,
    cursor_specs: list[dict[str, object]],
) -> str:
    updated = str(text or "")

    for spec in cursor_specs:
        cursor_name = str(spec.get("cursor_name", "")).strip()
        open_paragraph = str(spec.get("open_paragraph", "")).strip()
        fetch_paragraph = str(spec.get("fetch_paragraph", "")).strip()
        close_paragraph = str(spec.get("close_paragraph", "")).strip()

        if not cursor_name:
            continue

        replacements = {
            "OPEN": open_paragraph,
            "FETCH": fetch_paragraph,
            "CLOSE": close_paragraph,
        }

```

```

    for operation, target_paragraph in replacements.items():
        if not target_paragraph:
            continue

        updated = self._replace_unnumbered_perform(
            text=updated,
            operation=operation,
            cursor_name=cursor_name,
            target_paragraph=target_paragraph,
        )

        updated = self._replace_numbered_perform(
            text=updated,
            operation=operation,
            cursor_name=cursor_name,
            target_paragraph=target_paragraph,
        )

        updated = self._replace_legacy_spec_perform_calls(
            text=updated,
            spec=spec,
        )

    return updated

def _replace_unnumbered_perform(
    self,
    text: str,
    operation: str,
    cursor_name: str,
    target_paragraph: str,
) -> str:
    pattern = re.compile(
        rf"(?P<prefix>\bPERFORM\s+)"
        rf"{operation}-{re.escape(cursor_name)}"
        rf"(?P<suffix>\.?)",
        flags=re.IGNORECASE,
    )

    return pattern.sub(
        rf"\g<prefix>{target_paragraph}\g<suffix>",
        text,
    )

def _replace_numbered_perform(
    self,
    text: str,
    operation: str,
    cursor_name: str,
    target_paragraph: str,
) -> str:
    pattern = re.compile(
        rf"(?P<prefix>\bPERFORM\s+)"
        rf"d{{3}}-{operation}-{re.escape(cursor_name)}"
        rf"(?P<suffix>\.?)",
        flags=re.IGNORECASE,
    )

    return pattern.sub(
        rf"\g<prefix>{target_paragraph}\g<suffix>",
        text,
    )

def _replace_legacy_spec_perform_calls(
    self,
    text: str,
    spec: dict[str, object],
) -> str:
    updated = str(text or "")

    legacy_replacements = {
        str(spec.get("old_open_paragraph", "")).strip(): str(
            spec.get("open_paragraph", "")
        ).strip(),
        str(spec.get("old_fetch_paragraph", "")).strip(): str(

```

```

        spec.get("fetch_paragraph", "")
    ).strip(),
    str(spec.get("old_close_paragraph", "").strip(): str(
        spec.get("close_paragraph", "")
    ).strip(),
}

for old_paragraph, new_paragraph in legacy_replacements.items():
    if not old_paragraph:
        continue

    if not new_paragraph:
        continue

    if old_paragraph == new_paragraph:
        continue

    pattern = re.compile(
        rf"(?P<prefix>\bPERFORM\s+)"
        rf"{re.escape(old_paragraph)}"
        rf"(?P<suffix>\.?)",
        flags=re.IGNORECASE,
    )

    updated = pattern.sub(
        rf"\g<prefix>{new_paragraph}\g<suffix>",
        updated,
    )

    return updated

def _normalize_existing_sql_error_references(
    self,
    text: str,
) -> str:
    updated = str(text or "")

    updated = re.sub(
        r"\bPERFORM\s+SQL-ERROR\s*\.\.?",
        "PERFORM SQLERROR.",
        updated,
        flags=re.IGNORECASE,
    )

    updated = re.sub(
        r"(?m)^(\\s*)(?:\\d{6}\\s+)?SQL-ERROR\\.\\s*(?:\\d{8})?\\s*$",
        r"\\1SQLERROR.",
        updated,
        flags=re.IGNORECASE,
    )

    return updated

def _clean_host_variables(
    self,
    host_variables: list[str],
) -> list[str]:
    output: list[str] = []

    for host in host_variables:
        clean_host = self.host_variable_resolver.normalize_host_reference(
            host
        )

        if not clean_host:
            continue

        if clean_host in output:
            continue

        output.append(clean_host)

    return output

def _fetch_host_lines(
    self,

```

```

        host_variables: list[str],
    ) -> list[str]:
        output: list[str] = []

        for index, host in enumerate(host_variables):
            suffix = "," if index < len(host_variables) - 1 else ""
            output.append(f"        {host}{suffix}")

        return output

def _insert_paragraph_block(
    self,
    text: str,
    block: str,
) -> str:
    end_program_match = END_PROGRAM_PATTERN.search(text)

    if end_program_match:
        return (
            text[: end_program_match.start()].rstrip()
            + "\n\n"
            + block.rstrip()
            + "\n\n"
            + text[end_program_match.start():].lstrip()
        ).rstrip() + "\n"

    return text.rstrip() + "\n\n" + block.rstrip() + "\n"

def _comment_block(
    self,
    title: str,
) -> list[str]:
    return [
        f"* {title:<62}*",
    ]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\generators\db2_infrastructure_generator.py
ENCODING: utf-8
=====

```

```

import re

from catalogs.output_sections import (
    DB2_CURSOR_DECLARATIONS_MARKER,
    DB2_CURSOR_FLAGS_MARKER,
    DB2_INFRASTRUCTURE_MARKER,
    DB2_SQL_ERROR_LOCATION_MARKER,
    SQLCA_INCLUDE_NAME,
    SQLERRWS_INCLUDE_NAME,
)

try:
    from catalogs.output_sections import (
        SQL_LOCATION_FIELD_NAME,
        SQL_LOCATION_PICTURE,
    )
except ImportError:
    SQL_LOCATION_FIELD_NAME = "SQL-LOCATION"
    SQL_LOCATION_PICTURE = "PIC X(40) VALUE SPACES."

from idms_db2_phase2.analyzers.field_usage_analyzer import FieldUsageAnalyzer
from idms_db2_phase2.domain.models import IdmsOperation
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_column_resolver import CursorColumnResolver
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.relationship_resolver import RelationshipResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer

class Db2InfrastructureGenerator:
    """
    Generates DB2 infrastructure in DATA DIVISION.

```

```

Generic behavior:
- Include SQLERRWS and SQLCA.
- Include DCLGEN views/tables actually used by cursor specs.
- Generate SQL-LOCATION.
- Generate cursor EOC flags using PIC X and 88-level values.
- Generate cursor declarations using WITH HOLD.
- Reduce SELECT columns using actual field usage and relationship keys.
- Generate child cursor WHERE from FK mapping.
- Generate child cursor ORDER BY from non-FK primary/order key.
- Do not generate parent/root ORDER BY unless explicitly required.
- Add FOR READ ONLY.
- Initialize generated DCLGEN groups in PROCEDURE DIVISION.
"""

CURSOR_OPERATIONS = {
    "OBTAIN_FIRST",
    "OBTAIN_NEXT",
    "FIND_FIRST",
}

PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^s*(?:\d{6}\s+)?PROCEDURE\s+DIVISION\b.*\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

LINKAGE_SECTION_PATTERN = re.compile(
    r"^s*(?:\d{6}\s+)?LINKAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

WORKING_STORAGE_PATTERN = re.compile(
    r"^s*(?:\d{6}\s+)?WORKING-STORAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

DATA_DIVISION_PATTERN = re.compile(
    r"^s*(?:\d{6}\s+)?DATA\s+DIVISION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

def __init__(
    self,
    table_name_resolver: TableNameResolver,
    column_name_resolver: ColumnNameResolver,
    host_variable_resolver: HostVariableResolver,
    cursor_name_resolver: CursorNameResolver,
) -> None:
    self.table_name_resolver = table_name_resolver
    self.column_name_resolver = column_name_resolver
    self.host_variable_resolver = host_variable_resolver
    self.cursor_name_resolver = cursor_name_resolver

    self.mapping_repository = table_name_resolver.mapping_repository

    self.relationship_resolver = RelationshipResolver(
        mapping_repository=self.mapping_repository,
        table_name_resolver=self.table_name_resolver,
        host_variable_resolver=self.host_variable_resolver,
    )

    self.cursor_column_resolver = CursorColumnResolver(
        mapping_repository=self.mapping_repository,
        table_name_resolver=self.table_name_resolver,
        column_name_resolver=self.column_name_resolver,
        relationship_resolver=self.relationship_resolver,
    )

    self.last_cursor_specs: list[dict[str, object]] = []

def apply(
    self,
    cobol_text: str,
    operations: list[IdmsOperation],
) -> tuple[str, list[str]]:
    messages: list[str] = []

```

```

text = str(cobol_text or "")

if not text:
    self.last_cursor_specs = []
    return text, messages

field_usage_analysis = FieldUsageAnalyzer(
    mapping_repository=self.mapping_repository,
    table_name_resolver=self.table_name_resolver,
).analyze(text)

messages.extend(field_usage_analysis.diagnostics)

cursor_specs = self.cursor_specs(
    operations=operations,
    cobol_text=text,
    field_usage_analysis=field_usage_analysis,
)

self.last_cursor_specs = cursor_specs

include_names = self.include_names(cursor_specs)

if include_names:
    messages.append(
        "DB2 infrastructure: DCLGEN includes selected: "
        + ", ".join(include_names)
    )
else:
    messages.append(
        "DB2 infrastructure: no operation-specific DCLGEN includes resolved."
    )

for message in self._cursor_spec_messages(cursor_specs):
    messages.append(message)

if DB2_INFRASTRUCTURE_MARKER in text:
    messages.append(
        "DB2 infrastructure: existing generated DB2 infrastructure block detected; not inserted again."
    )
    updated = text
else:
    block = self.infrastructure_block(
        include_names=include_names,
        cursor_specs=cursor_specs,
    )

    updated = self._insert_in_data_division(
        text=text,
        block=block,
    )

    messages.append(
        "DB2 infrastructure: generated infrastructure block inserted."
    )

updated = self._ensure_dclgen_initialization(
    text=updated,
    include_names=include_names,
)

return updated, messages

def cursor_specs(
    self,
    operations: list[IdmsOperation],
    cobol_text: str = "",
    field_usage_analysis=None,
) -> list[dict[str, object]]:
    output: list[dict[str, object]] = []
    seen: set[tuple[str, str, str]] = set()
    cursor_order = 1

    if field_usage_analysis is None:
        field_usage_analysis = FieldUsageAnalyzer(
            mapping_repository=self.mapping_repository,

```



```

        table_name_resolver=self.table_name_resolver,
    ).analyze(cobol_text)

for operation in operations or []:
    operation_name = str(operation.operation or "").upper()

    if operation_name not in self.CURSOR_OPERATIONS:
        continue

    record_name = NameNormalizer.normalize(operation.record_name)
    set_name = NameNormalizer.normalize(operation.set_name)

    if not record_name:
        continue

    table_name = self.table_name_resolver.table_for_record(record_name)

    if not table_name:
        continue

    key = (
        set_name,
        record_name,
        table_name,
    )

    if key in seen:
        continue

    seen.add(key)

    cursor_name = self.cursor_name_resolver.cursor_name_from_table(
        table_name
    )

    paragraph_spec = self._cursor_paragraph_spec(
        cursor_order=cursor_order,
        table_name=table_name,
    )

    select_columns = self.cursor_column_resolver.select_columns_for_record(
        record_name=record_name,
        field_usage_analysis=field_usage_analysis,
    )

    relationship = self.relationship_resolver.resolve_for_child_record(
        record_name
    )

    where_conditions = self._where_conditions_from_relationship(
        relationship.conditions
    )

    order_by_columns = self.cursor_column_resolver.order_by_columns_for_record(
        record_name
    )

    host_variables = self.host_variable_resolver.host_references_for_columns(
        table_name=table_name,
        columns=select_columns,
    )

    output.append(
        {
            "set_name": set_name,
            "record_name": record_name,
            "table_name": table_name,
            "cursor_name": cursor_name,
            "select_columns": select_columns,
            "host_variables": host_variables,
            "where_conditions": where_conditions,
            "order_by_columns": order_by_columns,
            "open_paragraph": paragraph_spec["open_paragraph"],
            "fetch_paragraph": paragraph_spec["fetch_paragraph"],
            "close_paragraph": paragraph_spec["close_paragraph"],
            "cursor_order": cursor_order,
        }
    )

```

```

        }
    )

    cursor_order += 1

    return output

def include_names(
    self,
    cursor_specs: list[dict[str, object]],
) -> list[str]:
    output: list[str] = []
    seen: set[str] = set()

    for spec in cursor_specs or []:
        table_name = NameNormalizer.normalize(
            str(spec.get("table_name", ""))
        )

        if not table_name:
            continue

        include_name = self._normalize_include_name(table_name)

        if not include_name:
            continue

        if include_name in seen:
            continue

        seen.add(include_name)
        output.append(include_name)

    return output

def infrastructure_block(
    self,
    include_names: list[str],
    cursor_specs: list[dict[str, object]],
) -> str:
    lines: list[str] = []

    lines.extend(
        self._comment_block(DB2_INFRASTRUCTURE_MARKER)
    )

    lines.extend(
        self._include_lines(
            include_names=[
                SQLERRWS_INCLUDE_NAME,
                SQLCA_INCLUDE_NAME,
                *include_names,
            ]
        )
    )

    lines.append("")

    lines.extend(
        self._comment_block(DB2_SQL_ERROR_LOCATION_MARKER)
    )

    lines.append(
        f"01 {SQL_LOCATION_FIELD_NAME:<30} {SQL_LOCATION_PICTURE}"
    )

    if cursor_specs:
        lines.append("")
        lines.extend(
            self._comment_block(DB2_CURSOR_FLAGS_MARKER)
        )

        lines.extend(
            self._cursor_flag_lines(cursor_specs)
        )

```

```

        lines.append("")
        lines.extend(
            self._comment_block(DB2_CURSOR_DECLARATIONS_MARKER)
        )

        for spec in cursor_specs:
            lines.extend(
                self._cursor_declaration(spec)
            )
            lines.append("")

        return "\n".join(lines).rstrip() + "\n"

def _cursor_flag_lines(
    self,
    cursor_specs: list[dict[str, object]],
) -> list[str]:
    lines: list[str] = []

    for spec in cursor_specs:
        cursor_name = NameNormalizer.to_cobol(
            str(spec.get("cursor_name", ""))
        )

        if not cursor_name:
            continue

        flag_name = f"WS-{cursor_name}-FLAG"
        not_eoc_name = f"{cursor_name}-NOT-EOC"
        eoc_name = f"{cursor_name}-EOC"

        lines.append(
            f"01 {flag_name:<30} PIC X."
        )
        lines.append(
            f"      88 {not_eoc_name:<26} VALUE 'N'."
        )
        lines.append(
            f"      88 {eoc_name:<26} VALUE 'Y'."
        )
        lines.append("")

    return lines

def _cursor_declaration(
    self,
    spec: dict[str, object],
) -> list[str]:
    cursor_name = NameNormalizer.to_cobol(
        str(spec.get("cursor_name", ""))
    )

    table_name = NameNormalizer.normalize(
        str(spec.get("table_name", ""))
    )

    select_columns = [
        NameNormalizer.normalize(str(column))
        for column in list(spec.get("select_columns", []))
        if NameNormalizer.normalize(str(column))
    ]

    where_conditions = [
        str(condition or "").strip()
        for condition in list(spec.get("where_conditions", []))
        if str(condition or "").strip()
    ]

    order_by_columns = [
        str(column or "").strip()
        for column in list(spec.get("order_by_columns", []))
        if str(column or "").strip()
    ]

    if not cursor_name:
        return [

```

```

        "** DB2 WARNING: Unable to declare cursor; missing cursor name."
    ]

    if not table_name:
        return [
            f"** DB2 WARNING: Unable to declare cursor {cursor_name}; missing DB2 table mapping."
        ]

    if not select_columns:
        select_columns = ["*"]

    lines: list[str] = [
        "EXEC SQL",
        f"    DECLARE {cursor_name} CURSOR WITH HOLD FOR",
        "    SELECT",
    ]

    lines.extend(
        self._select_lines(
            columns=select_columns,
        )
    )

    lines.append(
        f"    FROM {table_name}"
    )

    if where_conditions:
        lines.append("    WHERE")
        lines.extend(
            self._and_lines(
                items=where_conditions,
                indent="    ",
            )
        )

    if order_by_columns:
        lines.append("    ORDER BY")
        lines.extend(
            self._comma_lines(
                items=order_by_columns,
                indent="    ",
            )
        )

    lines.append("    FOR READ ONLY")
    lines.append("END-EXEC.")

    return lines

def _where_conditions_from_relationship(
    self,
    conditions,
) -> list[str]:
    output: list[str] = []

    for condition in conditions or []:
        child_column = NameNormalizer.normalize(
            getattr(condition, "child_column", "")
        )
        parent_host = str(
            getattr(condition, "parent_host_reference", "") or ""
        ).strip()

        if not child_column or not parent_host:
            continue

        output.append(
            f"{child_column} = {parent_host}"
        )

    return output

def _select_lines(
    self,
    columns: list[str],

```

```

) -> list[str]:
    clean_columns = [
        NameNormalizer.normalize(column)
        for column in columns
        if NameNormalizer.normalize(column)
    ]

    output: list[str] = []

    for index, column in enumerate(clean_columns):
        if index == 0:
            output.append(f"        {column}")
        else:
            output.append(f"        , {column}")

    return output

def _include_lines(
    self,
    include_names: list[str],
) -> list[str]:
    output: list[str] = []
    seen: set[str] = set()

    for include_name in include_names:
        normalized = self._normalize_include_name(include_name)

        if not normalized:
            continue

        if normalized in seen:
            continue

        seen.add(normalized)

        output.extend(
            [
                "EXEC SQL",
                f"        INCLUDE {normalized}",
                "END-EXEC.",
            ]
        )

    return output

def _ensure_dclgen_initialization(
    self,
    text: str,
    include_names: list[str],
) -> str:
    clean_include_names = [
        self._normalize_include_name(include_name)
        for include_name in include_names
        if self._normalize_include_name(include_name)
    ]

    if not clean_include_names:
        return text

    missing_initialize = []

    for include_name in clean_include_names:
        group_name = "DCL" + NameNormalizer.to_cobol(include_name)

        if re.search(
            rf"\bINITIALIZE\s+{re.escape(group_name)}\b",
            text,
            flags=re.IGNORECASE,
        ):
            continue

        missing_initialize.append(group_name)

    if not missing_initialize:
        return text

```

```

initialize_lines = [
    "* DB2: Initialize generated DCLGEN host groups."
]

for group_name in missing_initializes:
    initialize_lines.append(
        f"INITIALIZE {group_name}."
    )

block = "\n".join(initialize_lines)

return self._insert_after_procedure_division(
    text=text,
    block=block,
)

def _insert_after_procedure_division(
    self,
    text: str,
    block: str,
) -> str:
    lines = str(text or "").splitlines()
    output: list[str] = []
    inserted = False

    for line in lines:
        output.append(line)

        if inserted:
            continue

        logical = self._logical_line(line)

        if self.PROCEDURE_DIVISION_PATTERN.match(logical):
            output.append(block)
            inserted = True

    if inserted:
        return "\n".join(output).rstrip() + "\n"

    return str(text or "").rstrip() + "\n\n" + block + "\n"

def _insert_in_data_division(
    self,
    text: str,
    block: str,
) -> str:
    for pattern in [
        self.LINKAGE_SECTION_PATTERN,
        self.PROCEDURE_DIVISION_PATTERN,
    ]:
        match = pattern.search(text)

        if match:
            return (
                text[: match.start()].rstrip()
                + "\n\n"
                + block.rstrip()
                + "\n\n"
                + text[match.start():].lstrip()
            ).rstrip() + "\n"

    working_storage_match = self.WORKING_STORAGE_PATTERN.search(text)

    if working_storage_match:
        return (
            text[: working_storage_match.end()].rstrip()
            + "\n\n"
            + block.rstrip()
            + "\n\n"
            + text[working_storage_match.end():].lstrip()
        ).rstrip() + "\n"

    data_division_match = self.DATA_DIVISION_PATTERN.search(text)

    if data_division_match:

```

```

        return (
            text[: data_division_match.end()].rstrip()
            + "\n\n"
            + "WORKING-STORAGE SECTION."
            + "\n\n"
            + block.rstrip()
            + "\n\n"
            + text[data_division_match.end():].lstrip()
        ).rstrip() + "\n"

    return block.rstrip() + "\n\n" + text.rstrip() + "\n"

def _cursor_paragraph_spec(
    self,
    cursor_order: int,
    table_name: str,
) -> dict[str, str]:
    if hasattr(self.cursor_name_resolver, "cursor_spec"):
        return self.cursor_name_resolver.cursor_spec(
            cursor_order=cursor_order,
            table_name=table_name,
        )

    cursor_name = self.cursor_name_resolver.cursor_name_from_table(
        table_name
    )

    base = 710 + ((cursor_order - 1) * 100)

    return {
        "cursor_name": cursor_name,
        "open_paragraph": f"{{base}}-OPEN-{{cursor_name}}",
        "fetch_paragraph": f"{{base + 10}}-FETCH-{{cursor_name}}",
        "close_paragraph": f"{{base + 20}}-CLOSE-{{cursor_name}}",
    }

def _cursor_spec_messages(
    self,
    cursor_specs: list[dict[str, object]],
) -> list[str]:
    messages: list[str] = []

    for spec in cursor_specs:
        cursor_name = str(spec.get("cursor_name", ""))
        record_name = str(spec.get("record_name", ""))
        table_name = str(spec.get("table_name", ""))
        select_columns = list(spec.get("select_columns", []))
        host_variables = list(spec.get("host_variables", []))

        if not table_name:
            messages.append(
                f"DB2 infrastructure: cursor {cursor_name} has no resolved DB2 table for record {record_name}."
            )

        if not select_columns:
            messages.append(
                f"DB2 infrastructure: cursor {cursor_name} has no resolved SELECT columns for record {record_name}."
            )

        if not host_variables:
            messages.append(
                f"DB2 infrastructure: cursor {cursor_name} has no resolved FETCH host variables for record {record_name}."
            )

    return messages

def _comma_lines(
    self,
    items: list[str],
    indent: str,
) -> list[str]:
    clean_items = [
        str(item or "").strip()
        for item in items
    ]

```

```

        if str(item or "").strip()
    ]

    output: list[str] = []

    for index, item in enumerate(clean_items):
        if index == 0:
            output.append(f"{indent}{item}")
        else:
            output.append(f"{indent}, {item}")

    return output

def _and_lines(
    self,
    items: list[str],
    indent: str,
) -> list[str]:
    clean_items = [
        str(item or "").strip()
        for item in items
        if str(item or "").strip()
    ]

    output: list[str] = []

    for index, item in enumerate(clean_items):
        prefix = "AND " if index > 0 else ""
        output.append(f"{indent}{prefix}{item}")

    return output

def _comment_block(
    self,
    title: str,
) -> list[str]:
    return [
        f"* {title:<62}*",
    ]

def _normalize_include_name(
    self,
    value: str,
) -> str:
    text = NameNormalizer.normalize(value)

    if not text:
        return ""

    return text

def _logical_line(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    if len(text) >= 80:
        left = text[:6]
        body = text[7:72]

        if left.strip().isdigit():
            return body.strip()

    if len(text) > 6 and text[:6].strip().isdigit():
        return text[6:].strip()

    return text.strip()

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\generators\sql_error_generator.py
ENCODING: utf-8
=====

```

```
import re
```



```
from patterns.db2_patterns import END_PROGRAM_PATTERN
```

```
class SqlErrorGenerator:
```

```
    """
```

```
    Generates and ensures SQLERROR paragraph.
```

```
    Generic rule:
```

- Use SQLERROR, not SQL-ERROR.
- Generated DB2 cursor paragraphs and SQL blocks perform SQLERROR.
- SQLERROR paragraph wraps the standard DB2 SQLERROR include.

```
    Generated paragraph:
```

```
        SQLERROR.  
        EXEC SQL  
            INCLUDE SQLERROR  
        END-EXEC.
```

```
    """
```

```
    DEFAULT_PARAGRAPH_NAME = "SQLERROR"
```

```
    SQLERROR_PARAGRAPH_PATTERN = re.compile(  
        r"^\s*(?:\d{6}\s+)?(?:SQLERROR|SQL-ERROR)\.\s*(?:\d{8})?\s*$",  
        flags=re.IGNORECASE | re.MULTILINE,  
    )
```

```
    SQLERROR_PERFORM_PATTERN = re.compile(  
        r"\bPERFORM\s+(?:SQLERROR|SQL-ERROR)\s*\.?\"",  
        flags=re.IGNORECASE,  
    )
```

```
    SQLERROR_INCLUDE_PATTERN = re.compile(  
        r"\bINCLUDE\s+SQLERROR\b",  
        flags=re.IGNORECASE,  
    )
```

```
    CUSTOM_SQLERROR_DISPLAY_PATTERN = re.compile(  
        r"DISPLAY\s+'DB2\s+SQL\s+ERROR",  
        flags=re.IGNORECASE,  
    )
```

```
    def paragraph_name(  
        self,  
        cobol_text: str,  
    ) -> str:  
        return self.DEFAULT_PARAGRAPH_NAME
```

```
    def has_sql_error_paragraph(  
        self,  
        cobol_text: str,  
    ) -> bool:  
        return bool(  
            self.SQLERROR_PARAGRAPH_PATTERN.search(cobol_text or "")  
        )
```

```
    def paragraph_block(  
        self,  
        paragraph_name: str = DEFAULT_PARAGRAPH_NAME,  
    ) -> str:  
        clean_name = self.DEFAULT_PARAGRAPH_NAME  
  
        return "\n".join(  
            [  
                f"{clean_name}.",  
                "    EXEC SQL",  
                "        INCLUDE SQLERROR",  
                "    END-EXEC.",  
            ]  
        )
```

```
    def ensure_sql_error_paragraph(  
        self,  
        cobol_text: str,  
    ) -> str:
```

```

text = str(cobol_text or "")

if not text:
    return ""

text = self.normalize_sql_error_references(text)

if self._has_sqlerror_paragraph(text):
    text = self._replace_existing_sqlerror_paragraph(text)
    return text.rstrip() + "\n"

block = self.paragraph_block()

end_program_match = END_PROGRAM_PATTERN.search(text)

if end_program_match:
    return (
        text[: end_program_match.start()].rstrip()
        + "\n\n"
        + block
        + "\n\n"
        + text[end_program_match.start():].lstrip()
        ).rstrip() + "\n"

return text.rstrip() + "\n\n" + block + "\n"

def normalize_sql_error_references(
    self,
    text: str,
) -> str:
    updated = str(text or "")

    updated = re.sub(
        r"\bPERFORM\s+SQL-ERROR\s*\.\?",
        "PERFORM SQLERROR.",
        updated,
        flags=re.IGNORECASE,
    )

    updated = re.sub(
        r"\bPERFORM\s+SQLERROR\s*\.\?",
        "PERFORM SQLERROR.",
        updated,
        flags=re.IGNORECASE,
    )

    updated = re.sub(
        r"(?m)^(?!\s*)(?:\d{6}\s+)?SQL-ERROR\.\s*(?:\d{8})?\s*$",
        r"\1SQLERROR.",
        updated,
        flags=re.IGNORECASE,
    )

    return updated

def _has_sqlerror_paragraph(
    self,
    text: str,
) -> bool:
    return bool(
        re.search(
            r"^\s*(?:\d{6}\s+)?SQLERROR\.\s*(?:\d{8})?\s*$",
            text or "",
            flags=re.IGNORECASE | re.MULTILINE,
        )
    )

def _replace_existing_sqlerror_paragraph(
    self,
    text: str,
) -> str:
    lines = str(text or "").splitlines()

    output: list[str] = []
    index = 0
    replaced = False

```

```

while index < len(lines):
    line = lines[index]

    if not self._is_sqlerror_paragraph_header(line):
        output.append(line)
        index += 1
        continue

    output.extend(self.paragraph_block().splitlines())
    replaced = True
    index += 1

    while index < len(lines):
        current = lines[index]

        if self._is_next_paragraph_or_program_boundary(current):
            break

        if self._line_belongs_to_old_sqlerror_block(current):
            index += 1
            continue

        if not current.strip():
            index += 1
            continue

        break

    continue

if not replaced:
    return text

return "\n".join(output).rstrip() + "\n"

def _is_sqlerror_paragraph_header(
    self,
    line: str,
) -> bool:
    logical = self._logical_line(line)

    return bool(
        re.fullmatch(
            r"(?:SQLERROR|SQL-ERROR)\.",
            logical,
            flags=re.IGNORECASE,
        )
    )

def _is_next_paragraph_or_program_boundary(
    self,
    line: str,
) -> bool:
    logical = self._logical_line(line)

    if not logical:
        return False

    if re.match(
        r"END\s+PROGRAM\b",
        logical,
        flags=re.IGNORECASE,
    ):
        return True

    if re.fullmatch(
        r"[A-Z0-9][A-Z0-9-]*\.",
        logical,
        flags=re.IGNORECASE,
    ):
        return True

    return False

def _line_belongs_to_old_sqlerror_block(

```

```

        self,
        line: str,
    ) -> bool:
        logical = self._logical_line(line)

        if not logical:
            return True

        if logical.upper().startswith("DISPLAY "):
            return True

        if logical.upper().startswith("CONTINUE"):
            return True

        if logical.upper().startswith("EXEC SQL"):
            return True

        if logical.upper().startswith("INCLUDE SQLERROR"):
            return True

        if logical.upper().startswith("END-EXEC"):
            return True

        return False

    def _logical_line(
        self,
        line: str,
    ) -> str:
        text = str(line or "").rstrip()

        if len(text) >= 80 and text[:6].isdigit() and text[72:80].isdigit():
            return text[7:72].strip()

        if len(text) > 6 and text[:6].isdigit():
            text = text[6:].strip()

        if len(text) >= 8 and text[-8:].isdigit():
            text = text[:-8].rstrip()

        return text.strip()

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\generators\sql_generator.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.timestamp_audit_rules import INSERT_EXCLUDE_AUDIT_PREFIXES

```

```

class SqlGenerator:
    """
    Generates DB2 embedded SQL snippets.

    Authority:
    - Sheet Mapping determines intended DB2 table and column names.
    - DCLGEN determines final table availability, host variable spelling, and group names.
    - TableNameResolver resolves TB/TV mismatches using DCLGEN.
    """

    SELECT_EXCLUDE_PREFIXES = (
        "TS_CREATE",
        "TS_UPDATE",
        "ID_USERID",
        "NR_USERID",
    )

    FALLBACK_KEY_PREFIXES = (
        "CT_",

```

```

        "NR_CIO",
        "DA_CR",
        "NR_ID",
        "NS_",
    )

    def __init__(
        self,
        mapping_repository: MappingRepository,
        dclgen_repository: DclgenRepository,
        table_name_resolver: TableNameResolver,
        column_name_resolver: ColumnNameResolver,
        host_variable_resolver: HostVariableResolver,
    ) -> None:
        self.mapping_repository = mapping_repository
        self.dclgen_repository = dclgen_repository
        self.table_name_resolver = table_name_resolver
        self.column_name_resolver = column_name_resolver
        self.host_variable_resolver = host_variable_resolver

    def select_by_key(
        self,
        record_name: str,
    ) -> list[str]:
        record = NameNormalizer.normalize(record_name)
        table = self.table_name_resolver.table_for_record(record)

        if not table:
            return self._missing_mapping(record, "Missing SELECT table mapping")

        columns = self.column_name_resolver.columns_for_record(record)

        if not columns:
            columns = self._dclgen_column_names_for_table(table)

        columns = self._filter_select_columns(columns)

        if not columns:
            return self._missing_mapping(record, "Missing SELECT column mapping")

        hosts = self.host_variable_resolver.host_references_for_columns(
            table_name=table,
            columns=columns,
        )

        if not hosts:
            return self._missing_mapping(record, "Missing SELECT host mapping")

        key_columns = self.column_name_resolver.key_columns_for_record(record)

        if not key_columns:
            key_columns = self._fallback_key_columns(columns)

        if not key_columns and columns:
            key_columns = columns[:1]

        where_conditions = self._where_conditions(
            table_name=table,
            columns=key_columns,
        )

        lines: list[str] = [
            f"MOVE {NameNormalizer.to_cobol(record)} TO SQL-LOCATION.",
            "EXEC SQL",
            " SELECT",
        ]

        lines.extend(
            self._comma_lines(
                items=columns,
                indent="    ",
            )
        )

        lines.append(" INTO")

```

```

        lines.extend(
            self._comma_lines(
                items=hosts,
                indent="    ",
            )
        )

    lines.append(f" FROM {table}")

    if where_conditions:
        lines.append(" WHERE")
        lines.extend(
            self._and_lines(
                items=where_conditions,
                indent="    ",
            )
        )

    lines.append("END-EXEC.")

    return lines

def insert(
    self,
    record_name: str,
) -> list[str]:
    record = NameNormalizer.normalize(record_name)
    table = self.table_name_resolver.table_for_record(record)

    if not table:
        return self._missing_mapping(record, "Missing INSERT table mapping")

    columns = self.column_name_resolver.columns_for_record(record)

    if not columns:
        columns = self._dclgen_column_names_for_table(table)

    columns = [
        column
        for column in columns
        if column
        and self._column_exists_in_table(
            table_name=table,
            column_name=column,
        )
        and not self._is_insert_excluded_audit_column(column)
    ]

    if not columns:
        return self._missing_mapping(record, "Missing INSERT column mapping")

    hosts = self.host_variable_resolver.host_references_for_columns(
        table_name=table,
        columns=columns,
    )

    if not hosts:
        return self._missing_mapping(record, "Missing INSERT host mapping")

    lines: list[str] = [
        f"MOVE 'INSERT-{NameNormalizer.to_cobol(record)}' TO SQL-LOCATION.",
        "EXEC SQL",
        f" INSERT INTO {table}",
        " (",
    ]

    lines.extend(
        self._comma_lines(
            items=columns,
            indent="    ",
        )
    )

    lines.extend(
        [
            " )",

```

```

        " VALUES",
        " (",
    ]
)

lines.extend(
    self._comma_lines(
        items=hosts,
        indent="    ",
    )
)

lines.extend(
    [
        " )",
        "END-EXEC.",
    ]
)

return lines

def update(
    self,
    record_name: str,
    changed_fields: list[str] | None = None,
) -> list[str]:
    record = NameNormalizer.normalize(record_name)
    table = self.table_name_resolver.table_for_record(record)

    if not table:
        return self._missing_mapping(record, "Missing UPDATE table mapping")

    all_columns = self.column_name_resolver.columns_for_record(record)

    if not all_columns:
        all_columns = self._dclgen_column_names_for_table(table)

    all_columns = [
        column
        for column in all_columns
        if column
        and self._column_exists_in_table(
            table_name=table,
            column_name=column,
        )
    ]

    if not all_columns:
        return self._missing_mapping(record, "Missing UPDATE column mapping")

    key_columns = self.column_name_resolver.key_columns_for_record(record)

    if not key_columns:
        key_columns = self._fallback_key_columns(all_columns)

    if not key_columns:
        return self._missing_mapping(record, "Missing UPDATE key mapping")

    normalized_key_columns = {
        NameNormalizer.normalize(column)
        for column in key_columns
        if NameNormalizer.normalize(column)
    }

    set_columns = [
        column
        for column in all_columns
        if NameNormalizer.normalize(column) not in normalized_key_columns
        and not self._is_audit_column(column)
    ]

    if changed_fields:
        changed = {
            NameNormalizer.normalize(field)
            for field in changed_fields
            if NameNormalizer.normalize(field)

```

```

    }

    set_columns = [
        column
        for column in set_columns
        if NameNormalizer.normalize(column) in changed
    ]

    if not set_columns:
        return self._missing_mapping(record, "Missing UPDATE SET mapping")

    set_lines = self._set_lines(
        table_name=table,
        columns=set_columns,
    )

    where_conditions = self._where_conditions(
        table_name=table,
        columns=key_columns,
    )

    if not set_lines:
        return self._missing_mapping(record, "Missing UPDATE host mapping")

    if not where_conditions:
        return self._missing_mapping(record, "Missing UPDATE WHERE mapping")

    lines: list[str] = [
        f"MOVE 'UPDATE-{NameNormalizer.to_cobol(record)}' TO SQL-LOCATION.",
        "EXEC SQL",
        f"UPDATE {table}",
        "SET",
    ]

    lines.extend(
        self._comma_lines(
            items=set_lines,
            indent="    ",
        )
    )

    lines.append(" WHERE")

    lines.extend(
        self._and_lines(
            items=where_conditions,
            indent="    ",
        )
    )

    lines.append("END-EXEC.")

    return lines

def delete(
    self,
    record_name: str,
) -> list[str]:
    record = NameNormalizer.normalize(record_name)
    table = self.table_name_resolver.table_for_record(record)

    if not table:
        return self._missing_mapping(record, "Missing DELETE table mapping")

    columns = self.column_name_resolver.columns_for_record(record)

    if not columns:
        columns = self._dclgen_column_names_for_table(table)

    key_columns = self.column_name_resolver.key_columns_for_record(record)

    if not key_columns:
        key_columns = self._fallback_key_columns(columns)

    if not key_columns:
        return self._missing_mapping(record, "Missing DELETE key mapping")

```



```

        where_conditions = self._where_conditions(
            table_name=table,
            columns=key_columns,
        )

    if not where_conditions:
        return self._missing_mapping(record, "Missing DELETE WHERE mapping")

    lines: list[str] = [
        f"MOVE 'DELETE-{NameNormalizer.to_cobol(record)}' TO SQL-LOCATION.",
        "EXEC SQL",
        f"DELETE FROM {table}",
        "WHERE",
    ]

    lines.extend(
        self._and_lines(
            items=where_conditions,
            indent="    ",
        )
    )

    lines.append("END-EXEC.")

    return lines

def commit(
    self,
) -> list[str]:
    return [
        "MOVE 'COMMIT' TO SQL-LOCATION.",
        "EXEC SQL",
        "COMMIT",
        "END-EXEC.",
    ]

def rollback(
    self,
) -> list[str]:
    return [
        "MOVE 'ROLLBACK' TO SQL-LOCATION.",
        "EXEC SQL",
        "ROLLBACK",
        "END-EXEC.",
    ]

def _where_conditions(
    self,
    table_name: str,
    columns: list[str],
) -> list[str]:
    output: list[str] = []

    for column in columns:
        normalized_column = NameNormalizer.normalize(column)

        if not normalized_column:
            continue

        host = self.host_variable_resolver.host_reference_for_column(
            table_name=table_name,
            column_name=normalized_column,
        )

        if not host:
            continue

        output.append(f"{normalized_column} = {host}")

    return self._unique_non_empty(output)

def _set_lines(
    self,
    table_name: str,
    columns: list[str],

```

```

) -> list[str]:
    output: list[str] = []

    for column in columns:
        normalized_column = NameNormalizer.normalize(column)

        if not normalized_column:
            continue

        host = self.host_variable_resolver.host_reference_for_column(
            table_name=table_name,
            column_name=normalized_column,
        )

        if not host:
            continue

        output.append(f"{normalized_column} = {host}")

    return self._unique_non_empty(output)

def _comma_lines(
    self,
    items: list[str],
    indent: str,
) -> list[str]:
    clean_items = [
        str(item or "").strip()
        for item in items
        if str(item or "").strip()
    ]

    output: list[str] = []

    for index, item in enumerate(clean_items):
        suffix = "," if index < len(clean_items) - 1 else ""
        output.append(f"{indent}{item}{suffix}")

    return output

def _and_lines(
    self,
    items: list[str],
    indent: str,
) -> list[str]:
    clean_items = [
        str(item or "").strip()
        for item in items
        if str(item or "").strip()
    ]

    output: list[str] = []

    for index, item in enumerate(clean_items):
        prefix = "AND " if index > 0 else ""
        output.append(f"{indent}{prefix}{item}")

    return output

def _missing_mapping(
    self,
    record_name: str,
    reason: str,
) -> list[str]:
    record = NameNormalizer.to_cobol(record_name)

    if record:
        return [
            f"* DB2: Conversion skipped because Sheet Mapping entry does not exist.",
            f"* DB2: Missing Sheet Mapping metadata for record {record}.",
            f"* DB2: Reason: {reason}.",
            "CONTINUE.",
        ]

    return [
        f"* DB2: Conversion skipped because required Sheet Mapping metadata does not exist.",
    ]

```

```

        f"* DB2: Reason: {reason}.",
        "CONTINUE.",
    ]

def _dclgen_column_names_for_table(
    self,
    table_name: str,
) -> list[str]:
    resolved_table = self.table_name_resolver.resolve_table(table_name)

    if not resolved_table:
        return []

    return self.dclgen_repository.column_names_for_table(resolved_table)

def _column_exists_in_table(
    self,
    table_name: str,
    column_name: str,
) -> bool:
    resolved_table = self.table_name_resolver.resolve_table(table_name)
    normalized_column = NameNormalizer.normalize(column_name)

    if not resolved_table or not normalized_column:
        return False

    return self.dclgen_repository.has_column(
        table_name=resolved_table,
        column_name=normalized_column,
    )

def _fallback_key_columns(
    self,
    columns: list[str],
) -> list[str]:
    output: list[str] = []

    for column in columns:
        normalized_column = NameNormalizer.normalize(column)

        if not normalized_column:
            continue

        if not normalized_column.startswith(self.FALLBACK_KEY_PREFIXES):
            continue

        if normalized_column in output:
            continue

        output.append(normalized_column)

        if len(output) >= 8:
            break

    return output

def _filter_select_columns(
    self,
    columns: list[str],
) -> list[str]:
    output: list[str] = []

    for column in columns:
        normalized_column = NameNormalizer.normalize(column)

        if not normalized_column:
            continue

        if self._is_select_excluded_column(normalized_column):
            continue

        if normalized_column in output:
            continue

        output.append(normalized_column)

```

```

        return output

    def _is_select_excluded_column(
        self,
        column_name: str,
    ) -> bool:
        normalized = NameNormalizer.normalize(column_name)

        return normalized.startswith(self.SELECT_EXCLUDE_PREFIXES)

    def _is_audit_column(
        self,
        column_name: str,
    ) -> bool:
        normalized = NameNormalizer.normalize(column_name)

        return normalized.startswith(
            (
                "TS_CREATE",
                "TS_UPDATE",
                "ID_USERID",
                "NR_USERID",
            )
        )

    def _is_insert_excluded_audit_column(
        self,
        column_name: str,
    ) -> bool:
        column = NameNormalizer.normalize(column_name)

        return any(
            column.startswith(prefix)
            for prefix in INSERT_EXCLUDE_AUDIT_PREFIXES
        )

    def _unique_non_empty(
        self,
        values: list[str],
    ) -> list[str]:
        output: list[str] = []

        for value in values:
            text = str(value or "").strip()

            if not text:
                continue

            if text in output:
                continue

            output.append(text)

        return output

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\generators\timestamp_generator.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

import re

```

```

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver

```

```

class TimestampGenerator:

```

```

    """

```

```

    Generates timestamp and audit support.

```

```

    This generator is intentionally conservative.

```

For retrieval-only programs, it does not inject timestamp working storage, timestamp paragraphs, or audit MOVE statements.

For update/insert/delete programs, it adds a safe timestamp working-storage block and a safe timestamp paragraph only when DB2 write activity is found.

The full audit-field MOVE generation can be enhanced later after retrieval execution is stable.

```
WS_MARKER = "DB2 GENERATED TIMESTAMP AND AUDIT WORKING STORAGE"
PARAGRAPH_NAME = "600-GET-TIMESTAMP"
PARAGRAPH_EXIT_NAME = "600-GET-TIMESTAMP-EXIT"
```

```
PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?PROCEDURE\\s+DIVISION\\b.*\\.\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
WORKING_STORAGE_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?WORKING-STORAGE\\s+SECTION\\.\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
LINKAGE_SECTION_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?LINKAGE\\s+SECTION\\.\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
END_PROGRAM_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?END\\s+PROGRAM\\b.*\\.\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
STOP_RUN_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?STOP\\s+RUN\\.\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE,
)
```

```
EXEC_SQL_PATTERN = re.compile(
    r"^\\s*EXEC\\s+SQL\\b",
    flags=re.IGNORECASE,
)
```

```
END_EXEC_PATTERN = re.compile(
    r"^\\s*END-EXEC\\.\\s*$",
    flags=re.IGNORECASE,
)
```

```
DB2_WRITE_OPERATION_PATTERN = re.compile(
    r"^\\s*(INSERT|UPDATE|DELETE)\\b",
    flags=re.IGNORECASE,
)
```

```
COBOL_WRITE_OPERATION_PATTERN = re.compile(
    r"^\\s*(STORE|MODIFY|ERASE)\\b",
    flags=re.IGNORECASE,
)
```

```
def __init__(
    self,
    mapping_repository: MappingRepository,
    table_name_resolver: TableNameResolver,
    host_variable_resolver: HostVariableResolver,
) -> None:
    self.mapping_repository = mapping_repository
    self.table_name_resolver = table_name_resolver
    self.host_variable_resolver = host_variable_resolver
    self.messages: list[str] = []

def apply(
    self,
    cobol_text: str,
    target_program_id: str = "",
) -> tuple[str, list[str]]:
    self.messages = []
```

```

if not cobol_text:
    return "", self.messages

if not self._has_db2_write_activity(cobol_text):
    self.messages.append(
        "Timestamp generator: no DB2 write activity found; timestamp audit generation skipped."
    )
    return cobol_text.rstrip() + "\n", self.messages

updated_text = cobol_text

updated_text = self._ensure_timestamp_working_storage(
    text=updated_text,
    target_program_id=target_program_id,
)

updated_text = self._ensure_timestamp_paragraph(
    text=updated_text,
)

updated_text = self._ensure_timestamp_perform(
    text=updated_text,
)

self.messages.append(
    "Timestamp generator: timestamp working storage and paragraph ensured."
)

return updated_text.rstrip() + "\n", self.messages

def _has_db2_write_activity(
    self,
    text: str,
) -> bool:
    in_exec_sql = False

    for line in str(text or "").splitlines():
        logical = self._logical_line(line)

        if self.EXEC_SQL_PATTERN.match(logical):
            in_exec_sql = True
            continue

        if in_exec_sql and self.DB2_WRITE_OPERATION_PATTERN.search(logical):
            return True

        if self.END_EXEC_PATTERN.match(logical):
            in_exec_sql = False
            continue

        if self.COBOL_WRITE_OPERATION_PATTERN.search(logical):
            return True

    return False

def _ensure_timestamp_working_storage(
    self,
    text: str,
    target_program_id: str,
) -> str:
    if self.WS_MARKER in text:
        return text

    block = self._timestamp_working_storage_block(
        target_program_id=target_program_id,
    )

    lines = text.splitlines()
    insert_index = self._working_storage_insert_index(lines)

    if insert_index < 0:
        return block.rstrip() + "\n\n" + text.rstrip() + "\n"

    updated_lines = (
        lines[:insert_index]

```

```

        + [""]
        + block.splitlines()
        + [""]
        + lines[insert_index:]
    )

    return "\n".join(updated_lines).rstrip() + "\n"

def _ensure_timestamp_paragraph(
    self,
    text: str,
) -> str:
    if re.search(
        rf"^\s*(?:\d{{6}}\s+)?{re.escape(self.PARAGRAPH_NAME)}\.",
        text,
        flags=re.IGNORECASE | re.MULTILINE,
    ):
        return text

    block = self._timestamp_paragraph_block()

    lines = text.splitlines()
    insert_index = self._timestamp_paragraph_insert_index(lines)

    if insert_index < 0:
        return text.rstrip() + "\n\n" + block + "\n"

    updated_lines = (
        lines[:insert_index]
        + block.splitlines()
        + [""]
        + lines[insert_index:]
    )

    return "\n".join(updated_lines).rstrip() + "\n"

def _ensure_timestamp_perform(
    self,
    text: str,
) -> str:
    if re.search(
        rf"\bPERFORM\s+{re.escape(self.PARAGRAPH_NAME)}\b",
        text,
        flags=re.IGNORECASE,
    ):
        return text

    lines = text.splitlines()
    procedure_index = self._find_pattern_index(
        lines=lines,
        pattern=self.PROCEDURE_DIVISION_PATTERN,
    )

    if procedure_index < 0:
        return text

    insert_index = procedure_index + 1

    while insert_index < len(lines):
        logical = self._logical_line(lines[insert_index])

        if not logical:
            insert_index += 1
            continue

        if logical.startswith("*"):
            insert_index += 1
            continue

        break

    updated_lines = (
        lines[:insert_index]
        + [f"    PERFORM {self.PARAGRAPH_NAME}."]
        + lines[insert_index:]
    )

```

```

        return "\n".join(updated_lines).rstrip() + "\n"

def _timestamp_working_storage_block(
    self,
    target_program_id: str,
) -> str:
    program_id = str(target_program_id or "").strip().upper()

    if not program_id:
        program_id = "DB2PGM"

    if len(program_id) > 8:
        program_id = program_id[:8]

    return "\n".join(
        [
            f"* {self.WS_MARKER}",
            f"01 CS-PROGRAM                                PIC X(8) VALUE '{program_id:<8}'.",
            "01 WS-TIMESTAMP-FIELDS.",
            "    05 TS-SYSTEM.",
            "        10 DA-SYS.",
            "            15 DA-SYS-CCYY.",
            "                20 CC                                PIC X(2).",
            "                20 YY                                PIC X(2).",
            "                    15 MM                            PIC X(2).",
            "                    15 DD                            PIC X(2).",
            "        10 HR-SYS.",
            "            15 HH                                PIC X(2).",
            "            15 MI                                PIC X(2).",
            "            15 SS                                PIC X(2).",
            "            15 TT                                PIC X(2).",
            "    05 TS-TIMESTAMP.",
            "        10 DA-TIMESTAMP.",
            "            15 DA-TIMESTAMP-CCYY.",
            "                20 CC                                PIC X(2).",
            "                20 YY                                PIC X(2).",
            "                    15 MM                            PIC X(2).",
            "                    15 DD                            PIC X(2).",
            "        10 HR-TIMESTAMP.",
            "            15 HH                                PIC X(2).",
            "            15 MI                                PIC X(2).",
            "            15 SS                                PIC X(2).",
            "            15 TT                                PIC X(2).",
        ]
    )

def _timestamp_paragraph_block(
    self,
) -> str:
    return "\n".join(
        [
            f"{self.PARAGRAPH_NAME}.",
            "    ACCEPT TS-SYSTEM FROM DATE.",
            "    ACCEPT HR-SYS FROM TIME.",
            "    MOVE DA-SYS TO DA-TIMESTAMP.",
            "    MOVE HR-SYS TO HR-TIMESTAMP.",
            "    DISPLAY 'TIMESTAMP: ' TS-TIMESTAMP.",
            f"{self.PARAGRAPH_EXIT_NAME}.",
            "    EXIT.",
        ]
    )

def _working_storage_insert_index(
    self,
    lines: list[str],
) -> int:
    linkage_index = self._find_pattern_index(
        lines=lines,
        pattern=self.LINKAGE_SECTION_PATTERN,
    )

    if linkage_index >= 0:
        return linkage_index

    procedure_index = self._find_pattern_index(

```



```

        lines=lines,
        pattern=self.PROCEDURE_DIVISION_PATTERN,
    )

    if procedure_index >= 0:
        return procedure_index

    working_storage_index = self._find_pattern_index(
        lines=lines,
        pattern=self.WORKING_STORAGE_PATTERN,
    )

    if working_storage_index >= 0:
        return working_storage_index + 1

    return -1

def _timestamp_paragraph_insert_index(
    self,
    lines: list[str],
) -> int:
    for index, line in enumerate(lines):
        logical = self._logical_line(line)

        if self.STOP_RUN_PATTERN.match(logical):
            return index

        if self.END_PROGRAM_PATTERN.match(logical):
            return index

    return -1

def _find_pattern_index(
    self,
    lines: list[str],
    pattern: re.Pattern,
) -> int:
    for index, line in enumerate(lines):
        logical = self._logical_line(line)

        if pattern.match(logical):
            return index

    return -1

def _logical_line(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    if len(text) > 6 and text[:6].strip().isdigit():
        return text[6:].strip()

    return text.strip()

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\infrastructure\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\infrastructure\file_loader.py
ENCODING: utf-8
=====

```

```

from io import BytesIO

import pdfplumber

class FileLoader:

```

```

"""
Loads readable text from uploaded-file-like objects.

Expected uploaded-file-like object:
- .name
- .getvalue()

Supported:
- PDF files through pdfplumber
- Text-like files through UTF-8 decode with errors ignored
"""

```

```

def read_uploaded_text(
    self,
    uploaded_file,
) -> str:
    if uploaded_file is None:
        return ""

    raw_bytes = uploaded_file.getvalue()
    file_name = str(uploaded_file.name or "").lower()

    if file_name.endswith(".pdf"):
        return self.read_pdf_bytes(raw_bytes)

    return raw_bytes.decode(
        "utf-8",
        errors="ignore",
    )

def read_pdf_bytes(
    self,
    raw_bytes: bytes,
) -> str:
    lines: list[str] = []

    with pdfplumber.open(BytesIO(raw_bytes)) as pdf:
        for page in pdf.pages:
            page_text = page.extract_text() or ""

            for raw_line in page_text.splitlines():
                cleaned_line = self._clean_line(raw_line)

                if cleaned_line:
                    lines.append(cleaned_line)

    return "\n".join(lines).strip()

def _clean_line(
    self,
    line: str | None,
) -> str:
    if line is None:
        return ""

    cleaned = str(line).rstrip()
    cleaned = cleaned.replace("\t", " ")
    cleaned = cleaned.replace("\u00a0", " ")

    while " " in cleaned:
        cleaned = cleaned.replace(" ", " ")

    return cleaned.strip()

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\infrastructure\local_uploaded_file.py
ENCODING: utf-8
=====

```

```

from pathlib import Path

```

```

class LocalUploadedFile:
    """
    Local file wrapper.

```

Existing parser flow expects uploaded-file-like objects with:

- .name
- .getvalue()

This wrapper allows local runner scripts to reuse the same parser logic used by the Streamlit file uploader flow.

"""

```
def __init__(
    self,
    file_path: Path,
) -> None:
    self.file_path = Path(file_path)
    self.name = self.file_path.name

def getvalue(
    self,
) -> bytes:
    return self.file_path.read_bytes()
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\infrastructure\logger_factory.py
ENCODING: utf-8
=====
```

```
import logging
from datetime import datetime
from pathlib import Path
```

```
from config.logging_settings import (
    DEFAULT_LOG_LEVEL,
    LOG_DATE_FORMAT,
    LOG_DATE_TIME_FORMAT,
    LOG_FILE_PREFIX,
    LOG_MESSAGE_FORMAT,
)
from config.path_settings import LOGS_DIR
```

```
class LoggerFactory:
```

"""

Creates production-style loggers that write to logs folder.

One log file is created per run:

logs/idms_db2_conversion_DD-MM-YYYY_HHMMSS.log
"""

@staticmethod

```
def create_logger(
    name: str,
    logs_dir: Path | None = None,
    level: str = DEFAULT_LOG_LEVEL,
) -> logging.Logger:
    target_logs_dir = Path(logs_dir or LOGS_DIR)
    target_logs_dir.mkdir(
        parents=True,
        exist_ok=True,
    )

    timestamp = datetime.now().strftime(LOG_DATE_TIME_FORMAT)
    log_file = target_logs_dir / f"{LOG_FILE_PREFIX}_{timestamp}.log"

    logger = logging.getLogger(name)
    logger.setLevel(LoggerFactory._level_value(level))
    logger.handlers.clear()
    logger.propagate = False

    formatter = logging.Formatter(
        fmt=LOG_MESSAGE_FORMAT,
        datefmt=LOG_DATE_FORMAT,
    )

    file_handler = logging.FileHandler(
        log_file,
```

```

        encoding="utf-8",
    )
    file_handler.setLevel(LoggerFactory._level_value(level))
    file_handler.setFormatter(formatter)

    console_handler = logging.StreamHandler()
    console_handler.setLevel(LoggerFactory._level_value(level))
    console_handler.setFormatter(formatter)

    logger.addHandler(file_handler)
    logger.addHandler(console_handler)

    logger.info("Logger initialized.")
    logger.info("Log file: %s", log_file)

    return logger

@staticmethod
def _level_value(
    level: str,
) -> int:
    return getattr(
        logging,
        str(level or DEFAULT_LOG_LEVEL).upper(),
        logging.INFO,
    )

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\infrastructure\runtime_context.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass, field
from pathlib import Path

```

```

@dataclass
class RuntimeContext:
    """
    Runtime context for local executions.

    Keep runtime paths outside parser and service logic.
    Parser and service classes should receive data, not know where the files
    came from.
    """

    project_root: Path
    input_dir: Path
    output_dir: Path
    sheet_mapping_path: Path | None = None
    dclgen_paths: list[Path] = field(default_factory=list)
    copybook_paths: list[Path] = field(default_factory=list)
    idms_cobol_source_path: Path | None = None
    target_program_id: str = ""
    auto_fix_pic_length_mismatches: bool = False

    def ensure_output_dir(
        self,
    ) -> None:
        self.output_dir.mkdir(
            parents=True,
            exist_ok=True,
        )

    def validate_required_files(
        self,
    ) -> None:
        if self.sheet_mapping_path is None:
            raise ValueError("Sheet Mapping path is required.")

        if self.idms_cobol_source_path is None:
            raise ValueError("IDMS COBOL source path is required.")

        self._validate_file_exists(
            self.sheet_mapping_path,

```

```

        "Sheet Mapping",
    )

    self._validate_file_exists(
        self.idms_cobol_source_path,
        "IDMS COBOL Source",
    )

    if not self.dclgen_paths:
        raise ValueError("At least one DCLGEN file path is required.")

    for index, dclgen_path in enumerate(self.dclgen_paths, start=1):
        self._validate_file_exists(
            dclgen_path,
            f"DCLGEN {index}",
        )

    for index, copybook_path in enumerate(self.copybook_paths, start=1):
        self._validate_file_exists(
            copybook_path,
            f"Copybook {index}",
        )

    self.ensure_output_dir()

    def _validate_file_exists(
        self,
        file_path: Path,
        label: str,
    ) -> None:
        if not file_path.exists():
            raise FileNotFoundError(f"{label} file not found: {file_path}")

        if not file_path.is_file():
            raise FileNotFoundError(f"{label} path is not a file: {file_path}")

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\logging_utils\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\logging_utils\logger_factory.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\orchestration\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\orchestration\conversion_service.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.analyzers.program_flow_analyzer import ProgramFlowAnalyzer
from idms_db2_phase2.composers.cobol_formatter import CobolFormatter
from idms_db2_phase2.composers.cursor_flow_composer import CursorFlowComposer
from idms_db2_phase2.composers.cursor_order_cleanup_composer import (
    CursorOrderCleanupComposer,
)
from idms_db2_phase2.composers.db2_date_comparison_composer import (
    Db2DateComparisonComposer,
)
from idms_db2_phase2.composers.fixed_format_composer import FixedFormatComposer

```

```

from idms_db2_phase2.composers.manual_layout_composer import ManualLayoutComposer
from idms_db2_phase2.composers.manual_style_preserver import ManualStylePreserver
from idms_db2_phase2.composers.sqlcode_wrapper_cleanup_composer import (
    SqlcodeWrapperCleanupComposer,
)
from idms_db2_phase2.composers.update_restart_skip_composer import (
    UpdateRestartSkipComposer,
)
from idms_db2_phase2.domain.models import ConversionInput, ConversionResult
from idms_db2_phase2.generators.cursor_paragraph_generator import (
    CursorParagraphGenerator,
)
from idms_db2_phase2.generators.db2_infrastructure_generator import (
    Db2InfrastructureGenerator,
)
from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.generators.sql_generator import SqlGenerator
from idms_db2_phase2.generators.timestamp_generator import TimestampGenerator
from idms_db2_phase2.repositories.copybook_repository import CopybookRepository
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.record_context_resolver import RecordContextResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.transformers.cobol_transformer import CobolTransformer
from idms_db2_phase2.transformers.field_reference_rewriter import (
    FieldReferenceRewriter,
)
from idms_db2_phase2.transformers.idms_statement_transformer import (
    IdmsStatementTransformer,
)
from idms_db2_phase2.transformers.pic_length_auto_fixer import PicLengthAutoFixer
from idms_db2_phase2.validators.input_validator import InputValidator
from idms_db2_phase2.validators.mapping_validator import MappingValidator
from idms_db2_phase2.validators.production_validator import ProductionValidator

```

```

class ConversionService:
    """
    Orchestrates IDMS COBOL to DB2 COBOL conversion.
    """

    def convert(
        self,
        conversion_input: ConversionInput,
    ) -> ConversionResult:
        validation_messages: list[str] = []

        input_validator = InputValidator()
        validation_messages.extend(
            input_validator.validate(conversion_input)
        )

        if validation_messages:
            return ConversionResult(
                converted_cobol="",
                validation_messages=self._unique_messages(validation_messages),
                operations=[],
            )

        repositories = self._repositories(conversion_input)
        resolvers = self._resolvers(repositories)

        mapping_validator = MappingValidator(
            mapping_repository=repositories["mapping"],
            dclgen_repository=repositories["dclgen"],
            table_name_resolver=resolvers["table_name"],
        )
        validation_messages.extend(mapping_validator.validate())

        generators = self._generators(
            repositories=repositories,
            resolvers=resolvers,
        )

```

```

transformers = self._transformers(
    repositories=repositories,
    resolvers=resolvers,
    generators=generators,
)

composers = self._composers()

converted_cobol, transform_messages, operations = transformers[
    "cobol"
].transform(
    cobol_text=conversion_input.idms_cobol_text,
    target_program_id=conversion_input.target_program_id,
)

validation_messages.extend(transform_messages)

converted_cobol = composers["update_restart_skip"].compose(
    converted_cobol
)

validation_messages.extend(
    composers["update_restart_skip"].messages
)

flow_analyzer = ProgramFlowAnalyzer(
    mapping_rows=conversion_input.sheet_mapping_rows,
    dclgen_columns=conversion_input.dclgen_columns,
)

flow_analysis = flow_analyzer.analyze(
    cobol_text=conversion_input.idms_cobol_text,
    operations=operations,
)

validation_messages.extend(flow_analysis.diagnostics)

converted_cobol = transformers["field_reference"].rewrite(
    converted_cobol
)

validation_messages.extend(
    transformers["field_reference"].rewrite_messages
)

converted_cobol, infrastructure_messages = generators[
    "db2_infrastructure"
].apply(
    cobol_text=converted_cobol,
    operations=operations,
)

validation_messages.extend(infrastructure_messages)

converted_cobol = composers["cursor_order_cleanup"].compose(
    converted_cobol
)

converted_cobol, cursor_messages = generators[
    "cursor_paragraph"
].apply(
    cobol_text=converted_cobol,
    operations=operations,
)

validation_messages.extend(cursor_messages)

converted_cobol = composers["cursor_flow"].compose(
    converted_cobol
)

converted_cobol = composers["sqlcode_cleanup"].compose(
    converted_cobol
)

```

```

converted_cobol = composers["cursor_order_cleanup"].compose(
    converted_cobol
)

converted_cobol = composers["date_compare"].compose(
    converted_cobol
)

converted_cobol, timestamp_messages = generators["timestamp"].apply(
    cobol_text=converted_cobol,
    target_program_id=conversion_input.target_program_id,
)

validation_messages.extend(timestamp_messages)

converted_cobol = generators["sql_error"].ensure_sql_error_paragraph(
    converted_cobol
)

if conversion_input.auto_fix_pic_length_mismatches:
    converted_cobol = transformers["pic_length"].fix(
        source_cobol_text=conversion_input.idms_cobol_text,
        converted_cobol_text=converted_cobol,
    )

    validation_messages.extend(
        transformers["pic_length"].messages
    )

converted_cobol = composers["formatter"].format(
    converted_cobol
)

converted_cobol = composers["manual_layout"].compose(
    converted_cobol
)

converted_cobol = composers["style_preserver"].preserve(
    original_text=conversion_input.idms_cobol_text,
    converted_text=converted_cobol,
)

converted_cobol = composers["fixed_format"].format(
    converted_cobol
)

converted_cobol = composers["cursor_flow"].compose(
    converted_cobol
)

converted_cobol = composers["sqlcode_cleanup"].compose(
    converted_cobol
)

converted_cobol = composers["cursor_order_cleanup"].compose(
    converted_cobol
)

converted_cobol = composers["date_compare"].compose(
    converted_cobol
)

converted_cobol = composers["fixed_format"].format(
    converted_cobol
)

production_validator = ProductionValidator(
    dclgen_repository=repositories["dclgen"],
)

validation_messages.extend(
    production_validator.validate(converted_cobol)
)

return ConversionResult(
    converted_cobol=converted_cobol,

```



```

        validation_messages=self._unique_messages(validation_messages),
        operations=operations,
    )

def _repositories(
    self,
    conversion_input: ConversionInput,
) -> dict[str, object]:
    return {
        "mapping": MappingRepository(
            conversion_input.sheet_mapping_rows,
        ),
        "dclgen": DclgenRepository(
            conversion_input.dclgen_columns,
        ),
        "copybook": CopybookRepository(
            conversion_input.copybook_fields,
        ),
    }

def _resolvers(
    self,
    repositories: dict[str, object],
) -> dict[str, object]:
    mapping_repository = repositories["mapping"]
    dclgen_repository = repositories["dclgen"]
    copybook_repository = repositories["copybook"]

    table_name_resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    column_name_resolver = ColumnNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_name_resolver,
    )

    host_variable_resolver = HostVariableResolver(
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_name_resolver,
    )

    cursor_name_resolver = CursorNameResolver()

    record_context_resolver = RecordContextResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    return {
        "table_name": table_name_resolver,
        "column_name": column_name_resolver,
        "host_variable": host_variable_resolver,
        "cursor_name": cursor_name_resolver,
        "record_context": record_context_resolver,
        "copybook": copybook_repository,
    }

def _generators(
    self,
    repositories: dict[str, object],
    resolvers: dict[str, object],
) -> dict[str, object]:
    sql_error_generator = SqlErrorGenerator()

    sql_generator = SqlGenerator(
        mapping_repository=repositories["mapping"],
        dclgen_repository=repositories["dclgen"],
        table_name_resolver=resolvers["table_name"],
        column_name_resolver=resolvers["column_name"],
        host_variable_resolver=resolvers["host_variable"],
    )

    db2_infrastructure_generator = Db2InfrastructureGenerator(

```

```

        table_name_resolver=resolvers["table_name"],
        column_name_resolver=resolvers["column_name"],
        host_variable_resolver=resolvers["host_variable"],
        cursor_name_resolver=resolvers["cursor_name"],
    )

    cursor_paragraph_generator = CursorParagraphGenerator(
        db2_infrastructure_generator=db2_infrastructure_generator,
        host_variable_resolver=resolvers["host_variable"],
        sql_error_generator=sql_error_generator,
    )

    timestamp_generator = TimestampGenerator(
        mapping_repository=repositories["mapping"],
        table_name_resolver=resolvers["table_name"],
        host_variable_resolver=resolvers["host_variable"],
    )

    return {
        "sql": sql_generator,
        "sql_error": sql_error_generator,
        "db2_infrastructure": db2_infrastructure_generator,
        "cursor_paragraph": cursor_paragraph_generator,
        "timestamp": timestamp_generator,
    }

def _transformers(
    self,
    repositories: dict[str, object],
    resolvers: dict[str, object],
    generators: dict[str, object],
) -> dict[str, object]:
    idms_statement_transformer = IdmsStatementTransformer(
        sql_generator=generators["sql"],
        sql_error_generator=generators["sql_error"],
        table_name_resolver=resolvers["table_name"],
        cursor_name_resolver=resolvers["cursor_name"],
    )

    cobol_transformer = CobolTransformer(
        idms_statement_transformer=idms_statement_transformer,
    )

    field_reference_rewriter = FieldReferenceRewriter(
        mapping_repository=repositories["mapping"],
        table_name_resolver=resolvers["table_name"],
        host_variable_resolver=resolvers["host_variable"],
    )

    pic_length_auto_fixer = PicLengthAutoFixer()

    return {
        "idms_statement": idms_statement_transformer,
        "cobol": cobol_transformer,
        "field_reference": field_reference_rewriter,
        "pic_length": pic_length_auto_fixer,
    }

def _composers(
    self,
) -> dict[str, object]:
    return {
        "formatter": CobolFormatter(),
        "cursor_flow": CursorFlowComposer(),
        "cursor_order_cleanup": CursorOrderCleanupComposer(),
        "date_compare": Db2DateComparisonComposer(),
        "sqlcode_cleanup": SqlcodeWrapperCleanupComposer(),
        "manual_layout": ManualLayoutComposer(),
        "style_preserver": ManualStylePreserver(),
        "fixed_format": FixedFormatComposer(),
        "update_restart_skip": UpdateRestartSkipComposer(),
    }

def _unique_messages(
    self,
    messages: list[str],

```

```

) -> list[str]:
    output: list[str] = []
    seen: set[str] = set()

    for message in messages:
        clean_message = str(message or "").strip()

        if not clean_message:
            continue

        if clean_message in seen:
            continue

        seen.add(clean_message)
        output.append(clean_message)

    return output

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\parsers\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\parsers\base_text_parser.py
ENCODING: utf-8
=====

```

```

class BaseTextParser:
    """
    Shared parser helper methods.

    This base class intentionally avoids business rules, schema names, and
    regex definitions. It contains only generic text cleanup utilities.
    """

    def normalize_line_endings(
        self,
        text: str,
    ) -> str:
        return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

    def normalize_spaces(
        self,
        text: str,
    ) -> str:
        output = str(text or "").replace("\t", " ")
        output = output.replace("\u00a0", " ")

        while " " in output:
            output = output.replace(" ", " ")

        return output

    def clean_text(
        self,
        text: str,
    ) -> str:
        output = self.normalize_line_endings(text)
        output = self.normalize_spaces(output)
        output = output.replace("'", "'").replace("'''", "'")
        output = output.replace('"', '"').replace('"', '"')

        return output

    def clean_cell(
        self,
        value,
    ) -> str:
        if value is None:
            return ""

```

```

        text = str(value)
        text = text.replace("\uffff", "")
        text = text.replace("\xa0", " ")
        text = text.replace("\r\n", "\n")
        text = text.replace("\r", "\n")
        text = self.normalize_spaces(text)

    return text.strip()

def strip_sequence_area(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    if len(text) > 6:
        indicator = text[6:7]

        if indicator in ("*", "/"):
            return indicator

        if text[:6].strip().isdigit():
            return text[6:]

    return text

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\parsers\cobol_parser.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import IdmsOperation
from idms_db2_phase2.parsers.base_text_parser import BaseTextParser
from patterns.cobol_patterns import PROGRAM_ID_PATTERN
from patterns.idms_patterns import (
    ERASE_PATTERN,
    FIND_FIRST_PATTERN,
    MODIFY_PATTERN,
    OBTAIN_CALC_PATTERN,
    OBTAIN_CALC_REVERSED_PATTERN,
    OBTAIN_FIRST_NEXT_PATTERN,
    OBTAIN_OWNER_PATTERN,
    READY_UPDATE_PATTERN,
    STORE_PATTERN,
)

```

```

class CobolParser(BaseTextParser):
    """
    Parses IDMS COBOL source and identifies IDMS database operations.

    This parser does not convert COBOL. It only identifies operations for
    later transformation, generation, diagnostics, and validation.

    Regex patterns are stored in patterns/cobol_patterns.py and
    patterns/idms_patterns.py.
    """

    def program_id(
        self,
        cobol_text: str,
    ) -> str:
        match = PROGRAM_ID_PATTERN.search(cobol_text or "")

        if not match:
            return ""

        return match.group(1).upper()

    def analyze(
        self,
        cobol_text: str,
    ) -> list[IdmsOperation]:
        operations: list[IdmsOperation] = []

```

```

for line_number, line in enumerate(
    str(cobol_text or "").splitlines(),
    start=1,
):
    clean_line = self.strip_sequence_area(line)
    upper = clean_line.upper()

    operation = self._operation_from_line(
        line=clean_line,
        upper=upper,
        line_number=line_number,
    )

    if operation is not None:
        operations.append(operation)

return operations

def _operation_from_line(
    self,
    line: str,
    upper: str,
    line_number: int,
) -> IdmsOperation | None:
    match = OBTAIN_CALC_PATTERN.search(upper)

    if match:
        return IdmsOperation(
            operation="OBTAIN_CALC",
            record_name=match.group("record").upper(),
            line_number=line_number,
            raw_line=line,
        )

    match = OBTAIN_CALC_REVERSED_PATTERN.search(upper)

    if match:
        return IdmsOperation(
            operation="OBTAIN_CALC",
            record_name=match.group("record").upper(),
            line_number=line_number,
            raw_line=line,
        )

    match = OBTAIN_FIRST_NEXT_PATTERN.search(upper)

    if match:
        return IdmsOperation(
            operation=f"OBTAIN_{match.group('mode').upper()}",
            record_name=match.group("record").upper(),
            set_name=match.group("set").upper(),
            line_number=line_number,
            raw_line=line,
        )

    match = OBTAIN_OWNER_PATTERN.search(upper)

    if match:
        return IdmsOperation(
            operation="OBTAIN_OWNER",
            set_name=match.group("set").upper(),
            line_number=line_number,
            raw_line=line,
        )

    match = FIND_FIRST_PATTERN.search(upper)

    if match:
        return IdmsOperation(
            operation="FIND_FIRST",
            record_name=(match.group("record") or "").upper(),
            set_name=match.group("set").upper(),
            line_number=line_number,
            raw_line=line,
        )

```

```

match = STORE_PATTERN.search(upper)

if match:
    return IdmsOperation(
        operation="STORE",
        record_name=match.group("record").upper(),
        line_number=line_number,
        raw_line=line,
    )

match = MODIFY_PATTERN.search(upper)

if match:
    return IdmsOperation(
        operation="MODIFY",
        record_name=match.group("record").upper(),
        line_number=line_number,
        raw_line=line,
    )

match = ERASE_PATTERN.search(upper)

if match:
    return IdmsOperation(
        operation="ERASE",
        record_name=match.group("record").upper(),
        line_number=line_number,
        raw_line=line,
    )

match = READY_UPDATE_PATTERN.search(upper)

if match:
    return IdmsOperation(
        operation="READY_UPDATE",
        record_name=match.group("record").upper(),
        line_number=line_number,
        raw_line=line,
    )

return None

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\parsers\copybook_parser.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import CopybookField
from idms_db2_phase2.parsers.base_text_parser import BaseTextParser
from patterns.copybook_patterns import (
    COPYBOOK_COMMENT_OR_SKIP_PATTERN,
    COPYBOOK_FIELD_PATTERN,
    COPYBOOK_OCCURS_PATTERN,
    COPYBOOK_PIC_PATTERN,
    COPYBOOK_USAGE_PATTERN,
)

```

```

class CopybookParser(BaseTextParser):
    """
    Parses COBOL copybook fields.

    This parser contains parsing logic only. Regex patterns live in
    patterns/copybook_patterns.py.
    """

    def parse(
        self,
        text: str,
    ) -> list[CopybookField]:
        if not str(text or "").strip():
            return []

        output: list[CopybookField] = []
        logical_lines = self._logical_lines(text)

```

```

for line in logical_lines:
    match = COPYBOOK_FIELD_PATTERN.search(line)

    if not match:
        continue

    name = str(match.group("name") or "").strip().upper()
    rest = str(match.group("rest") or "")

    if not name:
        continue

    if name == "FILLER":
        continue

    output.append(
        CopybookField(
            level=str(match.group("level") or "").strip(),
            name=name,
            picture=self._picture(rest),
            usage=self._usage(rest),
            occurs=self._occurs(rest),
        )
    )

return output

def _picture(
    self,
    text: str,
) -> str:
    match = COPYBOOK_PIC_PATTERN.search(str(text or ""))

    if not match:
        return ""

    return str(match.group("pic") or "").strip().upper()

def _usage(
    self,
    text: str,
) -> str:
    match = COPYBOOK_USAGE_PATTERN.search(str(text or ""))

    if not match:
        return ""

    return str(match.group("usage") or "").strip().upper()

def _occurs(
    self,
    text: str,
) -> str:
    match = COPYBOOK_OCCURS_PATTERN.search(str(text or ""))

    if not match:
        return ""

    return str(match.group("occurs") or "").strip()

def _logical_lines(
    self,
    text: str,
) -> list[str]:
    output: list[str] = []
    buffer = ""

    for raw_line in str(text or "").splitlines():
        line = self.strip_sequence_area(raw_line).rstrip()

        if not line.strip():
            continue

        if COPYBOOK_COMMENT_OR_SKIP_PATTERN.search(line):
            continue

```

```

    if buffer:
        buffer = f"{buffer} {line.strip()}"
    else:
        buffer = line.strip()

    if "." in line:
        parts = buffer.split(".")

        for part in parts[:-1]:
            clean = part.strip()

            if clean:
                output.append(clean + ".")

        buffer = parts[-1].strip()

    if buffer.strip():
        output.append(buffer.strip())

    return output

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\parsers\dclgen_parser.py
ENCODING: utf-8
=====

```

```

from catalogs.db2_sql_types import DB2_SQL_SKIP_WORDS, DB2_SQL_TYPE_STARTERS
from catalogs.dclgen_schema import DCLGEN_HOST_FIELD_SUFFIXES_TO_IGNORE
from idms_db2_phase2.domain.models import DclgenColumn
from idms_db2_phase2.parsers.base_text_parser import BaseTextParser
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.dclgen_patterns import (
    COBOL_FIELD_PATTERN,
    COBOL_GROUP_PATTERN,
    DECLARE_TABLE_ALT_PATTERN,
    DECLARE_TABLE_PATTERN,
    DCLGEN_PIC_PATTERN,
    DCLGEN_TABLE_COMMENT_PATTERN,
    DCLGEN_USAGE_PATTERN,
    MULTIPLE_UNDERSCORE_PATTERN,
    SQL_NAME_CLEANUP_PATTERN,
    VALID_SQL_COLUMN_NAME_PATTERN,
)

```

```

class DclgenParser(BaseTextParser):
    """
    Parses DB2 DCLGEN text files.

    This parser contains parsing logic only.

    Static content is externalized:
    - SQL type names: catalogs/db2_sql_types.py
    - DCLGEN suffix rules: catalogs/dclgen_schema.py
    - Regex patterns: patterns/dclgen_patterns.py
    """

    def __init__(self) -> None:
        self.diagnostics: list[str] = []

    def parse_many_texts(
        self,
        texts: list[str],
    ) -> list[DclgenColumn]:
        self.diagnostics = []
        output: list[DclgenColumn] = []

        self.diagnostics.append(
            f"DCLGEN input file count: {len(texts)}"
        )

        for index, text in enumerate(texts, start=1):
            source_label = f"DCLGEN file {index}"

```



```

        self.diagnostics.append(
            f"{source_label} input length: {len(text or '')}"
        )

        parsed = self.parse(
            text=text,
            source_label=source_label,
        )

        self.diagnostics.append(
            f"{source_label} parsed columns: {len(parsed)}"
        )

        output.extend(parsed)

    self.diagnostics.append(
        f"DCLGEN total parsed columns: {len(output)}"
    )

    return output

def parse(
    self,
    text: str,
    source_label: str = "DCLGEN",
) -> list[DclgenColumn]:
    if not text or not text.strip():
        self.diagnostics.append(
            f"{source_label}: empty text."
        )
        return []

    cleaned_text = self.clean_text(text)
    table_name = self._find_table_name(cleaned_text)

    if table_name:
        self.diagnostics.append(
            f"{source_label}: resolved table name: {table_name}"
        )
    else:
        self.diagnostics.append(
            f"{source_label}: table name not found."
        )

    sql_columns = self._parse_sql_declare_columns(
        text=cleaned_text,
        table_name=table_name,
        source_label=source_label,
    )

    cobol_fields = self._parse_cobol_host_fields(
        text=cleaned_text,
        source_label=source_label,
    )

    if sql_columns:
        output = self._merge_sql_columns_with_cobol_hosts(
            sql_columns=sql_columns,
            cobol_fields=cobol_fields,
            fallback_table_name=table_name,
        )

        self.diagnostics.append(
            f"{source_label}: SQL columns parsed: {len(sql_columns)}"
        )
        self.diagnostics.append(
            f"{source_label}: COBOL host fields parsed: {len(cobol_fields)}"
        )
        self.diagnostics.append(
            f"{source_label}: merged DCLGEN columns: {len(output)}"
        )

        return output

    if cobol_fields:
        self.diagnostics.append(

```

```

        f"{source_label}: no SQL DECLARE columns found; using COBOL host fields as fallback."
    )

    return self._fallback_columns_from_cobol_fields(
        table_name=table_name,
        cobol_fields=cobol_fields,
    )

self.diagnostics.append(
    f"{source_label}: no SQL columns or COBOL host fields found."
)

return []

def _find_table_name(
    self,
    text: str,
) -> str:
    for pattern in [
        DECLARE_TABLE_PATTERN,
        DECLARE_TABLE_ALT_PATTERN,
        DCLGEN_TABLE_COMMENT_PATTERN,
    ]:
        match = pattern.search(text)

        if match:
            return self._normalize_sql_name(match.group(1))

    return ""

def _parse_sql_declare_columns(
    self,
    text: str,
    table_name: str,
    source_label: str,
) -> list[dict[str, object]]:
    sections = self._extract_declare_sections(text)

    if not sections:
        self.diagnostics.append(
            f"{source_label}: no EXEC SQL DECLARE TABLE section found."
        )
        return []

    output: list[dict[str, object]] = []

    for section_index, section in enumerate(sections, start=1):
        section_table = self._find_table_name(section) or table_name
        body = self._extract_parenthesized_body(section)

        if not body:
            self.diagnostics.append(
                f"{source_label}: DECLARE section {section_index} has no column body."
            )
            continue

        parsed = self._parse_declare_body(
            body=body,
            table_name=section_table,
        )

        self.diagnostics.append(
            f"{source_label}: DECLARE section {section_index} parsed SQL columns: {len(parsed)}"
        )

        output.extend(parsed)

    return output

def _extract_declare_sections(
    self,
    text: str,
) -> list[str]:
    lines = text.splitlines()
    sections: list[str] = []
    current: list[str] = []

```

```

inside = False

for line in lines:
    upper = line.upper()

    if "DECLARE" in upper and "TABLE" in upper:
        inside = True
        current = [line]
        continue

    if inside:
        current.append(line)

        if "END-EXEC" in upper:
            sections.append("\n".join(current))
            current = []
            inside = False
            continue

        if upper.strip() == ")" or upper.strip().startswith(")"):
            sections.append("\n".join(current))
            current = []
            inside = False
            continue

    if inside and current:
        sections.append("\n".join(current))

return sections

def _extract_parenthesized_body(
    self,
    section: str,
) -> str:
    start = section.find("(")

    if start < 0:
        return ""

    depth = 0
    body_chars: list[str] = []

    for index in range(start, len(section)):
        char = section[index]

        if char == "(":
            depth += 1

            if depth == 1:
                continue

        if char == ")":
            depth -= 1

            if depth == 0:
                break

        if depth >= 1:
            body_chars.append(char)

    return "".join(body_chars)

def _parse_declare_body(
    self,
    body: str,
    table_name: str,
) -> list[dict[str, object]]:
    logical_items = self._split_sql_items(body)

    inline_columns: list[dict[str, object]] = []
    column_names: list[str] = []
    db2_types: list[str] = []

    for item in logical_items:
        normalized_item = self._normalize_sql_item(item)

```

```

        if not normalized_item:
            continue

        inline = self._parse_inline_column_definition(
            normalized_item,
            table_name=table_name,
        )

        if inline:
            inline_columns.append(inline)
            continue

        if self._looks_like_column_name(normalized_item):
            column_names.append(
                self._normalize_sql_name(normalized_item)
            )
            continue

        if self._looks_like_db2_type(normalized_item):
            db2_types.append(
                self._normalize_db2_type(normalized_item)
            )
            continue

    if inline_columns:
        return inline_columns

    output: list[dict[str, object]] = []

    for index, column_name in enumerate(column_names):
        db2_type = ""

        if index < len(db2_types):
            db2_type = db2_types[index]

        output.append(
            {
                "table_name": table_name,
                "column_name": column_name,
                "db2_type": db2_type,
                "nullable": self._is_nullable_db2_type(db2_type),
            }
        )

    return output

def _split_sql_items(
    self,
    body: str,
) -> list[str]:
    items: list[str] = []
    current: list[str] = []
    depth = 0

    normalized = body.replace("\n", " ")

    for char in normalized:
        if char == "(":
            depth += 1
            current.append(char)
            continue

        if char == ")":
            if depth > 0:
                depth -= 1

            current.append(char)
            continue

        if char == "," and depth == 0:
            item = "".join(current).strip()

            if item:
                items.append(item)

            current = []

```

```

        continue

        current.append(char)

    item = "".join(current).strip()

    if item:
        items.append(item)

    expanded: list[str] = []

    for item in items:
        expanded.extend(
            self._split_possible_stacked_column_or_type_lines(item)
        )

    return expanded

def _split_possible_stacked_column_or_type_lines(
    self,
    item: str,
) -> list[str]:
    text = " ".join(str(item or "").split())

    if not text:
        return []

    inline = self._parse_inline_column_definition(
        text,
        table_name="",
    )

    if inline:
        return [text]

    tokens = text.split()

    if len(tokens) <= 1:
        return [text]

    if self._looks_like_db2_type(text):
        return [text]

    if all(self._looks_like_column_name(token) for token in tokens):
        return tokens

    return [text]

def _parse_inline_column_definition(
    self,
    item: str,
    table_name: str,
) -> dict[str, object] | None:
    tokens = item.split()

    if len(tokens) < 2:
        return None

    first = self._normalize_sql_name(tokens[0])

    if not self._looks_like_column_name(first):
        return None

    remaining = " ".join(tokens[1:])

    if not self._looks_like_db2_type(remaining):
        return None

    db2_type = self._normalize_db2_type(remaining)

    return {
        "table_name": table_name,
        "column_name": first,
        "db2_type": db2_type,
        "nullable": self._is_nullable_db2_type(db2_type),
    }

```

```

def _looks_like_column_name(
    self,
    value: str,
) -> bool:
    text = self._normalize_sql_name(value)

    if not text:
        return False

    if text in DB2_SQL_SKIP_WORDS:
        return False

    if text.split(" ")[0] in DB2_SQL_TYPE_STARTERS:
        return False

    return bool(VALID_SQL_COLUMN_NAME_PATTERN.fullmatch(text))

def _looks_like_db2_type(
    self,
    value: str,
) -> bool:
    text = str(value or "").strip().upper()

    if not text:
        return False

    first = text.split()[0]
    first = first.split("(", 1)[0]

    return first in DB2_SQL_TYPE_STARTERS

def _normalize_sql_item(
    self,
    value: str,
) -> str:
    text = str(value or "").strip()
    text = text.strip(",").strip()
    text = " ".join(text.split())

    return text

def _normalize_db2_type(
    self,
    value: str,
) -> str:
    text = " ".join(
        str(value or "").replace(" ", ",").split()
    )

    text = text.rstrip(",")

    return text.upper()

def _is_nullable_db2_type(
    self,
    db2_type: str,
) -> bool:
    return "NOT NULL" not in str(db2_type or "").upper()

def _parse_cobol_host_fields(
    self,
    text: str,
    source_label: str,
) -> list[dict[str, str]]:
    fields: list[dict[str, str]] = []

    current_group = ""
    seen_group = False

    for raw_line in text.splitlines():
        line = raw_line.rstrip()
        group_match = COBOL_GROUP_PATTERN.match(line)

        if group_match:
            current_group = group_match.group(1).upper()

```

```

        seen_group = True
        continue

    if not seen_group:
        continue

    match = COBOL_FIELD_PATTERN.match(line)

    if not match:
        continue

    level = match.group(1)
    name = match.group(2).upper()
    body = match.group("body") or ""
    nullable = name.upper().endswith("-NULL")

    pic = self._picture(body)
    usage = self._usage(body)

    if not pic and not usage:
        continue

    fields.append(
        {
            "group": current_group,
            "level": level,
            "name": name,
            "picture": pic,
            "usage": usage,
            "nullable_indicator": "Y" if nullable else "N",
        }
    )

    self.diagnostics.append(
        f"{source_label}: COBOL host field scan found {len(fields)} field(s)."
    )

    return fields

def _picture(
    self,
    text: str,
) -> str:
    match = DCLGEN_PIC_PATTERN.search(str(text or ""))

    if not match:
        return ""

    return match.group("pic").strip().upper()

def _usage(
    self,
    text: str,
) -> str:
    match = DCLGEN_USAGE_PATTERN.search(str(text or ""))

    if not match:
        return ""

    return match.group(1).strip().upper()

def _merge_sql_columns_with_cobol_hosts(
    self,
    sql_columns: list[dict[str, object]],
    cobol_fields: list[dict[str, str]],
    fallback_table_name: str,
) -> list[DclgenColumn]:
    output: list[DclgenColumn] = []
    usable_hosts = self._usable_host_fields(cobol_fields)

    for index, sql_column in enumerate(sql_columns):
        table_name = self._normalize_sql_name(
            str(
                sql_column.get("table_name", "")
                or fallback_table_name
                or ""
            )

```

```

    )
)

column_name = self._normalize_sql_name(
    str(sql_column.get("column_name", ""))
)

db2_type = str(sql_column.get("db2_type", "") or "").strip()
nullable = bool(sql_column.get("nullable", True))

host = self._best_host_for_column(
    column_name=column_name,
    index=index,
    usable_hosts=usable_hosts,
)

output.append(
    DclgenColumn(
        table_name=table_name,
        column_name=column_name,
        db2_type=db2_type,
        cobol_host_name=host.get("name", column_name),
        cobol_picture=host.get("picture", ""),
        cobol_usage=host.get("usage", ""),
        nullable=nullable,
    )
)

return output

def _usable_host_fields(
    self,
    cobol_fields: list[dict[str, str]],
) -> list[dict[str, str]]:
    output: list[dict[str, str]] = []

    for field in cobol_fields:
        name = str(field.get("name", "")).upper()

        if not name:
            continue

        if any(
            name.endswith(suffix)
            for suffix in DCLGEN_HOST_FIELD_SUFFIXES_TO_IGNORE
        ):
            continue

        output.append(field)

    return output

def _best_host_for_column(
    self,
    column_name: str,
    index: int,
    usable_hosts: list[dict[str, str]],
) -> dict[str, str]:
    normalized_column = self._normalize_compare_name(column_name)

    for host in usable_hosts:
        host_name = self._normalize_compare_name(host.get("name", ""))

        if host_name == normalized_column:
            return host

    for host in usable_hosts:
        host_name = self._normalize_compare_name(host.get("name", ""))

        if normalized_column and normalized_column in host_name:
            return host

        if host_name and host_name in normalized_column:
            return host

    if index < len(usable_hosts):

```



```

        return usable_hosts[index]

    return {
        "name": NameNormalizer.to_cobol(column_name),
        "picture": "",
        "usage": "",
    }

def _fallback_columns_from_cobol_fields(
    self,
    table_name: str,
    cobol_fields: list[dict[str, str]],
) -> list[DclgenColumn]:
    output: list[DclgenColumn] = []

    for field in self._usable_host_fields(cobol_fields):
        host_name = field.get("name", "")

        if not host_name:
            continue

        output.append(
            DclgenColumn(
                table_name=table_name,
                column_name=self._normalize_cobol_name_to_db2(host_name),
                db2_type="",
                cobol_host_name=host_name,
                cobol_picture=field.get("picture", ""),
                cobol_usage=field.get("usage", ""),
                nullable=True,
            )
        )

    return output

def _normalize_sql_name(
    self,
    value: str,
) -> str:
    text = str(value or "").strip()
    text = text.strip("'").strip('"')
    text = text.strip(">").strip("[").strip("]")

    if "." in text:
        text = text.split(".")[1]

    text = text.replace("-", "_")
    text = SQL_NAME_CLEANUP_PATTERN.sub("_", text.upper())
    text = MULTIPLE_UNDERSCORE_PATTERN.sub("_", text)

    return text.strip("_")

def _normalize_cobol_name_to_db2(
    self,
    value: str,
) -> str:
    text = str(value or "").strip().upper()
    text = text.replace("-", "_")
    text = SQL_NAME_CLEANUP_PATTERN.sub("_", text)
    text = MULTIPLE_UNDERSCORE_PATTERN.sub("_", text)

    return text.strip("_")

def _normalize_compare_name(
    self,
    value: str,
) -> str:
    text = self._normalize_cobol_name_to_db2(value)

    if text.startswith("DCL_"):
        return text[4:]

    if text.startswith("DCL"):
        return text[3:]

    return text

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\parsers\sheet_mapping_parser.py
ENCODING: utf-8
=====

import csv
from io import BytesIO
from io import StringIO

from openpyxl import load_workbook

from catalogs.sheet_mapping_schema import (
    SHEET_MAPPING_CANONICAL_COLUMNS,
    SHEET_MAPPING_FIELD_ALIASES,
    SHEET_MAPPING_HEADER_DETECTION_GROUPS,
    SHEET_MAPPING_MODEL_FIELD_MAP,
)
from patterns.sheet_mapping_patterns import (
    CELL_WHITESPACE_PATTERN,
    HEADER_NON_ALPHANUMERIC_PATTERN,
    HEADER_WHITESPACE_PATTERN,
)
from src.idms_db2_phase2.domain.models import SheetMappingRow

class SheetMappingParser:
    """
    Parses Sheet Mapping files from CSV/text or XLSX.

    This class intentionally does not own:
    - Sheet Mapping column names
    - Sheet Mapping aliases
    - Header detection groups
    - Regex patterns

    Those belong to:
    - catalogs/sheet_mapping_schema.py
    - patterns/sheet_mapping_patterns.py
    """

    def __init__(self) -> None:
        self.diagnostics: list[str] = []

    def parse_uploaded_file(
        self,
        uploaded_file,
    ) -> list[SheetMappingRow]:
        self.diagnostics = []

        if uploaded_file is None:
            self.diagnostics.append("No Sheet Mapping file supplied.")
            return []

        file_name = str(uploaded_file.name or "").lower()
        raw_bytes = uploaded_file.getvalue()

        self.diagnostics.append(f"Sheet Mapping file name: {file_name}")
        self.diagnostics.append(f"Sheet Mapping file size bytes: {len(raw_bytes)}")

        if file_name.endswith(".xlsx"):
            return self.parse_xlsx_bytes(raw_bytes)

        if file_name.endswith(".xls"):
            self.diagnostics.append(
                "Unsupported .xls file detected. Save the file as .xlsx or .csv."
            )
            return []

        text = raw_bytes.decode(
            "utf-8-sig",
            errors="ignore",
        )

        self.diagnostics.append(f"CSV/text decoded length: {len(text)}")

```

```

        if text:
            sample = text[:500].replace("\r", "\\r").replace("\n", "\\n")
            self.diagnostics.append(f"CSV/text sample: {sample}")

        return self.parse_csv_text(text)

def parse_csv_text(
    self,
    text: str,
) -> list[SheetMappingRow]:
    if not str(text or "").strip():
        self.diagnostics.append("CSV/text Sheet Mapping is empty.")
        return []

    stream = StringIO(text)
    reader = csv.reader(stream)
    raw_rows = [tuple(row) for row in reader]

    if not raw_rows:
        self.diagnostics.append("CSV/text Sheet Mapping has no rows.")
        return []

    headers = [
        self._cell_to_string(value)
        for value in raw_rows[0]
    ]

    self.diagnostics.append(f"CSV detected headers: {headers}")

    output: list[SheetMappingRow] = []

    for row in raw_rows[1:]:
        raw_row = self._row_to_dict(
            headers=headers,
            row=row,
        )

        mapping_row = self._to_mapping_row(raw_row)

        if self._has_useful_content(mapping_row):
            output.append(mapping_row)

    self.diagnostics.append(f"CSV parsed useful rows: {len(output)}")
    self._add_population_diagnostics(output)

    return output

def parse_xlsx_bytes(
    self,
    raw_bytes: bytes,
) -> list[SheetMappingRow]:
    if not raw_bytes:
        self.diagnostics.append("XLSX Sheet Mapping is empty.")
        return []

    workbook = load_workbook(
        BytesIO(raw_bytes),
        data_only=True,
        read_only=True,
    )

    output: list[SheetMappingRow] = []

    for worksheet in workbook.worksheets:
        rows = list(worksheet.iter_rows(values_only=True))

        if not rows:
            continue

        header_index = self._find_header_row(
            rows=rows,
            sheet_title=worksheet.title,
        )

        if header_index < 0:

```

```

        self.diagnostics.append(
            f"Sheet {worksheet.title}: no Sheet Mapping header row detected."
        )
        continue

    parsed_rows = self._parse_xlsx_rows_from_header(
        raw_rows=rows,
        header_index=header_index,
    )

    self.diagnostics.append(
        f"Sheet {worksheet.title}: parsed useful rows: {len(parsed_rows)}"
    )

    output.extend(parsed_rows)

    self.diagnostics.append(f"XLSX parsed useful rows: {len(output)}")
    self._add_population_diagnostics(output)

    return output

def _parse_xlsx_rows_from_header(
    self,
    raw_rows: list[tuple],
    header_index: int,
) -> list[SheetMappingRow]:
    headers = [
        self._cell_to_string(value)
        for value in raw_rows[header_index]
    ]

    output: list[SheetMappingRow] = []

    for row in raw_rows[header_index + 1:]:
        raw_row = self._row_to_dict(
            headers=headers,
            row=row,
        )

        mapping_row = self._to_mapping_row(raw_row)

        if self._has_useful_content(mapping_row):
            output.append(mapping_row)

    return output

def _row_to_dict(
    self,
    headers: list[str],
    row: tuple,
) -> dict[str, str]:
    raw_row: dict[str, str] = {}

    for index, header in enumerate(headers):
        clean_header = self._cell_to_string(header)

        if not clean_header:
            continue

        value = row[index] if index < len(row) else ""
        raw_row[clean_header] = self._cell_to_string(value)

    return raw_row

def _find_header_row(
    self,
    rows: list[tuple],
    sheet_title: str,
) -> int:
    for index, row in enumerate(rows[:100]):
        normalized_cells = {
            self._normalize_header(self._cell_to_string(value))
            for value in row
            if value is not None
        }

```

```

        if index < 10:
            self.diagnostics.append(
                f"Sheet {sheet_title}: row {index} normalized cells: "
                f"{sorted(normalized_cells)}"
            )

        if self._normalize_header(SHEET_MAPPING_CANONICAL_COLUMNS[0]) in normalized_cells:
            return index

        if self._row_has_any_header(
            normalized_cells=normalized_cells,
            aliases=SHEET_MAPPING_HEADER_DETECTION_GROUPS[0],
        ):
            return index

        if (
            self._row_has_any_header(
                normalized_cells=normalized_cells,
                aliases=SHEET_MAPPING_HEADER_DETECTION_GROUPS[1],
            )
            and self._row_has_any_header(
                normalized_cells=normalized_cells,
                aliases=SHEET_MAPPING_HEADER_DETECTION_GROUPS[2],
            )
        ):
            return index

    return -1

def _row_has_any_header(
    self,
    normalized_cells: set[str],
    aliases: list[str],
) -> bool:
    for alias in aliases:
        if self._normalize_header(alias) in normalized_cells:
            return True

    return False

def _to_mapping_row(
    self,
    raw_row: dict[str, str],
) -> SheetMappingRow:
    values: dict[str, str] = {}

    for model_field, canonical_column in SHEET_MAPPING_MODEL_FIELD_MAP.items():
        values[model_field] = self._get(
            row=raw_row,
            canonical_name=canonical_column,
        )

    return SheetMappingRow(**values)

def _get(
    self,
    row: dict[str, str],
    canonical_name: str,
) -> str:
    aliases = SHEET_MAPPING_FIELD_ALIASES.get(
        canonical_name,
        [canonical_name],
    )

    normalized_lookup = {
        self._normalize_header(key): value
        for key, value in row.items()
    }

    for alias in aliases:
        normalized_alias = self._normalize_header(alias)
        value = normalized_lookup.get(normalized_alias)

        if value is not None:
            return str(value).strip()

```

```

        return ""

def _cell_to_string(
    self,
    value,
) -> str:
    if value is None:
        return ""

    text = str(value)
    text = text.replace("\uffff", "")
    text = text.replace("\xa0", " ")
    text = text.replace("\r\n", "\n")
    text = text.replace("\r", "\n")
    text = CELL_WHITESPACE_PATTERN.sub(" ", text)

    return text.strip()

def _normalize_header(
    self,
    value: str,
) -> str:
    text = self._cell_to_string(value)
    text = text.upper()
    text = text.replace("_", " ")
    text = HEADER_NON_ALPHANUMERIC_PATTERN.sub("", text)
    text = HEADER_WHITESPACE_PATTERN.sub(" ", text)

    return text.strip()

def _has_useful_content(
    self,
    row: SheetMappingRow,
) -> bool:
    values = [
        getattr(row, field_name)
        for field_name in SHEET_MAPPING_MODEL_FIELD_MAP
    ]

    return any(str(value or "").strip() for value in values)

def _add_population_diagnostics(
    self,
    rows: list[SheetMappingRow],
) -> None:
    record_count = sum(
        1
        for row in rows
        if str(row.cobol_record_idms or "").strip()
    )

    source_field_count = sum(
        1
        for row in rows
        if str(row.cobol_zone or "").strip()
        or str(row.reference_field_name_copybook or "").strip()
    )

    db2_table_count = sum(
        1
        for row in rows
        if str(row.new_db2_record or "").strip()
        or str(row.cross_application_db2_table or "").strip()
    )

    db2_column_count = sum(
        1
        for row in rows
        if str(row.new_db2_field_name or "").strip()
        or str(row.cross_application_db2_field_name or "").strip()
    )

    useful_context_count = sum(
        1
        for row in rows
        if (

```

```

        str(row.cobol_record_idms or "").strip()
        and (
            str(row.cobol_zone or "").strip()
            or str(row.reference_field_name_copybook or "").strip()
        )
        and (
            str(row.new_db2_record or "").strip()
            or str(row.cross_application_db2_table or "").strip()
        )
        and (
            str(row.new_db2_field_name or "").strip()
            or str(row.cross_application_db2_field_name or "").strip()
        )
    )
)

self.diagnostics.append(
    f"Sheet Mapping populated Cobol Record IDMS rows: {record_count}"
)
self.diagnostics.append(
    f"Sheet Mapping populated source field rows: {source_field_count}"
)
self.diagnostics.append(
    f"Sheet Mapping populated DB2 table rows: {db2_table_count}"
)
self.diagnostics.append(
    f"Sheet Mapping populated DB2 column rows: {db2_column_count}"
)
self.diagnostics.append(
    f"Sheet Mapping useful source-to-target rows: {useful_context_count}"
)
)

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\parsers\text_loader.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.infrastructure.file_loader import FileLoader

```

```

class TextLoader(FileLoader):
    """
    Backward-compatible alias for FileLoader.

    Existing UI and testing code may import TextLoader from parsers.
    Clean architecture keeps actual file loading in infrastructure.
    """

    pass

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\repositories\__init__.py
ENCODING: utf-8
=====

```

```

[Empty file]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\repositories\copybook_repository.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import CopybookField
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class CopybookRepository:
    """
    Repository wrapper for optional copybook fields.
    """

    def __init__(
        self,
    )

```

```

        fields: list[CopybookField] | None = None,
    ) -> None:
        self.fields = fields or []

    def all(
        self,
    ) -> list[CopybookField]:
        return list(self.fields)

    def count(
        self,
    ) -> int:
        return len(self.fields)

    def names(
        self,
    ) -> list[str]:
        values = {
            NameNormalizer.normalize(field.name)
            for field in self.fields
            if NameNormalizer.normalize(field.name)
        }

        return sorted(values)

    def find(
        self,
        field_name: str,
    ) -> CopybookField | None:
        target = NameNormalizer.normalize(field_name)

        if not target:
            return None

        for field in self.fields:
            if NameNormalizer.normalize(field.name) == target:
                return field

        return None

    def picture_for(
        self,
        field_name: str,
    ) -> str:
        field = self.find(field_name)

        if field is None:
            return ""

        return field.picture

    def usage_for(
        self,
        field_name: str,
    ) -> str:
        field = self.find(field_name)

        if field is None:
            return ""

        return field.usage

    def occurs_for(
        self,
        field_name: str,
    ) -> str:
        field = self.find(field_name)

        if field is None:
            return ""

        return field.occurs

    def has_field(
        self,
        field_name: str,

```



```

) -> bool:
    return self.find(field_name) is not None

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\repositories\dclgen_repository.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import DclgenColumn
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class DclgenRepository:
    """
    Repository wrapper for DCLGEN columns.

    DCLGEN is the authority for COBOL host-variable spelling, group names,
    and PIC clauses.
    """

    def __init__(
        self,
        columns: list[DclgenColumn] | None = None,
    ) -> None:
        self.columns = columns or []

    def all(
        self,
    ) -> list[DclgenColumn]:
        return list(self.columns)

    def count(
        self,
    ) -> int:
        return len(self.columns)

    def tables(
        self,
    ) -> list[str]:
        values = {
            NameNormalizer.normalize(column.table_name)
            for column in self.columns
            if NameNormalizer.normalize(column.table_name)
        }

        return sorted(values)

    def columns_for_table(
        self,
        table_name: str,
    ) -> list[DclgenColumn]:
        target = NameNormalizer.normalize(table_name)

        if not target:
            return []

        return [
            column
            for column in self.columns
            if NameNormalizer.normalize(column.table_name) == target
        ]

    def column_names_for_table(
        self,
        table_name: str,
    ) -> list[str]:
        output: list[str] = []
        seen: set[str] = set()

        for column in self.columns_for_table(table_name):
            name = NameNormalizer.normalize(column.column_name)

            if not name:
                continue

```

```

        if name in seen:
            continue

        seen.add(name)
        output.append(name)

    return output

def has_table(
    self,
    table_name: str,
) -> bool:
    return bool(self.columns_for_table(table_name))

def has_column(
    self,
    table_name: str,
    column_name: str,
) -> bool:
    target_column = NameNormalizer.normalize(column_name)

    if not target_column:
        return False

    return target_column in self.column_names_for_table(table_name)

def host_for_column(
    self,
    table_name: str,
    column_name: str,
) -> str:
    target_column = NameNormalizer.normalize(column_name)

    if not target_column:
        return ""

    for column in self.columns_for_table(table_name):
        if NameNormalizer.normalize(column.column_name) == target_column:
            return NameNormalizer.to_cobol(
                column.cobol_host_name or column.column_name
            )

    return ""

def group_for_table(
    self,
    table_name: str,
) -> str:
    table = NameNormalizer.normalize(table_name)

    if not table:
        return ""

    return "DCL" + NameNormalizer.to_cobol(table)

def host_reference_for_column(
    self,
    table_name: str,
    column_name: str,
) -> str:
    group = self.group_for_table(table_name)
    host = self.host_for_column(
        table_name=table_name,
        column_name=column_name,
    )

    if not group or not host:
        return ""

    return f":{host} OF {group}"

def valid_host_references(
    self,
) -> set[str]:
    output: set[str] = set()

```

```

    for column in self.columns:
        table = NameNormalizer.normalize(column.table_name)
        group = self.group_for_table(table)
        host = NameNormalizer.to_cobol(
            column.cobol_host_name or column.column_name
        )

        if group and host:
            output.add(f"{group}.{host}")
            output.add(f"{group} {host}")
            output.add(host)

    return output

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\repositories\mapping_repository.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import SheetMappingRow
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class MappingRepository:
    """
    Repository wrapper for Sheet Mapping rows.

    Services should use this repository instead of repeatedly scanning raw
    SheetMappingRow lists.
    """

    def __init__(
        self,
        rows: list[SheetMappingRow] | None = None,
    ) -> None:
        self.rows = rows or []

    def all(
        self,
    ) -> list[SheetMappingRow]:
        return list(self.rows)

    def count(
        self,
    ) -> int:
        return len(self.rows)

    def records(
        self,
    ) -> list[str]:
        values = {
            NameNormalizer.normalize(row.cobol_record_idms)
            for row in self.rows
            if NameNormalizer.normalize(row.cobol_record_idms)
        }

        return sorted(values)

    def tables(
        self,
    ) -> list[str]:
        values = {
            NameNormalizer.normalize(
                row.new_db2_record or row.cross_application_db2_table
            )
            for row in self.rows
            if NameNormalizer.normalize(
                row.new_db2_record or row.cross_application_db2_table
            )
        }

        return sorted(values)

    def rows_for_record(
        self,

```

```

        record_name: str,
    ) -> list[SheetMappingRow]:
        target = NameNormalizer.normalize(record_name)

        if not target:
            return []

        return [
            row
            for row in self.rows
            if NameNormalizer.normalize(row.cobol_record_idms) == target
        ]

def rows_for_table(
    self,
    table_name: str,
) -> list[SheetMappingRow]:
    target = NameNormalizer.normalize(table_name)

    if not target:
        return []

    return [
        row
        for row in self.rows
        if NameNormalizer.normalize(row.new_db2_record) == target
        or NameNormalizer.normalize(row.cross_application_db2_table) == target
    ]

def db2_columns_for_table(
    self,
    table_name: str,
) -> list[str]:
    rows = self.rows_for_table(table_name)
    output: list[str] = []
    seen: set[str] = set()

    for row in rows:
        column = NameNormalizer.normalize(
            row.new_db2_field_name
            or row.cross_application_db2_field_name
        )

        if not column:
            continue

        if column in seen:
            continue

        seen.add(column)
        output.append(column)

    return output

def db2_table_for_record(
    self,
    record_name: str,
) -> str:
    rows = self.rows_for_record(record_name)

    for row in rows:
        table = NameNormalizer.normalize(row.new_db2_record)

        if table:
            return table

    for row in rows:
        table = NameNormalizer.normalize(row.cross_application_db2_table)

        if table:
            return table

    return ""

def key_columns_for_record(
    self,

```

```

        record_name: str,
    ) -> list[str]:
        rows = self.rows_for_record(record_name)
        output: list[str] = []
        seen: set[str] = set()

        for row in rows:
            key_text = " ".join(
                [
                    str(row.idms_key or ""),
                    str(row.db2_key or ""),
                ]
            ).upper()

            if "KEY" not in key_text and "PRIMARY" not in key_text and "CALC" not in key_text:
                continue

            column = NameNormalizer.normalize(row.new_db2_field_name)

            if not column:
                continue

            if column in seen:
                continue

            seen.add(column)
            output.append(column)

        return output

    def has_record(
        self,
        record_name: str,
    ) -> bool:
        return bool(self.rows_for_record(record_name))

    def has_table(
        self,
        table_name: str,
    ) -> bool:
        return bool(self.rows_for_table(table_name))

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\resolvers\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\resolvers\column_name_resolver.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class ColumnNameResolver:
    """
    Resolves DB2 column names.

    Authority:
    - Sheet Mapping provides DB2 column names.
    - DCLGEN validates whether a column exists for the resolved table.
    """

    def __init__(
        self,
        mapping_repository: MappingRepository,
        dclgen_repository: DclgenRepository,
        table_name_resolver: TableNameResolver,
    ):

```

```

) -> None:
    self.mapping_repository = mapping_repository
    self.dclgen_repository = dclgen_repository
    self.table_name_resolver = table_name_resolver

def columns_for_record(
    self,
    record_name: str,
) -> list[str]:
    table_name = self.table_name_resolver.table_for_record(record_name)

    if not table_name:
        return []

    mapped_columns = self.mapping_repository.db2_columns_for_table(table_name)

    if not mapped_columns:
        mapped_table = self.mapping_repository.db2_table_for_record(record_name)
        mapped_columns = self.mapping_repository.db2_columns_for_table(mapped_table)

    return self._filter_to_existing_dclgen_columns(
        table_name=table_name,
        columns=mapped_columns,
    )

def columns_for_table(
    self,
    table_name: str,
) -> list[str]:
    resolved_table = self.table_name_resolver.resolve_table(table_name)

    if not resolved_table:
        return []

    mapped_columns = self.mapping_repository.db2_columns_for_table(table_name)

    if mapped_columns:
        return self._filter_to_existing_dclgen_columns(
            table_name=resolved_table,
            columns=mapped_columns,
        )

    return self.dclgen_repository.column_names_for_table(resolved_table)

def key_columns_for_record(
    self,
    record_name: str,
) -> list[str]:
    table_name = self.table_name_resolver.table_for_record(record_name)

    if not table_name:
        return []

    mapped_keys = self.mapping_repository.key_columns_for_record(record_name)

    return self._filter_to_existing_dclgen_columns(
        table_name=table_name,
        columns=mapped_keys,
    )

def has_column(
    self,
    table_name: str,
    column_name: str,
) -> bool:
    resolved_table = self.table_name_resolver.resolve_table(table_name)
    normalized_column = NameNormalizer.normalize(column_name)

    if not resolved_table or not normalized_column:
        return False

    return self.dclgen_repository.has_column(
        table_name=resolved_table,
        column_name=normalized_column,
    )

```

```

def _filter_to_existing_dclgen_columns(
    self,
    table_name: str,
    columns: list[str],
) -> list[str]:
    resolved_table = self.table_name_resolver.resolve_table(table_name)

    if not resolved_table:
        return []

    dclgen_columns = set(
        self.dclgen_repository.column_names_for_table(resolved_table)
    )

    if not dclgen_columns:
        return self._unique(columns)

    output: list[str] = []

    for column in columns:
        normalized = NameNormalizer.normalize(column)

        if not normalized:
            continue

        if normalized not in dclgen_columns:
            continue

        output.append(normalized)

    return self._unique(output)

def _unique(
    self,
    values: list[str],
) -> list[str]:
    output: list[str] = []
    seen: set[str] = set()

    for value in values:
        normalized = NameNormalizer.normalize(value)

        if not normalized:
            continue

        if normalized in seen:
            continue

        seen.add(normalized)
        output.append(normalized)

    return output

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\resolvers\cursor_column_resolver.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.analyzers.field_usage_analyzer import FieldUsageAnalysis
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.relationship_resolver import RelationshipResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.timestamp_audit_rules import AUDIT_COLUMN_PREFIXES

```

```

class CursorColumnResolver:
    """
    Resolves minimal cursor SELECT and ORDER BY columns generically.

    SELECT priority:
    1. DCLGEN host fields used in procedure logic.
    2. Fields used in procedure conditions.
    3. Fields used as MOVE sources for output writes.
    """

```

4. Parent key columns required by child cursor relationships.
5. Child order key columns.
6. Fallback to mapped non-audit columns only if usage is unavailable.

ORDER BY rule:

- Parent/root cursors do not get ORDER BY unless explicit order metadata exists in Sheet Mapping.
- Child cursors may order by non-FK primary/sequence key.
- DESC is added for child sequence/order key where metadata indicates sequence/event/latest-first semantics.

"""

ORDER_HINT_WORDS = (

"ORDER",
"ORDERBY",
"SORT",
"SORTKEY",
"RANK",

)

DESC_HINT_WORDS = (

"SEQ",
"SEQUENCE",
"IDMSKEY",
"IDENTIFIERSEQ",
"IDENTIFIERSEQSPECIAL",
"EVENT",
"EVPR",
"EVEF",
"AUTO",
"INCREMENT",

)

def __init__(

self,
mapping_repository: MappingRepository,
table_name_resolver: TableNameResolver,
column_name_resolver: ColumnNameResolver,
relationship_resolver: RelationshipResolver,

) -> None:

self.mapping_repository = mapping_repository
self.table_name_resolver = table_name_resolver
self.column_name_resolver = column_name_resolver
self.relationship_resolver = relationship_resolver

def select_columns_for_record(

self,
record_name: str,
field_usage_analysis: FieldUsageAnalysis | None = None,

) -> list[str]:

record = NameNormalizer.normalize(record_name)

if not record:

return []

selected: list[str] = []

selected.extend(

self._columns_from_field_usage(
record_name=record,
field_usage_analysis=field_usage_analysis,

)

)

selected.extend(

self.relationship_resolver.parent_key_columns_required_by_children(record)

)

if self.relationship_resolver.has_foreign_keys(record):

selected.extend(
self._order_by_base_columns_for_record(record)

)

selected = self._valid_non_audit_columns(

record_name=record,
columns=selected,


```

    )

    if selected:
        return selected

    fallback = self.column_name_resolver.columns_for_record(record)

    return self._valid_non_audit_columns(
        record_name=record,
        columns=fallback,
    )

def order_by_columns_for_record(
    self,
    record_name: str,
) -> list[str]:
    record = NameNormalizer.normalize(record_name)

    if not record:
        return []

    is_child = self.relationship_resolver.has_foreign_keys(record)

    if not is_child:
        return self._explicit_parent_order_columns(record)

    order_columns = self._order_by_base_columns_for_record(record)

    output: list[str] = []

    for column in order_columns:
        expression = column

        if self._should_order_desc(
            record_name=record,
            column_name=column,
        ):
            expression = f"{column} DESC"

        output.append(expression)

    return self._unique(output)

def _explicit_parent_order_columns(
    self,
    record_name: str,
) -> list[str]:
    """
    Parent/root cursor gets ORDER BY only if Sheet Mapping explicitly
    indicates order/sort/rank semantics.

    This prevents automatic ORDER BY on normal parent PK columns.
    """

    rows = self.mapping_repository.rows_for_record(record_name)
    output: list[str] = []

    for row in rows:
        text = " ".join(
            [
                str(getattr(row, "remarks", "")) or "",
                str(getattr(row, "hopex_expression_type_remark", "")) or "",
                str(getattr(row, "relation", "")) or "",
                str(getattr(row, "basetype", "")) or "",
            ]
        ).upper()

        if not any(word in text for word in self.ORDER_HINT_WORDS):
            continue

        column = NameNormalizer.normalize(
            getattr(row, "new_db2_field_name", "")
        )

        if not column:
            continue

```

```

        if not self._column_exists_for_record(record_name, column):
            continue

        if self._is_audit_column(column):
            continue

        output.append(column)

    return self._unique(output)

def _order_by_base_columns_for_record(
    self,
    record_name: str,
) -> list[str]:
    return self.relationship_resolver.order_by_columns_for_record(record_name)

def _should_order_desc(
    self,
    record_name: str,
    column_name: str,
) -> bool:
    record = NameNormalizer.normalize(record_name)
    column = NameNormalizer.normalize(column_name)

    rows = self.mapping_repository.rows_for_record(record)

    for row in rows:
        row_column = NameNormalizer.normalize(
            getattr(row, "new_db2_field_name", "")
        )

        if row_column != column:
            continue

        combined_text = " ".join(
            [
                str(getattr(row, "db2_key", "") or ""),
                str(getattr(row, "hopex_expression_type_remark", "") or ""),
                str(getattr(row, "remarks", "") or ""),
                str(getattr(row, "new_db2_data_type", "") or ""),
                str(getattr(row, "basetype", "") or ""),
                column,
            ]
        ).upper()

        if any(word in combined_text for word in self.DESC_HINT_WORDS):
            return True

    return False

def _columns_from_field_usage(
    self,
    record_name: str,
    field_usage_analysis: FieldUsageAnalysis | None,
) -> list[str]:
    if field_usage_analysis is None:
        return []

    usage = field_usage_analysis.usage_by_record.get(
        NameNormalizer.normalize(record_name)
    )

    if usage is None:
        return []

    output: list[str] = []

    for field_name in usage.all_fields:
        column = self._column_for_source_or_host_field(
            record_name=record_name,
            source_or_host_field=field_name,
        )

        if column:
            output.append(column)

```

```

        return self._unique(output)

def _column_for_source_or_host_field(
    self,
    record_name: str,
    source_or_host_field: str,
) -> str:
    record = NameNormalizer.normalize(record_name)
    source = NameNormalizer.normalize(source_or_host_field)

    if not record or not source:
        return ""

    rows = self.mapping_repository.rows_for_record(record)

    for row in rows:
        target_column = NameNormalizer.normalize(
            getattr(row, "new_db2_field_name", "")
            or getattr(row, "cross_application_db2_field_name", "")
        )

        if not target_column:
            continue

        source_candidates = {
            NameNormalizer.normalize(
                self._extract_source_field(getattr(row, "cobol_zone", ""))
            ),
            NameNormalizer.normalize(
                self._extract_source_field(
                    getattr(row, "reference_field_name_copybook", "")
                )
            ),
            target_column,
            NameNormalizer.normalize(NameNormalizer.to_cobol(target_column)),
        }

        source_candidates = {
            candidate
            for candidate in source_candidates
            if candidate
        }

        if source in source_candidates:
            return target_column

    return ""

def _valid_non_audit_columns(
    self,
    record_name: str,
    columns: list[str],
) -> list[str]:
    output: list[str] = []
    seen: set[str] = set()

    for column in columns:
        normalized = NameNormalizer.normalize(column)

        if not normalized:
            continue

        if normalized in seen:
            continue

        if self._is_audit_column(normalized):
            continue

        if not self._column_exists_for_record(record_name, normalized):
            continue

        seen.add(normalized)
        output.append(normalized)

    return output

```

```

def _column_exists_for_record(
    self,
    record_name: str,
    column_name: str,
) -> bool:
    table = self.table_name_resolver.table_for_record(record_name)

    if not table:
        return False

    return self.column_name_resolver.has_column(
        table_name=table,
        column_name=column_name,
    )

def _extract_source_field(
    self,
    value: str,
) -> str:
    text = str(value or "").strip()

    if not text:
        return ""

    text = text.replace(".", " ")

    tokens = text.split()

    if tokens and tokens[0].isdigit() and len(tokens) > 1:
        return tokens[1]

    if tokens:
        return tokens[0]

    return ""

def _is_audit_column(
    self,
    column_name: str,
) -> bool:
    column = NameNormalizer.normalize(column_name)

    return any(
        column.startswith(prefix)
        for prefix in AUDIT_COLUMN_PREFIXES
    )

def _unique(
    self,
    values: list[str],
) -> list[str]:
    output: list[str] = []
    seen: set[str] = set()

    for value in values:
        normalized = str(value or "").strip()

        if not normalized:
            continue

        key = NameNormalizer.normalize(normalized)

        if key in seen:
            continue

        seen.add(key)
        output.append(normalized)

    return output

```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\resolvers\cursor_name_resolver.py
ENCODING: utf-8
=====
```

```
from catalogs.db2_naming_catalog import DB2_CURSOR_SUFFIX, DB2_DEFAULT_CURSOR_NAME
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.cursor_rules import cursor_paragraph_number
```

```
class CursorNameResolver:
    """
    Resolves cursor names and cursor paragraph names.

    Authority:
    - Cursor names are derived from DB2 table or record names.
    - Cursor names are not derived from IDMS set names.
    """

    def cursor_name_from_table(
        self,
        table_name: str,
    ) -> str:
        table = NameNormalizer.normalize(table_name)

        if not table:
            return DB2_DEFAULT_CURSOR_NAME

        base = self._remove_table_suffix(table)

        if not base:
            return DB2_DEFAULT_CURSOR_NAME

        return NameNormalizer.to_cobol(base + DB2_CURSOR_SUFFIX)

    def paragraph_names(
        self,
        cursor_order: int,
        cursor_name: str,
    ) -> dict[str, str]:
        clean_cursor = NameNormalizer.to_cobol(cursor_name)

        return {
            "open": f"{cursor_paragraph_number(cursor_order, 'open')}-OPEN-{clean_cursor}",
            "fetch": f"{cursor_paragraph_number(cursor_order, 'fetch')}-FETCH-{clean_cursor}",
            "close": f"{cursor_paragraph_number(cursor_order, 'close')}-CLOSE-{clean_cursor}",
        }

    def paragraph_numbers(
        self,
        cursor_order: int,
    ) -> dict[str, int]:
        return {
            "open": cursor_paragraph_number(cursor_order, "open"),
            "fetch": cursor_paragraph_number(cursor_order, "fetch"),
            "close": cursor_paragraph_number(cursor_order, "close"),
        }

    def cursor_spec(
        self,
        cursor_order: int,
        table_name: str,
    ) -> dict[str, str | int]:
        cursor_name = self.cursor_name_from_table(table_name)
        paragraph_names = self.paragraph_names(
            cursor_order=cursor_order,
            cursor_name=cursor_name,
        )
        paragraph_numbers = self.paragraph_numbers(cursor_order)

        return {
            "cursor_name": cursor_name,
            "open_paragraph": paragraph_names["open"],
            "fetch_paragraph": paragraph_names["fetch"],
            "close_paragraph": paragraph_names["close"],
            "open_number": paragraph_numbers["open"],
        }
```

```

        "fetch_number": paragraph_numbers["fetch"],
        "close_number": paragraph_numbers["close"],
    }

def _remove_table_suffix(
    self,
    table_name: str,
) -> str:
    table = NameNormalizer.normalize(table_name)

    if table.endswith("_TV") or table.endswith("_TB"):
        return table[:-3] + "_"

    if table.endswith("TV") or table.endswith("TB"):
        return table[:-2]

    return table + "_"

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\resolvers\host_variable_resolver.py
ENCODING: utf-8
=====

import re

from catalogs.db2_naming_catalog import DB2_HOST_REFERENCE_PREFIX
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer

class HostVariableResolver:
    """
    Resolves DCLGEN host variables.

    Authority:
    - DCLGEN determines COBOL host-variable spelling.
    - DCLGEN determines DCLGEN group names.
    - Final DB2 embedded SQL host reference format is:

        :DCLGROUP.HOST-FIELD

    Example:

        :DCLDZBEFFTV.NR-IDSTOCK-479BEFF

    This resolver also normalizes older generated forms:

        :HOST-FIELD OF DCLGROUP
        HOST-FIELD OF DCLGROUP
        :DCLGROUP.HOST-FIELD
        DCLGROUP.HOST-FIELD

    into the canonical DB2 style:

        :DCLGROUP.HOST-FIELD
    """

    HOST_OF_GROUP_PATTERN = re.compile(
        r"^:?\s*"
        r"(?P<host>[A-Z][A-Z0-9-]*)"
        r"\s+OF\s+"
        r"(?P<group>DCL[A-Z0-9-]+)"
        r"\s*$",
        flags=re.IGNORECASE,
    )

    GROUP_DOT_HOST_PATTERN = re.compile(
        r"^:?\s*"
        r"(?P<group>DCL[A-Z0-9-]+)"
        r"\s*\.\s*"
        r"(?P<host>[A-Z][A-Z0-9-]*)"
        r"\s*$",
        flags=re.IGNORECASE,
    )

```

```

DOUBLE_COLON_PATTERN = re.compile(
    r"^:+",
    flags=re.IGNORECASE,
)

def __init__(
    self,
    dclgen_repository: DclgenRepository,
    table_name_resolver: TableNameResolver,
) -> None:
    self.dclgen_repository = dclgen_repository
    self.table_name_resolver = table_name_resolver

def host_for_column(
    self,
    table_name: str,
    column_name: str,
) -> str:
    resolved_table = self.table_name_resolver.resolve_table(table_name)

    if not resolved_table:
        return ""

    return self.dclgen_repository.host_for_column(
        table_name=resolved_table,
        column_name=column_name,
    )

def group_for_table(
    self,
    table_name: str,
) -> str:
    resolved_table = self.table_name_resolver.resolve_table(table_name)

    if not resolved_table:
        return ""

    group = self.dclgen_repository.group_for_table(resolved_table)

    if group:
        return NameNormalizer.to_cobol(group)

    return "DCL" + NameNormalizer.to_cobol(resolved_table)

def host_reference_for_column(
    self,
    table_name: str,
    column_name: str,
) -> str:
    group = self.group_for_table(table_name)
    host = self.host_for_column(
        table_name=table_name,
        column_name=column_name,
    )

    if not group or not host:
        return ""

    return self.normalize_host_reference(
        f"{DB2_HOST_REFERENCE_PREFIX}{group}.{host}"
    )

def host_references_for_columns(
    self,
    table_name: str,
    columns: list[str],
) -> list[str]:
    output: list[str] = []
    seen: set[str] = set()

    for column in columns:
        reference = self.host_reference_for_column(
            table_name=table_name,
            column_name=column,
        )

```

```

        if not reference:
            continue

        key = reference.upper()

        if key in seen:
            continue

        seen.add(key)
        output.append(reference)

    return output

def normalize_host_reference(
    self,
    value: str,
) -> str:
    text = str(value or "").strip()

    if not text:
        return ""

    text = text.replace(":", ";")
    text = self.DOUBLE_COLON_PATTERN.sub(":", text)

    of_match = self.HOST_OF_GROUP_PATTERN.match(text)

    if of_match:
        group = NameNormalizer.to_cobol(of_match.group("group"))
        host = NameNormalizer.to_cobol(of_match.group("host"))

        if not group or not host:
            return ""

        return f"{DB2_HOST_REFERENCE_PREFIX}{group}.{host}"

    dot_match = self.GROUP_DOT_HOST_PATTERN.match(text)

    if dot_match:
        group = NameNormalizer.to_cobol(dot_match.group("group"))
        host = NameNormalizer.to_cobol(dot_match.group("host"))

        if not group or not host:
            return ""

        return f"{DB2_HOST_REFERENCE_PREFIX}{group}.{host}"

    if text.startswith(DB2_HOST_REFERENCE_PREFIX):
        text = text[1:].strip()

    if "." in text:
        parts = text.split(".", 1)
        group = NameNormalizer.to_cobol(parts[0])
        host = NameNormalizer.to_cobol(parts[1])

        if group and host and group.upper().startswith("DCL"):
            return f"{DB2_HOST_REFERENCE_PREFIX}{group}.{host}"

    if text.upper().startswith("DCL") and " " in text:
        parts = text.split()

        if len(parts) >= 2:
            group = NameNormalizer.to_cobol(parts[0])
            host = NameNormalizer.to_cobol(parts[1])

            if group and host:
                return f"{DB2_HOST_REFERENCE_PREFIX}{group}.{host}"

    if text.upper().startswith("DCL"):
        return f"{DB2_HOST_REFERENCE_PREFIX}{NameNormalizer.to_cobol(text)}"

    return f"{DB2_HOST_REFERENCE_PREFIX}{NameNormalizer.to_cobol(text)}"

def valid_host_references(
    self,

```



```

) -> set[str]:
    output: set[str] = set()

    for reference in self.dclgen_repository.valid_host_references():
        normalized = self.normalize_host_reference(reference)

        if normalized:
            output.add(normalized)
            output.add(normalized[1:])

    return output

def has_host_for_column(
    self,
    table_name: str,
    column_name: str,
) -> bool:
    return bool(
        self.host_for_column(
            table_name=table_name,
            column_name=column_name,
        )
    )

def host_reference_key(
    self,
    table_name: str,
    column_name: str,
) -> str:
    reference = self.host_reference_for_column(
        table_name=table_name,
        column_name=column_name,
    )

    if not reference:
        return ""

    return reference[1:]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\resolvers\record_context_resolver.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class RecordContextResolver:
    """
    Resolves active source record context.

    This helper is used by transformers and field rewriters to avoid guessing
    the active record directly inside transformation logic.
    """

    def __init__(
        self,
        mapping_repository: MappingRepository,
        dclgen_repository: DclgenRepository,
    ) -> None:
        self.mapping_repository = mapping_repository
        self.dclgen_repository = dclgen_repository

    def known_records(
        self,
    ) -> list[str]:
        return self.mapping_repository.records()

    def record_from_text(
        self,
        text: str,
    ) -> str:
        normalized_text = NameNormalizer.normalize(text)

```

```

    if not normalized_text:
        return ""

    for record in self.known_records():
        if record and record in normalized_text:
            return record

    return ""

def record_from_paragraph_name(
    self,
    paragraph_name: str,
) -> str:
    return self.record_from_text(paragraph_name)

def record_from_move_target(
    self,
    move_target: str,
) -> str:
    return self.record_from_text(move_target)

def record_from_initialize_target(
    self,
    initialize_target: str,
) -> str:
    text = NameNormalizer.normalize(initialize_target)

    if not text:
        return ""

    if text.startswith("DCL"):
        table = text[3:]
        return self.record_from_table(table)

    return self.record_from_text(text)

def record_from_table(
    self,
    table_name: str,
) -> str:
    table = NameNormalizer.normalize(table_name)

    if not table:
        return ""

    for record in self.known_records():
        record_table = self.mapping_repository.db2_table_for_record(record)

        if NameNormalizer.normalize(record_table) == table:
            return record

    return ""

def record_from_dclgen_group(
    self,
    group_name: str,
) -> str:
    text = NameNormalizer.normalize(group_name)

    if not text:
        return ""

    if text.startswith("DCL"):
        return self.record_from_table(text[3:])

    return ""

def context_strength(
    self,
    record_name: str,
) -> int:
    if not record_name:
        return 0

    if self.mapping_repository.has_record(record_name):

```

```
        return 100
```

```
    return 0
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\resolvers\relationship_resolver.py
ENCODING: utf-8
=====
```

```
from dataclasses import dataclass
```

```
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
```

```
@dataclass
class RelationshipCondition:
    child_record: str
    child_table: str
    child_column: str
    parent_record: str
    parent_table: str
    parent_column: str
    parent_host_reference: str
```

```
@dataclass
class RelationshipResolution:
    child_record: str
    parent_record: str = ""
    child_table: str = ""
    parent_table: str = ""
    conditions: list[RelationshipCondition] = None
    diagnostics: list[str] = None

    def __post_init__(self) -> None:
        if self.conditions is None:
            self.conditions = []
        if self.diagnostics is None:
            self.diagnostics = []
```

```
class RelationshipResolver:
    """
    Resolves parent-child DB2 relationships using Sheet Mapping metadata.

    Generic rule:
    - Child FK columns come from rows marked FOREIGN KEY.
    - Parent columns are resolved by matching those FK columns against
      another record's primary key columns.
    - If Cross Application DB2 table/field is present, it is preferred.
    """

    def __init__(
        self,
        mapping_repository: MappingRepository,
        table_name_resolver: TableNameResolver,
        host_variable_resolver: HostVariableResolver,
    ) -> None:
        self.mapping_repository = mapping_repository
        self.table_name_resolver = table_name_resolver
        self.host_variable_resolver = host_variable_resolver

    def resolve_for_child_record(
        self,
        child_record: str,
    ) -> RelationshipResolution:
        child = NameNormalizer.normalize(child_record)
        result = RelationshipResolution(child_record=child)

        if not child:
            result.diagnostics.append("Relationship resolver: child record is empty.")
            return result
```

```

child_table = self.table_name_resolver.table_for_record(child)
result.child_table = child_table

rows = self.mapping_repository.rows_for_record(child)

fk_rows = [
    row
    for row in rows
    if self._is_foreign_key_row(row)
]

if not fk_rows:
    result.diagnostics.append(
        f"Relationship resolver: no foreign-key rows for child record {child}."
    )
    return result

parent_record = self._infer_parent_record(
    child_record=child,
    fk_rows=fk_rows,
)

parent_table = ""

if parent_record:
    parent_table = self.table_name_resolver.table_for_record(parent_record)

result.parent_record = parent_record
result.parent_table = parent_table

for row in fk_rows:
    child_column = NameNormalizer.normalize(
        getattr(row, "new_db2_field_name", "")
        or getattr(row, "cross_application_db2_field_name", "")
    )

    if not child_column:
        continue

    explicit_parent_table = self.table_name_resolver.resolve_table(
        getattr(row, "cross_application_db2_table", "")
    )

    explicit_parent_column = NameNormalizer.normalize(
        getattr(row, "cross_application_db2_field_name", "")
    )

    effective_parent_table = explicit_parent_table or parent_table
    effective_parent_column = explicit_parent_column or child_column

    if not effective_parent_table:
        effective_parent_table = self._find_parent_table_by_column(
            child_record=child,
            child_column=child_column,
        )

    if not effective_parent_table:
        result.diagnostics.append(
            f"Relationship resolver: no parent table found for {child}.{child_column}."
        )
        continue

    if not parent_record:
        parent_record = self._record_for_table(effective_parent_table)
        result.parent_record = parent_record

    if not result.parent_table:
        result.parent_table = effective_parent_table

    parent_host = self.host_variable_resolver.host_reference_for_column(
        table_name=effective_parent_table,
        column_name=effective_parent_column,
    )

    if not parent_host:

```

```

        result.diagnostics.append(
            "Relationship resolver: missing parent host for "
            f"child={child}, child_column={child_column}, "
            f"parent_table={effective_parent_table}, "
            f"parent_column={effective_parent_column}."
        )
        continue

    result.conditions.append(
        RelationshipCondition(
            child_record=child,
            child_table=child_table,
            child_column=child_column,
            parent_record=parent_record,
            parent_table=effective_parent_table,
            parent_column=effective_parent_column,
            parent_host_reference=parent_host,
        )
    )

    result.diagnostics.append(
        "Relationship resolver: "
        f"child={child}, parent={result.parent_record}, "
        f"conditions={len(result.conditions)}"
    )

    return result

def foreign_key_columns_for_record(
    self,
    record_name: str,
) -> list[str]:
    record = NameNormalizer.normalize(record_name)
    rows = self.mapping_repository.rows_for_record(record)
    output: list[str] = []
    seen: set[str] = set()

    for row in rows:
        if not self._is_foreign_key_row(row):
            continue

        column = NameNormalizer.normalize(
            getattr(row, "new_db2_field_name", "")
            or getattr(row, "cross_application_db2_field_name", "")
        )

        if not column or column in seen:
            continue

        seen.add(column)
        output.append(column)

    return output

def primary_key_columns_for_record(
    self,
    record_name: str,
) -> list[str]:
    record = NameNormalizer.normalize(record_name)
    rows = self.mapping_repository.rows_for_record(record)
    output: list[str] = []
    seen: set[str] = set()

    for row in rows:
        if not self._is_primary_key_row(row):
            continue

        column = NameNormalizer.normalize(
            getattr(row, "new_db2_field_name", "")
        )

        if not column or column in seen:
            continue

        seen.add(column)
        output.append(column)

```

```

    return output

def parent_key_columns_required_by_children(
    self,
    parent_record: str,
) -> list[str]:
    parent = NameNormalizer.normalize(parent_record)
    output: list[str] = []
    seen: set[str] = set()

    for child in self.mapping_repository.records():
        child = NameNormalizer.normalize(child)

        if child == parent:
            continue

        relation = self.resolve_for_child_record(child)

        if relation.parent_record != parent:
            continue

        for condition in relation.conditions:
            column = NameNormalizer.normalize(condition.parent_column)

            if not column or column in seen:
                continue

            seen.add(column)
            output.append(column)

    return output

def order_by_columns_for_record(
    self,
    record_name: str,
) -> list[str]:
    record = NameNormalizer.normalize(record_name)
    primary_keys = self.primary_key_columns_for_record(record)
    foreign_keys = set(self.foreign_key_columns_for_record(record))
    output: list[str] = []

    for column in primary_keys:
        normalized = NameNormalizer.normalize(column)

        if not normalized:
            continue

        if normalized in foreign_keys:
            continue

        if normalized not in output:
            output.append(normalized)

    return output

def has_foreign_keys(
    self,
    record_name: str,
) -> bool:
    return bool(self.foreign_key_columns_for_record(record_name))

def _is_primary_key_row(
    self,
    row,
) -> bool:
    db2_key = NameNormalizer.normalize(getattr(row, "db2_key", ""))
    idms_key = NameNormalizer.normalize(getattr(row, "idms_key", ""))

    return (
        "PRIMARY" in db2_key
        or db2_key == "KEY"
        or "CALC" in idms_key
    )

def _is_foreign_key_row(

```

```

        self,
        row,
    ) -> bool:
        db2_key = NameNormalizer.normalize(getattr(row, "db2_key", ""))
        relation = NameNormalizer.normalize(getattr(row, "relation", ""))

        return (
            "FOREIGN" in db2_key
            or "FK" in db2_key
            or "FOREIGN" in relation
        )

def _infer_parent_record(
    self,
    child_record: str,
    fk_rows: list,
) -> str:
    for row in fk_rows:
        parent_table = self.table_name_resolver.resolve_table(
            getattr(row, "cross_application_db2_table", "")
        )

        if parent_table:
            record = self._record_for_table(parent_table)
            if record:
                return record

    best_record = ""
    best_count = 0

    for record in self.mapping_repository.records():
        record = NameNormalizer.normalize(record)

        if record == child_record:
            continue

        parent_keys = set(self.primary_key_columns_for_record(record))

        if not parent_keys:
            continue

        child_fk_columns = {
            NameNormalizer.normalize(
                getattr(row, "new_db2_field_name", "")
                or getattr(row, "cross_application_db2_field_name", "")
            )
            for row in fk_rows
        }

        match_count = len(parent_keys.intersection(child_fk_columns))

        if match_count > best_count:
            best_count = match_count
            best_record = record

    return best_record

def _find_parent_table_by_column(
    self,
    child_record: str,
    child_column: str,
) -> str:
    column = NameNormalizer.normalize(child_column)

    if not column:
        return ""

    child_table = self.table_name_resolver.table_for_record(child_record)

    for record in self.mapping_repository.records():
        normalized_record = NameNormalizer.normalize(record)

        if normalized_record == child_record:
            continue

        table = self.table_name_resolver.table_for_record(normalized_record)

```

```

        if not table:
            continue

        if table == child_table:
            continue

        if column in self.primary_key_columns_for_record(normalized_record):
            return table

    return ""

def _record_for_table(
    self,
    table_name: str,
) -> str:
    table = self.table_name_resolver.resolve_table(table_name)

    if not table:
        return ""

    for record in self.mapping_repository.records():
        record_table = self.table_name_resolver.table_for_record(record)

        if record_table == table:
            return NameNormalizer.normalize(record)

    return ""

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\resolvers\table_name_resolver.py
ENCODING: utf-8
=====

```

```

from catalogs.db2_naming_catalog import DB2_TABLE_SUFFIX_EQUIVALENTS
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class TableNameResolver:
    """
    Resolves DB2 table names.

    Authority:
    - Sheet Mapping provides intended DB2 table names.
    - DCLGEN provides the final available DB2 table name.
    - If Sheet Mapping says TB but DCLGEN has TV, resolve to DCLGEN TV.

    Example:
    - Sheet Mapping: DZBFARTB
    - DCLGEN table: DZBFARTV
    - Final SQL table: DZBFARTV
    """

```

```

def __init__(
    self,
    mapping_repository: MappingRepository,
    dclgen_repository: DclgenRepository,
) -> None:
    self.mapping_repository = mapping_repository
    self.dclgen_repository = dclgen_repository

def resolve_table(
    self,
    table_name: str,
) -> str:
    normalized = NameNormalizer.normalize(table_name)

    if not normalized:
        return ""

    for candidate in self.table_candidates(normalized):
        if self.dclgen_repository.has_table(candidate):
            return candidate

```



```

        return normalized

def table_for_record(
    self,
    record_name: str,
) -> str:
    mapped_table = self.mapping_repository.db2_table_for_record(record_name)

    if not mapped_table:
        return ""

    return self.resolve_table(mapped_table)

def table_candidates(
    self,
    table_name: str,
) -> list[str]:
    normalized = NameNormalizer.normalize(table_name)

    if not normalized:
        return []

    output: list[str] = [normalized]

    for source_suffix, target_suffix in DB2_TABLE_SUFFIX_EQUIVALENTS:
        if normalized.endswith(source_suffix):
            output.append(
                normalized[:-len(source_suffix)] + target_suffix
            )

    cobol_name = NameNormalizer.to_cobol(normalized)
    compact_name = NameNormalizer.compact(normalized)

    if cobol_name:
        output.append(cobol_name)

    if compact_name:
        output.append(compact_name)

    final: list[str] = []

    for item in output:
        normalized_item = NameNormalizer.normalize(item)

        if not normalized_item:
            continue

        if normalized_item in final:
            continue

        final.append(normalized_item)

    return final

def known_dclgen_tables(
    self,
) -> list[str]:
    return self.dclgen_repository.tables()

def known_mapping_tables(
    self,
) -> list[str]:
    return self.mapping_repository.tables()

def has_resolved_table(
    self,
    table_name: str,
) -> bool:
    resolved = self.resolve_table(table_name)

    if not resolved:
        return False

    return self.dclgen_repository.has_table(resolved)

```

```

def context_strength(
    self,
    record_name: str,
) -> int:
    if not record_name:
        return 0

    if self.mapping_repository.has_record(record_name):
        return 100

    return 0

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\cobol_fixed_format_composer.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\cobol_formatter.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\cobol_transformer.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\conversion_service.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.orchestration.conversion_service import ConversionService
__all__ = [
    "ConversionService",
]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\conversion_service.py.broken_before_pdf_restore
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.orchestration.conversion_service import ConversionService

__all__ = [
    "ConversionService",
]

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\conversion_service.py.broken_before_pdf_restore_1
ENCODING: utf-8
=====

```

```

=====
from idms_db2_phase2.orchestration.conversion_service import ConversionService
__all__ = [

```

```
"ConversionService",  
]
```

```
=====
```

```
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\conversion_service.py.broken_before_pdf_restore_2  
ENCODING: utf-8  
=====
```

```
from idms_db2_phase2.orchestration.conversion_service import ConversionService  
__all__ = [  
    "ConversionService",  
]
```

```
=====
```

```
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\db2_cursor_paragraph_generator.py  
ENCODING: utf-8  
=====
```

[Empty file]

```
=====
```

```
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\db2_infrastructure_generator.py  
ENCODING: utf-8  
=====
```

[Empty file]

```
=====
```

```
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\field_reference_rewriter.py  
ENCODING: utf-8  
=====
```

[Empty file]

```
=====
```

```
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services>manual_layout_composer.py  
ENCODING: utf-8  
=====
```

[Empty file]

```
=====
```

```
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\metadata_service.py  
ENCODING: utf-8  
=====
```

```
from idms_db2_phase2.analyzers.metadata_service import MetadataService  
  
__all__ = [  
    "MetadataService",  
]
```

```
=====
```

```
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\name_normalizer.py  
ENCODING: utf-8  
=====
```

```
from patterns.naming_patterns import (  
    FOUR_DIGIT_RECORD_SUFFIX_PATTERN,  
    MULTIPLE_UNDERSCORE_PATTERN,  
    NON_COMPACT_NAME_CHARACTER_PATTERN,  
    NON_DB2_NAME_CHARACTER_PATTERN,  
)
```

```
class NameNormalizer:  
    """  
    Generic name normalization helper.
```

This service contains name transformation logic only.
Regex patterns are stored in patterns/naming_patterns.py.
"""

```
@staticmethod
def normalize(
    value: str | None,
) -> str:
    if value is None:
        return ""

    text = str(value).strip().upper()

    if not text:
        return ""

    text = text.replace("-", "_")
    text = text.replace(" ", "_")
    text = NON_DB2_NAME_CHARACTER_PATTERN.sub("_", text)
    text = MULTIPLE_UNDERSCORE_PATTERN.sub("_", text)

    return text.strip("_")

@staticmethod
def to_cobol(
    value: str | None,
) -> str:
    return NameNormalizer.normalize(value).replace("_", "-")

@staticmethod
def compact(
    value: str | None,
) -> str:
    text = NameNormalizer.normalize(value)

    return NON_COMPACT_NAME_CHARACTER_PATTERN.sub("", text)

@staticmethod
def remove_record_suffix(
    value: str | None,
) -> str:
    text = NameNormalizer.normalize(value)

    if not text:
        return ""

    return FOUR_DIGIT_RECORD_SUFFIX_PATTERN.sub("", text)
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\pic_length_auto_fixer.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\production_validator.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\program_flow_analyzer.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\sql_generator.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\validation_service.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.domain.models import ConversionInput

class ValidationService:
    """
    Validates the minimum required conversion inputs.

    This service performs input-level validation only. Production validation
    of converted COBOL output should remain in a separate production validator.
    """

    def validate(
        self,
        conversion_input: ConversionInput,
    ) -> list[str]:
        messages: list[str] = []

        if not conversion_input.sheet_mapping_rows:
            messages.append(
                "Sheet Mapping is required and must contain rows."
            )

        if not conversion_input.dclgen_columns:
            messages.append(
                "At least one DCLGEN file is required."
            )

        if not conversion_input.idms_cobol_text.strip():
            messages.append(
                "IDMS COBOL source text file is required and must contain readable COBOL code."
            )

        return messages
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\validation_service.py.broken_before_pdf_restore
ENCODING: utf-8
=====
```

```
"""
Input validation service.

This service performs input-level validation only.

Production validation of converted COBOL output should remain in the
production validator.
"""
```

```
from idms_db2_phase2.domain.models import ConversionInput
from rules.validation_rules import INPUT_VALIDATION_MESSAGES
```

```
class ValidationService:
    """
    Validates the minimum required conversion inputs.
    """

    def validate(
        self,
        conversion_input: ConversionInput,
    ) -> list[str]:
        """
```

Return input validation messages.

Required:

- Sheet Mapping rows
 - DCLGEN columns
 - IDMS COBOL source text
- """

messages: list[str] = []

if conversion_input is None:

```
    messages.extend(
        [
            INPUT_VALIDATION_MESSAGES["missing_sheet_mapping"],
            INPUT_VALIDATION_MESSAGES["missing_dclgen"],
            INPUT_VALIDATION_MESSAGES["missing_idms_cobol"],
        ]
    )
    return messages
```

if not conversion_input.sheet_mapping_rows:

```
    messages.append(INPUT_VALIDATION_MESSAGES["missing_sheet_mapping"])
```

if not conversion_input.dclgen_columns:

```
    messages.append(INPUT_VALIDATION_MESSAGES["missing_dclgen"])
```

if not str(conversion_input.idms_cobol_text or "").strip():

```
    messages.append(INPUT_VALIDATION_MESSAGES["missing_idms_cobol"])
```

return messages

=====

FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\validation_service.py.broken_before_pdf_restore_1

ENCODING: utf-8

=====

=====

```
from idms_db2_phase2.domain.models import ConversionInput
```

```
class ValidationService:
```

```
    """
```

Validates the minimum required conversion inputs.

This service performs input-level validation only. Production validation of converted COBOL output should remain in a separate production validator.

```
    """
```

```
    def validate(
```

```
        self,
```

```
        conversion_input: ConversionInput,
```

```
    ) -> list[str]:
```

```
        messages: list[str] = []
```

```
        if not conversion_input.sheet_mapping_rows:
```

```
            messages.append(
```

```
                "Sheet Mapping is required and must contain rows."
```

```
            )
```

```
        if not conversion_input.dclgen_columns:
```

```
            messages.append(
```

```
                "At least one DCLGEN file is required."
```

```
            )
```

```
        if not conversion_input.idms_cobol_text.strip():
```

```
            messages.append(
```

```
                "IDMS COBOL source text file is required and must contain readable COBOL code."
```

```
            )
```

```
        return messages
```

=====

FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\services\validation_service.py.broken_before_pdf_restore_2

ENCODING: utf-8

=====

```
from idms_db2_phase2.domain.models import ConversionInput
```

```
class ValidationService:
```

```
    """
```

Validates the minimum required conversion inputs.

This service performs input-level validation only. Production validation

of converted COBOL output should remain in a separate production validator.

```
"""
def validate(
    self,
    conversion_input: ConversionInput,
) -> list[str]:
    messages: list[str] = []
    if not conversion_input.sheet_mapping_rows:
        messages.append(
            "Sheet Mapping is required and must contain rows."
        )
    if not conversion_input.dclgen_columns:
        messages.append(
            "At least one DCLGEN file is required."
        )
    if not conversion_input.idms_cobol_text.strip():
        messages.append(
            "IDMS COBOL source text file is required and must contain readable COBOL code."
        )
    return messages
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\testing\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\testing\run_retrieval.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
import sys
from datetime import datetime
from pathlib import Path
```

```
CURRENT_FILE = Path(__file__).resolve()
SRC_DIR = CURRENT_FILE.parents[2]
PROJECT_ROOT = CURRENT_FILE.parents[3]
```

```
for path in [PROJECT_ROOT, SRC_DIR]:
    if str(path) not in sys.path:
        sys.path.insert(0, str(path))
```

```
from config.path_settings import (
    DEFAULT_COPYBOOK_CANDIDATE_PATHS,
    DEFAULT_DCLGEN_CANDIDATE_PATHS,
    DEFAULT_OUTPUT_DIR,
    DEFAULT_RETRIEVAL_SOURCE_PATH,
    DEFAULT_SHEET_MAPPING_PATH,
)
```

```
from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.orchestration.conversion_service import ConversionService
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser
```

```
TARGET_PROGRAM_ID = "VMDZ4420"
AUTO_FIX_PIC_LENGTH_MISMATCHES = False
```

```
SHEET_MAPPING_PATH = DEFAULT_SHEET_MAPPING_PATH
IDMS_COBOL_SOURCE_PATH = DEFAULT_RETRIEVAL_SOURCE_PATH
OUTPUT_DIR = DEFAULT_OUTPUT_DIR
```

```
DCLGEN_PATHS = [
```

```

    path
    for path in DEFAULT_DCLGEN_CANDIDATE_PATHS
    if path.exists() and path.is_file()
]

COPYBOOK_PATHS = [
    path
    for path in DEFAULT_COPYBOOK_CANDIDATE_PATHS
    if path.exists() and path.is_file()
]

def validate_file_exists(
    file_path: Path,
    label: str,
) -> None:
    if not file_path.exists():
        raise FileNotFoundError(f"{label} file not found: {file_path}")

    if not file_path.is_file():
        raise FileNotFoundError(f"{label} path is not a file: {file_path}")

def validate_inputs() -> None:
    validate_file_exists(SHEET_MAPPING_PATH, "Sheet Mapping")
    validate_file_exists(IDMS_COBOL_SOURCE_PATH, "IDMS COBOL Source")

    if not DCLGEN_PATHS:
        searched = "\n".join(
            str(path)
            for path in DEFAULT_DCLGEN_CANDIDATE_PATHS
        )

        raise ValueError(
            "At least one DCLGEN file path is required. "
            "No DCLGEN candidate file was found. Searched:\n"
            f"{searched}"
        )

    for index, dclgen_path in enumerate(DCLGEN_PATHS, start=1):
        validate_file_exists(dclgen_path, f"DCLGEN {index}")

    for index, copybook_path in enumerate(COPYBOOK_PATHS, start=1):
        validate_file_exists(copybook_path, f"Copybook {index}")

    OUTPUT_DIR.mkdir(
        parents=True,
        exist_ok=True,
    )

def load_inputs() -> tuple[list, list, list, str, list[str]]:
    diagnostics: list[str] = []

    sheet_parser = SheetMappingParser()
    dclgen_parser = DclgenParser()
    copybook_parser = CopybookParser()
    file_loader = FileLoader()

    diagnostics.append("START LOAD INPUTS")

    sheet_file = LocalUploadedFile(SHEET_MAPPING_PATH)
    sheet_rows = sheet_parser.parse_uploaded_file(sheet_file)

    diagnostics.append(f"Sheet Mapping file: {SHEET_MAPPING_PATH}")
    diagnostics.append(f"Sheet Mapping rows: {len(sheet_rows)}")

    if hasattr(sheet_parser, "diagnostics"):
        diagnostics.extend(sheet_parser.diagnostics)

    dclgen_texts: list[str] = []

    diagnostics.append(f"DCLGEN file count: {len(DCLGEN_PATHS)}")

    for index, dclgen_path in enumerate(DCLGEN_PATHS, start=1):
        dclgen_file = LocalUploadedFile(dclgen_path)

```



```

dclgen_text = file_loader.read_uploaded_text(dclgen_file)

dclgen_texts.append(dclgen_text)

diagnostics.append(f"DCLGEN {index}: {dclgen_path}")
diagnostics.append(f"DCLGEN {index} text length: {len(dclgen_text)}")

if hasattr(dclgen_parser, "parse_many_texts"):
    dclgen_columns = dclgen_parser.parse_many_texts(dclgen_texts)
else:
    dclgen_columns = []

    for index, dclgen_text in enumerate(dclgen_texts, start=1):
        parsed_columns = dclgen_parser.parse(
            text=dclgen_text,
            source_label=f"DCLGEN file {index}",
        )
        dclgen_columns.extend(parsed_columns)

diagnostics.append(f"DCLGEN total columns: {len(dclgen_columns)}")

if hasattr(dclgen_parser, "diagnostics"):
    diagnostics.extend(dclgen_parser.diagnostics)

copybook_text_parts: list[str] = []

diagnostics.append(f"Copybook file count: {len(COPYBOOK_PATHS)}")

for index, copybook_path in enumerate(COPYBOOK_PATHS, start=1):
    copybook_file = LocalUploadedFile(copybook_path)
    copybook_text = file_loader.read_uploaded_text(copybook_file)

    copybook_text_parts.append(copybook_text)

    diagnostics.append(f"Copybook {index}: {copybook_path}")
    diagnostics.append(f"Copybook {index} text length: {len(copybook_text)}")

copybook_text = "\n".join(copybook_text_parts)

if copybook_text.strip():
    try:
        copybook_fields = copybook_parser.parse(
            text=copybook_text,
            source_label="Copybook files",
        )
    except TypeError:
        copybook_fields = copybook_parser.parse(copybook_text)
else:
    copybook_fields = []

diagnostics.append(f"Copybook total fields: {len(copybook_fields)}")

if hasattr(copybook_parser, "diagnostics"):
    diagnostics.extend(copybook_parser.diagnostics)

source_file = LocalUploadedFile(IDMS_COBOL_SOURCE_PATH)
idms_cobol_text = file_loader.read_uploaded_text(source_file)

diagnostics.append(f"IDMS COBOL source file: {IDMS_COBOL_SOURCE_PATH}")
diagnostics.append(f"IDMS COBOL source text length: {len(idms_cobol_text)}")

return (
    sheet_rows,
    dclgen_columns,
    copybook_fields,
    idms_cobol_text,
    diagnostics,
)

```

```

def run_conversion() -> None:
    validate_inputs()

    (
        sheet_rows,
        dclgen_columns,

```

```

        copybook_fields,
        idms_cobol_text,
        diagnostics,
    ) = load_inputs()

    service = ConversionService()

    result = service.convert(
        ConversionInput(
            sheet_mapping_rows=sheet_rows,
            dclgen_columns=dclgen_columns,
            copybook_fields=copybook_fields,
            idms_cobol_text=idms_cobol_text,
            target_program_id=TARGET_PROGRAM_ID,
            auto_fix_pic_length_mismatches=AUTO_FIX_PIC_LENGTH_MISMATCHES,
        )
    )

    date_time = datetime.now().strftime("%d-%m-%Y_%H%M%S")
    code_name = IDMS_COBOL_SOURCE_PATH.stem
    output_cobol_path = OUTPUT_DIR / f"{code_name}_{date_time}.cbl"

    output_cobol_path.write_text(
        result.converted_cobol or "",
        encoding="utf-8",
    )

    print("DB2 COBOL generation completed.")
    print(f"Output file created: {output_cobol_path}")
    print("")
    print("Input Summary")
    print("-----")
    print(f"Project Root      : {PROJECT_ROOT}")
    print(f"SRC Directory     : {SRC_DIR}")
    print(f"Sheet Mapping Rows : {len(sheet_rows)}")
    print(f"DCLGEN Columns    : {len(dclgen_columns)}")
    print(f"Copybook Fields    : {len(copybook_fields)}")
    print(f"COBOL Text Length  : {len(idms_cobol_text)}")
    print(f"Target PROGRAM-ID  : {TARGET_PROGRAM_ID}")

    print("")
    print("Selected Input Files")
    print("-----")
    print(f"Sheet Mapping      : {SHEET_MAPPING_PATH}")
    print(f"IDMS COBOL Source  : {IDMS_COBOL_SOURCE_PATH}")

    print("DCLGEN Files:")
    for path in DCLGEN_PATHS:
        print(f" - {path}")

    if COPYBOOK_PATHS:
        print("Copybook Files:")
        for path in COPYBOOK_PATHS:
            print(f" - {path}")
    else:
        print("Copybook Files      : None")

    print("")
    print("Validation Messages")
    print("-----")

    if result.validation_messages:
        for message in result.validation_messages:
            print(f"- {message}")
    else:
        print("No validation messages.")

    print("")
    print("Diagnostics")
    print("-----")

    for diagnostic in diagnostics:
        print(f"- {diagnostic}")

if __name__ == "__main__":

```

```
run_conversion()
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\testing\run_update.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

import sys
from datetime import datetime
from pathlib import Path

CURRENT_FILE = Path(__file__).resolve()
SRC_DIR = CURRENT_FILE.parents[2]
PROJECT_ROOT = CURRENT_FILE.parents[3]

for path in [PROJECT_ROOT, SRC_DIR]:
    if str(path) not in sys.path:
        sys.path.insert(0, str(path))

from config.path_settings import (
    DEFAULT_COPYBOOK_CANDIDATE_PATHS,
    DEFAULT_DCLGEN_CANDIDATE_PATHS,
    DEFAULT_OUTPUT_DIR,
    DEFAULT_SHEET_MAPPING_PATH,
    DEFAULT_UPDATE_SOURCE_PATH,
    LOGS_DIR,
    PROJECT_ROOT,
    SRC_DIR,
)
from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.infrastructure.logger_factory import LoggerFactory
from idms_db2_phase2.orchestration.conversion_service import ConversionService
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser

TARGET_PROGRAM_ID = "VMDZ1567"
AUTO_FIX_PIC_LENGTH_MISMATCHES = False

SHEET_MAPPING_PATH = DEFAULT_SHEET_MAPPING_PATH
IDMS_COBOL_SOURCE_PATH = DEFAULT_UPDATE_SOURCE_PATH
OUTPUT_DIR = DEFAULT_OUTPUT_DIR

DCLGEN_PATHS = [
    path
    for path in DEFAULT_DCLGEN_CANDIDATE_PATHS
    if path.exists() and path.is_file()
]

COPYBOOK_PATHS = [
    path
    for path in DEFAULT_COPYBOOK_CANDIDATE_PATHS
    if path.exists() and path.is_file()
]

def validate_file_exists(
    file_path: Path,
    label: str,
) -> None:
    if not file_path.exists():
        raise FileNotFoundError(f"{label} file not found: {file_path}")

    if not file_path.is_file():
        raise FileNotFoundError(f"{label} path is not a file: {file_path}")

def validate_inputs() -> None:
    validate_file_exists(SHEET_MAPPING_PATH, "Sheet Mapping")
    validate_file_exists(IDMS_COBOL_SOURCE_PATH, "IDMS COBOL Source")
```

```

if not DCLGEN_PATHS:
    searched = "\n".join(
        str(path)
        for path in DEFAULT_DCLGEN_CANDIDATE_PATHS
    )
    raise ValueError(
        "At least one DCLGEN file path is required. "
        "No DCLGEN candidate file was found. Searched:\n"
        f"{searched}"
    )

for index, dclgen_path in enumerate(DCLGEN_PATHS, start=1):
    validate_file_exists(dclgen_path, f"DCLGEN {index}")

for index, copybook_path in enumerate(COPYBOOK_PATHS, start=1):
    validate_file_exists(copybook_path, f"Copybook {index}")

OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True,
)

def load_inputs(
    logger,
) -> tuple[list, list, list, str, list[str]]:
    diagnostics: list[str] = []

    sheet_parser = SheetMappingParser()
    dclgen_parser = DclgenParser()
    copybook_parser = CopybookParser()
    file_loader = FileLoader()

    diagnostics.append("START LOAD INPUTS")
    logger.info("START LOAD INPUTS")

    sheet_file = LocalUploadedFile(SHEET_MAPPING_PATH)
    sheet_rows = sheet_parser.parse_uploaded_file(sheet_file)

    diagnostics.append(f"Sheet Mapping file: {SHEET_MAPPING_PATH}")
    diagnostics.append(f"Sheet Mapping rows: {len(sheet_rows)}")
    logger.info("Sheet Mapping file: %s", SHEET_MAPPING_PATH)
    logger.info("Sheet Mapping rows: %s", len(sheet_rows))

    if hasattr(sheet_parser, "diagnostics"):
        diagnostics.extend(sheet_parser.diagnostics)

    dclgen_texts: list[str] = []

    diagnostics.append(f"DCLGEN file count: {len(DCLGEN_PATHS)}")
    logger.info("DCLGEN file count: %s", len(DCLGEN_PATHS))

    for index, dclgen_path in enumerate(DCLGEN_PATHS, start=1):
        dclgen_file = LocalUploadedFile(dclgen_path)
        dclgen_text = file_loader.read_uploaded_text(dclgen_file)
        dclgen_texts.append(dclgen_text)

        diagnostics.append(f"DCLGEN {index}: {dclgen_path}")
        diagnostics.append(f"DCLGEN {index} text length: {len(dclgen_text)}")
        logger.info("DCLGEN %s: %s", index, dclgen_path)
        logger.info("DCLGEN %s text length: %s", index, len(dclgen_text))

    if hasattr(dclgen_parser, "parse_many_texts"):
        dclgen_columns = dclgen_parser.parse_many_texts(dclgen_texts)
    else:
        dclgen_columns = []

        for index, dclgen_text in enumerate(dclgen_texts, start=1):
            parsed_columns = dclgen_parser.parse(
                text=dclgen_text,
                source_label=f"DCLGEN file {index}",
            )
            dclgen_columns.extend(parsed_columns)

    diagnostics.append(f"DCLGEN total columns: {len(dclgen_columns)}")

```

```

logger.info("DCLGEN total columns: %s", len(dclgen_columns))

if hasattr(dclgen_parser, "diagnostics"):
    diagnostics.extend(dclgen_parser.diagnostics)

copybook_text_parts: list[str] = []

diagnostics.append(f"Copybook file count: {len(COPYBOOK_PATHS)}")
logger.info("Copybook file count: %s", len(COPYBOOK_PATHS))

for index, copybook_path in enumerate(COPYBOOK_PATHS, start=1):
    copybook_file = LocalUploadedFile(copybook_path)
    copybook_text = file_loader.read_uploaded_text(copybook_file)
    copybook_text_parts.append(copybook_text)

    diagnostics.append(f"Copybook {index}: {copybook_path}")
    diagnostics.append(f"Copybook {index} text length: {len(copybook_text)}")
    logger.info("Copybook %s: %s", index, copybook_path)
    logger.info("Copybook %s text length: %s", index, len(copybook_text))

copybook_text = "\n".join(copybook_text_parts)

if copybook_text.strip():
    try:
        copybook_fields = copybook_parser.parse(
            text=copybook_text,
            source_label="Copybook files",
        )
    except TypeError:
        copybook_fields = copybook_parser.parse(copybook_text)
else:
    copybook_fields = []

diagnostics.append(f"Copybook total fields: {len(copybook_fields)}")
logger.info("Copybook total fields: %s", len(copybook_fields))

if hasattr(copybook_parser, "diagnostics"):
    diagnostics.extend(copybook_parser.diagnostics)

source_file = LocalUploadedFile(IDMS_COBOL_SOURCE_PATH)
idms_cobol_text = file_loader.read_uploaded_text(source_file)

diagnostics.append(f"IDMS COBOL source file: {IDMS_COBOL_SOURCE_PATH}")
diagnostics.append(f"IDMS COBOL source text length: {len(idms_cobol_text)}")
logger.info("IDMS COBOL source file: %s", IDMS_COBOL_SOURCE_PATH)
logger.info("IDMS COBOL source text length: %s", len(idms_cobol_text))

return (
    sheet_rows,
    dclgen_columns,
    copybook_fields,
    idms_cobol_text,
    diagnostics,
)

def run_conversion() -> None:
    validate_inputs()

    logger = LoggerFactory.create_logger(
        name="idms_db2_update_conversion",
        logs_dir=LOGS_DIR,
    )

    (
        sheet_rows,
        dclgen_columns,
        copybook_fields,
        idms_cobol_text,
        diagnostics,
    ) = load_inputs(logger)

    service = ConversionService()

    result = service.convert(
        ConversionInput(

```

```

        sheet_mapping_rows=sheet_rows,
        dclgen_columns=dclgen_columns,
        copybook_fields=copybook_fields,
        idms_cobol_text=idms_cobol_text,
        target_program_id=TARGET_PROGRAM_ID,
        auto_fix_pic_length_mismatches=AUTO_FIX_PIC_LENGTH_MISMATCHES,
    )
)

date_time = datetime.now().strftime("%d-%m-%Y_%H%M%S")
code_name = IDMS_COBOL_SOURCE_PATH.stem
output_cobol_path = OUTPUT_DIR / f"{code_name}_{date_time}.cbl"

output_cobol_path.write_text(
    result.converted_cobol or "",
    encoding="utf-8",
)

logger.info("DB2 COBOL generation completed.")
logger.info("Output file created: %s", output_cobol_path)

print("DB2 COBOL generation completed.")
print(f"Output file created: {output_cobol_path}")
print("")
print("Input Summary")
print("")
print(f"Project Root      : {PROJECT_ROOT}")
print(f"SRC Directory     : {SRC_DIR}")
print(f"Sheet Mapping Rows : {len(sheet_rows)}")
print(f"DCLGEN Columns    : {len(dclgen_columns)}")
print(f"Copybook Fields    : {len(copybook_fields)}")
print(f"COBOL Text Length  : {len(idms_cobol_text)}")
print(f"Target PROGRAM-ID  : {TARGET_PROGRAM_ID}")

print("")
print("Selected Input Files")
print("")
print(f"Sheet Mapping      : {SHEET_MAPPING_PATH}")
print(f"IDMS COBOL Source  : {IDMS_COBOL_SOURCE_PATH}")

print("DCLGEN Files:")
for path in DCLGEN_PATHS:
    print(f" - {path}")

if COPYBOOK_PATHS:
    print("Copybook Files:")
    for path in COPYBOOK_PATHS:
        print(f" - {path}")
else:
    print("Copybook Files      : None")

print("")
print("Validation Messages")
print("")

if result.validation_messages:
    for message in result.validation_messages:
        print(f"- {message}")
        logger.warning(message)
else:
    print("No validation messages.")

print("")
print("Diagnostics")
print("")

for diagnostic in diagnostics:
    print(f"- {diagnostic}")
    logger.info(diagnostic)

if __name__ == "__main__":
    run_conversion()

```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\transformers\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\transformers\cobol_transformer.py
ENCODING: utf-8
=====
```

```
import re

from idms_db2_phase2.domain.models import IdmsOperation
from idms_db2_phase2.parsers.cobol_parser import CobolParser
from idms_db2_phase2.transformers.idms_statement_transformer import (
    IdmsStatementTransformer,
)
from patterns.cobol_patterns import DIVISION_PATTERN, PROGRAM_ID_PATTERN
from patterns.db2_patterns import SQL_ERROR_PARAGRAPH_PATTERN

class CobolTransformer:
    """
    Converts IDMS COBOL statements to DB2-compatible COBOL.

    Responsibilities:
    - Preserve original COBOL business flow.
    - Convert CBL compiler option line to DB2-compatible option.
    - Replace target PROGRAM-ID.
    - Remove residual IDMS declarative/control lines.
    - Remove or convert residual IDMS executable lines.
    - Preserve sequence/spacing for one-line DB condition replacements.
    - Remove orphan IDMS-ABORT paragraph after PERFORM IDMS-ABORT is removed.
    """

    IDMS_DECLARATIVE_PATTERNS = [
        re.compile(
            r"^IDMS-CONTROL\s+SECTION\.\?$",
            flags=re.IGNORECASE,
        ),
        re.compile(
            r"^PROTOCOL\b.*$",
            flags=re.IGNORECASE,
        ),
        re.compile(
            r"^IDMS-RECORDS\s+WITHIN\s+WORKING-STORAGE\s+SECTION\.\?$",
            flags=re.IGNORECASE,
        ),
        re.compile(
            r"^SCHEMA\s+SECTION\.\?$",
            flags=re.IGNORECASE,
        ),
        re.compile(
            r"^DB\s+[A-Z0-9-]+\s+WITHIN\s+[A-Z0-9-]+\.\?$",
            flags=re.IGNORECASE,
        ),
        re.compile(
            r"^COPY\s+IDMS\b.*$",
            flags=re.IGNORECASE,
        ),
    ]

    IDMS_EXECUTABLE_PATTERNS = [
        re.compile(
            r"^BIND\b.*$",
            flags=re.IGNORECASE,
        ),
        re.compile(
            r"^FIND\s+CURRENT\b.*$",
            flags=re.IGNORECASE,
        ),
        re.compile(
            r"^FINISH\.\?$",

```

```

        flags=re.IGNORECASE,
    ),
]

IDMS_ABORT_PARAGRAPH_PATTERN = re.compile(
    r"^IDMS-ABORT\.?$",
    flags=re.IGNORECASE,
)

EXIT_LINE_PATTERN = re.compile(
    r"^EXIT\.?$",
    flags=re.IGNORECASE,
)

GENERATED_LINE_PREFIXES = (
    "* DB2:",
    "EXEC SQL",
    "END-EXEC",
    "MOVE '",
    "PERFORM ",
    "END-IF",
    "CONTINUE",
    "SELECT",
    "INSERT",
    "UPDATE",
    "DELETE",
    "COMMIT",
    "ROLLBACK",
)

def __init__(
    self,
    idms_statement_transformer: IdmsStatementTransformer,
) -> None:
    self.idms_statement_transformer = idms_statement_transformer
    self.parser = CobolParser()

def transform(
    self,
    cobol_text: str,
    target_program_id: str = "",
) -> tuple[str, list[str], list[IdmsOperation]]:
    operations = self.parser.analyze(cobol_text)

    validation_messages: list[str] = []
    output_lines: list[str] = []
    current_division = ""
    sql_error_paragraph = self._detect_sql_error_paragraph(cobol_text)
    skip_orphan_idms_abort_exit = False

    for raw_line in str(cobol_text or "").splitlines():
        line = raw_line.rstrip()
        logical = self._logical_line(line)
        logical_stripped = logical.strip()

        if skip_orphan_idms_abort_exit:
            if self._is_exit_line(logical_stripped):
                skip_orphan_idms_abort_exit = False
                continue

            skip_orphan_idms_abort_exit = False

        if not logical_stripped:
            output_lines.append(line)
            continue

        if self._is_cbl_line(logical_stripped):
            output_lines.append(
                self._replace_logical_body(
                    original_line=line,
                    replacement_body="CBL ARITH(EXTEND)",
                )
            )
            continue

        if self._is_idms_abort_paragraph(logical_stripped):

```



```

        output_lines.append(
            """ DB2: Removed orphan IDMS-ABORT paragraph."
        )
        skip_orphan_idms_abort_exit = True
        continue

    program_id_line = self._program_id_replacement(
        original_line=line,
        logical_line=logical_stripped,
        target_program_id=target_program_id,
    )

    if program_id_line is not None:
        output_lines.append(program_id_line)
        continue

    division_match = DIVISION_PATTERN.match(logical_stripped)

    if division_match:
        current_division = division_match.group(1).upper()
        output_lines.append(line)
        continue

    if self._is_idms_declarative_or_control(logical_stripped):
        output_lines.extend(
            self._removed_declarative_lines(logical_stripped)
        )
        continue

    if self._is_idms_executable_cleanup(logical_stripped):
        output_lines.extend(
            self._removed_executable_lines(
                logical_line=logical_stripped,
                current_division=current_division,
            )
        )
        continue

    transformed_lines, messages = (
        self.idms_statement_transformer.transform_line(
            line=logical_stripped,
            current_division=current_division,
            sql_error_paragraph=sql_error_paragraph,
        )
    )

    validation_messages.extend(messages)

    if self._transformer_changed_line(
        original_logical=logical_stripped,
        transformed_lines=transformed_lines,
    ):
        output_lines.extend(
            self._merge_transformed_lines_with_original_style(
                original_line=line,
                original_logical=logical_stripped,
                transformed_lines=transformed_lines,
            )
        )
    else:
        output_lines.append(line)

    converted_text = "\n".join(output_lines).rstrip() + "\n"

    converted_text = self._fix_program_id_period(
        text=converted_text,
        target_program_id=target_program_id,
    )

    return converted_text, validation_messages, operations

def _merge_transformed_lines_with_original_style(
    self,
    original_line: str,
    original_logical: str,
    transformed_lines: list[str],

```

```

) -> list[str]:
    if not transformed_lines:
        return [original_line]

    if len(transformed_lines) == 1:
        transformed = str(transformed_lines[0] or "").strip()

        if self._is_single_line_token_replacement(
            original_logical=original_logical,
            transformed_line=transformed,
        ):
            return [
                self._replace_logical_body(
                    original_line=original_line,
                    replacement_body=transformed,
                )
            ]

        return [transformed]

    return transformed_lines

def _is_single_line_token_replacement(
    self,
    original_logical: str,
    transformed_line: str,
) -> bool:
    original = str(original_logical or "").strip()
    transformed = str(transformed_line or "").strip()
    transformed_upper = transformed.upper()

    if not original or not transformed:
        return False

    if "SQLCODE = 100" in transformed_upper:
        return True

    if "SQLCODE NOT = 100" in transformed_upper:
        return True

    if "SQLCODE NOT = 0" in transformed_upper:
        return True

    if transformed_upper.startswith(self.GENERATED_LINE_PREFIXES):
        return False

    if transformed.startswith("*"):
        return False

    return False

def _program_id_replacement(
    self,
    original_line: str,
    logical_line: str,
    target_program_id: str,
) -> str | None:
    if not target_program_id:
        return None

    if not PROGRAM_ID_PATTERN.search(logical_line):
        return None

    program_id = str(target_program_id or "").strip().upper()

    if not program_id:
        return None

    replacement_body = f"PROGRAM-ID. {program_id}."

    return self._replace_logical_body(
        original_line=original_line,
        replacement_body=replacement_body,
    )

def _fix_program_id_period(

```

```

        self,
        text: str,
        target_program_id: str,
    ) -> str:
        program_id = str(target_program_id or "").strip().upper()

        if program_id:
            pattern = re.compile(
                rf"PROGRAM-ID\. \s*{re.escape(program_id)}\.\.?",
                flags=re.IGNORECASE,
            )

            return pattern.sub(
                f"PROGRAM-ID. {program_id}.",
                text,
                count=1,
            )

        return re.sub(
            r"(PROGRAM-ID\. \s*[A-Z0-9-]+\s*$",
            r"\1.",
            text,
            count=1,
            flags=re.IGNORECASE | re.MULTILINE,
        )

def _is_cbl_line(
    self,
    logical_line: str,
) -> bool:
    return str(logical_line or "").strip().upper().startswith("CBL ")

def _is_idms_declarative_or_control(
    self,
    logical_line: str,
) -> bool:
    statement = str(logical_line or "").strip().rstrip(".")

    for pattern in self.IDMS_DECLARATIVE_PATTERNS:
        if pattern.search(statement):
            return True

    return False

def _is_idms_executable_cleanup(
    self,
    logical_line: str,
) -> bool:
    statement = str(logical_line or "").strip().rstrip(".")

    for pattern in self.IDMS_EXECUTABLE_PATTERNS:
        if pattern.search(statement):
            return True

    return False

def _is_idms_abort_paragraph(
    self,
    logical_line: str,
) -> bool:
    statement = str(logical_line or "").strip()

    return bool(self.IDMS_ABORT_PARAGRAPH_PATTERN.match(statement))

def _is_exit_line(
    self,
    logical_line: str,
) -> bool:
    statement = str(logical_line or "").strip()

    return bool(self.EXIT_LINE_PATTERN.match(statement))

def _removed_declarative_lines(
    self,
    logical_line: str,
) -> list[str]:

```

```

    return [
        f"* DB2: Removed residual IDMS control statement: {logical_line}",
    ]

def _removed_executable_lines(
    self,
    logical_line: str,
    current_division: str,
) -> list[str]:
    statement = str(logical_line or "").strip()

    if re.search(
        r"^FINISH\.\?$",
        statement,
        flags=re.IGNORECASE,
    ):
        if current_division == "PROCEDURE":
            return [
                "* DB2: IDMS FINISH converted to COMMIT.",
                "MOVE 'COMMIT' TO SQL-LOCATION.",
                "EXEC SQL",
                "    COMMIT",
                "END-EXEC.",
            ]

            return [
                f"* DB2: Removed IDMS FINISH outside PROCEDURE DIVISION: {statement}",
            ]

        if re.search(
            r"^BIND\b",
            statement,
            flags=re.IGNORECASE,
        ):
            return self._procedure_safe_removal(
                message=f"* DB2: Removed IDMS BIND statement: {statement}",
                current_division=current_division,
            )

        if re.search(
            r"^FIND\s+CURRENT\b",
            statement,
            flags=re.IGNORECASE,
        ):
            return self._procedure_safe_removal(
                message=f"* DB2: Removed IDMS FIND CURRENT statement: {statement}",
                current_division=current_division,
            )

        return self._procedure_safe_removal(
            message=f"* DB2: Removed residual IDMS executable statement: {statement}",
            current_division=current_division,
        )

def _procedure_safe_removal(
    self,
    message: str,
    current_division: str,
) -> list[str]:
    if current_division == "PROCEDURE":
        return [
            message,
            "CONTINUE.",
        ]

    return [
        message,
    ]

def _transformer_changed_line(
    self,
    original_logical: str,
    transformed_lines: list[str],
) -> bool:
    if not transformed_lines:
        return False

```

```

    if len(transformed_lines) != 1:
        return True

    transformed = str(transformed_lines[0] or "").strip()
    original = str(original_logical or "").strip()

    return transformed != original

def _logical_line(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    text = self._remove_right_sequence(text)
    text = self._remove_left_sequence(text)

    return text.strip()

def _remove_left_sequence(
    self,
    line: str,
) -> str:
    text = str(line or "")

    if len(text) >= 6 and text[:6].strip().isdigit():
        return text[6:].strip()

    return text

def _remove_right_sequence(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    match = re.match(
        r"^(?P<body>.*?)(?:\s+(?P<right>\d{8}))\s*$",
        text,
        flags=re.IGNORECASE,
    )

    if match:
        return match.group("body").rstrip()

    return text

def _replace_logical_body(
    self,
    original_line: str,
    replacement_body: str,
) -> str:
    text = str(original_line or "").rstrip()

    left_sequence = ""
    body_with_possible_right = text

    left_match = re.match(
        r"^(?P<left>\s*\d{6}\s+)(?P<body>.*?)$",
        body_with_possible_right,
    )

    if left_match:
        left_sequence = left_match.group("left")
        body_with_possible_right = left_match.group("body")

    right_match = re.match(
        r"^(?P<body>.*?)(?P<spaces>\s+)(?P<right>\d{8})\s*$",
        body_with_possible_right,
    )

    if right_match:
        original_body = right_match.group("body")
        original_spaces = right_match.group("spaces")
        right_sequence = right_match.group("right")

```

```

        original_body_area_width = len(original_body) + len(original_spaces)

        replacement = str(replacement_body or "")

        if len(replacement) >= original_body_area_width:
            formatted_body = replacement + " "
        else:
            formatted_body = replacement.ljust(original_body_area_width)

        return f"{left_sequence}{formatted_body}{right_sequence}"

    if left_sequence:
        return f"{left_sequence}{replacement_body}"

    return str(replacement_body or "")

def _detect_sql_error_paragraph(
    self,
    cobol_text: str,
) -> str:
    match = SQL_ERROR_PARAGRAPH_PATTERN.search(str(cobol_text or ""))

    if not match:
        return "SQL-ERROR"

    return match.group(1).upper()

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\transformers\field_reference_rewriter.py
ENCODING: utf-8
=====

```

```

import re
from difflib import SequenceMatcher

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.field_reference_patterns import QUALIFIED_REFERENCE_PATTERN

```

```

class FieldReferenceRewriter:
    """
    Generic IDMS field reference to DB2 DCLGEN reference rewriter.

    No business names are hardcoded.

    Rules:
    - Qualified references are always eligible:
        FIELD OF RECORD
        FIELD IN RECORD

    - Bare references are eligible only when active record context is strong.

    - Date fields are protected only for bare references.
      Qualified references like DA-FIELD OF RECORD are eligible.
    """

    EXEC_SQL_PATTERN = re.compile(
        r"^\\s*EXEC\\s+SQL\\b",
        flags=re.IGNORECASE,
    )

    END_EXEC_PATTERN = re.compile(
        r"^\\s*END-EXEC\\.\\s*$",
        flags=re.IGNORECASE,
    )

    DIVISION_PATTERN = re.compile(
        r"^\\s*(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\\s+DIVISION\\b",
        flags=re.IGNORECASE,
    )

```

```

PARAGRAPH_PATTERN = re.compile(
    r"^\s*(?P<name>[A-Z0-9][A-Z0-9-]*)\.\s*$",
    flags=re.IGNORECASE,
)

STRING_PATTERN = re.compile(
    r"'[^']*'",
    flags=re.IGNORECASE,
)

HOST_REFERENCE_OF_PATTERN = re.compile(
    r":?\s*(?P<host>[A-Z][A-Z0-9-]*)\s+OF\s+(?P<group>DCL[A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

HOST_REFERENCE_DOT_PATTERN = re.compile(
    r":?\s*(?P<group>DCL[A-Z0-9-]*)\.(?P<host>[A-Z][A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

INITIALIZE_DCL_PATTERN = re.compile(
    r"\bINITIALIZE\s+(?P<group>DCL[A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

GENERATED_OBTAIN_COMMENT_PATTERN = re.compile(
    r"CONVERTED\s+(?:OBTAIN|FIND)\s+(?:FIRST|NEXT|CALC)?\s*(?P<record>[A-Z][A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

MOVE_SPACES_TO_RECORD_PATTERN = re.compile(
    r"\bMOVE\s+(?:SPACE|SPACES|ZEROES|ZEROS)\s+TO\s+(?P<record>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

PROTECTED_BARE_PREFIXES = (
    "WS-",
    "WK-",
    "W-",
    "SW-",
    "UIT-",
    "OUT-",
    "ES-",
    "D-",
    "DATE",
    "SQL",
    "DCL",
    "L-",
    "LS-",
    "PARAM",
    "ERROR-",
    "USER",
    "CS-",
    "TS-",
    "DA-",
    "HR-",
    "HELP-",
)

MIN_SIMILARITY_FOR_ALIAS = 0.94

def __init__(
    self,
    mapping_repository: MappingRepository,
    table_name_resolver: TableNameResolver,
    host_variable_resolver: HostVariableResolver,
) -> None:
    self.mapping_repository = mapping_repository
    self.table_name_resolver = table_name_resolver
    self.host_variable_resolver = host_variable_resolver
    self.rewrite_messages: list[str] = []

    self.record_field_map: dict[str, dict[str, str]] = {}
    self.record_group_map: dict[str, str] = {}
    self.table_record_map: dict[str, str] = {}
    self.group_record_map: dict[str, str] = {}

```

```

def rewrite(
    self,
    text: str,
) -> str:
    self.rewrite_messages = []

    if not text:
        return ""

    self._build_reference_maps()

    if not self.record_field_map:
        return str(text or "").rstrip() + "\n"

    output_lines: list[str] = []
    inside_exec_sql = False
    current_division = ""
    active_record = ""

    for raw_line in str(text or "").splitlines():
        line = raw_line.rstrip()
        logical = self._logical_line(line)

        division_match = self.DIVISION_PATTERN.match(logical)

        if division_match:
            current_division = division_match.group(1).upper()
            active_record = ""
            output_lines.append(line)
            continue

        detected_context = self._detect_active_record_from_line(
            logical_line=logical,
            current_active_record=active_record,
        )

        if detected_context:
            active_record = detected_context

        if self.EXEC_SQL_PATTERN.match(logical):
            inside_exec_sql = True
            output_lines.append(line)
            continue

        if inside_exec_sql:
            output_lines.append(line)

            if self.END_EXEC_PATTERN.match(logical):
                inside_exec_sql = False

            continue

        output_lines.append(
            self._rewrite_line(
                line=line,
                current_division=current_division,
                active_record=active_record,
            )
        )

    return "\n".join(output_lines).rstrip() + "\n"

def _rewrite_line(
    self,
    line: str,
    current_division: str,
    active_record: str,
) -> str:
    if self._is_comment_or_blank(line):
        return line

    if current_division != "PROCEDURE":
        return line

    if self._has_existing_dclgen_reference(line):

```



```

        return line

    segments = self._split_string_segments(line)
    rewritten_segments: list[str] = []

    for segment, is_string in segments:
        if is_string:
            rewritten_segments.append(segment)
            continue

        rewritten = self._rewrite_qualified_references(segment)
        rewritten = self._rewrite_bare_references(
            text=rewritten,
            active_record=active_record,
        )

        rewritten_segments.append(rewritten)

    return "".join(rewritten_segments)

def _rewrite_qualified_references(
    self,
    text: str,
) -> str:
    def repl(match):
        source_field = NameNormalizer.to_cobol(match.group("field"))
        source_record = NameNormalizer.normalize(match.group("record"))

        target = self._target_for_record_field(
            record_name=source_record,
            source_field=source_field,
        )

        if not target:
            return match.group(0)

        self.rewrite_messages.append(
            f"Qualified field rewrite: {source_field} OF "
            f"{NameNormalizer.to_cobol(source_record)} -> {target}"
        )

        return target

    return QUALIFIED_REFERENCE_PATTERN.sub(
        repl,
        text,
    )

def _rewrite_bare_references(
    self,
    text: str,
    active_record: str,
) -> str:
    record = NameNormalizer.normalize(active_record)

    if not record:
        return text

    field_map = self.record_field_map.get(record, {})

    if not field_map:
        return text

    updated = str(text or "")

    for source_key, target_reference in sorted(
        field_map.items(),
        key=lambda item: len(item[0]),
        reverse=True,
    ):
        source_name = NameNormalizer.to_cobol(source_key)

        if not source_name or not target_reference:
            continue

        if self._is_protected_bare_source_name(source_name):

```

```

        continue

    updated = self._replace_bare_identifier(
        text=updated,
        source_name=source_name,
        target_name=target_reference,
    )

    return updated

def _replace_bare_identifier(
    self,
    text: str,
    source_name: str,
    target_name: str,
) -> str:
    source = NameNormalizer.to_cobol(source_name)
    target = str(target_name or "").strip()

    if not source or not target:
        return text

    pattern = re.compile(
        rf"(?<![A-Z0-9-]){re.escape(source)}(?![A-Z0-9-])",
        flags=re.IGNORECASE,
    )

    def repl(match):
        before_text = text[: match.start()]
        after_text = text[match.end():]

        if self._is_part_of_qualified_reference(
            before_text=before_text,
            after_text=after_text,
        ):
            return match.group(0)

        self.rewrite_messages.append(
            f"Bare field rewrite: {source} -> {target}"
        )

        return target

    return pattern.sub(
        repl,
        text,
    )

def _target_for_record_field(
    self,
    record_name: str,
    source_field: str,
) -> str:
    record = NameNormalizer.normalize(record_name)
    field = self._field_key(source_field)

    if not record or not field:
        return ""

    return self.record_field_map.get(record, {}).get(field, "")

def _build_reference_maps(
    self,
) -> None:
    self.record_field_map = {}
    self.record_group_map = {}
    self.table_record_map = {}
    self.group_record_map = {}

    records = self.mapping_repository.records()

    for record in records:
        normalized_record = NameNormalizer.normalize(record)

        if not normalized_record:
            continue

```

```

        resolved_table = self.table_name_resolver.table_for_record(
            normalized_record
        )

        if resolved_table:
            table_key = NameNormalizer.normalize(resolved_table)
            group_name = "DCL" + NameNormalizer.to_cobol(table_key)

            self.record_group_map[normalized_record] = group_name
            self.table_record_map[table_key] = normalized_record
            self.group_record_map[NameNormalizer.normalize(group_name)] = (
                normalized_record
            )

        self.record_field_map[normalized_record] = (
            self._field_map_for_record(normalized_record)
        )

    self._add_redefines_aliases()
    self._add_strict_similarity_aliases()

def _field_map_for_record(
    self,
    record_name: str,
) -> dict[str, str]:
    output: dict[str, str] = {}

    rows = self.mapping_repository.rows_for_record(record_name)

    for row in rows:
        source_candidates = self._source_candidates_from_row(row)

        if not source_candidates:
            continue

        target_table = self.table_name_resolver.resolve_table(
            self._first_non_empty(
                getattr(row, "new_db2_record", ""),
                getattr(row, "cross_application_db2_table", ""),
            )
        )

        target_column = NameNormalizer.normalize(
            self._first_non_empty(
                getattr(row, "new_db2_field_name", ""),
                getattr(row, "cross_application_db2_field_name", ""),
            )
        )

        if not target_table or not target_column:
            continue

        target_reference = self.dclgen_reference_for_column(
            table_name=target_table,
            column_name=target_column,
        )

        if not target_reference:
            continue

        for source_candidate in source_candidates:
            source_key = self._field_key(source_candidate)

            if not source_key:
                continue

            if source_key not in output:
                output[source_key] = target_reference

    return output

def _source_candidates_from_row(
    self,
    row,
) -> list[str]:

```

```

candidates: list[str] = []

for value in [
    getattr(row, "cobol_zone", ""),
    getattr(row, "reference_field_name_copybook", ""),
]:
    field_name = self._extract_field_name(value)

    if not field_name:
        continue

    cobol_name = NameNormalizer.to_cobol(field_name)

    if not cobol_name:
        continue

    if cobol_name not in candidates:
        candidates.append(cobol_name)

    redefines_base = self._extract_redefines_base(value)

    if redefines_base:
        base_name = NameNormalizer.to_cobol(redefines_base)

        if base_name and base_name not in candidates:
            candidates.append(base_name)

return candidates

def _add_redefines_aliases(
    self,
) -> None:
    for record in self.mapping_repository.records():
        normalized_record = NameNormalizer.normalize(record)
        rows = self.mapping_repository.rows_for_record(normalized_record)
        field_map = self.record_field_map.get(normalized_record, {})

        if not field_map:
            continue

        for row in rows:
            cobol_zone = getattr(row, "cobol_zone", "") or ""
            alias_name = self._extract_field_name(cobol_zone)
            base_name = self._extract_redefines_base(cobol_zone)

            if not alias_name or not base_name:
                continue

            alias_key = self._field_key(alias_name)
            base_key = self._field_key(base_name)

            if not alias_key or not base_key:
                continue

            if alias_key in field_map:
                continue

            if base_key in field_map:
                field_map[alias_key] = field_map[base_key]

def _add_strict_similarity_aliases(
    self,
) -> None:
    for record, field_map in self.record_field_map.items():
        existing_keys = list(field_map.keys())

        rows = self.mapping_repository.rows_for_record(record)

        for row in rows:
            for source_candidate in self._source_candidates_from_row(row):
                source_key = self._field_key(source_candidate)

                if not source_key:
                    continue

                if source_key in field_map:

```

```

        continue

    best_key = ""
    best_score = 0.0

    for existing_key in existing_keys:
        score = self._similarity(source_key, existing_key)

        if score > best_score:
            best_score = score
            best_key = existing_key

    if (
        best_key
        and best_score >= self.MIN_SIMILARITY_FOR_ALIAS
        and best_key in field_map
    ):
        field_map[source_key] = field_map[best_key]

def _dclgen_reference_for_column(
    self,
    table_name: str,
    column_name: str,
) -> str:
    host_reference = self.host_variable_resolver.host_reference_for_column(
        table_name=table_name,
        column_name=column_name,
    )

    if not host_reference:
        return ""

    host_reference = str(host_reference or "").strip()

    of_match = self.HOST_REFERENCE_OF_PATTERN.search(host_reference)

    if of_match:
        host = NameNormalizer.to_cobol(of_match.group("host"))
        group = NameNormalizer.to_cobol(of_match.group("group"))
        return f"{host} OF {group}"

    dot_match = self.HOST_REFERENCE_DOT_PATTERN.search(host_reference)

    if dot_match:
        host = NameNormalizer.to_cobol(dot_match.group("host"))
        group = NameNormalizer.to_cobol(dot_match.group("group"))
        return f"{host} OF {group}"

    cleaned = host_reference.lstrip(":").strip()

    if " OF " in cleaned.upper():
        parts = re.split(
            r"\s+OF\s+",
            cleaned,
            flags=re.IGNORECASE,
        )
        if len(parts) == 2:
            host = NameNormalizer.to_cobol(parts[0])
            group = NameNormalizer.to_cobol(parts[1])
            return f"{host} OF {group}"

    return NameNormalizer.to_cobol(cleaned)

def _detect_active_record_from_line(
    self,
    logical_line: str,
    current_active_record: str,
) -> str:
    logical = str(logical_line or "").strip()

    if not logical:
        return current_active_record

    paragraph_record = self._record_from_paragraph(logical)

    if paragraph_record:

```

```

        return paragraph_record

    initialize_record = self._record_from_initialize(logical)

    if initialize_record:
        return initialize_record

    generated_comment_record = self._record_from_generated_comment(logical)

    if generated_comment_record:
        return generated_comment_record

    move_target_record = self._record_from_move_target(logical)

    if move_target_record:
        return move_target_record

    qualified_record = self._record_from_qualified_reference(logical)

    if qualified_record:
        return qualified_record

    return current_active_record

def _record_from_paragraph(
    self,
    logical_line: str,
) -> str:
    match = self.PARAGRAPH_PATTERN.match(logical_line)

    if not match:
        return ""

    paragraph = NameNormalizer.normalize(match.group("name"))

    for record in self.record_field_map:
        if record and record in paragraph:
            return record

    return ""

def _record_from_initialize(
    self,
    logical_line: str,
) -> str:
    match = self.INITIALIZE_DCL_PATTERN.search(logical_line)

    if not match:
        return ""

    group = NameNormalizer.normalize(match.group("group"))

    return self.group_record_map.get(group, "")

def _record_from_generated_comment(
    self,
    logical_line: str,
) -> str:
    match = self.GENERATED_OBTAIN_COMMENT_PATTERN.search(logical_line)

    if not match:
        return ""

    record = NameNormalizer.normalize(match.group("record"))

    if record in self.record_field_map:
        return record

    return ""

def _record_from_move_target(
    self,
    logical_line: str,
) -> str:
    match = self.MOVE_SPACES_TO_RECORD_PATTERN.search(logical_line)

```

```

    if not match:
        return ""

    record = NameNormalizer.normalize(match.group("record"))

    if record in self.record_field_map:
        return record

    return ""

def _record_from_qualified_reference(
    self,
    logical_line: str,
) -> str:
    match = QUALIFIED_REFERENCE_PATTERN.search(logical_line)

    if not match:
        return ""

    record = NameNormalizer.normalize(match.group("record"))

    if record in self.record_field_map:
        return record

    return ""

def _extract_field_name(
    self,
    value: str,
) -> str:
    text = str(value or "").strip()

    if not text:
        return ""

    text = text.replace(".", " ")

    level_match = re.match(
        r"^\s*(0[1-9]|[1-4][0-9]|66|77|88)\s+([A-Z][A-Z0-9-]*)\b",
        text,
        flags=re.IGNORECASE,
    )

    if level_match:
        return level_match.group(2)

    redefines_match = re.match(
        r"^\s*([A-Z][A-Z0-9-]*)\s+REDEFINES\s+[A-Z][A-Z0-9-]*",
        text,
        flags=re.IGNORECASE,
    )

    if redefines_match:
        return redefines_match.group(1)

    tokens = re.findall(
        r"[A-Z][A-Z0-9-]*",
        text,
        flags=re.IGNORECASE,
    )

    if not tokens:
        return ""

    return tokens[0]

def _extract_redefines_base(
    self,
    value: str,
) -> str:
    text = str(value or "")

    match = re.search(
        r"\bREDEFINES\s+([A-Z][A-Z0-9-]*)\b",
        text,
        flags=re.IGNORECASE,
    )

```

```

    )

    if not match:
        return ""

    return match.group(1)

def _field_key(
    self,
    value: str,
) -> str:
    return NameNormalizer.to_cobol(value)

def _is_protected_bare_source_name(
    self,
    source_name: str,
) -> bool:
    source = NameNormalizer.to_cobol(source_name).upper()

    if not source:
        return True

    for prefix in self.PROTECTED_BARE_PREFIXES:
        if source.startswith(prefix):
            return True

    return False

def _is_part_of_qualified_reference(
    self,
    before_text: str,
    after_text: str,
) -> bool:
    before = str(before_text or "").upper()
    after = str(after_text or "").upper()

    if re.search(r"\b(OF|IN)\s*$", before):
        return True

    if re.match(r"^s+(OF|IN)\s+[A-Z0-9-]+", after):
        return True

    return False

def _has_existing_dclgen_reference(
    self,
    line: str,
) -> bool:
    upper = str(line or "").upper()

    if " OF DCL" in upper:
        return True

    if re.search(
        r":\s*DCL[A-Z0-9-]+\.",
        upper,
    ):
        return True

    if re.search(
        r":\s*[A-Z0-9-]+\s+OF\s+DCL[A-Z0-9-]+",
        upper,
    ):
        return True

    return False

def _split_string_segments(
    self,
    line: str,
) -> list[tuple[str, bool]]:
    text = str(line or "")
    output: list[tuple[str, bool]] = []
    last_index = 0

    for match in self.STRING_PATTERN.finditer(text):

```



```

        if match.start() > last_index:
            output.append(
                (
                    text[last_index: match.start()],
                    False,
                )
            )

        output.append(
            (
                match.group(0),
                True,
            )
        )

        last_index = match.end()

    if last_index < len(text):
        output.append(
            (
                text[last_index:],
                False,
            )
        )

    return output

def _logical_line(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    if len(text) >= 80:
        left = text[:6]
        indicator = text[6:7]
        body = text[7:72]

        if left.strip().isdigit():
            if indicator in {"*", "/"}:
                return indicator + body.rstrip()

        return body.strip()

    if len(text) > 6 and text[:6].strip().isdigit():
        return text[6:].strip()

    return text.strip()

def _is_comment_or_blank(
    self,
    line: str,
) -> bool:
    logical = self._logical_line(line)
    stripped = str(logical or "").strip()

    if not stripped:
        return True

    if stripped.startswith("*"):
        return True

    if stripped.startswith("/"):
        return True

    return False

def _first_non_empty(
    self,
    *values: str,
) -> str:
    for value in values:
        text = str(value or "").strip()

        if text:
            return text

```

```

        return ""

def _similarity(
    self,
    left: str,
    right: str,
) -> float:
    return SequenceMatcher(
        None,
        NameNormalizer.compact(left),
        NameNormalizer.compact(right),
    ).ratio()

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\transformers\idms_statement_transformer.py
ENCODING: utf-8
=====

from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.generators.sql_generator import SqlGenerator
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.idms_patterns import (
    BIND_STATEMENT_PATTERN,
    COMMIT_PATTERN,
    CONNECT_STATEMENT_PATTERN,
    DB_END_OF_SET_TOKEN_PATTERN,
    DB_REC_NOT_FOUND_TOKEN_PATTERN,
    DISCONNECT_STATEMENT_PATTERN,
    ERASE_PATTERN,
    FIND_CURRENT_PATTERN,
    FIND_FIRST_PATTERN,
    FINISH_PATTERN,
    IDMS_ABORT_PERFORM_PATTERN,
    IDMS_DECLARATIVE_OR_CONTROL_PATTERNS,
    IDMS_STATUS_PERFORM_PATTERN,
    MODIFY_PATTERN,
    OBTAIN_CALC_PATTERN,
    OBTAIN_CALC_REVERSED_PATTERN,
    OBTAIN_FIRST_NEXT_PATTERN,
    ON_DB_REC_NOT_FOUND_PATTERN,
    READY_PATTERN,
    STORE_PATTERN,
    USAGE_MODE_PATTERN,
)

class IdmsStatementTransformer:
    """
    Converts executable IDMS statements to DB2-compatible COBOL.

    This transformer contains conversion logic only. Regex patterns live in
    patterns/idms_patterns.py.
    """

    def __init__(
        self,
        sql_generator: SqlGenerator,
        sql_error_generator: SqlErrorGenerator,
        table_name_resolver: TableNameResolver,
        cursor_name_resolver: CursorNameResolver,
    ) -> None:
        self.sql_generator = sql_generator
        self.sql_error_generator = sql_error_generator
        self.table_name_resolver = table_name_resolver
        self.cursor_name_resolver = cursor_name_resolver
        self.messages: list[str] = []

    def transform_line(
        self,
        line: str,
        current_division: str,
        sql_error_paragraph: str,

```

```

) -> tuple[list[str], str]:
    stripped_line = str(line or "").strip()
    upper = stripped_line.upper()

    if not stripped_line:
        return [line], ""

    if self._is_idms_declarative_or_control_statement(upper):
        return [
            f"* DB2: Removed residual IDMS control statement: {stripped_line}",
        ], ""

    if IDMS_STATUS_PERFORM_PATTERN.search(upper) or IDMS_ABORT_PERFORM_PATTERN.search(upper):
        return self._removed_idms_executable_lines(
            message=f"* DB2: Removed IDMS status/abort paragraph call: {stripped_line}",
            current_division=current_division,
        ), ""

    if BIND_STATEMENT_PATTERN.search(upper):
        return self._removed_idms_executable_lines(
            message=f"* DB2: Removed IDMS BIND statement: {stripped_line}",
            current_division=current_division,
        ), ""

    if USAGE_MODE_PATTERN.search(upper) or READY_PATTERN.search(upper):
        return self._removed_idms_executable_lines(
            message=f"* DB2: Removed IDMS usage/READY statement: {stripped_line}",
            current_division=current_division,
        ), ""

    if FIND_CURRENT_PATTERN.search(upper):
        return self._removed_idms_executable_lines(
            message=f"* DB2: Removed IDMS FIND CURRENT statement: {stripped_line}",
            current_division=current_division,
        ), ""

    if CONNECT_STATEMENT_PATTERN.search(upper) or DISCONNECT_STATEMENT_PATTERN.search(upper):
        return self._removed_idms_executable_lines(
            message=f"* DB2: Removed IDMS relationship statement. Relationship is handled by DB2 keys: {stripped_line}",
            current_division=current_division,
        ), ""

    finish_result = self._convert_finish_or_commit(
        upper=upper,
        stripped_line=stripped_line,
        current_division=current_division,
    )

    if finish_result is not None:
        return finish_result, ""

    on_not_found_result = self._convert_on_db_rec_not_found(stripped_line)

    if on_not_found_result is not None:
        return on_not_found_result, ""

    obtain_calc_result = self._convert_obtain_calc(
        upper=upper,
        stripped_line=stripped_line,
        current_division=current_division,
        sql_error_paragraph=sql_error_paragraph,
    )

    if obtain_calc_result is not None:
        return obtain_calc_result, ""

    obtain_set_result, opened_set = self._convert_obtain_set(
        upper=upper,
        stripped_line=stripped_line,
        current_division=current_division,
    )

    if obtain_set_result is not None:
        return obtain_set_result, opened_set

```

```

find_first_result, opened_set = self._convert_find_first(
    upper=upper,
    stripped_line=stripped_line,
    current_division=current_division,
)

if find_first_result is not None:
    return find_first_result, opened_set

store_result = self._convert_store_modify_erase(
    upper=upper,
    stripped_line=stripped_line,
    current_division=current_division,
    sql_error_paragraph=sql_error_paragraph,
)

if store_result is not None:
    return store_result, ""

return self._replace_idms_condition_tokens(line), ""

def _convert_finish_or_commit(
    self,
    upper: str,
    stripped_line: str,
    current_division: str,
) -> list[str] | None:
    if FINISH_PATTERN.search(upper):
        if current_division != "PROCEDURE":
            return [f"* DB2: FINISH ignored outside PROCEDURE DIVISION: {stripped_line}"]

        return [
            f"* DB2: IDMS FINISH converted to COMMIT.",
            f*self.sql_generator.commit(),
        ]

    if COMMIT_PATTERN.search(upper) and "EXEC SQL" not in upper:
        if current_division != "PROCEDURE":
            return [f"* DB2: COMMIT ignored outside PROCEDURE DIVISION: {stripped_line}"]

        return self.sql_generator.commit()

    return None

def _convert_on_db_rec_not_found(
    self,
    stripped_line: str,
) -> list[str] | None:
    match = ON_DB_REC_NOT_FOUND_PATTERN.match(stripped_line)

    if not match:
        return None

    statement = str(match.group("statement") or "").strip()

    if not statement:
        return [
            "IF SQLCODE = 100",
            "    CONTINUE",
            "END-IF.",
        ]

    return [
        "IF SQLCODE = 100",
        f"    {statement}",
        "END-IF.",
    ]

def _convert_obtain_calc(
    self,
    upper: str,
    stripped_line: str,
    current_division: str,
    sql_error_paragraph: str,
) -> list[str] | None:
    match = OBTAIN_CALC_PATTERN.search(upper)

```

```

if not match:
    match = OBTAIN_CALC_REVERSED_PATTERN.search(upper)

if not match:
    return None

record = match.group("record").upper()

if current_division != "PROCEDURE":
    return [
        f"* DB2: OBTAIN CALC ignored outside PROCEDURE DIVISION: {stripped_line}",
    ]

return [
    f"* DB2: Converted OBTAIN CALC for {record}.",
    *self.sql_generator.select_by_key(record),
    "IF SQLCODE NOT = 0 AND SQLCODE NOT = 100",
    f"    PERFORM {sql_error_paragraph}.",
    "END-IF.",
]

def _convert_obtain_set(
    self,
    upper: str,
    stripped_line: str,
    current_division: str,
) -> tuple[list[str] | None, str]:
    match = OBTAIN_FIRST_NEXT_PATTERN.search(upper)

    if not match:
        return None, ""

    mode = match.group("mode").upper()
    record = match.group("record").upper()
    set_name = match.group("set").upper()

    if current_division != "PROCEDURE":
        return [
            f"* DB2: OBTAIN {mode} ignored outside PROCEDURE DIVISION: {stripped_line}",
        ], ""

    table = self.table_name_resolver.table_for_record(record)
    cursor_name = self.cursor_name_resolver.cursor_name_from_table(table)

    return [
        f"* DB2: Converted OBTAIN {mode} {record} WITHIN {set_name}.",
        f"PERFORM OPEN-{cursor_name}.",
        f"PERFORM FETCH-{cursor_name}.",
    ], set_name

def _convert_find_first(
    self,
    upper: str,
    stripped_line: str,
    current_division: str,
) -> tuple[list[str] | None, str]:
    match = FIND_FIRST_PATTERN.search(upper)

    if not match:
        return None, ""

    record = (match.group("record") or "").upper()
    set_name = match.group("set").upper()

    if current_division != "PROCEDURE":
        return [
            f"* DB2: FIND FIRST ignored outside PROCEDURE DIVISION: {stripped_line}",
        ], ""

    if not record:
        self.messages.append(
            f"FIND FIRST WITHIN {set_name} needs record inference from source code or mapping."
        )
    return [
        f"* DB2: FIND FIRST WITHIN {set_name} could not be converted because record name was not found.",
    ], ""

```

```

        "CONTINUE.",
    ], ""

    table = self.table_name_resolver.table_for_record(record)
    cursor_name = self.cursor_name_resolver.cursor_name_from_table(table)

    return [
        f"* DB2: Converted FIND FIRST {record} WITHIN {set_name}.",
        f"PERFORM OPEN-{cursor_name}.",
        f"PERFORM FETCH-{cursor_name}.",
    ], set_name

def _convert_store_modify_erase(
    self,
    upper: str,
    stripped_line: str,
    current_division: str,
    sql_error_paragraph: str,
) -> list[str] | None:
    for pattern, operation_name, generator_method in [
        (STORE_PATTERN, "STORE", self.sql_generator.insert),
        (MODIFY_PATTERN, "MODIFY", self.sql_generator.update),
        (ERASE_PATTERN, "ERASE", self.sql_generator.delete),
    ]:
        match = pattern.search(upper)

        if not match:
            continue

        record = match.group("record").upper()

        if current_division != "PROCEDURE":
            return [
                f"* DB2: {operation_name} ignored outside PROCEDURE DIVISION: {stripped_line}",
            ]

        return [
            f"* DB2: Converted {operation_name} for {record}.",
            *generator_method(record),
            "IF SQLCODE NOT = 0",
            f"    PERFORM {sql_error_paragraph}.",
            "END-IF.",
        ]

    return None

def _replace_idms_condition_tokens(
    self,
    line: str,
) -> list[str]:
    updated = DB_REC_NOT_FOUND_TOKEN_PATTERN.sub("SQLCODE = 100", line)
    updated = DB_END_OF_SET_TOKEN_PATTERN.sub("SQLCODE = 100", updated)

    return [updated]

def _is_idms_declarative_or_control_statement(
    self,
    upper: str,
) -> bool:
    return any(pattern.search(upper) for pattern in IDMS_DECLARATIVE_OR_CONTROL_PATTERNS)

def _removed_idms_executable_lines(
    self,
    message: str,
    current_division: str,
) -> list[str]:
    if current_division == "PROCEDURE":
        return [
            message,
            "CONTINUE.",
        ]

    return [message]

```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\transformers\pic_length_auto_fixer.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.pic_patterns import (
    DATA_ENTRY_START_PATTERN,
    MOVE_PATTERN,
    NUMERIC_PIC_PATTERN,
)

class PicLengthAutoFixer:
    """
    Auto-fixes target COBOL PIC lengths when a MOVE source has more numeric
    digits than the MOVE target.

    This transformer is generic and does not hardcode business fields.
    """

    def __init__(self) -> None:
        self.messages: list[str] = []

    def fix(
        self,
        source_cobol_text: str,
        converted_cobol_text: str,
    ) -> str:
        self.messages = []

        if not converted_cobol_text:
            return converted_cobol_text

        combined_text = "\n".join(
            [
                source_cobol_text or "",
                converted_cobol_text or "",
            ]
        )

        pic_lengths = self._parse_numeric_pic_lengths(combined_text)

        if not pic_lengths:
            self.messages.append("Auto-fix PIC: no numeric PIC fields found.")
            return converted_cobol_text

        move_pairs = self._parse_move_pairs(converted_cobol_text)

        if not move_pairs:
            self.messages.append("Auto-fix PIC: no MOVE source/target pairs found.")
            return converted_cobol_text

        fixes: dict[str, int] = {}

        for source_name, target_name in move_pairs:
            source_key = self._normalize_move_identifier(source_name)
            target_key = self._normalize_move_identifier(target_name)

            source_digits = pic_lengths.get(source_key)
            target_digits = pic_lengths.get(target_key)

            if not source_digits or not target_digits:
                continue

            if source_digits > target_digits:
                fixes[target_key] = max(
                    fixes.get(target_key, 0),
                    source_digits,
                )

        if not fixes:
            self.messages.append("Auto-fix PIC: no target PIC expansion required.")
            return converted_cobol_text

        updated = self._apply_fixes(
```

```

        converted_cobol_text=converted_cobol_text,
        fixes=fixes,
    )

    return updated

def _parse_numeric_pic_lengths(
    self,
    text: str,
) -> dict[str, int]:
    output: dict[str, int] = {}

    for line in str(text or "").splitlines():
        match = DATA_ENTRY_START_PATTERN.match(line)

        if not match:
            continue

        name = self._normalize_move_identifier(match.group("name"))
        rest = match.group("rest") or ""

        digits = self._numeric_pic_digits(rest)

        if name and digits:
            output[name] = digits

    return output

def _parse_move_pairs(
    self,
    text: str,
) -> list[tuple[str, str]]:
    output: list[tuple[str, str]] = []

    for match in MOVE_PATTERN.finditer(str(text or "")):
        source = str(match.group("source") or "")
        target = str(match.group("target") or "")

        if source and target:
            output.append((source, target))

    return output

def _numeric_pic_digits(
    self,
    text: str,
) -> int | None:
    match = NUMERIC_PIC_PATTERN.search(str(text or ""))

    if not match:
        return None

    explicit_length = match.group("len")
    pic = match.group("pic") or ""

    if explicit_length:
        try:
            return int(explicit_length)
        except ValueError:
            return None

    return pic.upper().count("9")

def _apply_fixes(
    self,
    converted_cobol_text: str,
    fixes: dict[str, int],
) -> str:
    output_lines: list[str] = []

    for line in converted_cobol_text.splitlines():
        match = DATA_ENTRY_START_PATTERN.match(line)

        if not match:
            output_lines.append(line)
            continue

```



```

        field_name = self._normalize_move_identifier(match.group("name"))
        required_digits = fixes.get(field_name)

        if not required_digits:
            output_lines.append(line)
            continue

        updated_line = self._replace_pic_digits(
            line=line,
            required_digits=required_digits,
        )

        if updated_line != line:
            self.messages.append(
                f"Auto-fix PIC: expanded {field_name} to 9({required_digits})."
            )

        output_lines.append(updated_line)

    return "\n".join(output_lines).rstrip() + "\n"

def _replace_pic_digits(
    self,
    line: str,
    required_digits: int,
) -> str:
    def repl(match) -> str:
        pic = match.group("pic") or ""
        trailing = match.group("trailing") or ""
        dot = match.group("dot") or ""
        sign_prefix = "S" if pic.upper().startswith("S") else ""
        new_pic = f"{sign_prefix}9({required_digits})"

        return f"PIC {new_pic}{trailing}{dot}"

    return NUMERIC_PIC_PATTERN.sub(
        repl,
        line,
        count=1,
    )

def _normalize_move_identifier(
    self,
    value: str,
) -> str:
    text = str(value or "").strip()

    if "." in text:
        text = text.split(".")[1]

    return NameNormalizer.to_cobol(text)

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\transformers\qualified_reference_transformer.py
ENCODING: utf-8
=====

```

```

"""
Qualified IDMS field reference transformer.

```

Rewrites safe qualified references only:

- FIELD OF IDMS-RECORD
- FIELD IN IDMS-RECORD

Bare words are not globally rewritten.

```

"""

```

```

from patterns.field_usage_patterns import QUALIFIED_REFERENCE_PATTERN
from rules.field_mapping_rules import QUALIFIED_REFERENCE_REWRITE_RULES

```

```

class QualifiedReferenceTransformer:
    """

```

```
Rewrites qualified IDMS field references to DB2 DCLGEN host references when
mapping metadata exists.
"""
```

```
def __init__(
    self,
    mapping_repository,
    dclgen_repository,
) -> None:
    self.mapping_repository = mapping_repository
    self.dclgen_repository = dclgen_repository

def transform_line(
    self,
    line: str,
) -> str:
    if not line:
        return ""

    return QUALIFIED_REFERENCE_PATTERN.sub(
        self._replace_reference,
        line,
    )

def _replace_reference(
    self,
    match,
) -> str:
    field_name = match.group("field")
    record_name = match.group("record")

    mapped_column = self.mapping_repository.find_column_mapping(
        record_name=record_name,
        idms_field_name=field_name,
    )

    if not mapped_column:
        return match.group(0)

    host_reference = self.dclgen_repository.find_host_reference(
        table_name=mapped_column.db2_table_name,
        column_name=mapped_column.db2_column_name,
    )

    if not host_reference:
        return match.group(0)

    return host_reference
```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\ui\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\ui\main_page.py
ENCODING: utf-8
=====
```

```
import re
from pathlib import Path

import streamlit as st

from idms_db2_phase2.analyzers.metadata_service import MetadataService
from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.orchestration.conversion_service import ConversionService
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser
```

```

def initialize_session_state() -> None:
    defaults = {
        "sheet_mapping_rows": [],
        "dclgen_columns": [],
        "copybook_fields": [],
        "idms_cobol_text": "",
        "idms_cobol_source_name": "",
        "converted_cobol": "",
        "converted_cobol_file_name": "converted_db2_cobol.cbl",
        "validation_messages": [],
        "operations": [],
        "generated": False,
        "loaded": False,
        "diagnostics": [],
        "uploaded_file_names": {},
        "auto_fix_pic_length_mismatches": False,
    }

    for key, value in defaults.items():
        if key not in st.session_state:
            st.session_state[key] = value

def render_main_page() -> None:
    initialize_session_state()

    tabs = st.tabs(
        [
            "Main",
            "Metadata Overview",
            "Column Names",
            "Sets",
            "Sheet Mapping Rows",
            "Generated DB2 COBOL",
            "Validation",
            "Diagnostics",
        ]
    )

    with tabs[0]:
        render_main_tab()

    with tabs[1]:
        render_metadata_overview_tab()

    with tabs[2]:
        render_column_names_tab()

    with tabs[3]:
        render_sets_tab()

    with tabs[4]:
        render_sheet_mapping_rows_tab()

    with tabs[5]:
        render_generated_cobol_tab()

    with tabs[6]:
        render_validation_tab()

    with tabs[7]:
        render_diagnostics_tab()

def render_main_tab() -> None:
    st.markdown("## Upload Inputs")

    st.info(
        "Upload Sheet Mapping, one or more DCLGEN files, optional Copybook files, "
        "and the IDMS COBOL source text file to generate DB2 embedded SQL COBOL."
    )

    col1, col2 = st.columns(2)

    with col1:

```

```

sheet_mapping_file = st.file_uploader(
    "Sheet Mapping Excel or CSV",
    type=["xlsx", "csv"],
    key="sheet_mapping_file",
    help=(
        "Upload Sheet Mapping as .xlsx or .csv. "
        "If your file is .xls, save it as .xlsx or .csv first."
    ),
)

dclgen_files = st.file_uploader(
    "DCLGEN text file or files",
    type=["txt", "cbl", "cpy"],
    accept_multiple_files=True,
    key="dclgen_files",
)

with col2:
    copybook_files = st.file_uploader(
        "Optional Copybook text file or files",
        type=["txt", "cbl", "cpy"],
        accept_multiple_files=True,
        key="copybook_files",
    )

    idms_cobol_source_file = st.file_uploader(
        "IDMS COBOL source code",
        type=["txt", "cbl", "cob"],
        key="idms_cobol_source_file",
    )

st.session_state.auto_fix_pic_length_mismatches = st.checkbox(
    "Auto-fix PIC length mismatches",
    value=st.session_state.auto_fix_pic_length_mismatches,
    help="If enabled, the converter can expand generated target PIC lengths when needed.",
)

target_program_id = st.text_input(
    "Target PROGRAM-ID",
    value="",
    help="Optional. If provided, the generated COBOL PROGRAM-ID will be replaced.",
)

col_load, col_generate = st.columns(2)

with col_load:
    if st.button("Load and Analyze Inputs", type="primary"):
        load_and_analyze_inputs(
            sheet_mapping_file=sheet_mapping_file,
            dclgen_files=dclgen_files,
            copybook_files=copybook_files,
            idms_cobol_source_file=idms_cobol_source_file,
        )

with col_generate:
    if st.button("Generate DB2 COBOL"):
        generate_db2_cobol(target_program_id=target_program_id)

render_current_status()

def load_and_analyze_inputs(
    sheet_mapping_file,
    dclgen_files,
    copybook_files,
    idms_cobol_source_file,
) -> None:
    diagnostics = []
    uploaded_file_names = {}

    file_loader = FileLoader()
    sheet_mapping_parser = SheetMappingParser()
    dclgen_parser = DclgenParser()
    copybook_parser = CopybookParser()

    sheet_rows = []

```

```

dclgen_columns = []
copybook_fields = []

if sheet_mapping_file is None:
    diagnostics.append("Sheet Mapping file not uploaded.")
else:
    uploaded_file_names["sheet_mapping_file"] = str(sheet_mapping_file.name or "")

    try:
        sheet_rows = sheet_mapping_parser.parse(sheet_mapping_file)
        diagnostics.append(f"Sheet Mapping uploaded: {sheet_mapping_file.name}")
        diagnostics.append(f"Sheet Mapping parsed rows: {len(sheet_rows)}")
    except Exception as exc:
        diagnostics.append(f"Sheet Mapping parse failed: {exc}")
        sheet_rows = []

dclgen_file_names = []

if not dclgen_files:
    diagnostics.append("DCLGEN file not uploaded.")
else:
    dclgen_text_parts = []

    for file in dclgen_files:
        file_name = str(file.name or "")
        dclgen_file_names.append(file_name)

        try:
            text = file_loader.read_uploaded_text(file)
            diagnostics.append(f"DCLGEN uploaded: {file_name}")
            diagnostics.append(f"DCLGEN text length for {file_name}: {len(text)}")
            dclgen_text_parts.append(text)
        except Exception as exc:
            diagnostics.append(f"DCLGEN read failed for {file_name}: {exc}")

    uploaded_file_names["dclgen_files"] = dclgen_file_names

    dclgen_text = "\n".join(dclgen_text_parts)

    try:
        dclgen_columns = dclgen_parser.parse(dclgen_text)
        diagnostics.append(f"DCLGEN parsed columns: {len(dclgen_columns)}")
    except Exception as exc:
        diagnostics.append(f"DCLGEN parse failed: {exc}")
        dclgen_columns = []

copybook_file_names = []

if not copybook_files:
    diagnostics.append("Copybook file not uploaded. Continuing without copybook.")
else:
    copybook_text_parts = []

    for file in copybook_files:
        file_name = str(file.name or "")
        copybook_file_names.append(file_name)

        try:
            text = file_loader.read_uploaded_text(file)
            diagnostics.append(f"Copybook uploaded: {file_name}")
            diagnostics.append(f"Copybook text length for {file_name}: {len(text)}")
            copybook_text_parts.append(text)
        except Exception as exc:
            diagnostics.append(f"Copybook read failed for {file_name}: {exc}")

    uploaded_file_names["copybook_files"] = copybook_file_names

    copybook_text = "\n".join(copybook_text_parts)

    try:
        copybook_fields = copybook_parser.parse(copybook_text)
        diagnostics.append(f"Copybook parsed fields: {len(copybook_fields)}")
    except Exception as exc:
        diagnostics.append(f"Copybook parse failed: {exc}")
        copybook_fields = []

```

```

idms_cobol_text = ""
idms_cobol_source_name = ""

if idms_cobol_source_file is None:
    diagnostics.append("IDMS COBOL source file not uploaded.")
else:
    idms_cobol_source_name = str(idms_cobol_source_file.name or "")
    uploaded_file_names[idms_cobol_source_file] = idms_cobol_source_name

    try:
        idms_cobol_text = file_loader.read_uploaded_text(idms_cobol_source_file)
        diagnostics.append(f"IDMS COBOL source uploaded: {idms_cobol_source_name}")
        diagnostics.append(f"IDMS COBOL source text length: {len(idms_cobol_text)}")
    except Exception as exc:
        diagnostics.append(f"IDMS COBOL source read failed: {exc}")
        idms_cobol_text = ""

st.session_state.sheet_mapping_rows = sheet_rows
st.session_state.dclgen_columns = dclgen_columns
st.session_state.copybook_fields = copybook_fields
st.session_state.idms_cobol_text = idms_cobol_text
st.session_state.idms_cobol_source_name = idms_cobol_source_name
st.session_state.converted_cobol = ""
st.session_state.converted_cobol_file_name = "converted_db2_cobol.cbl"
st.session_state.validation_messages = []
st.session_state.operations = []
st.session_state.generated = False
st.session_state.loaded = True
st.session_state.diagnostics = diagnostics
st.session_state.uploaded_file_names = uploaded_file_names

st.success("Inputs loaded and analyzed.")

def generate_db2_cobol(target_program_id: str) -> None:
    if not st.session_state.loaded:
        st.warning("Load and analyze inputs before generating DB2 COBOL.")
        return

    service = ConversionService()

    result = service.convert(
        ConversionInput(
            sheet_mapping_rows=st.session_state.sheet_mapping_rows,
            dclgen_columns=st.session_state.dclgen_columns,
            copybook_fields=st.session_state.copybook_fields,
            idms_cobol_text=st.session_state.idms_cobol_text,
            target_program_id=target_program_id,
            auto_fix_pic_length_mismatches=st.session_state.auto_fix_pic_length_mismatches,
        )
    )

    st.session_state.converted_cobol = result.converted_cobol
    st.session_state.validation_messages = result.validation_messages
    st.session_state.operations = result.operations
    st.session_state.converted_cobol_file_name = build_converted_cobol_file_name(
        target_program_id=target_program_id,
        source_file_name=st.session_state.idms_cobol_source_name,
    )
    st.session_state.generated = True

    st.success("DB2 COBOL generation completed.")

def build_converted_cobol_file_name(
    target_program_id: str,
    source_file_name: str,
) -> str:
    target = sanitize_file_stem(target_program_id)

    if target:
        return f"{target}.cbl"

    source_stem = ""

    if source_file_name:

```

```

        source_stem = Path(source_file_name).stem
        source_stem = sanitize_file_stem(source_stem)

    if source_stem:
        return f"{source_stem}_db2.cbl"

    return "converted_db2_cobol.cbl"

def sanitize_file_stem(value: str) -> str:
    text = str(value or "").strip()

    if not text:
        return ""

    text = re.sub(r"^[A-Za-z0-9_-]+", "_", text)
    text = re.sub(r"_+", "_", text)

    return text.strip("_")

def render_current_status() -> None:
    st.markdown("## Current Status")

    col1, col2, col3, col4 = st.columns(4)

    with col1:
        st.metric("Sheet Mapping Rows", len(st.session_state.sheet_mapping_rows))

    with col2:
        st.metric("DCLGEN Columns", len(st.session_state.dclgen_columns))

    with col3:
        st.metric("Copybook Fields", len(st.session_state.copybook_fields))

    with col4:
        st.metric("IDMS COBOL Length", len(st.session_state.idms_cobol_text))

    if st.session_state.loaded:
        st.success("Inputs loaded. Review metadata and diagnostics before generating.")

    if st.session_state.generated:
        st.success("DB2 COBOL generated. Open the Generated DB2 COBOL tab to download.")

def render_metadata_overview_tab() -> None:
    st.markdown("## Metadata Overview")

    metadata_service = MetadataService()

    summaries = metadata_service.record_summaries(
        st.session_state.sheet_mapping_rows,
    )

    if summaries:
        st.dataframe(
            summaries,
            use_container_width=True,
            hide_index=True,
        )
    else:
        st.info("No metadata available. Upload Sheet Mapping first.")

def render_column_names_tab() -> None:
    st.markdown("## Column Names")

    metadata_service = MetadataService()

    rows = metadata_service.column_rows(
        st.session_state.sheet_mapping_rows,
    )

    if rows:
        st.dataframe(
            rows,

```

```

        use_container_width=True,
        hide_index=True,
    )
else:
    st.info("No column rows available. Upload Sheet Mapping first.")

def render_sets_tab() -> None:
    st.markdown("## Sets")

    metadata_service = MetadataService()

    rows = metadata_service.relationship_summaries(
        st.session_state.sheet_mapping_rows,
    )

    if rows:
        st.dataframe(
            rows,
            use_container_width=True,
            hide_index=True,
        )
    else:
        st.info("No relationship rows available. Upload Sheet Mapping first.")

def render_sheet_mapping_rows_tab() -> None:
    st.markdown("## Sheet Mapping Rows")

    metadata_service = MetadataService()

    rows = metadata_service.mapping_preview_rows(
        st.session_state.sheet_mapping_rows,
    )

    if rows:
        st.dataframe(
            rows,
            use_container_width=True,
            hide_index=True,
        )
    else:
        st.info("No Sheet Mapping rows available. Upload Excel or CSV first.")

def render_generated_cobol_tab() -> None:
    st.markdown("## Generated DB2 COBOL")

    if not st.session_state.converted_cobol:
        st.info("Generate DB2 COBOL from the Main tab.")
        return

    st.caption(
        f"Output file name: `{st.session_state.converted_cobol_file_name}`"
    )

    st.download_button(
        label="Download Generated COBOL",
        data=st.session_state.converted_cobol,
        file_name=st.session_state.converted_cobol_file_name,
        mime="text/plain",
        type="primary",
    )

    st.text_area(
        "Final DB2 COBOL Code",
        value=st.session_state.converted_cobol,
        height=760,
    )

def render_validation_tab() -> None:
    st.markdown("## Validation")

    if not st.session_state.validation_messages:
        st.success("No validation messages.")

```



```

        return

    for message in st.session_state.validation_messages:
        st.warning(message)

def render_diagnostics_tab() -> None:
    st.markdown("## Diagnostics")

    st.markdown("### Uploaded Files")
    st.json(st.session_state.uploaded_file_names)

    st.markdown("### Parser Diagnostics")

    diagnostics = st.session_state.diagnostics or []

    if not diagnostics:
        st.info("No diagnostics available. Click Load and Analyze Inputs first.")
        return

    st.code(
        "\n".join(diagnostics),
        language="text",
    )

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\validators\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\validators\input_validator.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import ConversionInput
from rules.validation_rules import INPUT_VALIDATION_MESSAGES

```

```

class InputValidator:
    """
    Validates required conversion inputs.

    This validator checks only input availability. It does not validate
    generated COBOL production quality.
    """

    def validate(
        self,
        conversion_input: ConversionInput,
    ) -> list[str]:
        messages: list[str] = []

        if not conversion_input.sheet_mapping_rows:
            messages.append(
                INPUT_VALIDATION_MESSAGES["missing_sheet_mapping"]
            )

        if not conversion_input.dclgen_columns:
            messages.append(
                INPUT_VALIDATION_MESSAGES["missing_dclgen"]
            )

        if not conversion_input.idms_cobol_text.strip():
            messages.append(
                INPUT_VALIDATION_MESSAGES["missing_idms_cobol"]
            )

        return messages

```

```
=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\validators\mapping_validator.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from rules.validation_rules import MAPPING_VALIDATION_MESSAGES
```

```
class MappingValidator:
```

```
    """
```

```
    Validates Sheet Mapping and DCLGEN consistency.
```

```
    This validator checks metadata availability and resolver alignment.
```

```
    """
```

```
    def __init__(
```

```
        self,
```

```
        mapping_repository: MappingRepository,
```

```
        dclgen_repository: DclgenRepository,
```

```
        table_name_resolver: TableNameResolver,
```

```
    ) -> None:
```

```
        self.mapping_repository = mapping_repository
```

```
        self.dclgen_repository = dclgen_repository
```

```
        self.table_name_resolver = table_name_resolver
```

```
    def validate(
```

```
        self,
```

```
    ) -> list[str]:
```

```
        messages: list[str] = []
```

```
        if self.mapping_repository.count() == 0:
```

```
            messages.append(
```

```
                MAPPING_VALIDATION_MESSAGES["missing_mapping_rows"]
```

```
            )
```

```
        if self.dclgen_repository.count() == 0:
```

```
            messages.append(
```

```
                MAPPING_VALIDATION_MESSAGES["missing_dclgen_columns"]
```

```
            )
```

```
        messages.extend(self._validate_record_table_resolution())
```

```
        messages.extend(self._validate_table_column_resolution())
```

```
        return messages
```

```
    def _validate_record_table_resolution(
```

```
        self,
```

```
    ) -> list[str]:
```

```
        messages: list[str] = []
```

```
        for record in self.mapping_repository.records():
```

```
            table = self.table_name_resolver.table_for_record(record)
```

```
            if table:
```

```
                continue
```

```
            messages.append(
```

```
                f"{MAPPING_VALIDATION_MESSAGES['missing_record_mapping']} Record={record}"
```

```
            )
```

```
        return messages
```

```
    def _validate_table_column_resolution(
```

```
        self,
```

```
    ) -> list[str]:
```

```
        messages: list[str] = []
```

```
        for table in self.mapping_repository.tables():
```

```
            resolved_table = self.table_name_resolver.resolve_table(table)
```

```
            if not resolved_table:
```

```
                continue
```

```

        columns = self.mapping_repository.db2_columns_for_table(table)

        if columns:
            continue

        messages.append(
            f"{MAPPING_VALIDATION_MESSAGES['missing_column_mapping']} Table={resolved_table}"
        )

    return messages

```

```

=====
FILE: IDMS-DB2-Code-Conversion\src\idms_db2_phase2\validators\production_validator.py
ENCODING: utf-8
=====

```

```

"""
Production validator.

Performs production-focused validation for generated DB2 COBOL.

```

```

It detects:
- Missing required DB2 constructs.
- Undefined TODO host variables.
- Generated DB2 error markers.
- Residual executable IDMS statements.
- Residual IDMS declarative/control statements.
- Generated DCLGEN host variables not found in uploaded DCLGEN metadata.
"""

```

```

from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.sequence_patterns import strip_sequence_numbers
from patterns.sql_patterns import HOST_REFERENCE_PATTERN
from patterns.validation_patterns import (
    ERROR_DB2_PATTERN,
    FORBIDDEN_EXECUTABLE_IDMS_PATTERNS,
    FORBIDDEN_IDMS_DECLARATIVE_PATTERNS,
    NO_FETCH_HOST_VARIABLES_PATTERN,
    TODO_DB2_PATTERN,
    TODO_HOST_VARIABLE_PATTERN,
    UNABLE_TO_DECLARE_CURSOR_PATTERN,
)
from rules.db2_validation_rules import REQUIRED_DB2_TOKENS
from rules.validation_rules import PRODUCTION_VALIDATION_MESSAGES

```

```

class ProductionValidator:
    def __init__(
        self,
        dclgen_repository: DclgenRepository,
    ) -> None:
        self.dclgen_repository = dclgen_repository

    def validate(
        self,
        converted_cobol_text: str,
    ) -> list[str]:
        messages: list[str] = []
        text = str(converted_cobol_text or "")

        if not text.strip():
            return ["Production validation: converted COBOL text is empty."]

        self._validate_required_db2_tokens(text, messages)
        self._validate_no_todo_or_generated_error(text, messages)
        self._validate_forbidden_idms_patterns(text, messages)
        self._validate_forbidden_idms_declaratives(text, messages)
        self._validate_generated_dclgen_host_variables(text, messages)

        return messages

    def _validate_required_db2_tokens(
        self,
        converted_cobol_text: str,

```

```

        messages: list[str],
    ) -> None:
        upper = converted_cobol_text.upper()

        for token in REQUIRED_DB2_TOKENS:
            if token not in upper:
                messages.append(
                    f"Production validation: required DB2 token missing: {token}"
                )

def _validate_no_todo_or_generated_error(
    self,
    converted_cobol_text: str,
    messages: list[str],
) -> None:
    if TODO_HOST_VARIABLE_PATTERN.search(converted_cobol_text):
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["todo_host_variable"]
        )

    if TODO_DB2_PATTERN.search(converted_cobol_text):
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["todo_db2"]
        )

    if ERROR_DB2_PATTERN.search(converted_cobol_text):
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["error_db2"]
        )

    if UNABLE_TO_DECLARE_CURSOR_PATTERN.search(converted_cobol_text):
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["unable_to_declare_cursor"]
        )

    if NO_FETCH_HOST_VARIABLES_PATTERN.search(converted_cobol_text):
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["no_fetch_host_variables"]
        )

def _validate_forbidden_idms_patterns(
    self,
    converted_cobol_text: str,
    messages: list[str],
) -> None:
    for line_number, line in enumerate(
        converted_cobol_text.splitlines(),
        start=1,
    ):
        logical = self._logical_line(line)

        if self._is_comment_or_blank(logical):
            continue

        for pattern in FORBIDDEN_EXECUTABLE_IDMS_PATTERNS:
            if pattern.search(logical):
                messages.append(
                    "Production validation: residual executable IDMS statement remains "
                    f"near line {line_number}: {logical}"
                )
                break

def _validate_forbidden_idms_declaratives(
    self,
    converted_cobol_text: str,
    messages: list[str],
) -> None:
    for line_number, line in enumerate(
        converted_cobol_text.splitlines(),
        start=1,
    ):
        logical = self._logical_line(line)

        if self._is_comment_or_blank(logical):
            continue

```

```

        for pattern in FORBIDDEN_IDMS_DECLARATIVE_PATTERNS:
            if pattern.search(logical):
                messages.append(
                    "Production validation: residual IDMS declarative/control statement remains "
                    f"near line {line_number}: {logical}"
                )
                break

def _validate_generated_dclgen_host_variables(
    self,
    converted_cobol_text: str,
    messages: list[str],
) -> None:
    generated_hosts = self._generated_dclgen_host_references(
        converted_cobol_text,
    )

    if not generated_hosts:
        return

    valid_hosts = self.dclgen_repository.valid_host_references()

    if not valid_hosts:
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["no_dclgen_hosts"]
        )
        return

    missing_hosts = sorted(
        host for host in generated_hosts if host not in valid_hosts
    )

    for host in missing_hosts:
        messages.append(
            f"Production validation: generated host variable {host} "
            "was not found in uploaded DCLGEN columns."
        )

def _generated_dclgen_host_references(
    self,
    converted_cobol_text: str,
) -> set[str]:
    output: set[str] = set()

    for match in HOST_REFERENCE_PATTERN.finditer(converted_cobol_text):
        group = NameNormalizer.to_cobol(match.group("group"))
        field = NameNormalizer.to_cobol(match.group("field"))

        if group and field:
            output.add(f"{group} . {field}")

    return output

def _logical_line(
    self,
    line: str,
) -> str:
    return strip_sequence_numbers(line)

def _is_comment_or_blank(
    self,
    line: str,
) -> bool:
    stripped = str(line or "").strip()

    if not stripped:
        return True

    if stripped.startswith("*"):
        return True

    return False

```

```
=====
FILE: IDMS-DB2-Code-Conversion\tests\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\tests\test_conversion_service.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.domain.models import ConversionInput, DclgenColumn, SheetMappingRow
from idms_db2_phase2.orchestration.conversion_service import ConversionService
```

```
def test_conversion_service_returns_empty_when_source_missing():
    service = ConversionService()

    result = service.convert(ConversionInput())

    assert result.converted_cobol == ""
    assert result.validation_messages
    assert result.operations == []

def test_conversion_service_generates_db2_infrastructure_and_sql_error():
    service = ConversionService()

    conversion_input = ConversionInput(
        sheet_mapping_rows=[
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                cobol_zone="NR-ID",
                idms_key="CALC",
                db2_key="PRIMARY KEY",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            ),
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                cobol_zone="CD-NAME",
                new_db2_record="DZBFARTB",
                new_db2_field_name="CD_NAME",
            ),
        ],
        dclgen_columns=[
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            ),
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="CD_NAME",
                cobol_host_name="CD-NAME",
            ),
        ],
        idms_cobol_text="""
IDENTIFICATION DIVISION.
PROGRAM-ID. OLDPROG.
DATA DIVISION.
WORKING-STORAGE SECTION.
PROCEDURE DIVISION.
OBTAIN VMB-FAR CALC.
STORE VMB-FAR.
END PROGRAM OLDPROG.
""",
        target_program_id="NEWPROG",
    )

    result = service.convert(conversion_input)
    text = result.converted_cobol

    assert "PROGRAM-ID. NEWPROG." in text
```

```

assert "EXEC SQL" in text
assert "INCLUDE SQLCA" in text
assert "SQL-ERROR." in text
assert "FROM DZBFARTV" in text
assert "INSERT INTO DZBFARTV" in text
assert result.operations

```

```

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_copybook_parser.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_dclgen_parser.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_generators.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import DclgenColumn, IdmsOperation, SheetMappingRow
from idms_db2_phase2.generators.cursor_paragraph_generator import CursorParagraphGenerator
from idms_db2_phase2.generators.db2_infrastructure_generator import Db2InfrastructureGenerator
from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.generators.sql_generator import SqlGenerator
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver

```

```

def build_generator_context():
    mapping_repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                idms_key="CALC",
                db2_key="PRIMARY KEY",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            ),
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                new_db2_record="DZBFARTB",
                new_db2_field_name="CD_NAME",
            ),
        ]
    )

    dclgen_repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            ),
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="CD_NAME",
                cobol_host_name="CD-NAME",
            ),
        ]
    )

    table_resolver = TableNameResolver(

```

```

        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    column_resolver = ColumnNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    host_resolver = HostVariableResolver(
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    cursor_resolver = CursorNameResolver()

    return {
        "mapping_repository": mapping_repository,
        "dclgen_repository": dclgen_repository,
        "table_resolver": table_resolver,
        "column_resolver": column_resolver,
        "host_resolver": host_resolver,
        "cursor_resolver": cursor_resolver,
    }

def test_sql_error_generator_adds_paragraph_before_end_program():
    generator = SqlErrorGenerator()

    text = "IDENTIFICATION DIVISION.\nPROGRAM-ID. TEST.\nEND PROGRAM TEST.\n"

    result = generator.ensure_sql_error_paragraph(text)

    assert "SQL-ERROR." in result
    assert "DISPLAY 'DB2 SQL ERROR SQLCODE='" in result
    assert result.index("SQL-ERROR.") < result.index("END PROGRAM")

def test_sql_generator_select_by_key_uses_resolved_tv_table():
    context = build_generator_context()

    generator = SqlGenerator(
        mapping_repository=context["mapping_repository"],
        dclgen_repository=context["dclgen_repository"],
        table_name_resolver=context["table_resolver"],
        column_name_resolver=context["column_resolver"],
        host_variable_resolver=context["host_resolver"],
    )

    lines = generator.select_by_key("VMB-FAR")
    text = "\n".join(lines)

    assert "FROM DZBFARTV" in text
    assert ":NR-ID OF DCLDZBFARTV" in text

def test_sql_generator_insert_uses_resolved_tv_table():
    context = build_generator_context()

    generator = SqlGenerator(
        mapping_repository=context["mapping_repository"],
        dclgen_repository=context["dclgen_repository"],
        table_name_resolver=context["table_resolver"],
        column_name_resolver=context["column_resolver"],
        host_variable_resolver=context["host_resolver"],
    )

    lines = generator.insert("VMB-FAR")
    text = "\n".join(lines)

    assert "INSERT INTO DZBFARTV" in text
    assert ":NR-ID OF DCLDZBFARTV" in text

def test_sql_generator_update_uses_resolved_tv_table():

```



```

context = build_generator_context()

generator = SqlGenerator(
    mapping_repository=context["mapping_repository"],
    dclgen_repository=context["dclgen_repository"],
    table_name_resolver=context["table_resolver"],
    column_name_resolver=context["column_resolver"],
    host_variable_resolver=context["host_resolver"],
)

lines = generator.update("VMB-FAR")
text = "\n".join(lines)

assert "UPDATE DZBFARTV" in text
assert "WHERE" in text

def test_db2_infrastructure_generator_inserts_sqlca_and_dclgen_include():
    context = build_generator_context()

    generator = Db2InfrastructureGenerator(
        table_name_resolver=context["table_resolver"],
        column_name_resolver=context["column_resolver"],
        host_variable_resolver=context["host_resolver"],
        cursor_name_resolver=context["cursor_resolver"],
    )

    cobol = "IDENTIFICATION DIVISION.\nPROGRAM-ID. TEST.\nDATA DIVISION.\nPROCEDURE DIVISION.\nEND PROGRAM TEST.\n"

    updated, messages = generator.apply(
        cobol_text=cobol,
        operations=[
            IdmsOperation(
                operation="OBTAIN_FIRST",
                record_name="VMB-FAR",
                set_name="AR-VMB-FAR",
            )
        ],
    )

    assert "INCLUDE SQLCA" in updated
    assert "INCLUDE SQLERRWS" in updated
    assert "INCLUDE DZBFARTV" in updated
    assert "SQL-LOCATION" in updated
    assert messages

def test_cursor_paragraph_generator_creates_open_fetch_close():
    context = build_generator_context()

    db2_generator = Db2InfrastructureGenerator(
        table_name_resolver=context["table_resolver"],
        column_name_resolver=context["column_resolver"],
        host_variable_resolver=context["host_resolver"],
        cursor_name_resolver=context["cursor_resolver"],
    )

    generator = CursorParagraphGenerator(
        db2_infrastructure_generator=db2_generator,
        host_variable_resolver=context["host_resolver"],
        sql_error_generator=SqlErrorGenerator(),
    )

    cobol = "IDENTIFICATION DIVISION.\nPROGRAM-ID. TEST.\nPROCEDURE DIVISION.\nEND PROGRAM TEST.\n"

    updated, messages = generator.apply(
        cobol_text=cobol,
        operations=[
            IdmsOperation(
                operation="OBTAIN_FIRST",
                record_name="VMB-FAR",
                set_name="AR-VMB-FAR",
            )
        ],
    )

```

```

assert "710-OPEN-DZBFARC1." in updated
assert "720-FETCH-DZBFARC1." in updated
assert "730-CLOSE-DZBFARC1." in updated
assert "FETCH DZBFARC1" in updated
assert messages

```

```

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_imports.py
ENCODING: utf-8
=====

```

```

def test_core_imports():
    from idms_db2_phase2.domain.models import ConversionInput
    from idms_db2_phase2.orchestration.conversion_service import ConversionService
    from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser
    from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
    from idms_db2_phase2.parsers.copybook_parser import CopybookParser
    from idms_db2_phase2.parsers.cobol_parser import CobolParser

    assert ConversionInput is not None
    assert ConversionService is not None
    assert SheetMappingParser is not None
    assert DclgenParser is not None
    assert CopybookParser is not None
    assert CobolParser is not None

def test_clean_architecture_imports():
    from catalogs.sheet_mapping_schema import SHEET_MAPPING_MODEL_FIELD_MAP
    from patterns.idms_patterns import OBTAIN_CALC_PATTERN
    from rules.authority_rules import INPUT_AUTHORITY_RULES

    assert SHEET_MAPPING_MODEL_FIELD_MAP
    assert OBTAIN_CALC_PATTERN
    assert INPUT_AUTHORITY_RULES

def test_layer_imports():
    from idms_db2_phase2.repositories.mapping_repository import MappingRepository
    from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
    from idms_db2_phase2.generators.sql_generator import SqlGenerator
    from idms_db2_phase2.transformers.cobol_transformer import CobolTransformer
    from idms_db2_phase2.validators.production_validator import ProductionValidator
    from idms_db2_phase2.composers.cobol_formatter import CobolFormatter

    assert MappingRepository is not None
    assert TableNameResolver is not None
    assert SqlGenerator is not None
    assert CobolTransformer is not None
    assert ProductionValidator is not None
    assert CobolFormatter is not None

```

```

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_name_normalizer.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer

def test_normalize_replaces_hyphen_with_underscore():
    assert NameNormalizer.normalize("VMB-FAR") == "VMB_FAR"

def test_to_cobol_replaces_underscore_with_hyphen():
    assert NameNormalizer.to_cobol("NR_ID_479_FAR") == "NR-ID-479-FAR"

def test_remove_record_suffix_removes_four_digit_suffix():
    assert NameNormalizer.remove_record_suffix("FIELD_NAME_0410") == "FIELD_NAME"

def test_compact_removes_non_alphanumeric_characters():
    assert NameNormalizer.compact("DZ-BFAR_TV") == "DZBFARTV"

```

```

def test_normalize_none_returns_empty_string():
    assert NameNormalizer.normalize(None) == ""

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_parsers.py
ENCODING: utf-8
=====

from idms_db2_phase2.parsers.cobol_parser import CobolParser
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser

def test_cobol_parser_detects_program_id():
    parser = CobolParser()

    assert parser.program_id("PROGRAM-ID. VMDZ1567.") == "VMDZ1567"

def test_cobol_parser_detects_obtain_calc():
    parser = CobolParser()

    operations = parser.analyze("OBTAIN VMB-FAR CALC.")

    assert len(operations) == 1
    assert operations[0].operation == "OBTAIN_CALC"
    assert operations[0].record_name == "VMB-FAR"

def test_cobol_parser_detects_store():
    parser = CobolParser()

    operations = parser.analyze("STORE VMB-FAR.")

    assert len(operations) == 1
    assert operations[0].operation == "STORE"
    assert operations[0].record_name == "VMB-FAR"

def test_copybook_parser_pareses_field():
    parser = CopybookParser()

    fields = parser.parse(
        """
        10 NR-ID PIC 9(8).
        """
    )

    assert len(fields) == 1
    assert fields[0].name == "NR-ID"
    assert fields[0].picture == "9(8)"

def test_dclgen_parser_pareses_inline_sql_declare():
    parser = DclgenParser()

    columns = parser.parse(
        """
        EXEC SQL DECLARE DZBFARTV TABLE
        (
            NR_ID DECIMAL(8,0) NOT NULL,
            CD_NAME CHAR(4)
        )
        END-EXEC.

        01 DCLDZBFARTV.
            10 NR-ID PIC S9(8) COMP-3.
            10 CD-NAME PIC X(4).
        """
    ,
        source_label="unit-test",
    )

```

```

assert len(columns) == 2
assert columns[0].table_name == "DZBFARTV"
assert columns[0].column_name == "NR_ID"
assert columns[0].cobol_host_name == "NR-ID"

def test_sheet_mapping_parser_parses_csv_text():
    parser = SheetMappingParser()

    rows = parser.parse_csv_text(
        "Cobol Record IDMS,Cobol Zone,IDMS Key,DB2 Key,New DB2 Record,New DB2 Field name\n"
        "VMB-FAR,NR-ID,CALC,PRIMARY KEY,DZBFARTB,NR_ID\n"
    )

    assert len(rows) == 1
    assert rows[0].cobol_record_idms == "VMB-FAR"
    assert rows[0].new_db2_record == "DZBFARTB"
    assert rows[0].new_db2_field_name == "NR_ID"

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_patterns.py
ENCODING: utf-8
=====

from patterns.cobol_patterns import PROGRAM_ID_PATTERN
from patterns.dclgen_patterns import DECLARE_TABLE_PATTERN
from patterns.idms_patterns import OBTAIN_CALC_PATTERN, STORE_PATTERN
from patterns.sql_patterns import EXEC_SQL_PATTERN, UPDATE_SQL_PATTERN
from patterns.validation_patterns import TODO_DB2_PATTERN

def test_program_id_pattern_matches():
    assert PROGRAM_ID_PATTERN.search("PROGRAM-ID. VMDZ1567.")

def test_dclgen_declare_table_pattern_matches():
    text = "EXEC SQL DECLARE DZBFARTV TABLE"

    match = DECLARE_TABLE_PATTERN.search(text)

    assert match
    assert match.group(1).strip() == "DZBFARTV"

def test_obtain_calc_pattern_matches():
    match = OBTAIN_CALC_PATTERN.search("OBTAIN VMB-FAR CALC.")

    assert match
    assert match.group("record") == "VMB-FAR"

def test_store_pattern_matches():
    match = STORE_PATTERN.search("STORE VMB-FAR.")

    assert match
    assert match.group("record") == "VMB-FAR"

def test_exec_sql_pattern_matches():
    assert EXEC_SQL_PATTERN.match("EXEC SQL")

def test_update_sql_pattern_matches_table():
    match = UPDATE_SQL_PATTERN.search("UPDATE DZBFARTV")

    assert match
    assert match.group("table") == "DZBFARTV"

def test_todo_db2_validation_pattern_matches():
    assert TODO_DB2_PATTERN.search("TODO DB2 mapping missing")

```

```
=====
FILE: IDMS-DB2-Code-Conversion\tests\test_repositories.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.domain.models import CopybookField, DclgenColumn, SheetMappingRow
from idms_db2_phase2.repositories.copybook_repository import CopybookRepository
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
```

```
def test_mapping_repository_returns_records():
    repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            )
        ]
    )

    assert repository.records() == ["VMB-FAR"]
```

```
def test_mapping_repository_finds_rows_for_record():
    repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            )
        ]
    )

    rows = repository.rows_for_record("VMB-FAR")

    assert len(rows) == 1
    assert rows[0].new_db2_record == "DZBFARTB"
```

```
def test_mapping_repository_returns_table_for_record():
    repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                new_db2_record="DZBFARTB",
            )
        ]
    )

    assert repository.db2_table_for_record("VMB-FAR") == "DZBFARTB"
```

```
def test_dclgen_repository_has_table():
    repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            )
        ]
    )

    assert repository.has_table("DZBFARTV")
```

```
def test_dclgen_repository_host_for_column():
    repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            )
        ]
    )
```

```

    )
]
)

assert repository.host_for_column("DZBFARTV", "NR_ID") == "NR-ID"

def test_copybook_repository_finds_field():
    repository = CopybookRepository(
        [
            CopybookField(
                level="10",
                name="NR-ID",
                picture="9(8)",
            )
        ]
    )

    assert repository.has_field("NR-ID")
    assert repository.picture_for("NR-ID") == "9(8)"

```

```

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_resolvers.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import DclgenColumn, SheetMappingRow
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver

```

```

def build_repositories():
    mapping_repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                idms_key="CALC",
                db2_key="PRIMARY KEY",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            )
        ]
    )

    dclgen_repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            )
        ]
    )

    return mapping_repository, dclgen_repository

def test_table_name_resolver_resolves_tb_to_tv():
    mapping_repository, dclgen_repository = build_repositories()

    resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    assert resolver.resolve_table("DZBFARTB") == "DZBFARTV"

def test_table_name_resolver_table_for_record():
    mapping_repository, dclgen_repository = build_repositories()

```

```

resolver = TableNameResolver(
    mapping_repository=mapping_repository,
    dclgen_repository=dclgen_repository,
)

assert resolver.table_for_record("VMB-FAR") == "DZBFARTV"

def test_column_name_resolver_columns_for_record():
    mapping_repository, dclgen_repository = build_repositories()

    table_resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    column_resolver = ColumnNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    assert column_resolver.columns_for_record("VMB-FAR") == ["NR_ID"]

def test_host_variable_resolver_host_reference_for_column():
    mapping_repository, dclgen_repository = build_repositories()

    table_resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    host_resolver = HostVariableResolver(
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    assert host_resolver.host_reference_for_column("DZBFARTB", "NR_ID") == ":NR-ID OF DCLDZBFARTV"

def test_cursor_name_resolver_from_table():
    resolver = CursorNameResolver()

    assert resolver.cursor_name_from_table("DZBFARTV") == "DZBFARC1"

```

```

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_rules.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_sheet_mapping_parser.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_table_name_resolver.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: IDMS-DB2-Code-Conversion\tests\test_transformers.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.domain.models import DclgenColumn, SheetMappingRow
from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.generators.sql_generator import SqlGenerator
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.transformers.cobol_transformer import CobolTransformer
from idms_db2_phase2.transformers.field_reference_rewriter import FieldReferenceRewriter
from idms_db2_phase2.transformers.idms_statement_transformer import IdmsStatementTransformer
from idms_db2_phase2.transformers.pic_length_auto_fixer import PicLengthAutoFixer
```

```
def build_transformer_context():
    mapping_repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                cobol_zone="NR-ID",
                idms_key="CALC",
                db2_key="PRIMARY KEY",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            ),
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                cobol_zone="CD-NAME",
                new_db2_record="DZBFARTB",
                new_db2_field_name="CD_NAME",
            ),
        ]
    )

    dclgen_repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            ),
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="CD_NAME",
                cobol_host_name="CD-NAME",
            ),
        ]
    )

    table_resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )
    column_resolver = ColumnNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )
    host_resolver = HostVariableResolver(
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )
    cursor_resolver = CursorNameResolver()

    sql_generator = SqlGenerator(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
        column_name_resolver=column_resolver,
        host_variable_resolver=host_resolver,
```



```

)

statement_transformer = IdmsStatementTransformer(
    sql_generator=sql_generator,
    sql_error_generator=SqlErrorGenerator(),
    table_name_resolver=table_resolver,
    cursor_name_resolver=cursor_resolver,
)

return {
    "mapping_repository": mapping_repository,
    "dclgen_repository": dclgen_repository,
    "table_resolver": table_resolver,
    "host_resolver": host_resolver,
    "statement_transformer": statement_transformer,
}

def test_idms_statement_transformer_converts_obtain_calc():
    context = build_transformer_context()

    lines, _opened_set = context["statement_transformer"].transform_line(
        line="OBTAIN VMB-FAR CALC.",
        current_division="PROCEDURE",
        sql_error_paragraph="SQL-ERROR",
    )

    text = "\n".join(lines)

    assert "Converted OBTAIN CALC" in text
    assert "SELECT" in text
    assert "FROM DZBFARTV" in text

def test_idms_statement_transformer_converts_store_to_insert():
    context = build_transformer_context()

    lines, _opened_set = context["statement_transformer"].transform_line(
        line="STORE VMB-FAR.",
        current_division="PROCEDURE",
        sql_error_paragraph="SQL-ERROR",
    )

    text = "\n".join(lines)

    assert "Converted STORE" in text
    assert "INSERT INTO DZBFARTV" in text

def test_idms_statement_transformer_removes_bind():
    context = build_transformer_context()

    lines, _opened_set = context["statement_transformer"].transform_line(
        line="BIND RUN-UNIT.",
        current_division="PROCEDURE",
        sql_error_paragraph="SQL-ERROR",
    )

    text = "\n".join(lines)

    assert "Removed IDMS BIND statement" in text
    assert "CONTINUE." in text

def test_cobol_transformer_converts_program_id_and_idms_statement():
    context = build_transformer_context()

    transformer = CobolTransformer(
        idms_statement_transformer=context["statement_transformer"],
    )

    source = """
IDENTIFICATION DIVISION.
PROGRAM-ID. OLDPROG.
PROCEDURE DIVISION.
OBTAIN VMB-FAR CALC.

```

```
END PROGRAM OLDPROG.
```

```
"""

    converted, messages, operations = transformer.transform(
        cobol_text=source,
        target_program_id="NEWPROG",
    )

    assert "PROGRAM-ID. NEWPROG." in converted
    assert "Converted OBTAIN CALC" in converted
    assert len(operations) == 1
    assert messages == []

def test_field_reference_rewriter_rewrites_qualified_reference():
    context = build_transformer_context()

    rewriter = FieldReferenceRewriter(
        mapping_repository=context["mapping_repository"],
        table_name_resolver=context["table_resolver"],
        host_variable_resolver=context["host_resolver"],
    )

    result = rewriter.rewrite("MOVE NR-ID OF VMB-FAR TO WS-NR-ID.")

    assert ":NR-ID OF DCLDZBFARTV" in result

def test_pic_length_auto_fixer_expands_target_pic():
    fixer = PicLengthAutoFixer()

    source = """
01 WS-SOURCE PIC 9(8).
01 WS-TARGET PIC 9(6).
"""

    converted = """
01 WS-TARGET PIC 9(6).
PROCEDURE DIVISION.
MOVE WS-SOURCE TO WS-TARGET.
"""

    result = fixer.fix(
        source_cobol_text=source,
        converted_cobol_text=converted,
    )

    assert "PIC 9(8)" in result
    assert fixer.messages

=====
FILE: IDMS-DB2-Code-Conversion\tests\test_validators.py
ENCODING: utf-8
=====

from idms_db2_phase2.domain.models import ConversionInput, DclgenColumn, SheetMappingRow
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.validators.input_validator import InputValidator
from idms_db2_phase2.validators.mapping_validator import MappingValidator
from idms_db2_phase2.validators.production_validator import ProductionValidator

def test_input_validator_detects_missing_inputs():
    validator = InputValidator()

    messages = validator.validate(ConversionInput())

    assert "Sheet Mapping is required and must contain rows." in messages
    assert "At least one DCLGEN file is required." in messages
    assert "IDMS COBOL source text file is required and must contain readable COBOL code." in messages

def test_input_validator_passes_with_required_inputs():
```

```

validator = InputValidator()

messages = validator.validate(
    ConversionInput(
        sheet_mapping_rows=[SheetMappingRow(cobol_record_idms="REC")],
        dclgen_columns=[DclgenColumn(table_name="TABLE")],
        idms_cobol_text="IDENTIFICATION DIVISION.",
    )
)

assert messages == []

def test_mapping_validator_detects_missing_metadata():
    mapping_repository = MappingRepository([])
    dclgen_repository = DclgenRepository([])
    table_resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    validator = MappingValidator(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    messages = validator.validate()

    assert "Mapping validation: Sheet Mapping rows are missing." in messages
    assert "Mapping validation: DCLGEN columns are missing." in messages

def test_mapping_validator_passes_basic_mapping():
    mapping_repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            )
        ]
    )
    dclgen_repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            )
        ]
    )
    table_resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    validator = MappingValidator(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    messages = validator.validate()

    assert "Mapping validation: Sheet Mapping rows are missing." not in messages
    assert "Mapping validation: DCLGEN columns are missing." not in messages

def test_production_validator_detects_missing_db2_tokens():
    dclgen_repository = DclgenRepository([])
    validator = ProductionValidator(dclgen_repository=dclgen_repository)

    messages = validator.validate("IDENTIFICATION DIVISION.")

    assert "Production validation: required DB2 token missing: EXEC SQL" in messages

```

```

assert "Production validation: required DB2 token missing: SQLCA" in messages
assert "Production validation: required DB2 token missing: END-EXEC" in messages

```

```

def test_production_validator_detects_residual_idms_statement():
    dclgen_repository = DclgenRepository([])
    validator = ProductionValidator(dclgen_repository=dclgen_repository)

    text = """
EXEC SQL
    INCLUDE SQLCA
END-EXEC.
PROCEDURE DIVISION.
OBTAIN VMB-FAR CALC.
"""

    messages = validator.validate(text)

    assert any("residual executable IDMS statement remains" in message for message in messages)

def test_production_validator_accepts_valid_host_reference():
    dclgen_repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            )
        ]
    )
    validator = ProductionValidator(dclgen_repository=dclgen_repository)

    text = """
EXEC SQL
    INCLUDE SQLCA
END-EXEC.
EXEC SQL
    SELECT NR_ID
    INTO :DCLDZBFARTV.NR-ID
    FROM DZBFARTV
END-EXEC.
"""

    messages = validator.validate(text)

    assert not any("was not found in uploaded DCLGEN columns" in message for message in messages)

```

```

=====
FILE: IDMS-DB2-Code-Conversion\tools\recover_files_from_pdf.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

import argparse
import re
import shutil
from pathlib import Path

```

```

import pdfplumber

```

```

DEFAULT_PDF_PATH = Path(r"C:\VSCode\Source-Code\output\phase2-17-08-2026.pdf")

```

```

PROJECT_ROOT_MARKER = "IDMS-DB2-Code-Conversion"

```

```

TARGET_FILES = {
    "patterns/field_usage_patterns.py",
    "src/idms_db2_phase2/analyzers/field_usage_analyzer.py",
    "rules/validation_rules.py",
    "src/idms_db2_phase2/services/validation_service.py",
    "src/idms_db2_phase2/orchestration/conversion_service.py",
    "src/idms_db2_phase2/services/conversion_service.py",
    "rules/fixed_format_rules.py",
}

```

```

"patterns/fixed_format_patterns.py",
"patterns/fixed_format_rules.py",
"src/idms_db2_phase2/composers/fixed_format_line_parser.py",
"src/idms_db2_phase2/composers/fixed_format_body_formatter.py",
"src/idms_db2_phase2/composers/fixed_format_wrapper.py",
"src/idms_db2_phase2/composers/fixed_format_sequence_manager.py",
"src/idms_db2_phase2/composers/fixed_format_composer.py",
}

def main() -> None:
    parser = argparse.ArgumentParser(
        description="Recover exported project files from the phase2 PDF export."
    )
    parser.add_argument(
        "pdf_path",
        nargs="?",
        default=str(DEFAULT_PDF_PATH),
        help=(
            "Path to phase2-17-08-2026.pdf. "
            f"Default: {DEFAULT_PDF_PATH}"
        ),
    )
    parser.add_argument(
        "--output",
        default="recovered_from_pdf",
        help="Output folder for recovered files. Default: recovered_from_pdf",
    )
    parser.add_argument(
        "--restore-targets",
        action="store_true",
        help="Also copy recovered Batch 1 and Batch 2 target files into the project.",
    )
    parser.add_argument(
        "--restore-all",
        action="store_true",
        help="Copy all recovered files into the current project. Use carefully.",
    )
    parser.add_argument(
        "--list",
        action="store_true",
        help="Only list recovered file paths.",
    )

    args = parser.parse_args()

    pdf_path = Path(args.pdf_path).resolve()
    output_dir = Path(args.output).resolve()
    project_root = Path.cwd().resolve()

    if not pdf_path.exists():
        raise FileNotFoundError(f"PDF not found: {pdf_path}")

    output_dir.mkdir(parents=True, exist_ok=True)

    extracted_text = extract_pdf_text(pdf_path)
    files = split_exported_files(extracted_text)

    if not files:
        raise RuntimeError(
            "No files were recovered. Check whether the PDF text contains FILE markers."
        )

    written = write_recovered_files(
        files=files,
        output_dir=output_dir,
    )

    print("")
    print("PDF recovery completed.")
    print(f"PDF path : {pdf_path}")
    print(f"Project root : {project_root}")
    print(f"Recovery output : {output_dir}")
    print(f"Recovered file count : {written}")
    print("")

```

```

if args.list:
    print("Recovered files:")
    for relative_path in sorted(files):
        print(f" - {relative_path}")
    return

target_matches = list_target_matches(files)

print("Batch 1 and Batch 2 files found in PDF:")
if target_matches:
    for relative_path in target_matches:
        print(f" - {relative_path}")
else:
    print(" - None")

missing_targets = missing_target_files(target_matches)

if missing_targets:
    print("")
    print("Batch 1 and Batch 2 files NOT found exactly by path:")
    for relative_path in missing_targets:
        print(f" - {relative_path}")

if args.restore_targets:
    restore_files(
        files_to_restore=target_matches,
        output_dir=output_dir,
        project_root=project_root,
    )

if args.restore_all:
    restore_files(
        files_to_restore=sorted(files),
        output_dir=output_dir,
        project_root=project_root,
    )

def extract_pdf_text(
    pdf_path: Path,
) -> str:
    page_texts: list[str] = []

    with pdfplumber.open(pdf_path) as pdf:
        for page_index, page in enumerate(pdf.pages, start=1):
            text = page.extract_text(
                x_tolerance=1,
                y_tolerance=3,
                layout=False,
            )

            if not text:
                print(f"Warning: no text extracted from page {page_index}.")
                continue

            page_texts.append(text)

    return "\n".join(page_texts)

def split_exported_files(
    extracted_text: str,
) -> dict[str, str]:
    lines = extracted_text.splitlines()

    files: dict[str, list[str]] = {}
    current_path = ""
    current_lines: list[str] = []

    for raw_line in lines:
        line = raw_line.rstrip()

        file_path = parse_file_marker(line)

        if file_path:
            if current_path:

```

```

        files.setdefault(current_path, []).extend(current_lines)

        current_path = normalize_export_path(file_path)
        current_lines = []
        continue

    if not current_path:
        continue

    if is_export_noise_line(line):
        continue

    if is_encoding_line(line):
        continue

    current_lines.append(clean_export_line(line))

if current_path:
    files.setdefault(current_path, []).extend(current_lines)

cleaned_files: dict[str, str] = {}

for path, content_lines in files.items():
    normalized_path = normalize_export_path(path)

    if not normalized_path:
        continue

    cleaned_files[normalized_path] = cleanup_file_content(
        "\n".join(content_lines)
    )

return cleaned_files


def parse_file_marker(
    line: str,
) -> str:
    text = str(line or "").strip()

    if not text.startswith("FILE:"):
        return ""

    text = text[len("FILE:") :].strip()

    encoding_index = text.upper().find("ENCODING:")

    if encoding_index >= 0:
        text = text[:encoding_index].strip()

    return text


def normalize_export_path(
    file_path: str,
) -> str:
    path = str(file_path or "").strip()

    path = path.replace("\\", "/")
    path = path.replace("\u00a0", " ")
    path = re.sub(r"\s+", " ", path)

    marker = PROJECT_ROOT_MARKER + "/"

    if marker in path:
        path = path.split(marker, 1)[1]

    path = path.strip("/ ")

    path = fix_common_pdf_path_spacing(path)
    path = fix_init_file_name(path)
    path = fix_known_pdf_path_errors(path)

    return path

```

```

def fix_common_pdf_path_spacing(
    path: str,
) -> str:
    fixed = path

    fixed = re.sub(
        r"\s+\.(py|md|txt|toml|csv|json|yaml|yml)$",
        r"\1",
        fixed,
        flags=re.IGNORECASE,
    )

    fixed = re.sub(
        r"\s+/",
        "/",
        fixed,
    )

    fixed = re.sub(
        r"/\s+",
        "/",
        fixed,
    )

    fixed = fixed.replace(" .py", ".py")
    fixed = fixed.replace(" .md", ".md")
    fixed = fixed.replace(" .txt", ".txt")
    fixed = fixed.replace(" .toml", ".toml")
    fixed = fixed.replace(" .csv", ".csv")

    return fixed

def fix_init_file_name(
    path: str,
) -> str:
    parts = path.split("/")
    fixed_parts: list[str] = []

    for part in parts:
        clean = part.strip()

        normalized = clean.replace(" ", "")
        normalized = normalized.replace("__init__", "__init__")
        normalized = normalized.replace("__init_", "__init__")
        normalized = normalized.replace("_init_", "__init__")
        normalized = normalized.replace("_init_.py", "__init__.py")
        normalized = normalized.replace("__init_.py", "__init__.py")
        normalized = normalized.replace("_init_.py", "__init__.py")

        if normalized.lower() in {
            "__init__.py",
            "__init_py",
            "__init_.py",
        }:
            clean = "__init__.py"
        else:
            clean = normalized

        fixed_parts.append(clean)

    return "/".join(fixed_parts)

def fix_known_pdf_path_errors(
    path: str,
) -> str:
    fixed = path

    fixed = fixed.replace(
        "src/idms_db2_phase2/transformers /",
        "src/idms_db2_phase2/transformers/",
    )

    fixed = fixed.replace("catalogs /", "catalogs/")
    fixed = fixed.replace("config /", "config/")

```



```

return fixed

def is_export_noise_line(
    line: str,
) -> bool:
    text = str(line or "").strip()

    if not text:
        return False

    if text.startswith("<!-- PageNumber="):
        return True

    if text.startswith("<!-- PageBreak"):
        return True

    if text.startswith("<!-- PageHeader="):
        return True

    if text.startswith("<!-- PageFooter="):
        return True

    if re.fullmatch(r"Page\s+\d+", text, flags=re.IGNORECASE):
        return True

    if re.fullmatch(
        r"PageNumber\s*=\s*['\"]?Page\s+\d+['\"]?",
        text,
        flags=re.IGNORECASE,
    ):
        return True

    if re.fullmatch(r"PageBreak", text, flags=re.IGNORECASE):
        return True

    if re.fullmatch(r"PageHeader=.*", text, flags=re.IGNORECASE):
        return True

    if re.fullmatch(r"PageFooter=.*", text, flags=re.IGNORECASE):
        return True

    if text.startswith("[") and "result(s)" in text and "truncated" in text:
        return True

    if text == "[Empty file]":
        return False

    if is_separator_line(text):
        return True

    if text in {" ", ""}:
        return True

    return False

def is_separator_line(
    text: str,
) -> bool:
    clean = str(text or "").strip()

    if not clean:
        return False

    if len(clean) < 8:
        return False

    separator_chars = set(clean)

    allowed = {
        "=",
        "-",
        "_",
        "*",
        "#",
    }

```

```

        "-",
        "-",
    }

    return separator_chars.issubset(allowed)

def is_encoding_line(
    line: str,
) -> bool:
    text = str(line or "").strip().upper()

    return text.startswith("ENCODING:")

def clean_export_line(
    line: str,
) -> str:
    text = str(line or "").rstrip()

    if text == "[Empty file]":
        return ""

    text = text.replace("\u00a0", " ")

    return text

def cleanup_file_content(
    content: str,
) -> str:
    text = str(content or "")

    text = text.replace("\uffff", "")
    text = text.replace("\u00a0", " ")

    lines: list[str] = []

    for raw_line in text.splitlines():
        line = raw_line.rstrip()

        if is_export_noise_line(line):
            continue

        lines.append(line)

    text = "\n".join(lines)

    text = re.sub(
        r"\n{4,}",
        "\n\n\n",
        text,
    )

    return text.rstrip() + "\n"

def write_recovered_files(
    files: dict[str, str],
    output_dir: Path,
) -> int:
    count = 0

    for relative_path, content in sorted(files.items()):
        if not relative_path:
            continue

        target_path = output_dir / relative_path
        target_path.parent.mkdir(parents=True, exist_ok=True)
        target_path.write_text(
            content,
            encoding="utf-8",
        )
        count += 1

    return count

```

```

def list_target_matches(
    files: dict[str, str],
) -> list[str]:
    normalized_files = {
        normalize_compare_path(path): path
        for path in files
    }

    matches: list[str] = []

    for target in sorted(TARGET_FILES):
        normalized_target = normalize_compare_path(target)

        if normalized_target in normalized_files:
            matches.append(normalized_files[normalized_target])

    return matches

def missing_target_files(
    target_matches: list[str],
) -> list[str]:
    matched_normalized = {
        normalize_compare_path(path)
        for path in target_matches
    }

    missing: list[str] = []

    for target in sorted(TARGET_FILES):
        if normalize_compare_path(target) not in matched_normalized:
            missing.append(target)

    return missing

def normalize_compare_path(
    value: str,
) -> str:
    return str(value or "").replace("\\", "/").strip("/ ").lower()

def restore_files(
    files_to_restore: list[str],
    output_dir: Path,
    project_root: Path,
) -> None:
    print("")
    print("Restoring recovered files into current project...")

    for relative_path in files_to_restore:
        source_path = output_dir / relative_path
        target_path = project_root / relative_path

        if not source_path.exists():
            print(f"Skip missing recovered source: {source_path}")
            continue

        target_path.parent.mkdir(parents=True, exist_ok=True)

        if target_path.exists():
            backup_path = next_backup_path(target_path)
            shutil.copy2(target_path, backup_path)
            print(f"Backup current file: {backup_path}")

        shutil.copy2(source_path, target_path)
        print(f"Restored: {relative_path}")

    print("")
    print("Restore completed.")

def next_backup_path(
    target_path: Path,

```

```

) -> Path:
    base_backup_path = target_path.with_suffix(
        target_path.suffix + ".broken_before_pdf_restore"
    )

    if not base_backup_path.exists():
        return base_backup_path

    index = 1

    while True:
        candidate = target_path.with_suffix(
            target_path.suffix + f".broken_before_pdf_restore_{index}"
        )

        if not candidate.exists():
            return candidate

        index += 1

if __name__ == "__main__":
    main()

```

```

=====
FILE: main.py
ENCODING: utf-8
=====

```

```

def main() -> None:
    print("Hello from idms-db2-code-conversion!")

```

```

if __name__ == "__main__":
    main()

```

```

=====
FILE: pyproject.toml
ENCODING: utf-8
=====

```

```

[project]
name = "idms-db2-code-conversion"
version = "0.1.0"
description = "IDMS COBOL to DB2 COBOL Phase 2 converter"
readme = "README.md"
requires-python = ">=3.14"
dependencies = [
    "openpyxl>=3.1.5",
    "pdfplumber>=0.11.10",
    "pydantic>=2.13.4",
    "pydantic-settings>=2.15.0",
    "streamlit>=1.61.1",
]

[tool.setuptools]
package-dir = {"" = "src"}

[tool.setuptools.packages.find]
where = ["src"]

[dependency-groups]
dev = [
    "pytest>=9.1.1",
]

```

```

=====
FILE: README.md
ENCODING: utf-8
=====

```

```

# IDMS DB2 Phase 2 Converter

```

Standalone Phase 2 project for converting IDMS COBOL programs into DB2 embedded SQL COBOL.

Inputs

- Sheet Mapping Excel or CSV
- DCLGEN text file or multiple DCLGEN files
- Copybook text file
- Optional Copybook PDF
- IDMS COBOL source code

Output

- Converted DB2 COBOL code
- Validation messages
- Metadata overview
- Record summary
- Column summary
- Set / relationship summary
- Sheet mapping preview

Run

```
```bash
set PYTHONPATH=src
python -m streamlit run src/idms_db2_phase2/app.py --server.port 8502

python src/idms_db2_phase2/testing/run_retrieval.py
```

```
=====
FILE: requirements.txt
ENCODING: utf-8
=====
```

```
openpyxl>=3.1.5
pdfplumber>=0.11.10
pydantic>=2.13.4
pydantic-settings>=2.15.0
streamlit>=1.61.1
pytest>=9.1.1
```

## EXPORT SUMMARY

```
=====
Project folder: IDMS-DB2-CC-New
Text/code files processed: 192
```